

Código Limpo em PHP



**Dicas de especialistas e práticas recomendadas para
escrever um PHP bonito, amigável e sustentável**

Código Limpo em PHP



**Dicas de especialistas e práticas recomendadas para
escrever um PHP bonito, amigável e sustentável**

Código Limpo em PHP

Código Limpo em PHP

Dicas de especialistas e práticas recomendadas para escrever um PHP bonito, amigável e sustentável

Por Jideon F. Marques

© Copyright 2022 Jideon Marques - Todos os direitos reservados.

O conteúdo deste ebook não pode ser reproduzido, duplicado ou transmitido sem permissão direta por escrito do autor ou do editor.

Sob nenhuma circunstância qualquer culpa ou responsabilidade legal será imputada ao editor, ou autor, por quaisquer danos, reparações ou perdas monetárias devido às informações contidas neste ebook, direta ou indiretamente.

Notícia legal:

Este ebook é protegido por direitos autorais. É apenas para uso pessoal. Você não pode alterar, distribuir, vender, usar, citar ou parafrasear qualquer parte ou o conteúdo deste ebook sem o consentimento do autor ou editor.

Aviso de isenção de responsabilidade:

Observe que as informações contidas neste documento são apenas para fins educacionais e de entretenimento. Todo esforço foi feito para apresentar informações precisas, atualizadas, confiáveis e completas. Nenhuma garantia de qualquer tipo é declarada ou implícita. Os leitores reconhecem que o autor não está envolvido na prestação de aconselhamento jurídico, financeiro, médico ou profissional. O conteúdo deste ebook foi derivado de várias fontes. Consulte um profissional médico licenciado antes de tentar este programa ou qualquer técnica descrita neste ebook.

Ao ler este documento, o leitor concorda que em nenhuma circunstância o autor é responsável por quaisquer lesões, morte, perdas, diretas ou indiretas, que sejam incorridas como resultado do uso das informações contidas neste documento, incluindo, mas não limitado a a, erros, omissões ou imprecisões.

Prefácio

PHP evoluiu ao longo de várias décadas de uma linguagem de script simples para criar páginas HTML para uma linguagem rica em recursos com um ecossistema extenso. Como a grande maioria dos sites ainda usa PHP, ele é um dos pilares da Internet.

Embora ainda seja amigável para iniciantes, ele pode ser usado para implementar tudo, desde pequenos sites até aplicativos corporativos usados em todo o mundo. No entanto, as baixas barreiras de entrada do PHP às vezes levam a códigos difíceis de entender e impossíveis de manter.

Com este livro, queremos apresentá-lo ao mundo do Código Limpo. Você aprenderá muito sobre a teoria e também como aplicar o conhecimento que aprendeu no mundo real. Você aprenderá quais ferramentas irão apoiá-lo nessa jornada e quais práticas recomendadas você deve usar para conseguir implementar o Clean Code com sucesso em sua equipe.

Para quem é este livro

Este livro destina-se a desenvolvedores PHP em início de carreira que desejam entender os fundamentos do código PHP de alta qualidade e a desenvolvedores PHP experientes que desejam se atualizar com as práticas recomendadas mais recentes.

O que este livro cobre

Capítulo 1, O que é código limpo e por que você deveria se importar?, apresenta o assunto principal do livro.

Capítulo 2, Quem decide o que são "boas práticas"?, explica como essas "regras" são decididas.

Capítulo 3, Code, Don't Do Stunts, mostra por que você deve considerar ser pragmático em vez de tentar exibir habilidades.

Capítulo 4, É mais do que apenas código, explica por que um perímetro de código limpo é maior do que apenas escrever código-fonte.

capítulo 5, Otimizando Seu Tempo e Separando Responsabilidades, explica como se tornar mais produtivo criando novos hábitos.

Capítulo 6, PHP Is Evolving – Deprecations and Revolutions, fornece uma visão geral rápida dos recursos mais esperados introduzidos no PHP, ajudando a escrever código limpo.

Capítulo 7, Code Quality Tools, ensina sobre ferramentas que o ajudarão a escrever código limpo e de fácil manutenção.

Capítulo 8, Code Quality Metrics, analisa todas as métricas que você precisa saber para avaliar a qualidade do seu código.

Capítulo 9, Organizing PHP Quality Tools, mostra como manter suas ferramentas organizadas.

Capítulo 10, Automated Testing, apresenta o teste automatizado e explica por que você deve fazê-lo.

Capítulo 11, Integração Contínua, explora como manter a qualidade do código de forma consistente e ao longo do tempo.

Capítulo 12, Trabalhando em equipe, apresenta as melhores práticas para trabalhar em uma equipe de desenvolvedores.

Capítulo 13, Criando Documentação Eficaz, demonstra como criar documentação útil e viva.

Para obter o máximo deste livro

Você precisará do PHP 8.0 ou superior instalado em seu computador. Todos os exemplos de código foram testados usando PHP 8.1 no Linux e macOS. Com pequenos ajustes, eles devem funcionar no Windows também, dependendo da sua configuração.

Software/hardware abordado no livro	Requisitos do sistema operacional
PHP 8.0 e superior	Linux, macOS ou Windows

Para acompanhar todos os capítulos, você também precisará instalar o Composer. Você encontrará mais informações sobre isso em Capítulo 9, Organizando PHP Quality Tools, ou em <https://getcomposer.org>.

Se você estiver usando a versão digital deste livro, aconselhamos que você mesmo digite o código ou acesse o código do repositório GitHub do livro (um link está disponível na próxima seção). Isso ajudará você a evitar possíveis erros relacionados à cópia e colagem de código.

Baixe os arquivos de código de exemplo

Você pode baixar os arquivos de código de exemplo para este livro do GitHub em <https://github.com/PacktPublishing/Clean-Code-in-PHP>. Se houver uma atualização no código, ela será atualizada no repositório do GitHub.

Também temos outros pacotes de código de nosso rico catálogo de livros e vídeos disponíveis em <https://github.com/PacktPublishing/>. Confira!

Baixe as imagens coloridas

Também fornecemos um arquivo PDF com imagens coloridas das capturas de tela e diagramas usados neste livro. Você pode baixá-lo aqui: <https://packt.link/b08Jl>.

Convenções usadas

Há uma série de convenções de texto usadas ao longo deste livro.

Código em texto: indica palavras de código em texto, nomes de tabelas de banco de dados, nomes de pastas, nomes de arquivos, extensões de arquivos, nomes de caminho, URLs fictícios, entrada do usuário e identificadores do Twitter. Aqui está um exemplo: “Então, criaremos naturalmente um serviço como o `UserRemover`, que executará essas duas tarefas seguidas”.

Um bloco de código é definido da seguinte forma:

```
<?php
```

```
exemplo de classe
```

```
{
```

```
public function doSomething() bool
```

```
{
```

```
retornar verdadeiro;
```

```
}
```

```
}
```

Quando queremos chamar sua atenção para uma parte específica de um bloco de código, as linhas ou itens relevantes são colocados em **negrito**:

```
{  
  
...  
  
"scripts": {  
  "analisar": [  
    "ferramentas/fornecedor/bin/php-cs-fixer fix src",  
    "ferramentas/fornecedor/bin/phpstan analyse --level 1 src"  
  ],  
  "post-update-cmd": "ferramentas -d de atualização do compositor",  
  "post-install-cmd": "ferramentas de atualização do compositor -d"  
}  
}
```

Qualquer entrada ou saída de linha de comando é escrita da seguinte forma:

```
$ php phploc src
```

Audacioso: indica um novo termo, uma palavra importante ou palavras que você

vê na tela. Por exemplo, palavras em menus ou caixas de diálogo aparecem em negrito. Aqui está um exemplo: “Passar o ponteiro do mouse sobre **TestClass** mostrará uma janela pop-up com uma explicação dizendo **TestClass** de tipo indefinido.”

Parte 1 - Apresentando o Código Limpo

O objetivo desta primeira parte é apresentar os conceitos de código limpo e a teoria por trás dele. Esta parte pretende ser bastante teórica, embora exemplos concretos sejam discutidos à medida que os capítulos avançam. A primeira abordagem do código limpo é proposta para explicar sua utilidade e porque ele é necessário na vida de um desenvolvedor.

Esta seção compreende os seguintes capítulos:

O que é código limpo e por que você deve se importar?

Se você é novo em PHP: Hypertext Preprocessor (PHP) e desenvolvimento de software em geral, há uma boa chance de você ainda não ter encontrado um código limpo. Todos nós começamos aqui; todos devem estar cientes disso. O código limpo é mais do que apenas um conjunto de regras. É uma mentalidade real - uma maneira de pensar sobre o que você está criando.

Muitas vezes encontramos desenvolvedores que estão em tecnologia da informação (TI) há muitos anos, até décadas, e nunca tiveram que se preocupar com código limpo. Isso pode ser por vários motivos: trabalhar sozinho em um projeto por muitos anos ou desenvolver em uma base de código legado são alguns dos muitos motivos pelos quais o “código limpo” não chegou aos seus ouvidos ou não foi uma de suas preocupações.

Fique atento ao código limpo — quanto mais cedo melhor. Como acontece com muitos outros hábitos, uma vez que você adota uma forma de trabalhar com código, é difícil mudar isso. Pode ser porque você não está nem um pouco convencido sobre o que é o código limpo, não tem certeza se ele se aplica a você ou pode ser preguiçoso em aplicar novas "regras" porque as suas já estão funcionando para você (tenha certeza de que todos já passou por isso). Mas se você se preocupa com isso e está tomando nota - bem, você já fez a parte mais difícil, porque isso significa que você está pronto para mudar seus hábitos. Não nos entenda mal — o objetivo não é mudar tudo o que você aprendeu até agora; é mais um caso de melhorar as habilidades que você já possui.

Você pode não estar ciente disso, mas é certo que você já possui alguns hábitos que se adequam totalmente aos princípios do código limpo e que são naturais para você e completamente novos para outros desenvolvedores. Você deve observar que o objetivo dos princípios de código limpo não é ser controverso.

Os principais tópicos que abordaremos neste capítulo estão listados aqui:

O que este livro abordará

Você já deve ter ouvido falar de livros sobre criação de software, livros sobre código limpo e assim por diante. Qual é a diferença entre este livro e os outros? Bem, percebemos que quando somos um neófito em desenvolvimento e descobrimos novas linguagens de programação, ou mesmo entrando em programação e código-fonte pela primeira vez, pode ser difícil entender como aplicar esses princípios sobre os quais você lê em todos os lugares. As linguagens de programação não oferecem todos os mesmos recursos – algumas estão evoluindo rapidamente, enquanto outras não estão mais evoluindo. As noções e a maneira como as coisas são nomeadas em um idioma podem não ser válidas em outros idiomas. É por isso que este livro se concentra especialmente no código limpo em PHP.

Você não terá que pensar em como implementar esses princípios e regras em PHP, o que é algo a menos para se lembrar na hora de absorver conhecimento. Você encontrará exemplos e ferramentas que se aplicam diretamente a você, usados em projetos profissionais e comprovados pelo setor. Assim, podemos ir direto aos pontos importantes que gostaríamos que você entendesse. Você pode colocar seu conhecimento em prática imediatamente enquanto lê este livro e, como resultado, aprender mais rápido. Além disso, você não precisa de mais de 10 anos de experiência para entender o que será mostrado. Apenas conhecimento básico de PHP é necessário para entender completamente o que será exposto.

Este livro é uma compilação de anos de experiência - anos de prática no campo, confrontados com problemas reais que tiveram de ser resolvidos de acordo com restrições técnicas, funcionais e de tempo e dinheiro. Este não será mais um livro cheio de princípios utópicos que não podem ser aplicados na vida real.

Mais do que se adequar diretamente ao seu ambiente falando apenas de PHP, este livro pode ser lido na ordem que você quiser. Está dividida em duas partes distintas. A primeira (que você está lendo no momento) vai expor um pouco da teoria sobre o que é código limpo e quais são os princípios básicos dele, aplicados diretamente na linguagem PHP até a versão 8.2, e assim por diante. A segunda parte será focada em ferramentas práticas que você pode usar para garantir que está seguindo as regras certas da maneira correta, configurando um ambiente e seu ambiente de desenvolvimento integrado (IDE) para ser o mais eficiente e limpo possível e obtendo métricas em seu código, testes automatizados, redação de documentação e muito mais. Isso significa que você pode pular partes, pegá-las na ordem que quiser e aprender na velocidade que quiser.

Você não terá que ler toda a parte teórica antes de entrar na prática – você pode mergulhar em ferramentas e exemplos concretos agora mesmo. Você também pode se concentrar na primeira parte antes de entrar nos aspectos práticos para ter certeza de entender completamente o que será explicado mais tarde. É com você.

Entendendo o que é código limpo

Código limpo é o ato de escrever código pensando no futuro, ou seja, pensando nas pessoas que colaborarão com você ou no projeto sem você. E quando estamos falando de “as pessoas que vão colaborar com você”, pode até ser você mesmo. Se nunca o fez, você deve tentar ler e manter o código-fonte que escreveu alguns meses (ou anos) atrás. Não é desafiador? Bem, não se preocupe - é difícil para todos. Você certamente achará isso complicado por muitos anos, e não há solução rápida para isso. É o mesmo processo para todos.

Há muitas razões para isso, conforme descrito aqui:

Falando sobre PHP, é extremamente interessante ver que a linguagem está evoluindo muito e rapidamente nos últimos anos. Muitos pediram a morte do PHP alguns anos atrás. É verdade que antes do PHP 7, a linguagem realmente não ia bem. PHP 7 e versões mais recentes são uma grande lufada de ar puro para nós. Novas versões estão sendo lançadas com frequência e recursos estão sendo adicionados ao PHP (basta olhar para a expressão de correspondência e o suporte de enumerações no PHP 8.1!). A digitação estrita também foi introduzida, o que também é um gigantesco passo à frente. Iremos mergulhar nas evoluções do PHP que facilitam a escrita de código limpo posteriormente.

Realmente não importa qual linguagem de programação você está usando - inevitavelmente, você ficará assustado toda vez que olhar para algum código antigo que você mesmo criou. Escrever seu código de maneira “limpa” ajudará você a minimizar esse efeito e a entendê-lo melhor, instantaneamente, anos depois. Ao fazer isso, você está se dando um presente, porque estará codificando

de uma maneira que está cada vez mais acostumado - uma maneira de escrever com a qual muitas pessoas estão acostumadas.

Pode ser a coisa mais impressionante sobre o código limpo: não importa qual tecnologia você está usando, qual biblioteca ou mesmo qual idioma. Não depende do país em que você mora e não depende dos seus anos de experiência, ou mesmo se você está programando para ganhar a vida ou como hobby sem intenções financeiras por trás disso. O código limpo é uma habilidade que, em algum momento, todo desenvolvedor aprimorará e enfrentará em sua vida. Claro, você ficará melhor nisso se focar nisso, mas apenas aprender com sua experiência sobre o que funciona e o que não funciona em um projeto de TI já é, de certa forma, aprender código limpo. Mais do que as regras, ferramentas e dicas que você encontrará neste livro, o código limpo é uma questão de experiência e anos de prática. Durante sua carreira de desenvolvedor,

Você deve sempre lembrar que a codificação é um trabalho de equipe. Agora é raro trabalhar sozinho em um projeto sem a intervenção de outros desenvolvedores - se é que isso ainda acontece. Pode ser um projeto profissional, um projeto de código aberto ou até mesmo um projeto pessoal privado. Dizer que outros desenvolvedores ajudarão em seu projeto não significa necessariamente que eles escreverão algum código. Eles também podem tentar encontrar soluções com você para os desafios técnicos que você enfrentará, e isso se aplica a todos os tipos de projeto.

De qualquer forma, fazer as coisas corretamente facilitará uma etapa crucial pela qual todos passamos ao entrar em um novo projeto: o processo de integração.

A importância do código limpo nas equipes

O processo de integração nunca é fácil. Você está entrando em um novo projeto e muitas pessoas ao seu redor já o conhecem. Você deve aprender tudo: quem faz o quê na equipe, quais são os requisitos exatos, os desafios técnicos pelos quais a equipe passou e o que está por vir. Esse processo pode durar várias semanas e até vários meses em alguns casos (também pode durar mais de um ano em alguns projetos legados gigantescos, mas isso é outro assunto). Se você pode economizar o processo de aprender as convenções de codificação porque já as conhece graças aos princípios de código limpo, então é uma coisa em que você não terá que pensar e poderá se concentrar em outras coisas.

Ser capaz de escrever um código limpo é como falar fluentemente o mesmo idioma que as outras pessoas da equipe: facilita a comunicação e a escrita do código da mesma forma, sem perceber quem escreveu qual parte.

O código limpo ajudará você com a longevidade do código. Ao trabalhar em equipe em um projeto, há grandes chances de o código-fonte ser mantido por pelo menos alguns anos. O código continuará a ser mantido anos após a sua partida. É importante fazer as coisas direito porque você não vai ficar aqui para sempre para explicar tudo o que você fez para a pessoa que ainda está trabalhando no projeto.

Escrever código de forma limpa evitará o aparecimento do que chamamos de pontos únicos de falha (SPOFs). Esses SPOFs são o pior pesadelo para qualquer projeto. Eles também são sintomáticos de projetos legados. Eles podem ser descritos como uma entidade (um desenvolvedor, uma tecnologia, uma

biblioteca e assim por diante) que detém todo o projeto por si só. Se esta entidade falhar, tudo desmorona.

Um exemplo concreto de falha de uma entidade é a saída de um desenvolvedor. Se ele é a única pessoa que entende completamente como o projeto funciona e é considerado o “super-herói” do projeto, então é um grande problema. Isso significa que o projeto não poderá continuar sem eles e será uma dor absoluta manter o projeto após sua partida.

Um excelente exemplo de prevenção de SPoF foi dado por Fabien Potencier, o criador do framework Symfony, durante a convenção SymfonyWorld Online 2021 Winter Edition. Mesmo que o Symfony seja um projeto de código aberto, ele tem sido a pessoa que mais trabalhou nele desde a sua criação. Ele explicou durante uma conferência que uma de suas principais tarefas agora é transmitir o máximo de conhecimento sobre o framework para o máximo de pessoas para que o Symfony não dependa mais exclusivamente dele. Ele absolutamente quer evitar ser o SPOF do Symfony, e naturalmente quer que o Symfony seja mantido e desenvolvido, mesmo depois que ele não estiver mais trabalhando nele.

Uma das várias maneiras de evitar esse problema é escrever um código claro, conciso e compreensível - escrever código de forma que todos possam entender facilmente o que está acontecendo. Depois de entrar neste livro, você pode ter uma ideia melhor de como conseguir isso.

Falando em trabalho em equipe, outro ponto crítico são as revisões de código. Mesmo que entraremos em detalhes sobre isso mais adiante neste livro, vamos falar sobre eles rapidamente aqui. Se você nunca ouviu falar de revisão de código, é simplesmente um processo em que outros desenvolvedores dos projetos estão revisando, lendo e comentando as alterações que você deseja trazer para a base de código. Este processo é obrigatório na maioria dos projetos

de código aberto e altamente recomendado para qualquer projeto, dados os benefícios que traz. Agora você pode imaginar facilmente que, se cada um tiver sua própria maneira de escrever código, essas revisões levarão muito mais tempo. Isso vai desde a maneira como você formata seu código ou como separa arquivos, classes e assim por diante. Se todos falarmos a mesma linguagem, podemos focar no que realmente importa durante a revisão do código: se a necessidade funcional for respeitada,

Veremos nos próximos capítulos que a definição de código limpo geralmente depende da equipe com a qual você está trabalhando. Mesmo que existam algumas regras gerais, depende principalmente do que os desenvolvedores com os quais você está trabalhando estão acostumados. Novamente, o motivo são os hábitos, mas também a consistência. Dessa forma, a equipe pode trabalhar mais rápido e melhor em conjunto.

Se por acaso você trabalha em uma área onde várias equipes estão reunidas no mesmo local, inevitavelmente notará que cada equipe tem seu próprio conjunto de regras que funcionam bem para elas, mas também regras comuns que permitirão que você converse com todos em a mesma língua. Como as soluções para seus problemas nem sempre são encontradas em sua equipe direta, mas às vezes também em pessoas ao seu redor sem trabalhar diretamente com você no mesmo projeto, será muito mais fácil ajudar uns aos outros se as regras básicas forem compartilhadas entre grupos de pessoas.

A importância do código limpo em projetos pessoais

Você pode pensar que o código limpo é menos importante para projetos pessoais, mesmo que seja apenas um pouquinho. Você estaria errado se pensasse assim por muitas razões, conforme estabelecido aqui:

Como mencionado anteriormente, tente ler o código que você escreveu há alguns anos. Há uma grande chance de você não conseguir entender brevemente o que queria fazer naquele momento. Você pode pensar que os comentários que você colocou (se você fizer isso) estão aqui para ajudar, mas vamos fazer duas perguntas, a seguir:

É disso que se trata: ser capaz de ler o código-fonte como frases simples sem ter que parar em uma linha para entendê-lo. Além disso, para repetir o que foi dito sobre código limpo em equipes: só porque você está criando um projeto por conta própria, não significa que não precisará de ajuda externa. Na verdade, é quase certo que um dia você precisará da ajuda de alguém para alguma coisa. Novamente, se você compartilhar o mesmo conjunto de regras comuns, será muito mais fácil para o ajudante trazer suas ideias. Mesmo em momentos e projetos pessoais, o trabalho do desenvolvedor é o trabalho em equipe. Sempre.

O código limpo é uma mentalidade, e essa mentalidade inclui ter orgulho do que criamos – o sabor de um trabalho bem feito. Talvez demore alguns anos para entender completamente do que se trata, e sempre temos coisas novas a aprender e situações que enfrentamos pela primeira vez. O código limpo é mover-se para o momento em que você olha para o seu código e diz a si mesmo que está completamente relaxado em mostrar seu código a qualquer pessoa.

Como diz o ditado, “escreva código como se o próximo desenvolvedor a executá-lo soubesse seu endereço”. E isso também deve se aplicar a projetos pessoais porque você é o próximo desenvolvedor.

Resumo

Já terminamos com a teoria? Bem, tipo isso. Definimos juntos em que consiste o código limpo. Demos uma definição mútua de código limpo. Por ter a mesma definição de código limpo, estamos um passo mais fundo e prontos para mergulhar em princípios mais avançados na próxima seção.

Mas é claro que ainda não terminamos. Mesmo que você concorde com a definição que apresentamos, é claro que você já pode ter muitas perguntas. E as perguntas mais importantes devem ser as seguintes: Quem decide essas regras para você? Quem tem o poder de impor essa visão? Isso é o que veremos juntos no próximo capítulo.

Quem decide quais são as “boas práticas”?

Boas práticas são ótimas, mas saber quem as decide e de onde vêm é melhor. Não é segredo que, quando você entende completamente o que está fazendo, imediatamente se sente melhor e mais confortável. A mesma coisa se aplica às boas práticas. Por que você deve acreditar sem questionar esses preceitos decididos por pessoas que você não conhece e que nunca colaboraram com você em seu projeto?

Você poderia dizer que as pessoas que ditam esses princípios têm mais experiência do que você e conhecem este mundo melhor do que você. Duas coisas. Primeiro, talvez um dia você tenha mais experiência do que eles. Talvez você seja melhor. Talvez você já esteja. Segundo, anos de experiência não são tudo. É comum ver desenvolvedores com 20 ou 30 anos de experiência completamente desatualizados ou com hábitos do século passado. Anos de experiência podem ser um argumento, mas não o único. A computação evolui em uma velocidade excepcional, e o mundo da web é ainda mais afetado por isso.

Veremos juntos neste capítulo de onde as melhores práticas se originaram: elas foram realmente inventadas e decididas por um grupo específico de pessoas? Quais são os diferentes princípios de código limpo existentes que você já pode aplicar aos seus projetos a partir de agora? Sua maneira de pensar pode mudar quando você os conhecer.

Aqui estão os principais tópicos que abordaremos neste capítulo:

Quem decide essas coisas afinal?

Uma coisa que vamos ver é que você deve sempre questionar as “boas práticas” e nunca considerá-las uma verdade geral que você deve respeitar sem entender o porquê. Uma excelente maneira de melhorar a si mesmo é perguntar toda vez que você não concorda com a crítica ou ponto de vista de alguém. Os desenvolvedores são fascinantes porque podem encontrar maneiras ilimitadas de resolver um único problema - soluções ilimitadas para o mesmo resultado. Mesmo que isso possa parecer algo um pouco cansativo às vezes, é sempre interessante entender porque um desenvolvedor quer resolver um problema de uma forma diferente de você. Existem vários objetivos para isso, como segue:

Essa é uma lista bem longa. Muitas soft skills são aprimoradas apenas discutindo com outros desenvolvedores uma maneira de resolver um determinado problema. Ser capaz de explicar uma situação com clareza é bem mais difícil do que parece.

E é também por isso que você deve sempre pedir mais explicações quando discordar de alguém. Primeiro, pode haver um mal-entendido sobre o problema exposto. Esta é uma situação comum em que várias partes discordam: o problema não estava claro no início. Cada parte está tentando justificar o que entendeu. Você pode facilmente imaginar a bagunça que isso traz.

Ser um bom desenvolvedor é (também) ser capaz de justificar e explicar todas as suas escolhas. Não mais: “Sempre fizemos coisas assim; não há outras razões para fazê-lo desta maneira.” Quando você está decidindo ou dizendo a alguém para seguir algumas diretrizes, você deve sempre ser capaz de justificar e

explicar claramente porque seu caminho é o melhor para você. Pode não ser a melhor maneira de fazer isso objetivamente, mas se você for capaz de explicar por que é mais adequado para você, isso fará com que você pareça muito mais aberto.

Dito isso, você já pode entender aonde queremos chegar com isso: ninguém detém a verdade absoluta. Se alguém estiver confiante o suficiente para dizer isso, você deve sempre ter cuidado.

Melhores práticas – de onde elas realmente vêm?

Quando falamos de “melhores práticas”, podemos diferenciar três casos, como segue:

Princípios de padrão de design

O caso dos padrões de projeto descreve soluções objetivas para resolver problemas. Você pode não gostar deles e de como o código está sendo organizado por eles, mas não pode dizer que eles são objetivamente ruins. Porque seus pensamentos sobre eles não importam, eles funcionam.

Falando de princípios que estão presentes há décadas, podemos destacar quatro famosos: DRY, KISS, YAGNI e SOLID.

SECO

SECO significa Don't Repeat Yourself. Este princípio simplesmente afirma que você nunca deve ter, em sua aplicação, duas autoridades fazendo a mesma coisa. Pode parecer óbvio, mas aplicar isso nem sempre é um reflexo, especialmente quando você é novo na programação de aplicativos. Ter a mesma responsabilidade em dois lugares separados em seu código significa manter esses dois lugares toda vez que você consertar alguma coisa. Significa ter que pensar nesses dois lugares a cada mudança (e um dia você vai esquecer um, com certeza). Além disso, como qualquer desenvolvedor que mantém seu código-fonte deve saber qual usar se duas coisas têm a mesma responsabilidade?

BEIJO

BEIJO significa Keep It Simple, Stupid. Às vezes, complicamos nossas vidas. Podemos ver duas razões principais para isso, como segue:

Ambos os casos são recorrentes e nos impedem de ir direto ao ponto e simplificar. Quando você sentir que está indo “longe demais”, pense nessa sigla para voltar aos trilhos.

YAGNI

YAGNI significa You Aren't Gonna Need It. De alguma forma, anda de mãos dadas com o princípio KISS. É bastante comum (para não dizer parte do nosso dia a dia como desenvolvedores) querer encontrar uma solução para um problema pensando no futuro. De fato, temos regularmente este pensamento: “Se amanhã eu precisar fazer tal ou tal coisa, pelo menos isso já estará em vigor”. A realidade é que, em geral, não – nunca teremos a necessidade que queremos tentar prever. Então, ao tentar antecipar uma tarefa que pode nunca existir e cujas restrições funcionais são desconhecidas, não apenas perdemos tempo, mas também complicamos nossas vidas pensando muito à frente. Afastamo-nos do nosso objetivo inicial, que é encontrar uma solução rápida, viável e robusta.

Não somos psíquicos e não podemos saber todos os problemas que aparecerão se a tarefa que previmos aparecer. Você perceberá rapidamente que, se mantiver as coisas simples e sem adições supérfluas para tentar avançar no jogo no dia seguinte, terá uma base de código saudável e sem frescuras. Isso significa uma compreensão mais rápida do código, maior facilidade para navegar por ele e fazer alterações quando realmente forem necessárias. Além disso, você provavelmente evitará muitos bugs. É sempre complicado (se não impossível) justificar que um bug foi feito em seu código-fonte porque você desenvolveu e gastou tempo em algo que não foi solicitado a você. Se o seu trabalho como desenvolvedor exige que você colabore diretamente com o cliente, saiba que o cliente não pagará por algo que não pediu. Você terá trabalhado de graça,

Cuidado: Obviamente, é necessário considerar isso caso a caso. Podemos tomar como exemplo os números mágicos. Números mágicos são valores constantes, principalmente números, codificados e sem nenhuma explicação de seu significado.

Veredito: Duas semanas depois, todos se esqueceram a que esse número corresponde. Em seguida, pensaremos em usar constantes de código bem nomeadas. No entanto, há uma grande chance de que o valor dessa constante de código nunca mude porque as necessidades teriam mudado. À primeira vista, seria estranho querer declarar uma constante para usá-la em qualquer lugar. O objetivo das constantes de código é adicionar semântica a valores fixos e nos permitir alterar facilmente esse valor em todos os lugares em que é usado no código, de uma só vez. Isso é, de certa forma, contrário ao YAGNI (uma vez que esses valores provavelmente nunca mudarão).

No entanto, podemos ver o valor de usar constantes. Perspectiva e reflexão são sempre necessárias, quaisquer que sejam os princípios de código limpo aplicados.

SÓLIDO

Finalmente, talvez o mais famoso, SOLID. Vamos ver o que essas letras significam:

Podemos perceber rapidamente que esses princípios estão muito ligados. Eles trabalham juntos e permitem um forte desacoplamento, uma forte separação de responsabilidades e um pensamento fluido quando você tem esses princípios em mente ao escrever o código. Pode ser extremamente útil lembrar desses princípios; pelo menos, é uma coisa particularmente boa saber de sua existência. Além do princípio SOLID, você pode ver que KISS, DRY e YAGNI são bastante lógicos e de bom senso. Lembrá-los de vez em quando pode ser benéfico e nos ajudar a colocar barreiras quando nos desviamos um pouco.

Bônus – princípio dos escoteiros

Algo que também pode ser acrescentado, que também é um princípio de bom senso, é o “princípio dos escoteiros”. Todos nós conhecemos aqueles grupos de jovens e adolescentes que agem com um propósito benevolente e demonstram muito altruísmo. vão acampar na floresta, fazem uma fogueira e passam a noite lá. Assim que se levantam de manhã, podem apagar o fogo e guardar suas coisas, mas acima de tudo, limpam o local para deixá-lo mais limpo do que era antes de eles chegarem (pelo menos, em teoria).

Como desenvolvedor, é o mesmo. Seja um escoteiro. Ao explorar e navegar pelo código, se o tempo e o contexto permitirem, geralmente é uma ideia particularmente inteligente limpar onde você vê alguma dívida técnica. Se você está passando por lugares na base de código e pensa "isso é muito ruim", talvez esta seja uma oportunidade de torná-lo mais gerenciável e limpo. Se todos aderirem, a qualidade do código-fonte do projeto pode aumentar muito rapidamente.

Claro, este “princípio dos olheiros” deve ser feito de acordo com as restrições do seu projeto, restrições de tempo e necessidades do cliente. Além disso, é bastante arriscado e você deve saber quando parar. Quando você envia suas alterações para sua equipe para revisão, as alterações e a limpeza feitas devem ser consistentes. Você não quer reescrever metade do aplicativo toda vez que encontra uma coisinha, que leva a outra, depois a outra e assim por diante. É mais sobre limpar as coisas que são relevantes para o que você está fazendo. Pode ser extremamente complicado manter o foco e fixo no seu contexto. Não existe uma resposta genuína para “quando parar”; vai depender muito do tempo que você tem e da sua tarefa. No entanto, não há nada que o impeça de anotar coisas às quais deseja voltar, mas que infelizmente não estavam relacionadas ao que estava fazendo,

Por outro lado, estilo de código, convenções de nomenclatura e coisas semelhantes estão sujeitas a gostos e hábitos. Todo mundo tem seus próprios gostos e hábitos, então decisões precisam ser tomadas. Como discutimos anteriormente, é muito mais fácil conversar quando todos seguimos as mesmas regras.

Então, quem decide essas práticas recomendadas em uma equipe ou organização? Bom, geralmente é um consenso após longas discussões sobre o assunto no início de um projeto. Porque sim – “melhores práticas” não são algo que você sempre pode aplicar em todos os lugares, e você deve estar ciente disso. Você deve estar ciente do contexto em que está inserido.

Consciente do contexto

Aqui, entramos em uma das partes mais importantes quando falamos de código limpo. Se houvesse apenas uma coisa para lembrar, seria isso. Podemos falar regularmente sobre as regras definidas por outros desenvolvedores, princípios de objetos e princípios de código limpo, mas nada será tão bom quanto o que vamos falar aqui: trata-se de estar ciente do seu contexto. Uma coisa que falta em muitos livros e artigos sobre código limpo é a sensação de que ele é relevante para a vida cotidiana. A vida de um desenvolvedor é feita de eventos inesperados, restrições técnicas, impossibilidades de fazer algumas coisas ou ser forçado a fazer outras coisas.

Existem tantas maneiras de fazer as coisas quanto projetos. Cada projeto tem sua própria história, decisões técnicas e restrições. Como resultado, acabamos com muitos princípios teóricos que não são aplicáveis ou que quebrariam a coerência do projeto. Boas práticas podem ditar como você nomeia variáveis, como nomeia suas classes e métodos, como nomeia seus arquivos ou como cria a estrutura em árvore do seu projeto. No entanto, e se isso for contra o que foi estabelecido no projeto? Este é um problema que surge com muita regularidade, especialmente nos chamados projetos “legados”.

A resposta é simples e complicada. É simples porque pode ser resumido em uma frase: discuta com sua equipe e com as outras pessoas que trabalham no projeto. É aqui que a resposta pode se complicar, pois muito provavelmente iniciará debates (às vezes acalorados) dentro da equipe, e isso é bastante normal. Mas o importante é o seguinte: você deve não apenas encontrar um terreno comum que todos respeitarão para ser consistente com o projeto, mas também encontrar maneiras que funcionem melhor para você e sua equipe.

Esta pode ser a pedra angular de todas essas discussões: você pode ter boas práticas e princípios de código limpo, mas talvez valha a pena quebrar algumas regras porque funciona melhor para você e sua equipe. Esta é uma razão perfeitamente válida para se desviar de certos princípios (nem todos, claro, caso contrário, é uma anarquia total). Às vezes, nenhum terreno comum será encontrado, e é aí que as melhores práticas e outros princípios podem prevalecer ao ditar uma regra que todos terão de seguir, se não uma que funcione para todos.

E é aí que voltamos a um ponto levantado anteriormente. Ter regras comuns em uma equipe de desenvolvedores ajuda a facilitar o entendimento. Navegar pelo projeto é mais fácil, assim como navegar pelos arquivos. Ao falarmos todos a mesma língua, nos entendemos melhor. Seja ajudando alguém ou sendo ajudado, é sempre a mesma história. Se as regras comuns de nomenclatura e recuo forem respeitadas, você não apenas evitará debates estéreis que o distrairiam de seu objetivo inicial, mas se sentir que o código escrito por seu colega de equipe foi escrito por você, é uma situação em que todos saem ganhando.

Podemos resumir a escolha de decidir sobre boas práticas e princípios “clássicos” de código limpo desta forma:

Sua situação já ocorreu no passado no projeto?

Aqui, vemos muito claramente, mais uma vez, que a chave do sucesso será a comunicação e o debate. Todo mundo tem seu próprio ponto de vista e abordagem do problema, então nunca é tudo preto no branco. Mas lembre-se de uma coisa: você deve evitar tomar decisões sozinho sobre uma prática que pode ser debatida. Evitar tomar uma decisão sozinho não significa que você não deva

tomar uma. Em vez disso, o que queremos dizer com isso é pensar nas opções, pesando os prós e os contras de cada uma. E, novamente, seja capaz de justificar cada proposta que você oferece às outras pessoas que trabalham no projeto. Você pode ter certeza de que será muito apreciado por sua equipe se oferecer várias soluções para os problemas com uma boa justificativa de por que essas soluções são adequadas, mas também os riscos que essas soluções acarretam. Rapidamente você perceberá que esse trabalho pode ser complicado de fazer sozinho, daí a importância de discuti-lo com sua equipe. Cada cérebro tem sua própria maneira de trabalhar.

A propósito, tudo isso não se aplica apenas a TI, código limpo e PHP. É por isso que foi mencionado no capítulo anterior que o código limpo não é apenas um conjunto de regras: é um jeito de ser – é uma mentalidade.

Ser consistente – obter resultados mais rapidamente

Ser perfeitamente consistente no que faz o forçará a entender o que está fazendo. Então, tudo se tornará um hábito. Se você tem esses bons hábitos a ponto de se tornarem naturais para você, os resultados virão mais rapidamente em dois casos específicos, conforme descrito a seguir:

As tarefas de auditoria automatizadas podem incluir absolutamente qualquer tarefa que você desejar. Entraremos em mais detalhes sobre como realizar essas tarefas assim queCapítulo 7,Ferramentas de qualidade de código, deste livro (na seção Ferramentas de qualidade de código), mas aqui estão as mais comuns:

Você entendeu. Essas ferramentas permitem que você execute as tarefas que deseja e precisa. Na realidade, eles são apenas orquestradores que executarão os comandos que você definir. É tão simples quanto isso. Depois disso, se seus comandos forem complexos, é outra história. Mas você percebe como essas verificações automatizadas podem ser infinitas.

Sobre as ferramentas de análise de código-fonte

Tendo bons hábitos de código, você definitivamente acelerará o processo. Talvez o exemplo mais revelador e concreto disso seja a verificação do código de estilo. Se você não sabe como escrever código em sua equipe e sua equipe colocou verificações automáticas nele para tudo o que deseja alterar, você pode passar horas imaginando que faltou um espaço em um lugar, uma quebra de linha em outro e em breve. Não se preocupe, a maioria das ferramentas oferece opções para corrigir esses erros automaticamente.

No entanto, este não é o caso das ferramentas de análise estática, como as que apresentaremos na seção dedicada às ferramentas de análise estática em Capítulo 7, Ferramentas de qualidade de código. Na verdade, uma análise estática revisará seu código e garantirá que os erros mais comuns não sejam cometidos. Não estamos falando de verificar o número de espaços em suas abas, mas realmente analisar o PHP para tentar entendê-lo e garantir que tudo esteja em ordem. Essas ferramentas podem ser um pouco rígidas às vezes e pode levar muito tempo para entendê-las corretamente. Além disso, essas ferramentas não são perfeitas e a ferramenta de análise estática pode não ser capaz de entender o que você deseja fazer. Embora este seja um problema separado, você evitará muitos problemas se lidar com ele na fonte: desenvolva bons hábitos de codificação. Seja minucioso e não deixe nada ao acaso. PHP é uma linguagem muito relaxada que permite que você faça praticamente qualquer coisa com variáveis - por exemplo, permitindo que você digite sem vacilar. Como sabemos, os resultados desse tipo de operação podem ser aleatórios, até muito surpreendentes, e podem parecer completamente ilógicos. De qualquer forma, PHP é o que é. Embora as ferramentas de análise estática possam detectar esses casos de risco na maioria das vezes, você pode passar horas corrigindo essas pequenas coisas que podem chegar às dezenas muito rapidamente.

Além disso (e isso pode parecer bobo), quanto mais limpo você codificar implementando práticas de código limpo e o que se segue, menos bugs você implementa. Ao ser seguro de si, você evita muitos sustos. Além disso, você permite que os próximos desenvolvedores que passarão por cima do seu código também não sejam enganados e facilite a passagem deles. Se tiver sorte (ou se tiver aplicado os princípios do código limpo e tudo o que o envolve!), terá à sua disposição testes que garantem o bom funcionamento da aplicação. Talvez você não esteja familiarizado com os testes, então vamos passar por isso juntos.

Sobre testes e suas múltiplas formas

Os testes são principalmente linhas de código escritas pelos desenvolvedores. Esses testes garantem que, para alguns dados de entrada, uma saída específica seja retornada. Essas entradas e saídas (I/Os) podem ser de diversos tipos e tamanhos. Pode ser um número inteiro, bem como uma página HTML gerada ou até mesmo uma imagem. Para facilitar e simplificar o conceito, os testes automatizados geralmente são agrupados em três famílias principais, como segue:

Todos esses testes existem por um motivo específico: não regressão.

O teste de não regressão é sem dúvida o que salvará sua posição como desenvolvedor em uma empresa. Tudo bem - talvez isso seja um pouco exagerado. No entanto, não podemos contar o número de aplicativos que foram salvos graças a ele. Quando sua cobertura de teste é boa o suficiente (a proporção de linhas de código e recursos cobertos por um ou mais testes), você pode modificar quase tudo e certifique-se de nunca quebrar nada se os testes estiverem sempre verdes. Na verdade, você pode interromper os recursos para reescrevê-los de maneira diferente e testar novas maneiras de fazer as coisas. Enquanto os testes estiverem verdes, você pode ter certeza de que o aplicativo se comportará corretamente, como antes de suas modificações. Claro, várias coisas entram em jogo.

Primeiro, os testes têm que testar alguma coisa. Isso pode soar estranho, mas, na verdade, você acaba com muitos testes que na verdade não avaliam nada. O exemplo mais típico são os testes de unidade nos setters e getters de uma classe. Quando você escreve testes para isso, você está avaliando que uma atribuição de

variável foi feita e que uma chamada de método foi feita. Você está testando... PHP! E o PHP já tem seus próprios testes. Escrever testes relevantes é um livro em si e é uma arte que pode ser dominada e refinada ao longo dos anos. E para isso, não há nada como a prática - repetidamente.

A segunda coisa a considerar quando você modifica um aplicativo é simplesmente que os testes que você configurou não estão mais atualizados e devem ser modificados. Às vezes, pode ser difícil entender se um teste falha voluntariamente (ou seja, torna-se incompatível com suas alterações) ou involuntariamente (porque o aplicativo não se comporta da mesma maneira que antes, quando deveria). Depende inteiramente do seu caso. Lembre-se de uma coisa: se seu aplicativo for testado corretamente, você poderá alterar qualquer linha de código e implantar seu aplicativo de olhos fechados, a qualquer momento e sem hesitar. Bem interessante, não é?

Agora que você vê o benefício dos testes em seu aplicativo, podemos discutir uma prática bastante comum na prática de código limpo. Você já deve ter ouvido falar: TDD.

TDD significa desenvolvimento orientado a testes. É uma metodologia que consiste em escrever testes antes de escrever o restante do código. No começo, é muito confuso. É complicado entender como isso acontece, ou mesmo como é possível. Trata-se de pensar para trás e questionar nossos hábitos de pensamento, mas o princípio é bastante trivial. Primeiro, pensamos nos testes, ou seja, nos dados que vamos enviar (para nossos métodos em testes de unidade, para um endpoint de API no caso de testes funcionais e assim por diante), bem como na saída que queremos (um objeto ou valor preciso em testes de unidade; um retorno preciso de JavaScript Object Notation (JSON), por exemplo, no caso de um teste funcional de API). Obviamente, e porque você ainda não escreveu o restante do código, todos os testes falham. Isso é intencional. O objetivo agora é fazer com que esses testes fiquem verdes um por um.

Se você experimentar essa prática, perceberá que uma espécie de mágica acontece sem nem perceber. Você organizará seu código de uma forma que nunca teria feito no começo. Ele será recortado de maneira clara e precisa para que seus testes passem da forma mais rápida e fácil possível. Além da excepcional satisfação intelectual resultante, você terá um código instantaneamente mais legível e limpo. Além disso (e ao contrário da crença e intuição generalizadas), os desenvolvimentos serão muito mais rápidos. Existe sim um período de adaptação para que fique bem natural, e você pode ter a impressão de ser terrivelmente lento no começo. No entanto, graças a isso, seu código é mais simples e, portanto, mais rápido de escrever, entender, adaptar e estender. Você tem que experimentar para experimentar isso, pois pode soar um pouco milagroso. E realmente é de alguma forma.

Além disso, a cobertura do código se torna cada vez mais extensa à medida que você desenvolve. Isso tem um impacto direto na manutenção da sua aplicação, como já dissemos: você terá muito mais segurança para alterar seu código, assim como todas as pessoas que terão que lê-lo e modificá-lo. Os testes estarão lá para protegê-lo. Como um bônus, os testes de leitura podem ser inestimáveis para entrar no código complexo. Ao ler os testes, você pode entender pelas E/S fornecidas para onde o desenvolvedor que escreveu os testes estava indo. Isso não tem preço na grande maioria dos aplicativos, principalmente os legados. Além disso, muitos desenvolvedores estão olhando para os testes em primeiro lugar ao revisar seu código. É um ponto de entrada incrível ao entrar na alteração de alguém na base de código. Veja seus testes como a única maneira de provar que o que você acabou de fazer realmente funciona. Esta é uma realidade:

Pode-se notar, por exemplo, que para a maioria (se não todos) os projetos de código aberto, como PHP, você deve adicionar testes ao fazer alterações. Seja para adicionar um novo recurso ou corrigir um problema, o teste será obrigatório e suas alterações nunca serão aprovadas sem teste. Esses testes serão, mais uma vez, provas irrefutáveis do comportamento do seu novo recurso ou de que você realmente corrigiu o bug em questão. Tudo isso dá muito trabalho, mas com isso

instalado, a cobertura do código torna-se enorme e você contribui totalmente para a estabilidade do software.

O teste é inestimável, tanto pela qualidade do seu código quanto pela velocidade com que você obtém resultados. Graças a ele, você será consistente e obterá resultados mais rapidamente.

Resumo

Acabamos de ver muitos novos conhecimentos juntos. Se você entender, pode ter certeza de que já é um desenvolvedor melhor do que no capítulo anterior.

Conhecer os princípios SOLID é um verdadeiro trunfo no mundo profissional e em projetos de qualidade industrial. Mesmo que cada caso seja um caso e cada projeto tenha suas especificidades, esses princípios têm a vantagem de serem aplicáveis em quase todos os lugares, e pelo menos de serem muito fortemente inspirados por eles.

Ter em mente os princípios KISS, DRY e YAGNI permitirá que você mantenha os pés no chão e não se espalhe muito nos próximos desenvolvimentos. Eles enfatizam pensar no momento presente para ajudar a preparar o futuro, em vez de pensar no futuro para tentar se adaptar ao momento presente. Você definitivamente deve se lembrar disso. Não sabemos os constrangimentos futuros que nos vão ser impostos, sejam eles técnicos ou funcionais, pelo que faz mais sentido pensar em como facilitar o tratamento desses constrangimentos do que adivinhá-los. Porque, convenhamos: temos muito poucas chances de acertar na mosca.

Se você tiver a oportunidade e a possibilidade, implementar o “princípio dos Scouts” em uma estratégia de TDD pode ser mais do que benéfico e sempre será uma excelente ideia. Se você nunca praticou TDD, e apesar das explicações dadas neste capítulo, é normal ter dúvidas sobre sua utilidade e, principalmente, sobre sua eficácia. Isso é normal, e todos nós já passamos por isso. No entanto, os resultados estão aí, e os vários estudos de caso realizados sobre o assunto

demonstram isso drasticamente. Talvez seja hora de testar esse jeito de fazer as coisas, que é muito bem visto e apreciado pelos veteranos do código limpo!

Apesar de tudo isso, devemos ter em mente que o código limpo também é uma adaptação ao seu ambiente. Não se trata de reescrever totalmente o aplicativo e mudar todos os hábitos da equipe de desenvolvimento sob o pretexto de que alguém de fora do projeto decidiu fazê-lo. Você deve estar ciente de seu contexto e lidar com seu ambiente. Você deve ser capaz de se adaptar à necessidade e ao que está ao seu redor. É isso que fará de você um bom “clean-coder”. propor várias soluções, bem como as vantagens e desvantagens de cada uma.

Depois de toda essa teoria, podemos passar para uma parte um pouco mais prática. Quais são as maneiras de escrever código limpo? Qual é o propósito do código? Embora tenhamos visto alguns princípios avançados, não devemos esquecer os básicos, e também devemos questionar o que já sabemos.

Código, não faça acrobacias

Os princípios avançados de código limpo realmente ajudarão você a se tornar um desenvolvedor facilmente compreendido, capaz de codificar de forma mais limpa. Eles ensinam você a ser consistente em suas escolhas, a pensar nos outros desenvolvedores e em sua equipe e a fazer da comunicação a principal ferramenta do nosso trabalho. Mesmo antes do código-fonte.

É fato: embora o código-fonte tenha um lugar preponderante no trabalho do desenvolvedor, não devemos fazer dele a nossa principal razão de ser. É uma realidade: o trabalho do desenvolvedor não é escrever código. Trata-se de encontrar uma solução para um determinado problema enquanto se adapta às restrições que atrapalhariam. Esta é a base do nosso trabalho que devemos ter absolutamente em mente. E embora os princípios que acabamos de ver no capítulo anterior, como o SOLID, pareçam estar fortemente ligados ao código, devemos tentar ter uma perspectiva mais “meta” sobre tudo isso, pensando fora da caixa e dando um passo de volta. Os princípios mencionados são, objetivamente, ferramentas que nos permitirão resolver os problemas colocados de forma eficiente e direta.

Podemos então nos fazer as seguintes perguntas: Qual é o real propósito do código-fonte? Qual é o seu propósito e podemos nos permitir fazer qualquer coisa com as coisas mais básicas da linguagem?

Estes são os tópicos que abordaremos neste capítulo:

Entendendo o código

Vamos primeiro nos perguntar sobre a importância do código. Qual é realmente a sua importância para nós, desenvolvedores, em nosso dia a dia? Para isso, vamos voltar no tempo.

Um pouco de história

A programação de computadores é, na verdade, um transistor por onde passa ou não uma corrente elétrica. Assim, acabamos com um sistema binário, com valor 0 se a corrente elétrica não passar pelo transistor e 1 se a corrente elétrica passar. Se você multiplicar esse número de transistores por vários bilhões, acabará com os processadores de hoje. Funciona muito bem, e nosso mundo é governado por esse sistema há décadas. No entanto, há uma clara limitação: não é humanamente possível entender e criar aplicativos apenas com 0s e 1s. Então, tivemos que encontrar uma nova maneira de escrever esses programas para que se tornassem humanamente possíveis e gerenciáveis.

Em seguida, passamos para o primeiro código-fonte legível por humanos: a linguagem assembly (muitas vezes abreviada como ASM). Popularizada no final da década de 1940, essa linguagem finalmente possibilitou a leitura de arquivos com uma linguagem mais ou menos semelhante à nossa linguagem natural, embora o assembly seja uma linguagem de baixíssimo nível (ou seja, muito próxima da linguagem da máquina - ou seja, binário). Uma coisa levou à outra e surgiram as linguagens de alto nível, sendo a linguagem C a mais conhecida, tendo sua primeira versão oficial em 1972. O princípio é simples: poder escrever programas de computador com uma linguagem cada vez mais natural para seres humanos. Uma ferramenta traduz automaticamente essa linguagem de nível superior em linguagem assembly e binária que a máquina pode interpretar.

A linguagem C é um excelente exemplo da utilidade e propósito principal de uma linguagem de programação. De fato, essa linguagem, criada por Dennis Ritchie e Brian Kernighan, foi originalmente usada para desenvolver o sistema operacional Unix. A questão é que era mais fácil criar uma linguagem de programação como C para escrever o sistema operacional Unix do que escrever o sistema operacional com as ferramentas da época - a saber, assembler (mesmo

que algumas partes do sistema operacional Unix sejam escritas em assembly, a grande maioria do código-fonte é escrita na linguagem C). Veremos o propósito real das linguagens de programação e do código a seguir.

A finalidade do código

E é aqui que tudo faz sentido. As linguagens de programação existem para nos ajudar a transcrever nossas ideias da maneira mais fácil possível e são interpretáveis pela máquina. Mas as linguagens de programação não se limitam a transcrever nossas ideias para máquinas – seu objetivo também é que outras pessoas possam entender nossas ideias lendo nosso código e sem precisar de nós. As linguagens de programação são uma mistura sutil do potencial de ser compreendido por um ser humano, ao mesmo tempo em que nos dá liberdade para nos comunicarmos com a máquina e as possibilidades de explorar todo o seu potencial. É isso que define o nível de uma linguagem de programação quando falamos de linguagens de alto e baixo nível: a posição do cursor fica entre “facilidade de uso e entendimento” e “desempenho e possibilidades oferecidas pela linguagem”. Há compensações necessárias a serem feitas. Essas são coisas que você deve ter em mente ao escolher a linguagem de programação mais apropriada ao iniciar um novo projeto. Um exemplo perfeito são todas as ferramentas agrupadas no PHP: Linguagem Hypertext Preprocessor (PHP) para lidar com solicitações e respostas do Hypertext Transfer Protocol (HTTP), o que o torna uma excelente escolha para criar um aplicativo da web. A maioria das coisas que você precisa já estão presentes, prontas para uso e sem a necessidade de instalar nada para lidar com os recursos mais básicos e avançados de aplicativos da web. o que o torna uma excelente escolha para criar um aplicativo da web. A maioria das coisas que você precisa já estão presentes, prontas para uso e sem a necessidade de instalar nada para lidar com os recursos mais básicos e avançados de aplicativos da web. o que o torna uma excelente escolha para criar um aplicativo da web. A maioria das coisas que você precisa já estão presentes, prontas para uso e sem a necessidade de instalar nada para lidar com os recursos mais básicos e avançados de aplicativos da web.

PHP é escrito em C; é uma linguagem de nível superior à linguagem C. É, portanto, mais fácil de entender e mais permissivo, mas oferece, em comparação com C, menos desempenho estendido e menos possibilidades. Se você precisar

escrever algum código assembly em PHP para se comunicar com algum hardware personalizado, por exemplo, a afirmação é simples: você não pode. Você terá que escrever uma extensão PHP, que será escrita na linguagem C (o que permite escrever partes do código-fonte na linguagem assembly). Este é um caso super avançado, é claro, mas você entendeu.

Vamos além, comparando PHP e Hypertext Markup Language (HTML). Embora o HTML não seja uma linguagem de programação, mas sim uma linguagem de descrição, ainda possui semelhanças com o PHP: ambas as linguagens são usadas para expressar ideias humanas que podem ser interpretadas pela máquina. A questão é simples: para quem não conhece muito bem as linguagens técnicas e de programação, certamente conseguirá explicar o conteúdo de um arquivo HTML, o que ele representa, sua semântica e sua finalidade. Em PHP, é uma questão diferente. De fato, entre divisão de arquivos, divisão de classes, programação orientada a objetos (OOP) e todos esses outros conceitos, certamente você levará muito mais tempo para fazer seu interlocutor não técnico entender o objetivo de tudo isso. No entanto, embora o HTML não permita OOP, ele não permite ramificação condicional, gravação em um arquivo, gerenciamento de requisições enviadas ao servidor, e assim por diante. Assim, acabamos com uma linguagem muito mais compreensível para o ser humano porque é muito próxima da nossa linguagem natural, mas com muito menos possibilidades.

Apesar dessas diferenças, devemos ter em mente o seguinte: o objetivo principal das línguas é exatamente o mesmo: ser compreendido pelo maior número de pessoas e pelo computador. Escrever código significa ser compreensível. É expor ideias. E assim como no dia a dia quando você expõe suas ideias, quanto mais simples e direto você for, mais as pessoas conseguirão te entender.

Seja compreendido, não inteligente

Acontece com muita frequência que diante de desafios técnicos e principalmente de código-fonte, queremos fazer as coisas de uma maneira fina, bonita e até “sexy”, como diriam alguns. Isso é completamente normal - como o código é uma parte importante de nossas vidas como desenvolvedores, às vezes queremos mostrar a extensão de nossas habilidades. Embora isso possa ser justificado às vezes, muitas vezes é uma péssima ideia querer mostrar a extensão completa desses talentos. Obviamente, nosso ego sofre um golpe - devemos nos conter às vezes. Você acabou de aprender novas maneiras de fazer as coisas, novas maneiras de codificar e novos princípios dos quais você está ardentemente convencido. Você passou um fim de semana aprendendo esta nova forma de organizar seu código e seu projeto, você sente isso como uma revelação, e tem certeza disso: você deve mostrar esta nova descoberta para seus colegas e sua equipe; vai revolucionar o projeto e trazer só coisas boas. Além disso, você receberá o crédito por essa novidade e será o referente dela. No entanto, esta não é a abordagem certa.

Não se engane. Aprender todos os dias, no seu tempo ou não, é uma coisa excepcional. Se você tiver a oportunidade de fazer isso, sairá melhor. Em uma nota lateral: isso não é obrigatório de forma alguma! Nada deve se forçar a codificar em seu tempo livre. É totalmente normal continuar codificando e programando para o trabalho.

O desejo de compartilhar suas descobertas e experiências com seus colegas é normal — mais do que isso, é saudável. Compartilhar conhecimento para elevar seus entes queridos é a melhor coisa de todas, e explicar algo é a melhor maneira de aprender sozinho. O erro é querer aplicá-lo imediatamente, de todos os lugares, o tempo todo. Cada forma tem suas vantagens e desvantagens. É

absolutamente necessário estar ciente das desvantagens que isso traz. Em geral, os mais comuns são a aplicação no projeto atual e a resistência à mudança de outras pessoas envolvidas no projeto. Basta dar o exemplo dos famosos princípios SOLID: embora sua eficácia seja comprovada, eles podem ser de difícil acesso para os recém-chegados.

A resistência à mudança é normal e natural para todas as pessoas. Nosso cérebro gosta de regularidade – gosta de ciclos e não gosta do inesperado. Isso obviamente se reflete no ambiente de trabalho, mas também em todos os outros aspectos da vida: dieta, exercícios e sono. Para código e nossos hábitos de trabalho, é exatamente o mesmo. Mais uma vez, consistência e regularidade são fundamentais.

Se você trazer suas novas descobertas para o projeto, terá que considerar o treinamento das pessoas que já estão no local. Eles não vão necessariamente querer mudar seus hábitos se os hábitos já existentes forem apropriados e já atenderem à necessidade. Novos hábitos também significam treinar todas as pessoas que não conhecem essas maneiras de fazer as coisas. Isso requer investimento pessoal, até mesmo substancial em alguns casos. Trata-se então de horas de aprendizagem quer no tempo pessoal, quer no horário de trabalho, e essas horas serão, portanto, horas em que a produtividade será inegavelmente reduzida. Às vezes isso é necessário e às vezes é até uma ideia promissora. Mas então, você deve ser capaz de justificá-lo para todos, incluindo as partes não técnicas do projeto, e isso pode ser claramente uma parte crítica, especialmente se você for pressionado pelo tempo.

Além disso, algumas práticas de programação podem fazer maravilhas, ser comprovadas e tornar a vida muito mais fácil. No entanto, eles têm uma grande desvantagem: o tempo de integração no projeto. Basta tomar como exemplo a prática da “programação if-less”. Este método de programação diz para nunca, absolutamente nunca usar if e ramificação condicional em seu código. Isso exige o uso massivo e puro de programação orientada a objetos (OOP). No papel, parece bom, e a satisfação intelectual de tal técnica deve ser excepcional. Sua

eficiência, uma vez dominada, é bastante clara. Tudo fica mais inteligente e o código fica visivelmente claro. Resumindo, está tudo pronto para que seu próximo projeto adote a programação if-less desde o início.

Porém, no dia em que alguém vier ao seu projeto para te ajudar e tentar entender o que você está fazendo (ou mesmo para manter e evoluir o projeto com você), a constatação será avassaladora: se a pessoa não conhece essa técnica de programação (sendo uma técnica longe de ser uma generalidade), todo o processo de introdução do projeto será penoso. Além de ter que treinar a pessoa nas restrições funcionais do projeto, ela também terá que ser treinada em uma nova técnica de programação à qual certamente não está acostumada. Isso significa entender o projeto, entender o que está em jogo, mudar hábitos, mudar a forma de fazer e trabalhar, reformular a forma de pensar. Entendemos muito rapidamente o custo de tal operação. Pode ser justificado,

Estamos falando de programação if-less aqui, mas o mesmo vale para outras formas de fazer coisas que não são uma regra geral. O desenvolvimento orientado a testes (TDD) é um deles! Integrar o TDD em um projeto pode ser doloroso e complicado, como vimos antes. No entanto, o TDD influencia principalmente a ordem de fazer as coisas, mais do que aprender uma maneira completa de codificar. Cabe a você ver, dependendo do seu contexto e das suas restrições, até que ponto vale a pena correr esses riscos.

De qualquer forma, se você escolher uma nova técnica de programação que pode ser descrita como exótica, poderá ter um código exemplar, limpo e eficiente e super-sustentável. O problema é que ninguém será capaz de entendê-lo. Lembre-se do que foi dito na seção anterior: o código é usado para expressar e transmitir ideias. Costuma ser compreendido por máquinas e, principalmente, por seres humanos. Seria uma pena sacrificar o segundo ponto, e é por isso que as linguagens de programação de alto nível foram inventadas.

Uma nota sobre manutenibilidade

E é aqui que fica complicado. Seu código está pronto — funciona. Você seguiu um novo método de programação e os desenvolvimentos preliminares do projeto estão indo bem por vários meses. E é bastante óbvio: pode não haver uma base sobre a qual construir seu projeto; você teve a sorte de começar a partir de uma folha de papel em branco. No entanto, a questão da manutenção surgirá em breve. Seja qual for a técnica de programação que você escolher, sejam quais forem as pessoas que estiverem trabalhando nela, sempre aparecerão bugs. Você pode precisar de novas pessoas para consertar tudo isso (e, assim, ensinar-lhes seus métodos de trabalho). Tem a certeza que domina suficientemente as suas novas metodologias para garantir o seguimento de uma candidatura ao longo de vários anos? É bem possível, mas você deve estar atento a isso e saber o que fazer caso fique preso na manutenção do seu aplicativo.

O objetivo deste capítulo não é desencorajar a iniciativa e o teste de novas metodologias de trabalho. É mais uma questão de ter plena consciência dos riscos de optar por novas metodologias de trabalho, principalmente a longo prazo. Veremos mais adiante que devemos ser incrivelmente cuidadosos com as últimas tendências que podem desaparecer da noite para o dia.

O mesmo se aplica às sintaxes de programação que podem parecer elegantes à primeira vista, mas que na realidade são um pesadelo de manutenibilidade. Dentre essas práticas, podemos encontrar, de forma não exaustiva, as destacadas a seguir.

Usando operadores binários e notações octal, hexadecimal e binária

Em geral, o uso de operadores binários em números inteiros para executar operações (deslocamento à esquerda e à direita, AND lógico, OR lógico, inversão de bits e assim por diante) é mais inútil do que qualquer outra coisa. Sua raridade os torna uma sintaxe que pode parecer elegante para realizar determinadas operações. No entanto, este não é o caso, e dominar as operações binárias não deve ser um pré-requisito para entender o código PHP.

O uso de notações octal, hexadecimal e binária pode ser justificado às vezes. Por exemplo, a notação octal pode ser usada quando você quiser jogar com permissões de arquivo. Hexadecimal pode ser usado se você quiser usar sinalizadores em seus métodos, bem como notação binária. Mas, em geral, além de tornar seu código complicado de ler, não há muito o que fazer.

Atribuindo uma variável e usando gotos

Uma variável pode ser atribuída ao mesmo tempo que testa seu valor. Aqui está um exemplo:

```
se nulo === ($var = método()))
```

Na melhor das hipóteses, você economiza uma linha de código. Mas já faz muito tempo que não nos preocupamos com o tamanho dos arquivos de origem do aplicativo, que são otimizados em tempo de execução pelo interpretador PHP de qualquer maneira. Não há custo para atribuir a variável antes do teste e seu código ficará imediatamente mais legível.

Uma instrução goto permite pular partes inteiras do código e até mesmo “subir” no código. Embora possa ser útil em alguns casos extremamente específicos, definitivamente não deve ser usado na maioria dos casos. Por anos, o uso de instruções goto tem sido desaprovado na maioria das linguagens de programação. De fato, eles trazem grande complexidade para a compreensão do fluxo de código. Há um nome em que muitos usos de goto devem ser lamentados: código espaguete.

Uso excessivo de comentários

Às vezes, vemos o uso abusivo de comentários em centenas de linhas para explicar o tipo de todas as variáveis, sua utilidade, todas as exceções levantadas pela função, o valor de retorno em detalhes e assim por diante. Podemos até encontrar arquivos de origem onde há mais comentários do que código. Todos esses detalhes podem ser omitidos na maioria das vezes por uma nomeação clara de seus métodos e variáveis. Além disso, a digitação de variáveis, argumentos e retornos de função em versões recentes do PHP também resolve esse problema. No entanto, o uso de comentários para produzir documentação pode ser totalmente justificado e deve ser usado sempre que possível. Ninguém nunca vai reclamar porque há “muita documentação”. Sinta-se à vontade para escrever dezenas de linhas sobre o que é uma classe, interface, método e assim por diante; de forma mais geral, escolhas descendentes, técnicas e/ou funcionais sobre ele, e assim por diante. Quando falamos em “uso abusivo de comentários”, estamos falando de comentários que estão explicando o que está acontecendo no código.

Usando comparações ternárias

Aqui está um exemplo de uma comparação ternária:

```
$var === null ? 'é nulo': 'não é nulo'
```

Embora comparações ternárias possam tornar o código conciso e incluir uma condição - por exemplo - na passagem de um argumento em uma função, elas não devem ser abusadas, especialmente comparações ternárias aninhadas, que imediatamente se tornam ilegíveis e induzem a dor de cabeça no primeiro nível de aninhamento . Você pode ver a prova disso no exemplo a seguir:

```
$var === null ? 'é nulo': is_int($var) ? 'é int': 'não é nulo'
```

Esta não é uma linha de código muito clara ou legível, o que torna as condições ternárias complicadas de ler quando a condição não é básica e simples, como no primeiro exemplo.

Usando abreviaturas

Aqui está talvez a prática mais comum a ser desencorajada: o uso de abreviaturas por toda parte. Novamente, no final do último milênio, poderíamos ter motivos para usar abreviações: o espaço e o armazenamento eram muito mais limitados do que hoje e os editores de código não eram tão inteligentes com todos os recursos de preenchimento automático que temos hoje. Portanto, nomear uma variável `$userPasswordRequest` em vez de `$usr` facilitará a vida de todos: tanto a sua quanto a dos desenvolvedores que voltarão ao seu código e não precisarão perguntar o que significam suas abreviações. Novamente, com as ferramentas de preenchimento automático que temos hoje, não faz sentido nomear nossas variáveis dessa maneira.

Trazendo micro-otimizações para o seu código

As microotimizações são alterações muito pequenas feitas no código que podem prejudicar sua legibilidade, justificadas pela otimização do código. Assim, seria mais rápido para executar. O problema é que muitas vezes isso não é útil, primeiro porque você não precisa otimizar uma instrução ao nanossegundo (por causa do poder dos processadores hoje em dia), mas também porque muitas otimizações são feitas por interpretadores e compiladores de linguagem. Assim, você sacrifica parte da legibilidade do seu código por algo que não é útil e que será feito automaticamente. Além disso, isso muitas vezes provoca debates estéreis onde ninguém tem mais razão do que ninguém. Nesses debates de micro-otimização específicos do PHP, encontramos em particular a posição dos operadores de incremento (++) e decremento (--) em posição antes ou depois das variáveis, o uso da barra invertida na frente dos métodos da Standard PHP Library (SPL), ou a declaração estática ou não de funções anônimas. Novamente, a resposta é: pense na consistência com o restante do código e seja pragmático. Você certamente não precisa dos 10 nanossegundos que serão (talvez, em algumas circunstâncias) economizados por sua otimização que terá iniciado um debate acalorado dentro da equipe de desenvolvimento.

Recodificando os métodos da SPL

Temos muito uso para uma linguagem que possui uma biblioteca padrão muito extensa. A biblioteca padrão é um conjunto de classes e métodos fornecidos com cada instalação do PHP. Infelizmente, percebemos rapidamente que é bastante desconhecido e oferece mais possibilidades do que você imagina. Como resultado, muitas vezes nos deparamos com métodos SPL que são recodificados no projeto porque o desenvolvedor em questão não sabia da existência do método padrão. Isso é extremamente lamentável e, em alguns casos, um problema real pelos seguintes motivos:

Fique à vontade para dar uma olhada na documentação oficial do PHP; alguns métodos como `natsort()` podem surpreendê-lo e economizar horas de desenvolvimento!

A lista poderia continuar, mas o ponto é que, embora você goste de usar essas coisas, você será o único que sentirá alguma satisfação. Um desenvolvedor júnior pode ficar completamente perdido ao ver essas práticas, enquanto um desenvolvedor sênior não entenderá o valor de usar essas práticas quando outras mais claras e simples estiverem disponíveis. Seu código deve ser entendido pelo maior número de pessoas possível. Mostre a extensão de suas habilidades, conhecimento e proficiência produzindo algum código simples, trivial e legível para um problema que parecia excessivamente complexo no início.

Resumo

Você entende o que significa “construir sobre o que sabemos”? Percebemos com um pouco de retrospectiva que isso é novamente muito bom senso e altruísmo, pensando nos próximos desenvolvedores que passarão por cima do nosso código. Aqui, não havia dúvida de olheiros, SOLID, Keep It Simple, Stupid (KISS) ou outros princípios. Trata-se de repensar nossos próprios métodos de escrever código.

Devemos lembrar que o básico pode (e deve) ser questionado e não considerado imutável. A autoconfiança é uma coisa maravilhosa, e se conseguirmos aliar isso ao questionamento contínuo dos teus hábitos numa perspectiva de melhoria contínua, estás no caminho certo para te tornares um excelente programador, capaz de escrever código limpo com naturalidade, e trazendo seus colaboradores com você nesta prática.

Tomar a iniciativa é uma coisa notável; conhecer os riscos e avaliá-los em seu contexto é a chave para buscar a perfeição. Dessa forma, você consegue saber se realmente vale a pena, mas também pode justificar suas escolhas para as pessoas que executam o projeto. Mais uma vez, voltamos à capacidade de justificar todas as nossas escolhas e ações quando se trata de código limpo. Código limpo não é apenas evitar o uso de operadores binários ou o uso de notações hexadecimais. Significa pensar no meio ambiente, nas restrições e no entorno do nosso projeto. Código limpo não é apenas sobre código. Felizmente, isso é exatamente o que veremos no próximo capítulo.

É mais do que apenas código

Descrever PHP: Hypertext Preprocessor (PHP) como uma linguagem de programação não seria um pouco redutor quando você pensa sobre isso? Devemos encarar os fatos: PHP não é uma linguagem de programação simples. É um ecossistema completo, com uma comunidade gigantesca, milhares de colaboradores e novos recursos sendo propostos e lançados regularmente. Mas não apenas isso: milhões de bibliotecas e interfaces de programação de aplicativos (API) são escritas e lançadas graças ao PHP. Mesmo muitas ferramentas de linha de comando são totalmente desenvolvidas graças à linguagem PHP. PHP é um mundo inteiro por conta própria. Vamos começar examinando as razões pelas quais o PHP não é apenas uma linguagem para escrever um site.

Estes são os tópicos que abordaremos neste capítulo:

PHP como um ecossistema

Isso pode ser percebido em vários itens que podemos listar juntos, como segue:

PHP é uma linguagem que tem uma reputação comprovada. Sua robustez e eficiência o tornaram o idioma preferido de alguns dos maiores sites do mundo. Onde alternativas foram implementadas no passado ou atualmente, como Python (que é usado por cerca de 1,2% de todos os sites no momento da redação) para alguns sites do Google, PHP é a maioria. Obviamente, muitas tecnologias atraentes estão surgindo e conquistando mercado do PHP, como Node.js ou C# e o .NET Framework. O PHP ainda tem um bom futuro pela frente. Saber escrever um site em PHP garante que você saberá ler o código-fonte da esmagadora maioria dos sites existentes no mundo.

Por todas essas razões, o PHP é um ecossistema. Levando a reflexão um pouco mais longe... se PHP não é apenas uma linguagem de programação e se PHP não é apenas código, por que deveríamos limitar código limpo a código?

O código limpo também pode conter, por extensão, a escolha certa de dependências externas e bibliotecas para instalar em seu projeto. Vamos ver por que você deve escolher suas dependências com sabedoria e como escolhê-las bem para limitar os riscos.

Escolhendo as bibliotecas certas

Escolher a biblioteca externa certa para instalar pode ser um verdadeiro desafio. É um desafio que todos enfrentamos ou enfrentaremos um dia. A razão é simples: não adianta reinventar a roda. A razão pela qual queremos instalar uma biblioteca externa geralmente é a mesma. Temos um problema específico que queremos resolver da forma mais limpa possível. Aqui surgem duas situações:

É então interessante recorrer a uma biblioteca externa cujo papel é nos trazer uma solução bem específica para o nosso caso. As vantagens são múltiplas, conforme descrito aqui:

Podemos ver claramente que a escolha de dependências de código aberto é bastante inevitável. Se você deseja que o seguro não acabe com uma ferramenta inutilizável e indisponível da noite para o dia, o código aberto é feito para você, porque você pode armazenar uma cópia do código-fonte pelo tempo que quiser, sem medo. Esta é a primeira maneira excelente de escolher uma biblioteca externa.

Um segundo fator a considerar é a frequência das atualizações do projeto. Obviamente, se um projeto é de código aberto, mas não é atualizado há vários meses ou mesmo anos, cuidado: pode ser um abandono. Nesse caso, significa que o projeto pode não suportar as próximas versões do PHP, por exemplo, ou que os bugs e falhas de segurança não serão mais corrigidos. Existem duas maneiras fáceis de saber se um projeto ainda é mantido ou não, conforme definido aqui:

Um terceiro fator a considerar é a documentação da biblioteca. Você provavelmente desejará certificar-se de que o projeto tenha documentação mínima e suficiente para configurar o básico. Se nenhuma documentação for fornecida, você pode ter certeza de que usar a biblioteca externa será uma dor sistemática. De fato, toda a manutenção do código se tornará uma batalha para lembrar como o projeto funciona, sem documentação para ajudá-lo ou compartilhar conhecimento. Além disso, isso também pode estar muito relacionado à comunidade em torno da tecnologia que você deseja usar. Se muito poucas pessoas usarem o projeto que você deseja integrar ao seu e a comunidade for bastante inativa, ou mesmo inexistente, ninguém poderá ajudá-lo da maneira ideal. Essa pode ser uma maneira eficaz de decidir se deve usar esta ou aquela dependência em seu código.

Um quarto fator é o número de dependências das quais a própria biblioteca depende. De um modo geral, preferimos uma biblioteca que tenha poucas dependências. Menos dependências significam menos pacotes para atualizar e menos terceiros, portanto, há menos chance de problemas em uma dessas partes.

Finalmente, muitos projetos têm distintivos de integração contínua (CI) em sua página principal. Esses emblemas permitem que você saiba rapidamente a cobertura do teste (lembrete: a proporção do código coberto pelos testes), o número de testes, a versão mais recente e assim por diante. Obviamente, é melhor escolher um projeto com o maior número possível de testes e com uma alta cobertura de teste para limitar os problemas durante as atualizações.

Uma palavra sobre versionamento semântico

Falando em atualizações, vamos falar sobre controle de versão e, especialmente, controle de versão semântico. Se a biblioteca externa que você deseja usar seguir as regras do versionamento semântico, isso pode ter um impacto incrivelmente positivo e tranquilizador em seus desenvolvimentos e atualizações. Vamos dar uma olhada no que isso significa exatamente.

O que é versionamento semântico?

O controle de versão é simplesmente colocar um número em uma versão do código-fonte. Estamos todos familiarizados com versões como 1.0, 1.5.0, 2.0.0 e assim por diante. A versão semântica acrescenta uma semântica, ou seja, um significado preciso para cada um desses números. Vamos pegar a versão 2.3.15 como exemplo. Aqui está como o versionamento semântico divide este número de versão:

Podemos ver as vantagens do versionamento semântico: serenidade, lógica e consistência. Obviamente, existem outras variações, como Alpha, Beta, Release Candidate e Golden Master. Mas estes são mais raros.

Como lidar com versionamento semântico

O controle de versão semântico também inclui uma notação específica que permite que seu gerenciador de dependências saiba como instalar novas versões e quando atualizar suas dependências. Vamos pegar como exemplo este extrato do arquivo que o Composer usa para instalar dependências (e este é o mesmo princípio para muitos gerenciadores de dependências por aí):

```
{
```

```
"exigir": {
```

```
"php": ">=7.3",
```

```
"symfony/dotenv": "3.4.*",
```

```
"symfony/event-dispatcher-contracts": "~1.1",
```

```
"symfony/http-client": "^4.2.2"
```

}

}

Este trecho descreve quatro dependências: uma versão mínima para PHP, bem como três bibliotecas externas. Realmente não importa quais são essas bibliotecas. Podemos observar aqui quatro maneiras distintas de definir as versões que queremos aceitar em nossas dependências. Vamos ver o que eles são.

A primeira maneira de escrever a versão que observamos é usando o operador `>=`. Este é um dos mais fáceis de entender: queremos aceitar todas as versões maiores ou iguais à especificada. Aqui, nosso aplicativo aceita todas as versões do PHP superiores à versão 7.3, bem como a própria versão 7.3. Obviamente, os gerenciadores de dependência aceitam outros operadores: `=`, `<`, `>` e `<=`. Você também pode combinar esses operadores para obter restrições de versão muito precisas, por exemplo, escrevendo `">=1.2.0 <2.0.0"`.

O segundo operador é bastante conhecido porque é usado em muitos outros contextos. É o curinga, denotado `*`. Este símbolo simplesmente representa o fato de que você pode substituí-lo pelo que quiser. No exemplo anterior, aceitamos todas as versões de patch da versão 3.4 da dependência. Isso permite que ele se beneficie apenas de correções de bugs, sem atualizar a versão secundária. Esse curinga pode ser colocado em qualquer lugar no número da versão. Por exemplo, a notação `3.*` se beneficiará de todas as versões secundárias da versão principal 3.

A notação a seguir é o uso do operador til, `~`. Este operador significa que você só

se beneficiará dos patches da versão fornecida. No exemplo, nos beneficiaremos de todas as versões de patch da versão 1.1 da dependência (ou seja, 1.1.0, 1.1.1, 1.1.2 e assim por diante). Isso é notavelmente semelhante ao operador curinga, exceto que o operador curinga não pode ser colocado em nenhum lugar no número da versão e diz respeito apenas a patches. Além disso, vale a pena notar que o Composer interpreta o til de maneira um pouco diferente: ele também permite versões secundárias, não apenas patches. Se você estiver usando o Composer e quiser se beneficiar apenas das versões patch sem as versões secundárias, deverá usar o operador curinga.

Finalmente, o último operador que veremos é o operador circunflexo, denotado $\wedge$. No exemplo anterior, o operador circunflexo permite todas as versões de patch, bem como versões secundárias da versão principal 4 (ou seja, 4.2.2, 4.2.3, 4.4.0 e assim por diante). Se você deseja definir uma versão mínima de uma dependência enquanto aceita novos patches e versões secundárias, mas recusa as versões principais (que podem trazer alterações importantes) automaticamente durante a atualização de bibliotecas externas, esta é uma escolha particularmente boa. É por isso que é um dos operadores mais populares.

As possibilidades são infinitas e, depois de dominar essa notação, você pode ter certeza de atualizar as dependências do seu projeto. No que diz respeito às boas práticas, é sempre uma boa ideia aceitar todos os novos patches e versões secundárias de uma dependência. Você nunca deve bloquear uma dependência para uma versão extremamente específica sem quaisquer condições ou possibilidade de atualização. De fato, se uma biblioteca externa respeitar escrupulosamente o versionamento semântico, você não terá nenhum conflito com seu código existente. Alterações de última hora são reservadas para versões principais. Portanto, você não deve aceitar automaticamente as versões principais ao atualizar suas dependências: é provável que você tenha que adaptar seu código para que funcione corretamente.

Estabilidade versus tendências

Vamos terminar este capítulo com algumas palavras sobre as versões mais recentes, mas também sobre tecnologias e bibliotecas externas da moda.

Primeiro, vamos falar sobre as versões mais recentes das bibliotecas externas. Claro, podemos ficar tentados a usar os mais recentes, aqueles que foram lançados apenas algumas horas atrás. Vale lembrar que bugs podem aparecer, e uma nova versão de patch pode ser lançada em um futuro próximo se for o caso. Ou não. E, neste caso, o bug pode persistir por um tempo. Portanto, é particularmente importante escrever testes. Imagine o conforto: você atualiza todas as suas dependências, executa seu conjunto de testes e, se todas as luzes estiverem verdes (e seu aplicativo for devidamente testado), você pode ter certeza de que está tudo bem.

Dito isso, se algum teste ficar vermelho porque você atualizou uma biblioteca externa, será necessário investigar para descobrir de onde isso está vindo. De qualquer forma, você não deve pensar que está a salvo de qualquer problema se suas dependências estiverem bem corrigidas e restritas ou se você aceitar apenas patches e/ou versões menores. Os patches também podem trazer bugs - nunca se sabe.

No que diz respeito às versões Alpha, vamos ser claros: essas versões não são feitas para aplicativos de produção. As diferentes bibliotecas são claras neste ponto: o código pode mudar de um dia para o outro, trazendo mudanças sem aviso prévio. Em suma, você deve ser incrivelmente cuidadoso. Dito isso, se você quiser avaliar essas versões para ver por si mesmo, os desenvolvedores das

bibliotecas ficarão encantados em receber seu feedback. As versões Beta devem ser mais estáveis e não trazer mais mudanças significativas. Você ainda deve ter muito cuidado ao usá-los.

Como regra geral, use apenas as versões finais e estáveis em produção. Reserve as versões Alpha e Beta para ambientes de desenvolvimento e teste se quiser estar pronto no dia do lançamento estável para implantação de produção. Novos recursos são sempre empolgantes, mas nunca valem a pena sacrificar a estabilidade do seu aplicativo. Seus usuários não se importam com os novos recursos das bibliotecas externas que você usa: apenas a estabilidade importa - o fato de simplesmente funcionar.

Agora, vamos falar sobre as tecnologias da moda (uma biblioteca PHP externa, uma nova ferramenta ou até mesmo uma nova linguagem de programação). Você ouve todos ao seu redor falando sobre uma determinada tecnologia. Essa tecnologia está se espalhando como um incêndio, você ouve falar dela em todos os lugares na Internet, grandes empresas estão entrando nela e conferências de tecnologia são sobre isso. Você deve ter cuidado com esse tipo de coisa. Mesmo que as promessas dessas tecnologias possam ser empolgantes e revolucionárias, pense primeiro no que é importante: seus usuários.

Essa tecnologia fará uma diferença real para seus usuários finais? Realmente vale a pena treinar e levar semanas ou até meses para descobrir como funciona? Você precisa ter certeza de que terá um impacto positivo real em seu projeto. Você também deve ter em mente que a tecnologia inovadora terá uma pequena comunidade. Os impactos são imediatos, conforme descrito a seguir:

Por fim, você deve garantir que o projeto seja robusto para não acabar com uma tecnologia recente abandonada sem aviso prévio. Isso acontece com mais frequência do que você pensa, e muito (se não todo) do seu trabalho terá sido em

vão. Portanto, fique atento às últimas tecnologias não comprovadas e certifique-se da robustez e seriedade do projeto. Espere até que você tenha algum tempo para pensar sobre isso. Mais uma vez, seus usuários certamente poderão prescindir dessa tecnologia (da qual eles não estarão cientes) até que ela esteja madura.

Resumo

Limitar o PHP à linguagem de programação é redutor. Acabamos de ver – é um ecossistema real com uma comunidade rica e ativa, e muito longe de enterrar sua linguagem favorita. Os desenvolvimentos em torno do PHP são incontáveis, e a própria linguagem evoluiu da maneira mais bonita nos últimos anos. As contribuições das funcionalidades deram um verdadeiro segundo fôlego a esta, permitindo-lhe reivindicar – ainda hoje – o primeiro lugar entre as linguagens de programação mais utilizadas no lado do servidor para uma aplicação web.

Tudo isso não seria nada sem a explosão no número de bibliotecas externas disponíveis para a linguagem. Você tem um problema; existe uma solução. Temos a sorte de que a maioria das bibliotecas externas seja de código aberto. Milhares de desenvolvedores disponibilizam, de forma voluntária e gratuita, o fruto de horas, semanas ou anos de trabalho.

Fazer uma escolha entre essas bibliotecas pode ser difícil e desafiador. É importante, até mesmo obrigatório, fazer um verdadeiro trabalho de pesquisa antes para ter certeza de fazer a escolha certa. Não estamos imunes a obstáculos e incidentes, mas este capítulo forneceu ferramentas e soluções prontas para uso para limitar os riscos. Acima de tudo, não se precipite nas tecnologias mais modernas. Se você deseja atrair usuários e fazer com que eles continuem usando seu aplicativo mais do que outro, as palavras-chave são “robustez” e “estabilidade”!

Já falamos muito sobre o trabalho dos outros, mas não devemos esquecer nossas próprias conquistas. Como você consegue desenvolver bons hábitos para

encontrar seu caminho em seu código enquanto consegue encontrar seu caminho sem esforço no código-fonte de suas bibliotecas externas favoritas quando precisa entender seu funcionamento interno? Voltamos ao que dissemos nos primeiros capítulos: tendo os mesmos hábitos, nos entendemos com mais facilidade. Isso obviamente se aplica à organização de um projeto, na nomenclatura de arquivos, na estrutura de pastas e assim por diante. E é exatamente isso que veremos na prática no próximo capítulo.

Otimizando seu tempo e separando responsabilidades

É hora de um pouco de prática depois de toda essa teoria! Já vimos muito juntos: princípios avançados sobre código limpo, como escolher as bibliotecas externas certas para seus aplicativos e como aproveitar os patches mais recentes dessas bibliotecas sem correr o risco de fazer seu projeto explodir no ar. Mas não devemos esquecer que existe a palavra “código” em “código limpo” (obviamente). Neste capítulo, portanto, vamos nos concentrar um pouco mais no código-fonte do seu aplicativo e ver os seguintes pontos:

Nomenclatura e convenções organizacionais

Devemos adicionar um aviso antes de mais nada. As convenções de nomenclatura e ideias organizacionais dadas neste capítulo não são uma verdade absoluta. Como vimos antes, o mais importante é respeitar as convenções já existentes em seu projeto e ser consistente com sua equipe. Se achar necessário, é possível adaptar estas regras às suas necessidades. Novamente, o importante é usar o bom senso e a lógica e ser o mais claro possível.

Vamos primeiro falar sobre a nomenclatura dos arquivos de origem. Obviamente, as convenções de nomenclatura diferem de uma tecnologia para outra (dependendo se você usa um determinado framework ou outro, as boas práticas podem mudar, por exemplo). No entanto, podemos notar algumas convenções que podem ser encontradas em quase todos os lugares.

Arquivos de classe e arquivos de interface

PHP: pré-processador de hipertexto(PHP) os arquivos de origem que definem uma classe, uma classe abstrata ou uma interface devem ter o mesmo nome da classe ou interface em questão. Por exemplo, a classe Foo deve ser definida em um arquivo Foo.php. Mais do que uma convenção, esta técnica de nomenclatura tem um real interesse técnico. De fato, os mecanismos de carregamento automático do PHP assumirão que seu arquivo define uma classe com o mesmo nome. O carregamento automático permite que o PHP descubra automaticamente as classes definidas em sua aplicação e, em particular, graças aos namespaces (voltaremos a isso em alguns momentos, pois estão diretamente ligados à organização de arquivos em seu projeto). Se você nomear seus arquivos e as classes que eles definem de forma diferente, o carregamento automático provavelmente falhará e gerará um erro. O estilo de nomenclatura mais comum nas várias linguagens e na comunidade global de desenvolvedores para nomear classes é o PascalCase. Este estilo simplesmente adiciona uma letra maiúscula ao início de cada palavra. Portanto, uma classe chamada My super service class seria chamada MySuperServiceClass se usássemos PascalCase. Existem outros estilos de nomenclatura - veremos alguns deles e em quais casos eles se aplicam.

executáveis

Arquivos PHP, sendo scripts de linha de comando executáveis, têm uma forte tendência de serem nomeados em letras minúsculas. A estrutura de teste PHPUnit é um bom exemplo. Na prática, é usado como um comando do sistema em um terminal. Na realidade, é simplesmente um script PHP. Usamos o hífen como separador de palavras. Esse estilo de nomenclatura é chamado kebab-case. Este é o estilo de nomenclatura mais usado por programas de linha de comando. Os conhecidos apt-get, docker-compose e git cherry-pick são embaixadores perfeitos. Nada impede que você nomeie seu arquivo executável de outra maneira e tudo funcionará bem. No entanto, ao nomear seus aplicativos de linha de comando escritos em PHP dessa maneira, você fornece uma experiência de interface de linha de comando (CLI) uniforme com a grande maioria dos comandos. Isso é exatamente o que queremos quando desenvolvemos uma aplicação de linha de comando em PHP:

Ativos e recursos da Web

Há outro caso em que o estilo kebab-case é usado, principalmente para recursos públicos da Web, principalmente arquivos JavaScript, Cascading Style Sheets (CSS) e Hypertext Markup Language (HTML). De fato, kebab-case é um estilo de nomenclatura facilmente legível e compreensível por todos, esteja você no campo técnico ou não. Além disso, os mecanismos de pesquisa são mais capazes de entender a semântica do seu recurso usando essa notação. Portanto, é essencial se você deseja fazer otimização de mecanismos de pesquisa (SEO). Aqui está um exemplo: é muito mais comum - até natural - ver um localizador uniforme de recursos (URL) como `/contact-us`, em comparação com algo como `/contactus` ou `/ContactUs`. Isso pode ser particularmente importante se você precisar trabalhar em arquivos front-end.

Nomeando classes, interfaces e métodos

Como vimos nos capítulos anteriores, as abreviações devem ser banidas do seu código. Os ambientes de desenvolvimento integrado (IDEs) são atualmente poderosos o suficiente para autocompletar os nomes das classes e métodos que você usa. Portanto, não é útil abreviar repetidamente os nomes de suas classes, métodos e variáveis, pois isso só causa confusão. É também por isso que suas classes abstratas devem começar com o prefixo `Abstract` (por exemplo, `AbstractMailer`) e suas interfaces devem terminar com o sufixo `Interface` (por exemplo, `MailerInterface`). Isso torna o nome um pouco mais longo, mas não há confusão em seu uso. Seu propósito é claro, definido e visível imediatamente. Não tenha medo de dar nomes longos às suas classes se for necessário para sua compreensão. `AbstractWebDeveloperConsoleStreamWrapperExtension` pode parecer extremamente longo para um nome de classe, mas fica imediatamente claro para que serve em um contexto de projeto, sem ter que fazer muitas perguntas. Novamente, com o recurso de preenchimento automático do seu IDE, você poderá usá-lo em segundos digitando as primeiras letras. O mesmo vale para seus nomes de atributo e método. Seja explícito.

Falando sobre nomeação de atributos e métodos, tendemos a preferir o estilo de nomenclatura `camelCase`. O princípio é o mesmo do `PascalCase` (ou seja, colocar uma letra maiúscula no início de cada palavra), exceto que a primeira letra do nome está em minúscula (por exemplo, `myGreatMethod`). Algumas linguagens usam `PascalCase` para nomear métodos como `C#`. Sejamos honestos - não há nenhuma justificativa ou argumento real para isso, e ambos os estilos de nomenclatura são realmente iguais. Pela primeira vez, é realmente uma convenção - um hábito específico do idioma.

Nomeando pastas

As convenções de nomenclatura para pastas são semelhantes às dos arquivos. PascalCase é usado principalmente para pastas. Outros estilos de nomenclatura também podem ser usados para pastas expostas publicamente, como recursos da Web. Você não deve hesitar em criar uma grande estrutura de árvore e dar-lhe significado. Portanto, as pastas nomeadas simplesmente Manager, Service ou Wrapper devem ser evitadas. Esses termos são muito genéricos e não permitem uma compreensão fácil do que eles definem. Preferimos variantes mais explícitas, como Mail e suas subpastas Provider, Logger e assim por diante. Essas subpastas podem ter nomes mais genéricos, estando contidas em uma pasta cujo nome define um contexto: um domínio. Uma maneira inteligente de se orientar é usar pastas para separar seu código-fonte em diferentes domínios. Seu aplicativo será melhor fatiado e sua arquitetura clara. O que diz respeito ao mesmo tema estará no mesmo lugar. Você será muito mais eficiente à medida que esse hábito se tornar natural. Muitos projetos e bibliotecas de código aberto usam essa forma de fatiar suas fontes. Assim, você poderá navegar não apenas pelo seu próprio código, mas também pelo código de outras pessoas. Às vezes é extremamente complicado e demorado encontrar o nome certo para um elemento, seja uma classe, um arquivo, uma variável ou qualquer outra coisa. Esta etapa é, no entanto, particularmente importante e não deve ser negligenciada. Cuidado para não dizer para si mesmo: “Coloquei um nome que não necessariamente é muito claro, mas vou mudar depois”. As chances são de que você vai esquecer, e a dívida técnica será trazida sem que você perceba. Muitos projetos e bibliotecas de código aberto usam essa forma de fatiar suas fontes. Assim, você poderá navegar não apenas pelo seu próprio código, mas também pelo código de outras pessoas. Às vezes é extremamente complicado e demorado encontrar o nome certo para um elemento, seja uma classe, um arquivo, uma variável ou qualquer outra coisa. Esta etapa é, no entanto, particularmente importante e não deve ser negligenciada. Cuidado para não dizer para si mesmo: “Coloquei um nome que não necessariamente é muito claro, mas vou mudar depois”. As chances são de que você vai esquecer, e a dívida técnica será trazida sem que você perceba. Muitos projetos e bibliotecas de código aberto usam essa forma de fatiar suas fontes. Assim, você poderá navegar não

apenas pelo seu próprio código, mas também pelo código de outras pessoas. Às vezes é extremamente complicado e demorado encontrar o nome certo para um elemento, seja uma classe, um arquivo, uma variável ou qualquer outra coisa. Esta etapa é, no entanto, particularmente importante e não deve ser negligenciada. Cuidado para não dizer para si mesmo: “Coloquei um nome que não necessariamente é muito claro, mas vou mudar depois”. As chances são de que você vai esquecer, e a dívida técnica será trazida sem que você perceba. um arquivo, uma variável ou qualquer outra coisa. Esta etapa é, no entanto, particularmente importante e não deve ser negligenciada. Cuidado para não dizer para si mesmo: “Coloquei um nome que não necessariamente é muito claro, mas vou mudar depois”. As chances são de que você vai esquecer, e a dívida técnica será trazida sem que você perceba. um arquivo, uma variável ou qualquer outra coisa. Esta etapa é, no entanto, particularmente importante e não deve ser negligenciada. Cuidado para não dizer para si mesmo: “Coloquei um nome que não necessariamente é muito claro, mas vou mudar depois”. As chances são de que você vai esquecer, e a dívida técnica será trazida sem que você perceba.

Separando responsabilidades

Vejamos em que consiste a separação de responsabilidades no código, para torná-lo mais limpo, compreensível, sustentável e extensível. Este é o primeiro ponto dos princípios SOLID. No segundo capítulo, definimos assim o princípio da responsabilidade única: “Significa que uma classe em seu código deve responder a apenas uma tarefa”.

Como lembrete, SOLID é um conjunto de regras conhecidas de código limpo que, quando aplicadas juntas, tornarão seu código muito mais claro e preciso. Em vez de tentar seguir os cinco princípios descritos por cada um dos caps SOLID ao pé da letra, é mais importante ter uma ideia global de tudo isso em mente ao codificar.

O primeiro passo para respeitar isso é, na verdade... nomear, como acabamos de ver! De fato, ao nomear uma classe de maneira adequada, clara e, principalmente, precisa, você já está garantindo que ela não vire uma bagunça onde você coloca um pouco de tudo que possa imaginar. E é por isso mesmo que é necessário não nomear seus métodos com termos muito genéricos como Gerente e Serviço. Isso leva a um grande problema: se acabarmos com uma classe chamada EmailManager, obviamente todos teremos como primeiro pensamento adicionar todos os nossos próximos métodos que lidam com o gerenciamento de emails. E é assim que começa o caos. É por isso que preferimos criar classes como EmailFactory, AbstractEmailSender e assim por diante, para evitar absolutamente ter classes com centenas de métodos diferentes.

Começamos a entender melhor esse princípio de responsabilidade única. Vamos

repetir: o objetivo não é criar classes com um único método nelas. Não faz sentido. Você deve dividi-lo de forma inteligente. Não existe uma regra geral para divisão de classes. A maneira certa de dividir uma classe virá naturalmente com a experiência e virá por si só. Se isso ajudar, você pode ver as pastas como domínios e os arquivos como subdomínios. Usando os próximos exemplos, temos um domínio (ou pasta) que seria nomeado Email e subdomínios dedicados a tarefas específicas: criar um email, definir uma classe base para enviar um email com um provedor de email específico e assim por diante. Podemos ir ainda mais longe nessa separação de responsabilidades. De fato, existem ferramentas para nos ajudar a resolver esse problema sem esforço. Vamos descobrir (ou redescobrir!) o envio de eventos.

Despacho de eventos

O envio de eventos geralmente é implementado graças aos padrões de projeto Observer e Mediator, como é o caso do componente EventDispatcher do Symfony. Esta informação é apenas para conhecimento geral. De fato, os padrões de projeto podem parecer obscuros, até mesmo assustadores no começo. Além disso, explicá-los merece um livro próprio. Então, vamos vulgarizar tudo isso sem falar sobre padrões de projeto. Além disso, não vamos implementar um event dispatcher: trata-se de entender como ele pode nos ajudar.

Muito simplesmente, o princípio do envio de eventos é notificar todas as partes interessadas na mudança de estado de uma determinada entidade quando ela acontecer. As partes notificarão um mediador central dizendo: “Tenho interesse em saber quando esse evento específico ocorre; avise-me quando isso acontecer, porque tenho coisas para fazer se isso acontecer. O mediador reterá então esta informação. Ocorrendo tal evento, o mediador passará então pela lista de interessados e dirá: “O evento acabou de acontecer; faça o que você tem que fazer.” Indo um passo além, é até possível que os interessados declarem prioridade para ir antes de todos os outros, se necessário. Cuidado, estamos falando de eventos síncronos, ou seja, os interessados no evento irão executar um a um, seguindo os outros, e não em paralelo.

O envio de eventos é tão simples quanto isso. Mas então, como isso nos ajudará a reforçar nosso princípio de responsabilidade única? Vejamos um exemplo concreto: um usuário exclui sua conta de seu aplicativo. Você então precisa fazer duas tarefas, como segue:

Assim, criaremos naturalmente um serviço como o UserRemover, que executará

essas duas tarefas seguidas. Funciona muito bem. `UserRemover` é um nome explícito que define uma tarefa muito precisa. Nenhum problema até agora. Então, um dia, seu aplicativo ganha popularidade. Você deseja enviar um e-mail aos administradores para notificá-los de que o usuário saiu. Nossa classe `UserRemover` acaba excluindo dados e enviando dois e-mails, ambos com conteúdo e destinatários bem específicos.

Mais tarde, você deseja dar ao usuário a possibilidade de excluir sua conta para respeitar o Regulamento Geral de Proteção de Dados (GDPR) porque você mora na Europa. Somente você deseja ter estatísticas precisas e manter os dados anônimos do usuário para respeitar sua escolha de desaparecer de seu aplicativo, tendo a possibilidade de explorar seus dados anônimos para melhorar seu produto. Você então tem um serviço `UserRemover` que realmente não remove nada, envia e-mails e talvez muitas outras tarefas. Temos um problema real: a classe não faz o que deveria fazer e é provável que agora tenha milhares de linhas e dezenas de métodos.

As coisas teriam sido muito diferentes se você tivesse usado o envio de eventos desde o início. Aqui está um exemplo de uma resolução. Quando um usuário quiser sair de seu aplicativo, você despachará um evento chamado `UserRemovalRequestEvent`. Então, à medida que seu aplicativo cresce, você criará partes interessadas neste evento: event listeners. Teremos uma por tarefa, da seguinte forma:

E a anonimização? Nada mais simples: vamos criar um listener também para essa tarefa, e vamos “desligar” o listener para deletar os dados. Portanto, temos uma classe por tarefa. Cada classe tem sua responsabilidade exclusiva (enviar um e-mail para os administradores, anonimizar dados e assim por diante). Se no futuro você precisar adicionar uma tarefa, você criará uma nova classe (ou listener) com uma tarefa específica, sem nunca mexer nas outras classes. O código é então limpo e extremamente extensível. As classes permanecem bem nomeadas e concisas. Se uma tarefa estiver obsoleta, basta removê-la da lista de interessados no evento em questão. O princípio da responsabilidade única é

respeitado.

Se você quiser usar um distribuidor de eventos pronto para uso, recomendamos usar o pacote `symfony/event-dispatcher`. Este é exatamente o componente que é utilizado no framework para seu funcionamento. É muito robusto e eficiente e tem sido comprovado por vários anos.

Desmistificando o polimorfismo – interfaces e classes abstratas

No que diz respeito à separação de responsabilidades, o envio de eventos é um conceito que já está avançado. Você pode considerar que seu nível no mundo do código limpo aumentou consideravelmente se você conhece esse mecanismo, entende e tem a chance de usá-lo. Tudo isso obviamente requer um pouco de configuração. Ou você implementa esse sistema sozinho ou usa uma biblioteca externa. No segundo caso, há obviamente toda uma fase de aprendizado a ser incluída. De qualquer forma, esta obviamente não é a única maneira de melhorar a separação de suas responsabilidades. Existe uma maneira nativa do PHP de melhorar essa separação, às vezes não usada o suficiente, às vezes mal compreendida e muitas vezes subestimada. Estamos falando aqui de polimorfismo, ou vulgarmente: classes abstratas e interfaces.

Primeiro, por que a palavra polimorfismo? Poly vem do grego que significa muitos, e morphism significa forma/forma. Classes e interfaces abstratas são apenas uma maneira de implementar o polimorfismo no código e na programação orientada a objetos (OOP). Para simplificar, vamos pegar apenas o caso das interfaces.

Interfaces

As interfaces definem uma forma comum para as classes que as implementam posteriormente. Eles determinam os métodos que cada implementação deve definir para o seu caso. Uma implementação deve necessariamente definir todos os métodos de sua(s) interface(s). Muitas vezes é por isso que ouvimos a seguinte afirmação: “uma interface é um contrato”. Podemos ver desta forma: se você implementa uma interface, você se compromete a implementar os métodos que ela define. Você não tem outra alternativa.

É aqui que ocorre todo o poder do polimorfismo. Em seu código, você pode dizer ao PHP que um argumento fulano de um método será necessariamente uma instância de um objeto implementando uma interface precisa. Você pode manipular os diferentes métodos desta interface e chamá-los. Você não precisa se preocupar em como será implementado quando for usado: não estamos interessados nisso. Como exemplo, vale mais que mil palavras, vamos pegar nosso sistema de correio.

Você tem MailerInterface que define apenas um método: um método para enviar um e-mail. Poderíamos chamá-lo de sendEmail. Como antes, quando um usuário é excluído em seu aplicativo, o ouvinte de evento para enviar um e-mail de despedida é chamado. Neste caso, o que lhe interessa é que o e-mail seja simplesmente enviado, não o funcionamento interno do envio. A propósito, esses funcionamentos internos não poderiam ser diferentes dependendo de certas condições? Você não precisa procurar muito por um exemplo: seu provedor de e-mail principal pode estar fora do ar, mas você precisa enviar sua mensagem. Em seguida, você deve usar outro provedor de e-mail, com uma interface de programação de aplicativo (API) diferente, opções diferentes e assim por diante. Sem o polimorfismo, as coisas podem ficar extremamente complicadas muito rapidamente.

A solução é criar duas implementações de MailerInterface, cada uma definindo o método `sendEmail`, dependendo do provedor de e-mail utilizado. Mas o resultado é o mesmo: o e-mail é enviado. Quando um usuário exclui sua conta, você executa uma verificação para garantir que seu provedor de e-mail principal esteja ativo e instanciar sua implementação. Se estiver inativo, você instancia a implementação do provedor de e-mail de backup. Por outro lado, no listener do evento de envio de e-mail, basta você continuar chamando o método `sendEmail` definido em MailerInterface, sem se preocupar com o resto. O código é limpo, claro; as responsabilidades são separadas; você economiza tempo. E, além disso, tornou-se resiliente ao fracasso.

Você poderia fazer isso com 10, 15 ou 20 provedores de e-mail, se quisesse. A vantagem é que, se a API de um dos provedores mudar ou você encontrar um bug em sua implementação, basta tocar na implementação problemática. Todos os outros não se movem, como é o caso das chamadas feitas à interface. Você reduz consideravelmente o risco de bugs e seu código é muito mais testável: você pode escrever testes especificamente para cada implementação. Isso é muito mais robusto do que testes generalizados e intermináveis que tentam avaliar todos os casos possíveis! O tempo economizado é excepcional e inestimável.

aulas abstratas

E como as classes abstratas se encaixam nisso tudo? Podemos vê-los como uma camada intermediária entre uma interface e suas implementações. Embora obviamente não seja obrigatório que uma classe abstrata implemente uma interface, muitas vezes é uma boa ideia criar uma interface sobre uma classe abstrata. De fato, estes últimos são mais permissivos que as interfaces: você pode definir parcialmente os métodos que são declarados, declarar atributos e decidir sobre a visibilidade dos métodos e atributos da classe (onde as interfaces permitem apenas visibilidade pública e não privada ou protegida). Com uma interface, você tem um contrato limpo, sem nenhuma informação que estaria aqui “por via das dúvidas” e com apenas o essencial se você respeitar o “I” dos princípios SOLID – a segregação das interfaces. Como um lembrete e simplesmente colocar,

As classes abstratas permitem definir comportamentos comuns às classes que as estendem e que desejam aproveitar o poder do polimorfismo. Em particular, isso evita redundância de código, fontes de bugs e copiar e colar sem fim. De fato, em nosso exemplo anterior, é muito provável que as diferentes implementações de MailerInterface tenham comportamentos comuns, como a criação do cliente HTTP que se comunica com a API do provedor de e-mail ou a criação de um objeto comum Message usado na interface interna funcionamento das implementações.

Neste caso, declararíamos AbstractMailer implementando MailerInterface e definimos comportamentos comuns das diferentes implementações. Então, as diferentes implementações estenderiam o AbstractMailer para aproveitar o comportamento comum que você acabou de definir.

Tenha cuidado - isso não significa que seja necessário criar interfaces e classes abstratas em todos os lugares, o tempo todo e em todos os casos. Não devemos negligenciar o impacto que isso tem na complexidade do código em comparação com uma única classe. Além disso, só porque você não criou uma interface para um caso, não significa que ela seja imutável e imutável. Muitas vezes, nos encontramos refatorando nosso código e criando interfaces e classes abstratas e adaptando classes existentes para implementá-las e estendê-las. Como vimos, precisamos inicialmente manter o código simples (respeitando os princípios YAGNI e KISS). Não podemos prever o futuro, as restrições de negócios evoluem.

Se no momento de criar uma classe não houver nada que sugira que serão necessárias diferentes implementações, isso não é uma preocupação. Este é um trabalho que será feito mais tarde. Por outro lado, se durante o desenvolvimento você se pegar copiando código de um lado para o outro e perceber uma forte redundância, será um excelente reflexo pensar em polimorfismo.

Resumo

Acabamos de cobrir a parte mais avançada da seção teórica deste livro. Agora estamos armados com o conhecimento para cortar nosso código de forma limpa, mantendo-o sustentável e extensível para futuros desenvolvedores. Ele também estará pronto para o futuro por ser fortemente aberto à extensão e fechado à modificação (conforme descrito em um dos princípios SOLID).

Revisamos muitos dos casos que você pode encontrar em relação à nomenclatura de arquivos, classes e métodos ao desenvolver um aplicativo PHP. Além disso, vimos que as pastas devem ter nomes específicos e podem ser usadas para dividir seu aplicativo em diferentes domínios.

A separação de responsabilidades também foi um grande tópico. É particularmente importante entender por que essa separação é útil, até mesmo vital, em um projeto. É a verdadeira chave para um projeto bem arquitetado e fácil de navegar. O envio de eventos é uma excelente maneira de conseguir isso, como vimos. O envio de eventos é um dos pilares de alguns projetos críticos da web, como o framework Symfony, para citar apenas um. Este depende muito desse mecanismo, tornando-o uma ferramenta conhecida por sua robustez, eficiência e, principalmente, flexibilidade. Isso também se deve ao polimorfismo e às diferentes interfaces declaradas dentro dele. Você pode redeclarar praticamente qualquer parte da estrutura graças a isso e adaptar tudo às suas necessidades mais avançadas.

Nem sempre é fácil entender quando criar uma interface ou uma classe abstrata. Isso vem com prática e experiência. Em breve, parecerá natural. Em caso de

dúvida, converse com um de seus colegas!

Terminaremos a parte teórica deste livro com uma parte mais leve no próximo capítulo, tratando das novas funcionalidades do PHP. Isso nos torna desenvolvedores mais rigorosos e melhores, especialmente nos últimos anos.

O PHP está evoluindo – depreciações e revoluções

Fizemos bem. A comunidade PHP se saiu bem; nós tivemos sorte. De fato, o PHP vem evoluindo fortemente há alguns anos. Mas essa forte evolução nem sempre existiu. Isso ocorreu principalmente devido a problemas durante o desenvolvimento do PHP 6, razão pela qual esta versão nunca foi lançada. Isso explica porque tantos projetos estavam (e ainda estão) presos no PHP 5.

O PHP 7 limpou a lousa e trouxe um verdadeiro renascimento para a linguagem. Além disso, é uma verdadeira lufada de ar fresco que impulsionou a língua para novos horizontes.

O PHP deixou de ser uma linguagem quase morta para ser uma linguagem em recuperação e se projetando no futuro. Neste último capítulo, dedicado à teoria do código limpo, vamos nos concentrar nos seguintes pontos:

PHP antigo versus novo

O PHP provavelmente ajudou você a se tornar um desenvolvedor muito mais rigoroso ao longo dos anos. Se nas suas primeiras décadas de existência o PHP permitia escrever código à sua maneira e sem o impedir de o fazer, com as (pouquíssimas) vantagens que isso traz, em retrospecto, foi sobretudo a oportunidade de ter tantas maneiras de escrever código, pois existem desenvolvedores (que raramente levam a resultados excepcionais) que o tornaram popular. Como sabemos agora, isso pode ser uma fonte de bugs intermináveis e infernais para depurar. Felizmente, a evolução da linguagem nos últimos anos corrigiu muitos desses bugs, em benefício de nossos aplicativos.

Digitação estrita

Primeiro, vamos dar uma olhada em uma das coisas mais importantes que você deve usar nas versões mais recentes do PHP a partir da versão 7.4 – a tipagem estrita de propriedades.

Houve um tempo em que você tinha permissão para passar quaisquer dados para qualquer variável e lançar variáveis quantas quisesse, sem uma maneira real e nativa de evitar isso - convertendo uma variável de array em uma string, uma string em um inteiro e assim por diante . Isso pode ser bastante confuso e potencialmente a fonte de muitos problemas. O que aconteceria se você multiplicasse uma string por um inteiro, por exemplo? Bem, o resultado é totalmente inesperado. Isso não é bem gerenciado como no Python, onde esses tipos de operações são permitidos, mas bem controlados. Se o seu código PHP depende da capacidade de digitação fraca, pode haver um problema com a arquitetura do código e você deve revisar absolutamente as partes que precisam dessa digitação fraca.

O PHP agora permite que você digite estritamente variáveis em algumas situações. A partir do PHP 8.1, você não pode digitar uma variável se ela não for um atributo de classe ou um argumento de método. Dito isso, você deve digitar todos os atributos de suas classes e argumentos de método – menos confusão, mais rigor. Você pode ter que repensar algumas partes do seu código, mas obtém o benefício de um código mais limpo e compreensível. Não há surpresas em tempo de execução por causa de um elenco inesperado. Se você realmente precisa passar qualquer tipo de dado para seus métodos, ainda pode contar com a palavra-chave mista, que informa ao PHP que essa variável pode ser qualquer tipo de dado ou que o método pode retornar qualquer tipo de dado. Claro, você deve evitar isso se puder e só usá-lo em casos muito precisos (como definições de métodos de interface, onde as implementações da interface podem retornar

vários tipos de dados).

Relatório de erros

O PHP 8 agora exhibe depreciações e erros mais rígidos por padrão. O nível de relatório de erros era menor em versões anteriores do PHP. Com essa mudança, você poderá ver com muito mais facilidade onde precisa prestar atenção à depreciação, por exemplo. Você ficará grato por esta mudança quando tiver que operar uma atualização de versão do PHP, por exemplo. Se você cuidar de corrigir as descontinuações assim que elas aparecerem, atualizar para outra versão do PHP será uma tarefa fácil. Além disso, quase sempre há uma mensagem resultante da descontinuação, informando como corrigi-la com precisão. Cuidar desses erros e especialmente das depreciações na primeira oportunidade é definitivamente uma jogada inteligente e faz com que você adote uma mentalidade de código limpo.

Atributos

Os comentários no código-fonte foram inventados para fornecer uma melhor compreensão das partes complicadas do código. Isso significa que, se removermos todos os comentários no código-fonte, ele também funcionará perfeitamente, pois compiladores e interpretadores não devem considerar comentários. Essa é a principal e primeira razão pela qual os comentários de código-fonte foram inventados.

Em seguida, as anotações foram criadas. A lógica e os mecanismos foram introduzidos nas seções de comentários. Não me interpretem mal – as anotações são muito práticas. Você obtém todas as informações e metadados sobre um elemento onde e quando precisar. Mas é de alguma forma uma aberração quando você pensa sobre isso. E lembre-se do que dissemos nos capítulos anteriores: se você escrever código de forma limpa, há uma grande chance de nunca precisar escrever uma única linha de comentário ou, no máximo, apenas algumas (escrever partes complicadas de código nem sempre pode ser evitada, mesmo para os melhores codificadores limpos).

Os atributos fazem parte da linguagem PHP desde a versão 8.0. Simplificando, os atributos têm o mesmo papel que as anotações: adicionar metadados a diferentes elementos, incluindo classes, propriedades e métodos. A diferença é que eles usam outra sintaxe, que não é o tipo usado para comentários. Mais do que ter melhor legibilidade, os comentários voltarão ao seu primeiro uso: ser informativo. Em uma instância, você pode diferenciar imediatamente metadados (descritos com atributos) e comentários para entender melhor a parte do código em que está trabalhando.

Vamos ver como são os atributos:

```
<?php
```

```
namespace App\Controller;
```

```
class ExampleController
```

```
{
```

```
public function home([CurrentUser] User $user)
```

```
{
```

```
// ...
```

```
}
```

```
}
```

É claro que o usuário atualmente conectado provavelmente será injetado na variável \$user. O código parece nítido e temos todas as informações necessárias para entender rapidamente o que está acontecendo. Também temos um espaço perfeitamente em branco se precisarmos adicionar informações na seção de comentários acima do método home(). Agora você pode ver claramente como os atributos o ajudam a ser mais rigoroso – remova todos os blocos de comentários e pense duas vezes antes de adicionar um novo.

Obviamente ainda há muitas coisas acontecendo nas versões mais recentes do PHP, a partir da versão 8.0. Aqui está uma lista não exaustiva:

Vamos ver alguns deles na próxima seção.

A revolução da versão 8

Como vimos, o PHP experimentou um impulso excepcional em sua evolução nos últimos anos. Enquanto pensávamos que a versão 7 era um verdadeiro renascimento da linguagem, a versão 8 provou que era apenas o começo. Aqui estão os principais novos recursos que irão ajudá-lo a escrever um código claro e conciso, e que irão ajudá-lo a levar ainda mais longe os princípios do código limpo que vimos ao longo destes capítulos.

Sintaxe de correspondência

A sintaxe de correspondência é a versão condensada do switch/case clássico. Não deve ser usado em todos os lugares porque pode rapidamente se tornar ilegível. No entanto, se você escolher os locais onde usá-lo com moderação, seu código pode ficar muito mais claro em um instante. Aqui está um exemplo da sintaxe de correspondência:

```
$foo = match($var) {  
  
    <valor 1> => Barra::meumétodo1(),  
  
    <valor 2> => Barra::meumétodo2(),  
  
};
```

Funciona da mesma forma que o interruptor. No entanto, observe a diferença no comprimento do código e como a legibilidade é aumentada. Você também pode ver imediatamente as limitações de tal sintaxe: se houver mais de uma instrução para executar por caso, ela não é adaptável e você corre o risco de acabar com algo ilegível. É então preferível manter o bloco switch/case mais clássico.

Argumentos nomeados

Se você está acostumado a usar outras linguagens regularmente, pode estar familiarizado com essa evolução do PHP. Desde a versão 8.0, é possível passar argumentos para métodos na ordem que você quiser. Isso é feito especificando o nome do argumento logo antes de seu valor.

```
$this->myMethodCall(agulha: 'Barra', habilitada: true);
```

Podemos ver rapidamente dois casos em que isso pode ser útil:

Classes e propriedades somente leitura

Aqui está um pequeno enigma para introduzir classes e propriedades somente leitura. Em uma classe PHP, como você pode garantir que um atributo será atribuído apenas uma vez – ou seja, um valor pode ser atribuído a ele apenas uma vez e todas as tentativas futuras de atribuí-lo falharão? Observe que isso também se aplica a atribuições dentro da própria classe. Isso significa que um modificador lançando uma exceção no caso de uma chamada não pode funcionar porque não podemos ter 100% de certeza de que o desenvolvedor passará pelo modificador dentro do escopo da classe.

Na verdade, a resposta é bem simples: é impossível. Se você estiver usando PHP 8.0 ou anterior, simplesmente não é possível. O PHP 8.1 nos traz uma solução nativa para resolver esse problema: propriedades somente leitura. Ao declarar uma propriedade de classe com a palavra-chave `readonly`, a variável só pode ser atribuída uma vez, independentemente do contexto. Isso pode ser especialmente útil ao definir e usar DTOs e VOs. Ao restringir o acesso à mutação dessas propriedades, o desenvolvedor que as utilizar terá que pensar bem no uso do objeto. Se quiserem fazer alterações, terão que criar um novo objeto. O objeto original não pode ser modificado, o que pode garantir maior estabilidade e robustez do código. Aqui está como as propriedades somente leitura são declaradas:

```
<?php
```

```
namespace App\Modelo;
```

```
classe MeuValueObject
```

```
{
```

```
cadeia somente leitura protegida $foo;
```

```
função pública __construct(string $foo)
```

```
{
```

```
$this->foo = $foo; // Primeira tarefa, tudo bem
```

```
// Qualquer outra atribuição de $this->foo resultará
```

```
em um erro fatal
```

```
}
```

```
}
```

Desde o PHP 8.2, é possível declarar uma classe inteira como somente leitura colocando este modificador logo antes da palavra-chave `class`, para que você não precise usar a palavra-chave em todas as propriedades que a classe inclui. Isso também lhe dá uma grande vantagem: uma classe declarada como `readonly` não poderá ter novas propriedades declaradas dinamicamente. Embora esse comportamento esteja obsoleto e deva ser evitado a todo custo (a declaração dinâmica de propriedades de classe é um pesadelo em termos de depuração, estabilidade e complexidade do código), ainda é possível fazê-lo em versões até o PHP 9.0, nas quais um erro fatal será acionado se esse comportamento ocorrer. Se você não estiver usando o PHP 9.0, declarar sua classe como somente leitura irá protegê-lo desse comportamento.

Este é um grande passo para desenvolvedores PHP. De fato, tudo graças a esse recurso, precisaremos ser, mais uma vez, cada vez mais rigorosos na forma como interagimos com os objetos e suas propriedades.

Migrando recursos para classes apropriadas

Dependendo de quanto tempo você desenvolve em PHP, você deve estar mais ou menos familiarizado com o que chamamos de recursos. Um recurso é um tipo especial de variável, que representa uma referência a um recurso externo. Isso pode soar um pouco vago, mas na verdade é bastante trivial. Um recurso pode ser o seguinte, por exemplo:

Isso funcionou bem por algumas décadas, mas entendemos que o termo recurso não é mais adaptado por ser muito generalista e especialmente antiquado agora que classes e objetos são predominantes no código moderno. Por que esses recursos não são simplesmente objetos como qualquer outro? Bem, não há nenhuma razão particular (pelo menos não no momento) para justificar isso. E é por isso que o PHP 8.0 iniciou uma migração longa, mas necessária, para converter os recursos antigos em classes completas. Faz muito mais sentido.

A utilização dos recursos era complexa. Eles são complicados de depurar e entender como funcionam e seu estado interno. Eles só podem ser chamados por funções especiais que lidam com recursos. Este é um grande avanço para os desenvolvedores que terão mais controle e mais ferramentas para melhorar o rigor do desenvolvimento e a robustez de seu código.

A alteração é feita à medida que novas versões do PHP são lançadas. Por exemplo, o PHP 8.0 incorpora a migração de recursos como os seguintes:

Por outro lado, o PHP 8.1 incorpora a migração dos seguintes recursos:

As seguintes versões do PHP continuam a fazer o mesmo – para eventualmente remover o uso de recursos na biblioteca padrão do PHP. Além disso, os principais desenvolvedores do PHP decidiram desde o PHP 8.1 criar algumas dessas classes em namespaces. Podemos ver que a linguagem, com esse tipo de ação e desenvolvimento, está se recuperando depois de um longo atraso e fazendo todos os esforços para restaurar sua reputação. O PHP mostra isso claramente: a linguagem veio para ficar.

Protegendo seus argumentos sensíveis contra vazamentos

Se você desenvolve com PHP há algum tempo, certamente conhece as várias maneiras de exibir o conteúdo de uma variável ou um argumento de função. Você pode pensar em `var_dump` e `print_r` em particular. Há também outras ocasiões em que os argumentos e seus valores podem ser exibidos: quando o rastreamento de pilha (ou pilha de chamada) é exibido. Isso acontece por uma chamada manual para um método como `debug_print_backtrace`, mas também e frequentemente quando uma exceção é lançada. Em qualquer caso, se informações confidenciais estiverem contidas em algum lugar nas variáveis ou na pilha de chamadas com os argumentos das chamadas de método, isso pode ser problemático. Você pode pensar que isso só acontece em um ambiente de desenvolvimento, mas isso é um erro. É muito provável que você escreva suas mensagens de exceção em um log de erros em algum lugar do(s) seu(s) servidor(es). Isso pode resultar na exibição de informações confidenciais em seus logs. Isso obviamente não é recomendado. Informações confidenciais não devem ser escritas em texto não criptografado em nenhum lugar. Além disso, e embora seja um erro, a segurança dos logs do aplicativo muitas vezes não é tão boa quanto a de um banco de dados, por exemplo. Além disso, há uma grande chance de muitos desenvolvedores do projeto (se não todos) terem acesso a esses logs para depurar o aplicativo. A ameaça nem sempre vem de fora. há uma grande chance de muitos desenvolvedores do projeto (se não todos) terem acesso a esses logs para depurar o aplicativo. A ameaça nem sempre vem de fora. há uma grande chance de muitos desenvolvedores do projeto (se não todos) terem acesso a esses logs para depurar o aplicativo. A ameaça nem sempre vem de fora.

Felizmente, o PHP 8.2 inclui um novo atributo para solucionar esse problema. É realmente possível indicar o atributo `#[SensitiveParameter]` antes de qualquer argumento de função. Isso dirá ao PHP para não exibir o valor do parâmetro em `var_dump`, um rastreamento de pilha e assim por diante. Colocado de forma

inteligente, você terá a certeza de não vaziar um valor sensível em uma mensagem de erro, por exemplo. De fato, não é incomum que os sites exibam o erro do servidor diretamente no front-end. Obviamente, isso deve ser banido o mais rápido possível, mas pelo menos ajuda a limitar os danos. Vamos ver como esse novo atributo pode ser usado:

```
<?php
```

```
namespace App\Controller;
```

```
class SecurityController
```

```
{
```

```
    função pública autenticar(string $nome de usuário,
```

```
    #[SensitiveParameter] string $senha)
```

```
{
```

```
    // Caso ocorra alguma exceção ou var_dump
```


sendo chamado aqui, o valor de \$password será

ser escondido nas diferentes saídas

}

}

No funcionamento interno do PHP, este atributo substituirá o argumento por um objeto do tipo SensitiveParameterValue, que ocultará o valor real do argumento. O argumento será exibido e presente na saída, mas seu valor ficará oculto. Adicionar esse atributo aos seus argumentos de método confidenciais é uma maneira inteligente e bem-vinda de adicionar rigor ao seu código e torná-lo mais resistente a ataques.

Resumo

Nunca é demais repetir: o PHP está evoluindo da maneira mais bonita e alcançando seus concorrentes no mundo da web. A linguagem ouve a comunidade e os desenvolvedores, oferecendo-lhes as ferramentas necessárias para responder aos problemas modernos da maneira mais viável possível.

Percorremos um longo caminho desde uma linguagem que permitia tudo e era muito (muito) negligente para os desafios dos aplicativos da web de hoje. Apesar da explosão de frameworks frontend e tecnologias voltadas para a substituição de linguagens de servidor por linguagens destinadas ao frontend (como Node.js com JavaScript), o PHP não tem do que se envergonhar. Seu desempenho impressionante, sua velocidade de evolução e a reputação que construiu ao longo dos anos mostram que ainda tem um futuro brilhante pela frente.

Embora o código limpo seja, como vimos, um estado de espírito e, de certa forma, uma filosofia, as soluções nativas da linguagem estão chegando aos montes para nos ajudar a aplicá-las da melhor maneira possível. Melhor ainda, esses recursos trazidos para o PHP nos permitem ver novas possibilidades que não necessariamente teríamos pensado inicialmente para tornar nosso código robusto, sustentável e viável a longo prazo. Pense apenas em argumentos nomeados, classes e propriedades somente leitura, tipagem estrita ou simplesmente no último tópico abordado neste capítulo: proteger argumentos confidenciais contra vazamentos em logs de aplicativos e mensagens de exceção.

Dito isto, é hora de começar a trabalhar. Começaremos examinando as ferramentas que fornecerão uma visão geral numérica muito rápida da qualidade

do seu código. Ter métricas para esse tipo de coisa permite que você veja quais melhorias você fará em seu código ou se é realmente hora de agir porque a qualidade está diminuindo à medida que você desenvolve. Então, vamos mergulhar no próximo capítulo, que destaca ferramentas de qualidade de código dedicadas à linguagem PHP.

Parte 2 – Mantendo a qualidade do código

O objetivo da segunda parte é permitir que você melhore constantemente seus projetos e, eventualmente, mantenha um nível consistentemente alto de qualidade de código. Ele fornecerá orientações sobre o uso de ferramentas e tecnologias de ponta, o que ajudará a reduzir os esforços necessários para atingir esse objetivo. Por fim, apresentaremos várias práticas recomendadas que ajudarão você a trabalhar em conjunto com outros desenvolvedores em uma base de código limpa e sustentável.

Esta seção compreende os seguintes capítulos:

Ferramentas de qualidade de código

Nas partes anteriores deste livro, aprendemos os fundamentos do código limpo. Agora, é hora de aplicar esse conhecimento ao nosso trabalho diário. Existem literalmente dezenas de ferramentas disponíveis para o ecossistema PHP que podem nos ajudar a detectar falhas e possíveis bugs, aplicar o estilo de código correto e, em geral, nos informar sobre problemas de qualidade.

Para garantir um início rápido e fácil no mundo das ferramentas de qualidade de código, esta seção apresentará as mais usadas. Para cada um, você aprenderá como instalar, configurar e usar diretamente em seu código. Você também aprenderá sobre alguns recursos extras úteis que eles fornecem.

Veremos os seguintes grupos de ferramentas:

Requerimentos técnicos

Para este capítulo, você só precisa de um mínimo de ferramentas já configuradas. As chances são altas de que você já os tenha instalado se você já trabalhou com código PHP antes:

Observe que, no restante deste livro, todos os exemplos são baseados em um ambiente Linux, como Ubuntu ou macOS. Se você estiver usando o Windows para desenvolvimento, provavelmente precisará fazer alguns ajustes, conforme descrito aqui: <https://www.php.net/manual/en/install.windows.commandline.php>.

Os arquivos de código para este capítulo podem ser encontrados aqui: <https://github.com/PacktPublishing/Clean-Code-in-PHP>

Verificação de sintaxe e estilo de código

O primeiro grupo de ferramentas que queremos discutir nos ajuda a manter nosso código sintaticamente correto (ou seja, pode ser executado corretamente pelo PHP) e formatado de forma estruturada. Parece óbvio que o código precisa ser escrito sem erros, mas é sempre bom verificar novamente, pois algumas ferramentas podem alterar ativamente seu código. Ter uma maneira simples e rápida de garantir isso será essencial quando automatizarmos todo o processo de qualidade do código posteriormente neste livro.

Ter seu código formatado seguindo um guia de estilo comum reduz o esforço necessário para ler e entender seu código, bem como o código de outras pessoas. Especialmente quando você está trabalhando em equipe, um guia de estilo aceito economiza horas de discussões sobre como formatar o código corretamente.

Aprenderemos sobre as seguintes ferramentas:

O linter integrado do PHP

A primeira ferramenta que queremos ver não é, na verdade, uma ferramenta de qualidade de código própria, mas sim uma opção incorporada ao próprio binário do PHP: o Linter. Ele verifica qualquer código em busca de erros de sintaxe sem executá-lo. Isso é particularmente útil para garantir que o código funcione após sessões de refatoração ou quando seu código foi alterado por uma ferramenta externa.

Instalação e uso

Como o Linter já faz parte da instalação do PHP, podemos começar a usá-lo imediatamente observando um exemplo. Se você olhar com atenção, provavelmente notará o erro que o autor cometeu no seguinte exemplo de classe:

```
<?php  
  
exemplo de classe  
  
{  
  
public function doSomething() bool  
  
{  
  
retornar verdadeiro;  
  
}  
  
}
```

Não se preocupe se você não identificar o erro imediatamente – é exatamente para isso que o Linter existe! Basta passar o nome completo e o caminho do arquivo a ser verificado para o binário do PHP, usando a opção `-l`. Ao adicionar a opção `-f`, o PHP também verificará erros fatais, que é algo que queremos. Ambas as opções podem ser combinadas em `-lf`.

Vamos supor que a classe anterior pode ser encontrada no arquivo `example.php` na pasta atual – então, tudo o que precisamos digitar é o seguinte:

```
$ php -lf exemplo.php
```

Teremos a seguinte saída:

```
PHP Parse error: erro de sintaxe, identificador inesperado
```

```
"bool", esperando ";" ou "{" em example.php na linha 5
```

```
Erros ao analisar exemplo.php
```

Você pode dizer ao linter para verificar um diretório completo também:

```
$ php -lf src/*
```

Observação

O linter embutido do PHP para no primeiro erro - como em, ele não fornecerá uma lista completa de todos os erros detectados. Portanto, é melhor executar o comando novamente após resolver o problema.

Uma recapitulação do Linter embutido no PHP

O PHP linter integrado é uma ferramenta útil para verificações rápidas de código, mas não pode fazer muito mais do que isso. Existem outros linters mais sofisticados como <https://github.com/overtrue/phplint>. Este não apenas retornará uma lista completa de erros, mas também poderá executar vários processos em paralelo, o que será visivelmente mais rápido em grandes bases de código. No entanto, outras ferramentas de qualidade de código já incluem um linter, como a ferramenta que veremos na próxima seção.

PHP CS Fixer: um sniffer de código

Outra ferramenta essencial é um sniffer de código. Ele verifica o código PHP em busca de violações de padrões de codificação e outras más práticas. PHP CS Fixador (<https://github.com/FriendsOfPHP/PHP-CS-Fixer>) é uma escolha viável para começar, pois, como o nome já indica, ele não apenas relata as descobertas, mas também as corrige imediatamente.

Outros farejadores de código

PHP CS Fixador não é o único sniffer de código disponível. Outro bem conhecido é o PHP_CodeSniffer (https://github.com/squizlabs/PHP_CodeSniffer), que podemos recomendar totalmente o uso também.

Instalação e uso

Usando o Composer, a instalação é direta:

```
$ composer require friendsofphp/php-cs-fixer --dev
```

Alternativas ao Composer

Existem várias maneiras de instalar as ferramentas que apresentaremos neste livro. Também verificaremos mais opções mais adiante neste livro.

O caso de uso típico para farejadores de código é cuidar da colocação de colchetes e do número de recuos, sejam eles espaços em branco ou tabulações. Vamos verificar o seguinte arquivo PHP com seu formato feio:

```
<?php
```

exemplo de classe

```
{
```

```
public function doSomething(): bool { return true; }
```

```
}
```

Se executarmos o code sniffer com suas configurações padrão, o comando é bom e curto:

```
$ vendor/bin/php-cs-fixer fix example.php
```

Isso verificará e corrigirá o arquivo example.php de uma só vez, deixando nosso código limpo e brilhante:

```
<?php
```

exemplo de classe

```
{
```

```
public function doSomething(): bool
```

```
{
```

```
    retornar verdadeiro;
```

```
}
```

```
}
```

Se você não deseja corrigir o arquivo imediatamente, pode usar a opção `--dry-run` para apenas verificar se há problemas. Adicione também a opção `-v` para exibir as descobertas:

```
$ vendor/bin/php-cs-fixer fix example.php --dry-run -v
```

Como acontece com todas as ferramentas de qualidade de código, você também pode executá-lo em todos os arquivos de uma pasta. O comando a seguir verificará a pasta src recursivamente, para que todas as subpastas também sejam verificadas:

```
$ vendor/bin/php-cs-fixer fix src
```


Regras e conjuntos de regras

Até agora, usamos o PHP CS Fixer com suas configurações padrão. Antes de podermos alterar esses padrões, vamos dar uma olhada em como ele sabe o que verificar e corrigir.

Um padrão comum nas ferramentas de qualidade de código é a organização de regras em conjuntos de regras. Uma regra é uma instrução simples que informa ao PHP CS Fixer como nosso código deve ser formatado em relação a um determinado aspecto. Por exemplo, se quisermos usar tipos estritos em PHP, todo arquivo PHP deve conter o `declare(strict_types=1);` instrução.

Existe uma regra no PHP CS Fixer que pode ser usada para forçar isso:

```
$ vendor/bin/php-cs-fixer fix src
```

```
--rules=declare_strict_types
```

Este comando verificará cada arquivo em `src` e adicionará `declare(strict_types=1);` após a tag PHP de abertura.

Uma vez que um padrão de codificação como o PSR-12 (<https://www.php-fig.org/psr/psr-12/>) incluir muitas instruções sobre como o código deve ser

formatado, seria complicado adicionar todas essas regras ao comando anterior. É por isso que foram introduzidos conjuntos de regras, que são simplesmente uma combinação de regras e até mesmo outros conjuntos de regras.

Se quisermos formatar o código seguindo PSR-12 explicitamente, podemos apenas executar isto:

```
$ vendor/bin/php-cs-fixer fix src --rules=@PSR12
```

Como você pode ver, um conjunto de regras é indicado pelo símbolo @.

Regras e documentação do conjunto de regras

É impossível discutir todas as regras e conjuntos de regras do PHP CS Fixer dentro do escopo deste livro. Se você está curioso sobre o que mais ele tem a oferecer, confira o repositório oficial do
GitHub:<https://github.com/FriendsOfPHP/PHP-CS-Fixer/tree/master/doc>

Configuração

Executar comandos manualmente é bom para começar, mas em algum momento, não vamos querer lembrar de todas as opções todas as vezes. É aí que entram os arquivos de configuração: a maioria das ferramentas de qualidade de código PHP permite armazenar a configuração desejada em um ou mais arquivos e em vários formatos, como YAML, XML ou PHP simples.

Para PHP CS Fixer, todas as configurações relevantes podem ser controladas através do arquivo de configuração `.php-cs-fixer.dist.php`. Aqui, você encontrará um exemplo:

```
<?php

$finder = PhpCsFixer\Finder::create()
->em(__DIR__)
->exclude('modelos');

$config = new PhpCsFixer\Config();
return $config->setRules([
    '@PSR12' => verdadeiro,
    'declare_strict_types' => verdadeiro,
    'array_syntax' => ['syntaxe' => 'curto'],
]);
```

```
->setFinder($finder);
```

Várias coisas estão acontecendo aqui. Primeiramente, é criada uma instância de `PhpCsFixer\Finder`, que é configurada para utilizar o mesmo diretório para procurar os arquivos PHP onde este arquivo de configuração está localizado. Como a pasta raiz do aplicativo geralmente está localizada aqui, podemos excluir certos subdiretórios (como modelos neste exemplo) da verificação.

Em segundo lugar, uma instância de `PhpCsFixer\Config` é criada. Aqui, informamos ao PHP CS Fixer quais regras e conjuntos de regras aplicar. Já discutimos o conjunto de regras `@PSR-12`, bem como a regra `declare_strict_types`. A regra `array_syntax` força o uso da sintaxe short array.

Você deve ter notado que o nome do arquivo de configuração, `.php-cs-fixer.dist.php`, contém a abreviação `dist`. Isso significa distribuição e geralmente indica que esse arquivo é aquele com o qual o projeto é distribuído. Em outras palavras, este é o arquivo que é adicionado ao repositório Git e fica imediatamente disponível após o checkout.

Se quiser usar sua própria configuração em seu sistema local, você pode criar uma cópia dela e renomeá-la como `.php-cs-fixer.php`. Se este arquivo existir, o PHP CS Fixer irá usá-lo em vez do `dist-file`. É uma boa prática deixar o Git ignorar este arquivo. Caso contrário, você pode adicionar acidentalmente suas configurações locais ao repositório.

Uso avançado

A capacidade do PHP CS Fixer não para na correção automática de violações do padrão de codificação. Também pode ser usado para aplicar pequenas tarefas de refatoração. Um ótimo caso de uso, por exemplo, é a migração automatizada para uma versão superior do PHP: o PHP CS Fixer vem com conjuntos de regras de migração, que podem introduzir alguns novos recursos de linguagem em sua base de código.

Por exemplo, com o PHP 8.0, é possível usar a palavra-chave `class` em vez da função `get_class()`. O PHP CS Fixer pode escanear seu código e substituir certas linhas – por exemplo, veja o seguinte:

```
$classe = get_class($someClass);
```

Ele pode substituir a linha anterior por esta:

```
$classe = $algumaClasse::classe;
```

Os conjuntos de regras de migração são separados em sem risco e com risco. Conjuntos de regras arriscados podem potencialmente causar efeitos colaterais, enquanto os não arriscados geralmente não causam problemas. Um bom exemplo de mudança arriscada é a regra `declare_strict_types` que discutimos anteriormente. Certifique-se de testar seu aplicativo completamente após aplicá-los.

Os recursos dessas migrações são limitados – seu código não incluirá repentinamente todos os novos recursos da versão do PHP.

Corretores de código não podem corrigir erros de sintaxe para nós. Por exemplo, a classe `Example` que verificamos com o linter interno do PHP na seção anterior ainda exigiria que o desenvolvedor a corrigisse manualmente primeiro.

Linting

PHP CS Fixador verifica os arquivos que você deseja detectar em busca de erros de sintaxe como a primeira etapa e não aplicará nenhuma alteração caso encontre erros de sintaxe. Isso significa que você não precisa executar o linter interno do PHP como uma etapa adicional.

Uma recapitulação do PHP CS Fixer

Um sniffer de código como o PHP CS Fixer deve fazer parte de todo projeto PHP sério. A capacidade de corrigir violações de regras automaticamente economizará muitas horas de trabalho. Se você optar por não aplicar nenhuma correção arriscada, dificilmente causará problemas.

Agora aprendemos como garantir que nosso código esteja bem formatado e sintaticamente correto. Embora essa seja a base de qualquer código de alta qualidade, ela não nos ajuda a evitar bugs ou problemas de manutenção. Nesse ponto, as ferramentas de análise de código estático entram em ação.

Análise de Código Estático

Análise de código estático significa que a única fonte de informação é o próprio código. Apenas ao escanear o código-fonte, essas ferramentas encontrarão questões e problemas que até mesmo o desenvolvedor mais sênior de sua equipe não perceberia durante uma revisão de código.

Estas são as ferramentas que gostaríamos de apresentar a você nas próximas seções:

phpcpd – o detector de copiar e colar

A programação de copiar e colar pode ser qualquer coisa, desde simplesmente irritante até uma ameaça real aos seus projetos. Bugs, problemas de segurança e más práticas serão copiados e, portanto, mais difíceis de corrigir. Pense nisso como se fosse uma praga se espalhando pelo seu código.

Essa forma de programação é bastante comum, principalmente entre desenvolvedores menos experientes, ou em projetos onde os prazos são muito apertados. Felizmente, nosso kit de ferramentas de código limpo oferece uma solução – o detector de copiar e colar PHP (phpcpd).

Instalação e uso

Esta ferramenta só pode ser baixada como um arquivo PHP independente (phar), então não usaremos o Composer para instalá-la desta vez:

```
$ wget https://phar.phpunit.de/phpcpd.phar
```

Manipulando arquivos phar

DentroCapítulo 9, Organizando as ferramentas de qualidade do PHP, aprenderemos como manter os arquivos phar organizados. Por enquanto, basta apenas baixá-lo.

Uma vez baixado, o phpcpd pode ser usado imediatamente sem configuração adicional. Requer apenas o caminho do diretório de destino como parâmetro. O exemplo a seguir mostra como verificar o diretório src em busca dos chamados “clones” (ou seja, código que foi copiado várias vezes). Vamos executá-lo com as configurações padrão primeiro:

```
$ php phpcpd.phar src
```

phpcpd 6.0.3 por Sebastian Bergmann.

Nenhum clone encontrado.

Hora: 00:00, Memória: 2,00 MB

Se o phpcpd não detectar nenhum clone, vale a pena verificar as duas opções, min-lines e min-tokens, que controlam sua “seleção”:

```
$ php phpcpd.phar --min-lines 4 --min-tokens 20 src
```

phpcpd 6.0.3 por Sebastian Bergmann.

Encontrado 1 clone com 22 linhas duplicadas em 2 arquivos:

- /src/example.php:12-23 (11 linhas)

/src/example.php:28-39

/src/exemplo3.php:7-18

32,35% de linhas duplicadas de um total de 68 linhas de código.

O tamanho médio da duplicação é de 22 linhas, o maior clone tem

11 de linhas

Hora: 00:00.001, Memória: 2,00 MB

A opção min-lines nos permite definir o número mínimo de linhas que um pedaço de código precisa ter até que seja considerado um clone.

Para entender o uso de min-tokens, primeiro devemos esclarecer o significado de um token neste contexto: quando você executa um script, o PHP usará internamente o chamado “tokenizer” para dividir o código-fonte em tokens únicos. Um token é um componente independente do seu programa PHP, como

uma palavra-chave, um operador, uma constante ou uma string. Pense nelas como palavras na linguagem humana. A opção `min-tokens`, portanto, controla o número de instruções que um pedaço de código contém antes de ser considerado um clone.

Você pode querer brincar com ambos os parâmetros para encontrar um bom equilíbrio de “seleção” para sua base de código. Uma certa quantidade de redundância em seu código não é automaticamente um problema e você também não quer incomodar muito seus colegas desenvolvedores. Usar os padrões para começar, portanto, é uma boa escolha.

Mais opções

Existem mais duas opções que você deve conhecer:

Recapitulação do phpcpd

Embora o phpcpd seja fácil de usar, é uma ajuda significativa contra a propagação lenta de copiar e colar código em seus projetos. É por isso que recomendamos adicioná-lo ao seu kit de ferramentas de codificador limpo.

PHPMD: o detector de bagunça do PHP

Um detector de bagunça verificará o código em busca de possíveis problemas, também conhecidos como “odores de código” – partes do código que podem introduzir bugs, comportamento inesperado ou são, em geral, mais difíceis de manter. Assim como no estilo de código, existem certas regras que devem ser seguidas para evitar problemas. O detector de bagunça aplica essas regras ao nosso código. A ferramenta padrão no ecossistema PHP para isso é o PHPMD, que mostraremos nesta seção.

Instalação e uso

Antes de vermos mais de perto o que esta ferramenta tem a nos oferecer, vamos instalá-la primeiro usando o Composer:

```
$ composer require phpmd/phpmd --dev
```

Após a conclusão da instalação, podemos executar o PHPMD já na linha de comando. Requer três argumentos:

Para um início rápido, vamos verificar a pasta src, criar a saída como texto e usar os conjuntos de regras cleancode e codesize integrados. Podemos fazer isso executando o seguinte comando:

```
$ vendor/bin/phpmd src text cleancode,codesize
```

PHPMD grava todas as saídas na saída padrão (stdout), que está na linha de comando. No entanto, todos os formatos de saída, exceto texto, não devem ser lidos lá. Se você deseja obter uma primeira visão geral, pode usar a saída html, pois ela gera um relatório bem formatado e interativo. Para armazenar a saída em um arquivo, vamos redirecioná-lo para um arquivo usando o operador > da seguinte forma:


```
$ vendor/bin/phpmd src html cleancode,codesize > phpmd_report.html
```

Basta abrir o arquivo HTML em seu navegador e você verá um relatório semelhante ao mostrado na Figura 7.1:

PHPMD Report

Generated at 2022-04-27 08:18 with [PHP Mess Detector](#) on **PHP 8.0.8** on *curtis-desktop*

2 problems found

Summary

By priority

Count	%	Priority
2	100.0 %	Top (1)

By rule set

Count	%	Rule set
2	100.0 %	Clean Code Rules

By name

Count	%	Rule name
1	50.0 %	StaticAccess
1	50.0 %	MissingImport

Details

#1 Avoid using static access to class **'User'** in **method 'loadData'**. [\(help\)](#) Top (1)

File: [/home/curtis/clean-code/chapter7/phpmd/src/phpmd_suppress_example.php](#) [Show code ▼](#)

```
24 class ExampleClass {
25     public function loadData(string $id) {
26         $userData = User::find($id);
27
28         $test = new Order();
```

#2 Missing class import via use statement (line **'28'**, column **'21'**). [\(help\)](#) Top (1)

File: [/home/curtis/clean-code/chapter7/phpmd/src/phpmd_suppress_example.php](#) [Show code ▼](#)

Figura 7.1: Um relatório PHPMD HTML em um navegador

O relatório é interativo, portanto, certifique-se de clicar em botões como **Mostrar detalhes** ou **Mostrar código** para exibir todas as informações existentes.

Regras e conjuntos de regras

No exemplo anterior, usamos os conjuntos de regras internos `cleancode` e `codesize`. Em primeiro lugar, os conjuntos de regras são nomeados de acordo com o domínio do problema que as regras verificam – como na regra de código limpo, você encontrará apenas regras que ajudam a manter a base de código limpa. No entanto, você ainda pode acabar com classes enormes com muitas funções complexas. Para evitar isso, é necessário adicionar o conjunto de regras `codesize`.

A tabela a seguir mostra os conjuntos de regras disponíveis e seu uso:

conjunto de regras	Nome curto	Descrição
Regras de código limpo	código limpo	Aplica código limpo em
Regras de tamanho de código	tamanho do código	Verifica se há blocos de
regras controversas	controverso	Verifica as melhores e m
Regras de design	Projeto	Ajuda a encontrar proble
regras de nomenclatura	nomeando	Evita nomes muito longo
Regras de código não utilizadas	não utilizado	Detecta código não utiliz

Tabela 7.1: Conjuntos de regras do PHPMD

Essas regras internas podem ser usadas simplesmente fornecendo os nomes curtos mencionados como argumentos para a chamada de função, conforme visto no exemplo anterior.

Se você tiver a sorte de começar um projeto do zero (ou seja, do zero), você pode e deve impor o máximo de regras desde o início. Isso manterá sua base de código limpa desde o início. Para projetos existentes, o esforço é um pouco maior, como veremos na próxima seção.

Usando PHPMD em projetos legados

Muitas vezes, você deseja usar o PHPMD para um projeto existente. Nesse caso, você provavelmente ficará impressionado com os inúmeros avisos que ele lançará na primeira execução. Não desista - existem algumas opções para ajudá-lo!

Ajustando conjuntos de regras

Se você planeja adicionar PHPMD a um projeto existente, ir all-in com os conjuntos de regras certamente levará à frustração por causa de quantos problemas são relatados. Você pode querer se concentrar em um ou dois conjuntos de regras por vez.

Também é altamente provável que você acabe com regras que considera irritantes ou contraproducentes no início – por exemplo, a regra `ElseExpression`, que proíbe o uso de `else` em uma expressão `if`. Deixando de lado a discussão sobre a utilidade dessa regra, o esforço de reescrever inúmeras declarações que estão funcionando bem não vale a pena. Portanto, se você não quiser usar essa regra em seu projeto, precisará criar seu próprio conjunto de regras.

Os conjuntos de regras são configurados por meio de arquivos XML, que especificam as regras que pertencem a eles. Cada regra é basicamente uma classe PHP que contém a lógica da regra. O arquivo XML a seguir define um conjunto de regras personalizado que inclui apenas os conjuntos de regras `cleancode` e `codesize`:

```

<?xml versão="1.0"?>

<ruleset name="Conjunto de regras PHPMD personalizado"
xmlns=http://pmd.sf.net/ruleset/1.0.0
xmlns:xsi=http://www.w3.org/2001/XMLSchema-instance
xsi:schemaLocation=http://pmd.sf.net/ruleset/1.0.0
http://pmd.sf.net/ruleset_xml_schema.xsd
xsi:noNamespaceSchemaLocation=
"http://pmd.sf.net/ruleset_xml_schema.xsd">

<descrição>

Conjunto de regras que contém todo o tamanho do código e código limpo
as regras

</descrição>

<rule ref="rulesets/codesize.xml" />

<rule ref="rulesets/cleancode.xml" />

</conjunto de regras>

```

XML parece estar um pouco fora de moda hoje em dia, mas ainda serve bem ao seu propósito. Normalmente, você não precisa se preocupar com todos os atributos da tag <ruleset> – apenas certifique-se de que eles estejam presentes. A tag <description> pode conter qualquer texto que você considere uma boa descrição para o conjunto de regras.

A tag `<rule>` é importante para nós. No exemplo anterior, referenciamos as regras `codesize` e `cleancode`.

Dica

Neste ponto, é uma boa ideia pesquisar os conjuntos de regras integrados no repositório GitHub <https://github.com/phpmd/phpmd/tree/master/src/main/resources/rulesets>. Como o XML é um formato de arquivo bastante detalhado, você se familiarizará com ele rapidamente.

Imagine que queremos remover a regra `ElseExpression` mencionada de nossas verificações. Para conseguir isso, você só precisa adicionar uma tag `<exclude>` dentro da tag `<rule>` correspondente da seguinte forma:

```
<rule ref="rulesets/cleancode.xml">  
  
<exclude name="ElseExpression" />  
  
</regra>
```

Dessa forma, você pode excluir quantas regras forem necessárias de um conjunto de regras. Se você quiser apenas escolher certas regras de diferentes conjuntos de regras, também pode fazer o contrário e referenciar as regras desejadas diretamente. Se você deseja que seu conjunto de regras personalizado inclua apenas as regras `StaticAccess` e `UndefinedVariable`, seu arquivo XML deve conter as duas tags a seguir:


```
<rule ref="rulesets/cleancode.xml/StaticAccess" />
```

```
<rule ref="rulesets/cleancode.xml/UndefinedVariable" />
```

Uma última coisa importante a saber sobre os arquivos de configuração XML é como alterar as propriedades individuais de uma regra. Novamente, uma boa maneira de descobrir todas as propriedades é verificar o arquivo do conjunto de regras real. Como alternativa, você pode verificar as classes PHP reais de cada regra

em <https://github.com/phpmd/phpmd/tree/master/src/main/php/PHPMD/Rule>.

Um exemplo típico é definir exceções para a regra StaticAccess. Geralmente é uma boa prática evitar o acesso estático, mas muitas vezes você não pode evitá-lo. Digamos que sua equipe concordou em permitir o acesso estático para os objetos DateTime e DateTimezone – você pode simplesmente configurar isso da seguinte maneira:

```
<rule ref="rulesets/cleancode.xml/StaticAccess">
```

```
<propriedades>
```

```
<nome da propriedade="exceções">
```

```
<valor>
```

```
\Data hora,
```

```
\DateTimezone
```

```
</value>
```

```
</propriedade>
```

```
</propriedades>
```

</regra>

Para usar este conjunto de regras personalizado no futuro, simplesmente salve o XML anterior em um arquivo (geralmente chamado phpmd.xml) e passe-o para o PHPMD na próxima execução:

```
$ vendor/bin/phpmd src text phpmd.xml
```

Localização do arquivo de configuração

É uma prática comum colocar phpmd.xml com os conjuntos de regras que você deseja usar na pasta raiz do seu projeto e usá-lo como fonte única de configuração. Se houver modificações no futuro, você só precisará ajustar um arquivo central.

Suprimindo avisos

Outra ferramenta útil para lidar com código legado é a anotação `@SuppressWarnings` DocBlock. Vamos assumir que uma classe em seu projeto faz uso de uma chamada de método estático e isso não pode ser alterado agora. Por padrão, qualquer acesso estático lançará um aviso. Como você não deseja usar o acesso estático em nenhum outro lugar do seu código, mas apenas nesta classe, remover a regra `StaticAccess` seria contraproducente.

Nesses casos, você pode usar a anotação `@SuppressWarnings`:

```
/**  
  
 * @SuppressWarnings(PHPMD.StaticAccess)  
  
 */  
  
 classe ExemploClasse {  
  
 public function getUser(int $id): Usuário {  
  
 return Usuário::find($id);  
  
 }  
  
 }
```

Você pode usar várias anotações em um DocBlock, se necessário. Por fim, se você deseja suprimir qualquer aviso em uma classe, basta usar a anotação `@SuppressWarnings(PHPMD)`.

Esteja ciente de que usar as anotações Suprimir deve ser seu último recurso. É muito tentador apenas adicioná-lo em todos os lugares. No entanto, silenciará a saída, mas não resolverá os problemas.

Aceitar violações

Em vez de suprimir avisos no nível do arquivo ou excluir regras de conjuntos de regras, você também pode decidir reconhecer as violações existentes. Por exemplo, quando você deseja usar PHPMD em um projeto legado, pode decidir

ignorar todas as violações que já estão no código por enquanto. No entanto, se novas violações forem introduzidas por uma nova classe, elas serão relatadas.

Felizmente, o PHPMD torna essa tarefa bastante fácil, fornecendo o chamado arquivo de linha de base, que será gerado para você automaticamente executando o seguinte:

```
$ vendor/bin/phpmd src text phpmd.xml --generate-baseline
```

No comando anterior, esperamos que um arquivo `phpmd.xml` já exista na pasta raiz do projeto. Usando o comando anterior, o PHPMD agora criará um arquivo chamado `phpmd.baseline.xml`.

Agora, você pode executar o seguinte:

```
$ vendor/bin/phpmd src text phpmd.xml
```

Na próxima vez, o PHPMD detectará automaticamente o arquivo de linha de base gerado anteriormente e o usará para suprimir todos os avisos de acordo. No entanto, se uma nova violação de regra for introduzida em um novo local, ela ainda será detectada e relatada como uma violação.

Uma palavra de advertência: assim como a anotação `@SuppressWarnings`, o recurso de linha de base não é uma ferramenta que pode ser usada uma vez e

ignorada com segurança no futuro. Os blocos de código problemáticos ainda fazem parte do seu projeto como dívida técnica com todos os efeitos negativos. É por isso que, se você decidir usar o recurso de linha de base, certifique-se de não se esquecer de abordar esses problemas ocultos no futuro.

Discutiremos como lidar com esses problemas mais adiante neste livro. Por enquanto, é importante apenas para você atualizar o arquivo de linha de base de tempos em tempos. Novamente, o PHPMD torna isso uma tarefa fácil. Basta executar o seguinte:

```
$ vendor/bin/phpmd src text phpmd.xml --update-baseline
```

Todas as violações que não existem mais em seu código serão removidas do arquivo de linha de base.

Uma recapitulação do PHPMD

A menos que você esteja iniciando um projeto no verde, a configuração do PHPMD exigirá um pouco mais de tempo. Especialmente se você estiver trabalhando em equipe, passará mais tempo discutindo sobre quais regras usar e quais excluir. Uma vez feito isso, porém, você tem uma ferramenta poderosa à sua disposição que ajudará os desenvolvedores a escrever código de manutenção de alta qualidade.

PHPStan – um analisador estático para PHP

Você deve ter notado que o PHPMD, que examinamos na seção anterior, não era muito específico do PHP, mas geralmente cuidava das melhores práticas de codificação. Embora isso seja, obviamente, muito importante, queremos usar o PHPStan para analisar nosso código com as más práticas de PHP em mente agora.

Como acontece com todas as ferramentas de análise estática, o PHPStan só pode trabalhar com as informações que pode obter do código. Portanto, funciona melhor com código moderno orientado a objetos. Se, por exemplo, o código fizer uso intenso de digitação estrita, o analisador terá informações adicionais para processar e, portanto, retornará mais resultados. Mas para projetos mais antigos, será de grande ajuda também, como veremos na seção a seguir.

Instalação e uso

A instalação do PHPStan com o Composer é apenas uma linha novamente:

```
$ composer require phpstan/phpstan --dev
```

Assim como a maioria das ferramentas de qualidade de código, o PHPStan pode ser instalado usando o PHAR. No entanto, somente ao usar o Composer você também pode instalar extensões. Vamos dar uma olhada neles um pouco mais adiante nesta seção.

Vamos usar o seguinte exemplo simplificado e armazená-lo dentro da pasta src:

```
<?php
```

```
IVA de classe
```

```
{
```

```
float privado $vat = 0,19;
```

```
função pública getVat(): int
```

```
{
```

```
return $this->vat;
```



```

}

}

classe OrderPosition

{

função pública getGrossPrice(float $netPrice): float

{

$vatModel = new Vat();

$vat = $vatModel->getVat();

return $netPrice * (1 + $vat);

}

}

$posição do pedido = new Posição do pedido();

echo $orderPosition->getGrossPrice(100);

```

Para executar uma varredura, você precisa especificar a palavra-chave de análise, juntamente com o caminho para a varredura, que é src em nosso caso:

```
$ fornecedor/bin/phpstan analise src
```

Figura 7.2 mostra a saída produzida pelo PHPStan:

```
$ vendor/bin/phpstan analyse src
```

```
Note: Using configuration file /home/curtis/clean-code/chapter7/phpstan/phpstan.neon.
```

```
2/2 [████████████████████] 100%
```

```
[OK] No errors
```

```
$
```

Figura 7.2: Um exemplo de saída do PHPStan

Quando executamos o script PHP, a saída será 100. Infelizmente, isso não está correto porque adicionar 19% de impostos ao preço líquido deve retornar 119, e não 100. Portanto, deve haver um bug em algum lugar. Vamos ver como o PHPStan pode nos ajudar aqui.

Níveis de regras

Ao contrário do PHPMD, onde você configura em detalhes quais regras aplicar, usaremos diferentes níveis de relatório aqui. Esses níveis foram definidos pelos desenvolvedores do PHPStan, começando do nível 0 (apenas realizando verificações básicas) até o nível 9 (sendo muito rigoroso em questões). Para não sobrecarregar os usuários com erros no início, o PHPStan por padrão usará o nível 0, que executa apenas algumas verificações.

Você pode especificar o nível usando a opção `level` (`-l|--level`). Vamos tentar o próximo nível mais alto:

```
$ fornecedor/bin/phpstan analyse --nível 1 src
```

Usando a abordagem de nível, você pode facilmente aumentar a qualidade do seu código passo a passo, como demonstraremos usando o seguinte exemplo inventado. Os níveis 1 e 2 também não retornarão nenhum erro. Quando finalmente atingirmos o nível 3, no entanto, finalmente encontraremos um problema:

```
$ vendor/bin/phpstan analyse --level 3 src
```

```
Note: Using configuration file /home/curtis/clean-code/chapter7/phpstan/phpstan.neon.
```

```
2/2 [████████████████████] 100%
```

```
.....
```

```
Line  phpstan_example.php
```

```
.....
```

```
9      Method Vat::getVat() should return int but returns float.
```

```
.....
```

```
[ERROR] Found 1 error
```

Figura 7.3: PHPStan relata um erro com nível 3

Verificando nosso código novamente, podemos identificar o problema rapidamente: o método `getVat()` retorna um número float (0,19), mas usar o tipo de retorno `int` o converte em 0.

Digitação estrita

Se tivéssemos usado o modo estrito adicionando o `declare(strict_types=1);` declaração na parte superior do código de exemplo, o PHP teria lançado um erro em vez de lançar silenciosamente o valor de retorno para `int`.

Isso demonstra a beleza e o poder da análise de código estático: corrigir esse pequeno bug fará com que nosso código funcione conforme o esperado e leva apenas alguns segundos, pois ainda estamos em nosso ambiente de desenvolvimento. No entanto, se esse bug tivesse atingido o ambiente de produção, levaríamos muito mais tempo para corrigi-lo e deixaríamos alguns clientes irritados para trás.

Configuração

Você pode usar arquivos de configuração para garantir que o mesmo nível e as mesmas pastas sejam sempre verificados. A configuração é escrita em NEON (<https://ne-on.org/>), um formato de arquivo muito semelhante ao YAML; se você puder ler e escrever YAML, funcionará perfeitamente.

A configuração básica contém apenas o nível e as pastas a serem verificadas:

Parâmetros:

nível: 4

caminhos:

-src

É uma boa prática salvar esta configuração em um arquivo chamado `phpstan.neon` na pasta raiz do seu projeto. Esse é o local onde o PHPStan espera que esteja por padrão. Se você seguir esta convenção, na próxima vez que quiser executá-lo, basta especificar a ação desejada:

```
$ fornecedor/bin/phpstan analyse
```

Se você usou a configuração de exemplo acima, o PHPStan irá agora escanear a pasta src, usando todas as regras do nível 0 ao nível 4.

Isso não é tudo que você pode configurar aqui. Na próxima seção, aprenderemos sobre alguns parâmetros adicionais.

Usando PHPStan em projetos legados

Se você quiser usar o PHPStan em projetos existentes de uma certa idade, provavelmente acabará com centenas, senão milhares de erros, dependendo do nível escolhido. Claro, você pode decidir continuar usando um nível inferior; mas isso também significa que o analisador perderá mais bugs, não apenas os existentes, mas também em código novo ou modificado.

Em um mundo ideal, você começaria com o nível 0, resolveria todos os erros, continuaria com o nível 1, resolveria todos os novos erros e assim por diante. Porém, isso requer muito tempo e, se nenhum teste automatizado estiver disponível, será necessário executar um teste manual completo no final. Você provavelmente não terá tanto tempo, então vamos ver quais outras opções temos.

Existem duas maneiras pelas quais o PHPStan pode ser instruído a ignorar erros: primeiro, usando anotações do PHPDocs e, segundo, usando um parâmetro especial no arquivo de configuração.

Usando anotações do PHPDocs

Para ignorar uma linha de código, basta adicionar um comentário antes ou na linha afetada, usando as anotações especiais `@phpstan-ignore-next-line` e `@phpstan-ignore-line` PHPDocs:

```
// @phpstan-ignore-next-line
```

```
$exampleClass->foo();
```

```
$exampleClass->bar(); // @phpstan-ignore-line
```

Ambas as linhas de código não serão mais verificadas quanto a erros. Cabe a você escolher a forma que preferir. Porém, não é possível ignorar blocos de código maiores ou mesmo funções ou classes inteiras (a menos que você queira adicionar um comentário a cada linha).

Usando parâmetros ignoreErrors

As anotações do PHPDocs são perfeitas para correções rápidas em apenas alguns locais, mas você precisará mexer em muitos arquivos se quiser ignorar vários erros. Usar o parâmetro ignoreErrors no arquivo de configuração não é muito confortável, pois você precisa escrever uma expressão regular para cada erro que deseja ignorar.

O exemplo a seguir explicará como funciona. Vamos supor que continuamos recebendo um erro da seguinte forma:

O método `OrderPosition::getGrossPrice()` não possui nenhum tipo de retorno especificado.

Embora teoricamente isso seja fácil de consertar, a equipe decide não adicionar uma dica de tipo para não arriscar nenhum efeito colateral. A classe `OrderPosition` foi mal escrita e não foi coberta com testes, mas ainda funciona conforme o esperado. Como ele será substituído em breve, não estamos

dispostos a correr o risco de tocá-lo.

Para ignorar este erro, precisamos adicionar o parâmetro `ignoreErrors` ao nosso arquivo de configuração `phpstan.neon`:

Parâmetros:

nível: 6

caminhos:

-src

`ignoreErrors`:

- '#^Método OrderPosition\:\:getGrossPrice\(\) não tem nenhum tipo de retorno especificado\.\$#'

Em vez de definir uma regra ou conjunto de regras para ignorar, precisamos fornecer aqui uma expressão regular que corresponda à mensagem do erro que deve ser ignorado.

Dica

Escrever expressões regulares pode ser um desafio. Felizmente, o site PHPStan oferece uma pequena ferramenta muito útil para gerar a parte `phpstan.neon` necessária a partir da mensagem de erro: <https://phpstan.org/user-guide/ignoring-errors#generate-an-ignoreerrors-entry>.

Na próxima execução, o erro não será mais exibido independentemente de onde ocorrer, pois corresponde à expressão regular aqui.

PHPStanGenericName não informa sobre o fato de que os erros são ignorados. Não se esqueça de corrigi-los em algum momento! No entanto, se você melhorar seu código ainda mais ao longo do tempo, o PHPStan informará quando os erros configurados para serem ignorados não forem mais correspondidos. Você pode removê-los da lista com segurança.

Se você deseja ignorar certos erros completamente, mas apenas em um ou mais arquivos ou caminhos, pode fazê-lo usando uma notação ligeiramente diferente:

ignoreErrors:

-

mensagem: '#^Método

OrderPosition\::getGrossPrice\(\) não tem retorno

tipo especificado\.\$#'

caminho: src/OrderPosition.php

O caminho precisa ser relativo ao local do arquivo de configuração phpstan.neon. Quando informado, o erro só será ignorado se ocorrer em OrderPosition.php.

Linha de base

Como acabamos de ver na seção anterior, adicionar erros que você deseja que sejam ignorados manualmente em seu arquivo de configuração é uma tarefa complicada. Mas existe uma maneira mais fácil: semelhante ao PHPMD, é possível adicionar automaticamente todos os erros atuais à lista de erros ignorados de uma vez, executando o seguinte comando com a opção `--generate-baseline`:

```
$ fornecedor/bin/phpstan analyse --generate-baseline
```

O arquivo recém-gerado, `phpstan-baseline.neon`, está no mesmo diretório que o arquivo de configuração. O PHPStan não o usará automaticamente. Você deve incluí-lo manualmente no arquivo `phpstan.neon` da seguinte forma:

inclui:

- `phpstan-baseline.neon`

Parâmetros:

...

Da próxima vez que você executar o PHPStan agora, quaisquer erros relatados anteriormente não devem ser mais relatados.

Internamente, o arquivo `baseline` nada mais é do que uma lista criada

automaticamente dos parâmetros `ignoreErrors`. Sinta-se à vontade para modificá-lo de acordo com suas necessidades. Você sempre pode regenerá-lo executando `phpstan` usando a opção `--generate-baseline` novamente.

Extensões

É possível estender a funcionalidade do PHPStan. A comunidade vivid já criou um número respeitável de extensões úteis. Por exemplo, frameworks como Symfony, Laminas ou Laravel geralmente usam métodos mágicos (como `__get()` e `__set()`), que não podem ser analisados automaticamente. Existem extensões para esses frameworks que fornecem as informações necessárias ao PHPStan.

Embora não possamos abordar essas extensões neste livro, recomendamos que você verifique a biblioteca de extensões: <https://phpstan.org/user-guide/extension-library>. Também existem extensões para PHPUnit, phpspec e WordPress.

Uma recapitulação do PHPStan

PHPStan é uma ferramenta poderosa. Não podemos cobrir todas as suas funcionalidades em apenas algumas páginas, mas demos a você uma boa ideia de como começar a usá-lo. Quando estiver familiarizado com seu uso básico, confira <https://phpstan.org> para aprender mais!

Psalm: Uma máquina de linting de análise estática PHP

O próximo e último analisador de código estático que queremos apresentar é o Psalm. Ele verificará nossa base de código para os chamados problemas e relatará quaisquer violações. Além disso, ele pode resolver alguns desses problemas automaticamente. Então, vamos dar uma olhada mais de perto.

Instalação e uso

Mais uma vez, instalar o Psalm com o Composer é apenas uma questão de algumas teclas:

```
$ composer require --dev vimeo/psalm
```

Também está disponível como um arquivo phar.

Após a instalação, não podemos simplesmente começar - em vez disso, precisamos primeiro configurar um arquivo de configuração para o projeto atual. Podemos usar a confortável opção `--init` para criá-lo:

```
$ vendor/bin/salmo --init
```

Este comando gravará um arquivo de configuração chamado `psalm.xml` no diretório atual, que deve ser a raiz do projeto. Durante sua criação, o Psalm verifica se pode encontrar algum código PHP e decide qual nível de erro é adequado, para começar. A execução do Salmo não requer mais opções:

```
$ vendor/bin/salmo
```

Configuração

O arquivo de configuração já foi criado durante o processo de instalação e pode, por exemplo, ser semelhante a este:

```
<?xml versão="1.0"?>
```

```
<salmo
```

```
nível de erro="7"
```

```
resolveFromConfigFile="true"
```

```
xmlns:xsi=http://www.w3.org/2001/XMLSchema-instance
```

```
xmlns=https://getpsalm.org/schema/config
```

```
xsi:schemaLocation=https://getpsalm.org/schema/config  
vendor/vimeo/psalm/config.xsd
```

>

<projectFiles>

<nome do diretório="src" />

<ignoreArquivos>

<nome do diretório="fornecedor" />

</ignoreFiles>

</projectFiles>

</salmo>

Vamos dar uma olhada nos atributos do nó <psalm>. Você não precisa se preocupar com as informações relacionadas ao esquema e ao namespace, apenas com as duas coisas a seguir:

documentação do salmo

O Salmo oferece muito mais opções de configuração que não podemos cobrir neste livro. Como sempre, recomendamos verificar a documentação (<https://psalm.dev/docs>) para saber mais sobre esta ferramenta.

Dentro do nó `<psalm>`, você encontrará mais configurações. No exemplo anterior, o Psalm é instruído a verificar apenas a pasta `src` e ignorar todos os arquivos na pasta `vendor`. Ignorar a pasta do fornecedor é importante, pois não queremos escanear nenhum código de terceiros.

Usando o Salmo em projetos legados

Agora veremos como podemos ajustar o Salmo para lidar melhor com os projetos existentes. Como nas ferramentas anteriores, existem basicamente duas maneiras de ignorar os problemas: usando o arquivo de configuração ou as anotações do docblock.

Existem três níveis de emissão de código: informação, erro e supressão. Embora as informações apenas imprimam mensagens informativas se forem encontrados problemas menores, os problemas que estão no nível de um tipo de erro, por outro lado, exigem que você fique ativo. Um problema do tipo suprimir não será mostrado.

Integração contínua

A diferença entre informação e erro se torna mais importante ao construir um pipeline de Integração Contínua. problemas de informação deixariam a compilação passar, enquanto problemas de erro a interromperiam. Abordaremos este tema mais adiante.

Supressão de bloco de documentos

A anotação `@psalm-suppress` pode ser usada em um bloco de documentação de função ou em um comentário para a próxima linha. A classe `Vat` dos exemplos anteriores pode ter a seguinte aparência:

IVA de classe

```
{  
  
float privado $vat = 0,19;  
  
/**  
  
 * @psalm-suppress InvalidReturnType  
  
 */  
  
função pública getVat(): int  
  
{  
  
/**  
  
 * @psalm-suppress InvalidReturnStatement  
  
 */  
  
return $this->vat;  
  
}  
  
}
```

Supressão do arquivo de configuração

Se quisermos suprimir problemas, precisamos configurar o issueHandler para eles, onde podemos definir o tipo para suprimir manualmente. Isso é feito no arquivo de configuração adicionando um nó <issueHandler> dentro do nó <psalm>:

```
<issueHandlers>
```

```
<InvalidReturnType errorLevel="suppress" />
```

```
<InvalidReturnStatement errorLevel="suppress" />
```

```
</issueHandlers>
```

A configuração anterior suprimiria todos os problemas InvalidReturnType e InvalidReturnStatement em todo o projeto. Podemos tornar isso um pouco mais específico, no entanto:

```
<issueHandlers>
```

```
<InvalidReturnType>
```

```
<errorLevel type="suppress">
```

```
<nome do arquivo="Vat.php" />
```


</errorLevel>

</InvalidReturnType>

<InvalidReturnStatement>

<errorLevel type="suppress">

<nome do diretório="src/IVA" />

</errorLevel>

</InvalidReturnStatement>

</issueHandlers>

Na documentação (https://psalm.dev/docs/running_psalms/dealing_with_code_issues/), você encontrará ainda mais maneiras de suprimir problemas – por exemplo, pelo nome da variável.

Linha de base

Como nos analisadores de código estático anteriores que discutimos, o Psalm também fornece um recurso para gerar um arquivo de linha de base, que incluirá todos os erros atuais para que sejam ignorados durante a próxima execução. Observe que o recurso de linha de base funciona apenas para problemas de erro, mas não para problemas de informação. Vamos criar o arquivo primeiro:

```
$ vendor/bin/psalm --set-baseline=psalm-baseline.xml
```

Salmonão tem nome padrão para este arquivo, então você precisa passá-lo como opção para o comando:

```
$ vendor/bin/psalm --use-baseline=psalm-baseline.xml
```

Você também pode adicioná-lo como um atributo adicional ao nó `<psalm>` no arquivo de configuração:

```
<salmo
```

```
...
```

```
errorBaseline="./psalm-baseline.xml"
```

>

Por fim, você pode atualizar o arquivo de linha de base – por exemplo, depois de fazer algumas melhorias no código:

```
$ vendor/bin/psalm --update-baseline
```

Corrigindo problemas automaticamente

Salmonão apenas encontrará o problema, mas também poderá corrigir muitos deles automaticamente. Ele informará quando esse for o caso e você poderá usar a opção `--alter`:

Salmo pode corrigir automaticamente 1 problemas.

Execute o Salmo novamente com

```
--alter --issues=InvalidReturnType --dry-run
```

para ver o que pode consertar.

Vamos executar o comando como Salmo sugere:

```
$ vendor/bin/psalm --alter --issues=InvalidReturnType --dry-run
```

A opção `--dry-run` diz ao Psalm para mostrar apenas o que mudaria como diff, mas não para aplicar as alterações. Dessa forma, você pode verificar se a alteração está correta:


```
$ vendor/bin/psalm --alter --issues=InvalidReturnType --dry-run
Target PHP version: 8.1 (inferred from current PHP version)
Scanning files...
Analyzing files...
```

```
█
Altering files...
```

```
/home/curtis/clean-code/chapter7/psalm/src/psalm_example.php:
--- /home/curtis/clean-code/chapter7/psalm/src/psalm_example.php
+++ /home/curtis/clean-code/chapter7/psalm/src/psalm_example.php
@@ -4,7 +4,7 @@
 {
     private float $vat = 0.19;

-    public function getVat(): int
+    public function getVat(): float
     {
         return $this->vat;
     }
 }
```

```
-----
No errors found!
-----
```

```
Checks took 0.29 seconds and used 71.634MB of memory
Psalm was able to infer types for 100% of the codebase
```

```
$ █
```

Fig 7.4: Salmo mostrando as mudanças propostas

Se você remover a opção `--dry-run`, as alterações serão aplicadas.

Uma recapitulação do Salmo

Salmo é uma ferramenta padrão no kit de ferramentas do codificador limpo por um bom motivo. É rápido, fácil de usar e poderoso. Além disso, o recurso de manipulação de código economizará muito tempo. Claro, existem inúmeras semelhanças com o PHPStan, mas com bastante frequência, você encontrará ambas as ferramentas trabalhando juntas na mesma base de código sem problemas. Pelo menos, você deve considerar tentar.

Extensões IDE

As ferramentas que vimos até agora compartilham algo em comum: elas precisam ser aplicadas ao nosso código depois de escrevê-lo. Claro, isso é muito melhor do que nada, mas não seria ótimo se as ferramentas nos dessem seu feedback imediatamente no momento em que escrevemos o código?

Isso é o que muitos outros desenvolvedores também pensaram, então eles criaram extensões para os IDEs mais populares, que atualmente são Visual Studio Code (VS Code) e PhpStorm:

IDEs PHP alternativos

PhpStorm e VS Code não são os únicos IDEs que existem para PHP. Outras alternativas são o NetBeans (<https://netbeans.apache.org>), Eclipse PDT (<https://www.eclipse.org>) ou CodeLobster (<https://www.codelobster.com>).

Nesta seção, apresentaremos três extensões para esses dois IDEs:

Integração da ferramenta de qualidade de código no PhpStorm

O PhpStorm oferece integração perfeita para as seguintes ferramentas que discutimos: PHP CS Fixer, PHPMD, PHPStan e Psalm. Mais informações podem ser encontradas aqui: <https://www.jetbrains.com/help/phpstorm/php-code-quality-tools.html>.

Inspeções PHP (EA Estendido)

Este plugin (<https://github.com/kalessil/phpinspectionsea>) é para PhpStorm. Ele adicionará ainda mais tipos de inspeções ao conjunto de inspeções já existentes, abrangendo tópicos como estilo de código, arquitetura ou possíveis bugs.

Inspeções IDE

Os IDEs modernos já estão equipados com muitas verificações de código úteis. No PhpStorm, eles são chamados de Inspeções. Alguns já estão ativados por padrão – mais podem ser ativados manualmente (<https://www.jetbrains.com/help/phpstorm/code-inspection.html#access-inspections-and-settings>). Para o VS Code, você precisa instalar uma extensão primeiro. Confira a documentação (<https://code.visualstudio.com/docs/languages/php>) para obter mais informações.

Instalação

Como acontece com todos os plug-ins do PhpStorm, a instalação é feita por meio da caixa de diálogo Arquivo -> Configurações -> Plug-ins. Você encontrará informações detalhadas sobre como instalar um plug-in no site do fornecedor (<https://www.jetbrains.com/help/phpstorm/managing-plugins.html>). Basta pesquisar por EA Extended. Observe que existe uma segunda versão deste plug-in, EA Ultimate, pela qual você deve pagar. Não vamos cobri-lo neste livro.

Após a instalação, nem todas as inspeções ficam imediatamente ativas. Vamos dar uma olhada na configuração das inspeções do PhpStorm, conforme mostrado na Figura 7.4:

Q

> PHP

> Appearance & Behavior

Keymap

> Editor

> General

Code Editing

Font

> Color Scheme

> Code Style

Inspections

File and Code Templates

File Encodings

Live Templates

File Types

> Copyright

Inlay Hints

Duplicates

> Emmet

Intentions

Language Injections

> Natural Languages

Reader Mode

TextMate Bundles

TODO

Plugins

> Version Control

Editor > Inspections

Profile: Project Default Project

Q

> Naming conventions

> Php Inspections (EA Extended)

> Architecture

> Code Style

> Compatibility

> Confusing constructs

> Control Flow

'array_search(...)' could be replaced

'array_unique(...)' can be used

'compact(...)' can be used

Foreach usage possible

General exception is thrown

'gettype(...)' could be replaced with

If-return-return could be simplified

Inverted 'if-else' constructs

'isset(...)' constructs can be merged

'isset(...)' usage

'list(...)' = ' usage possible

Loop which does not loop

Non-optimal if conditions

'ob_get_clean()' can be used

One-time use variables

PDO API usage

Redundant 'else' keyword

Multiple inspections are selected. You can edit them as a single inspection.

Severity: Mixed In All Scopes

☐ Disable new inspections by default

Figura 7.5: A caixa de diálogo de configuração de Inspeções no PhpStorm

Todas as inspeções deste plugin podem ser encontradas na seção Php Inspections (EA Extended). As inspeções que não estão ativas por padrão podem ser facilmente ativadas marcando a caixa de seleção ao lado delas. Recomendamos a leitura da documentação (<https://github.com/kalessil/phpinspectionsea/tree/master/docs>) antes de ativar quaisquer inspeções adicionais – caso contrário, você pode acabar com muitas regras. Você pode revisitá-los mais tarde.

Uso

Inspeções PHP (EA Estendido) não apenas avisa sobre problemas, mas também oferece os chamados Quick-Fixes, que permitem que o IDE faça o trabalho para você. Aqui, você encontrará um exemplo. Observe a cláusula **if** destacada na linha 7:

```
1  <?php
2
3  class InspectionExample
4  {
5      public function checkGreaterThanOne(int $value): bool
6      {
7          if ($value > 1) {
8              return true;
9          }
10
11          return false;
12      }
13  }
```


Figura 7.6: Código de exemplo com um problema encontrado por PHP Inspections (EA Extended)

Ao passar o ponteiro do mouse sobre a área destacada, o PhpStorm mostrará uma janela pop-up com mais instruções sobre a melhoria sugerida:

```
1 <?php
2
3 class InspectionExample
4 {
5     public function checkGreaterThanOne(int $value): bool
6     {
7         if ($value > 1) {
8
9             [EA] The construct can be replaced with 'return $value > 1'.
10
11             [EA] Use return instead Alt+Shift+Enter More actions... Alt+Enter
12
13             return false;
14         }
15     }
16 }
```

Figura 7.7: PHP Inspections (EA Extended) sugerindo uma melhoria de código

Você pode optar por corrigir o problema diretamente pressionando Alt + Shift + Enter ao mesmo tempo ou pode clicar na área destacada para mostrar o balão de correção rápida. Se você clicar no balão, verá um menu com mais algumas opções. Você também pode invocar a seguinte caixa de diálogo pressionando Alt + Enter:

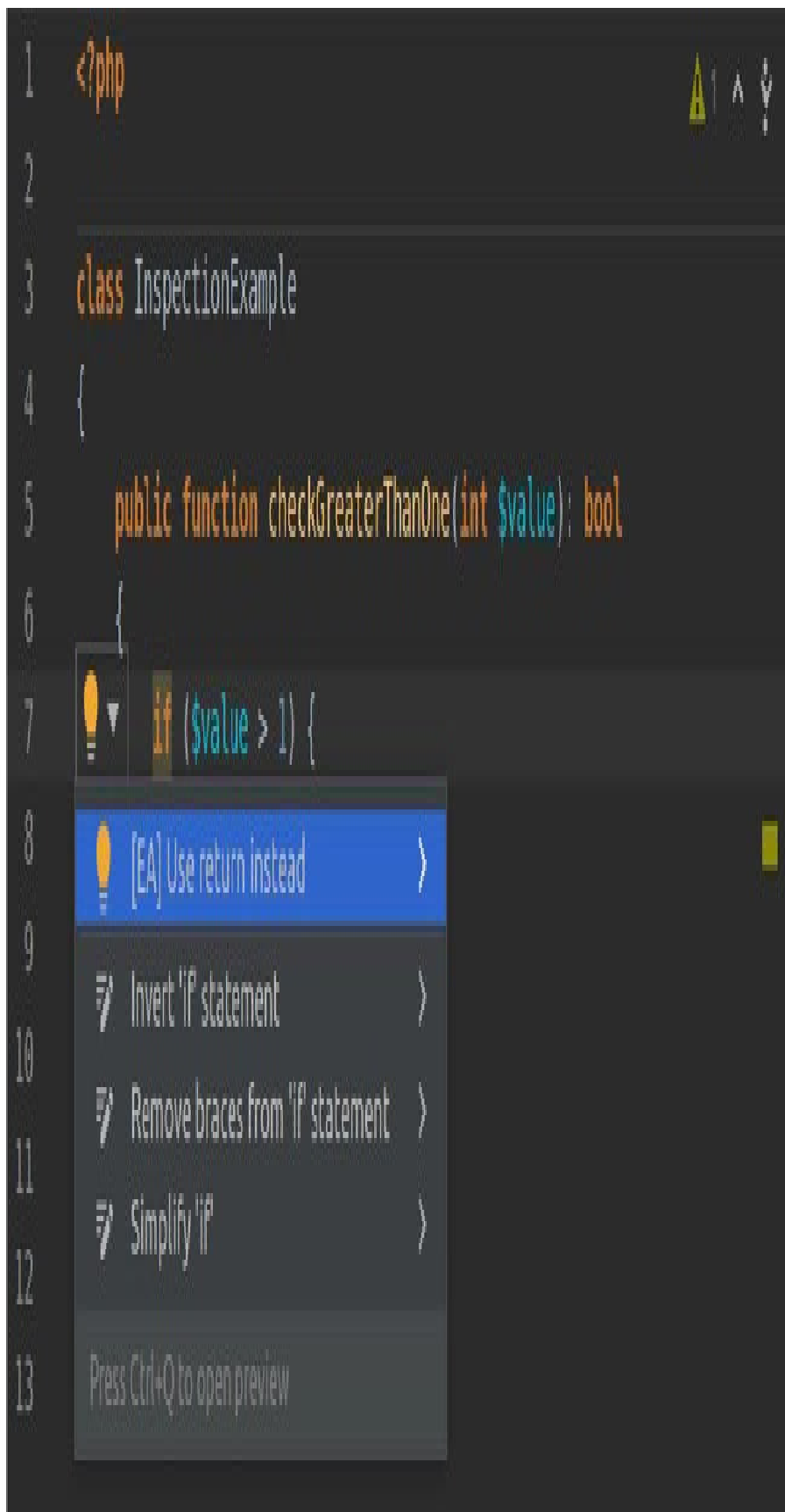


Figura 7.8: O menu de opções do Quick-Fix

PhpStorm oferece várias correções agora. O primeiro, marcado com [EA], é uma sugestão do plugin. Outro clique aplicará a correção:

```
1  <?php
2
3  class InspectionExample
4  {
5      public function checkGreaterThanOne(int $value): bool
6      {
7          return $value > 1;
8      }
9  }
```

Figura 7.9: O código após a aplicação de um Quick-Fix

É isso! Em apenas alguns segundos, você tornou seu código mais curto e fácil de ler. O PHP Inspections (EA Extended) é um ótimo complemento para o PhpStorm, pois oferece inspeções sensatas e as integra perfeitamente. Se você estiver usando este IDE, não hesite em instalá-lo.

Inspeções ao trabalhar em equipe

Essas inspeções são uma ótima maneira de melhorar seu código e educar-se sobre as melhores práticas. No entanto, há uma grande desvantagem: como você garante que todos os desenvolvedores trabalhando em seu projeto tenham as mesmas inspeções ativadas? Abordaremos esse tópico em Trabalho em equipe.

Intelefense

A segunda extensão que queremos apresentar é o Intelephense for VS Code. É a extensão PHP baixada com mais frequência para este editor e fornece muitas funcionalidades (como conclusão e formatação de código), o que transforma o VS Code em um PHP IDE totalmente capaz. Há também uma versão premium comercial dessa extensão que oferece ainda mais funcionalidades. Para instalá-lo, siga as instruções no site do Marketplace para este plug-in (<https://marketplace.visualstudio.com/items?itemName=bmewburn.vscode-intelephense-client>).

Intelefense não tem a gama de funcionalidades que um IDE comercial completo ofereceria de forma alguma, mas para um serviço gratuito, é uma escolha perfeita. Ele oferece os chamados Diagnósticos (que são semelhantes às Inspeções no PhpStorm) que podem ser configurados na tela de configurações do plugin, conforme Figura 7.9:

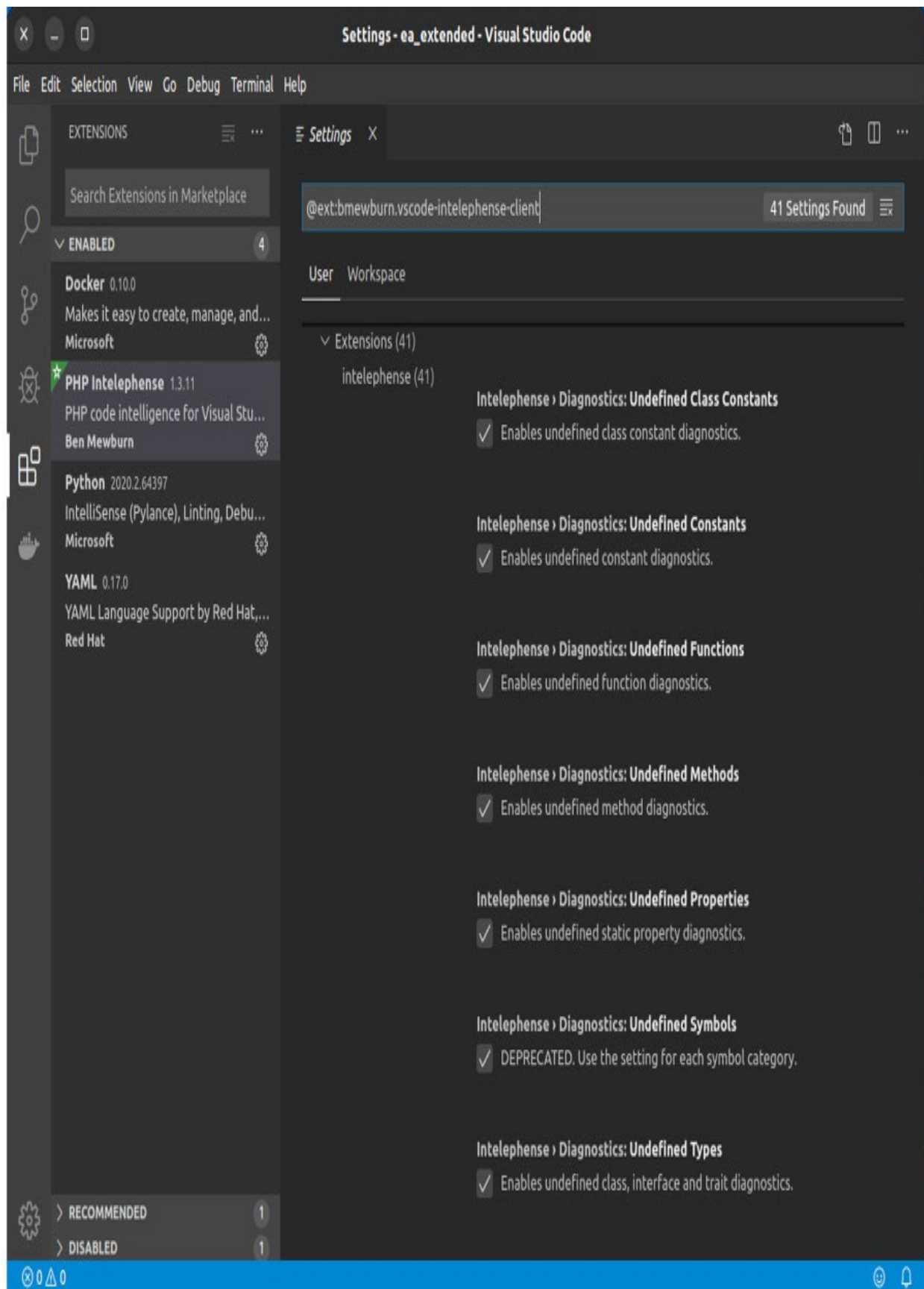


Figura 7.10: A tela de configurações do Intelephense

Uso

A figura a seguir mostra o diagnóstico no Intelephense em ação:

```
1 <?php
2
3 class DiagnosticsExample
4 {
5     private int $unusedAttribute = 1;
6     private int $usedAttribute = 2;
7
8     public function exampleFunction(): void
9     {
10         if ($this->usedAttribute == 2) {
11             echo $this->usedAttribute;
12         }
13
14         $testInstance = new TestClass();
15     }
16 }
17
```

Figura 7.11: Uma aula de exemplo mostrando como o Intelephense destaca problemas

Duas coisas podem ser vistas aqui. Em primeiro lugar, e mais obviamente, está a linha vermelha abaixo de `TestClass`. Passar o ponteiro do mouse sobre `TestClass` mostrará uma janela pop-up com uma explicação: `TestClass` de tipo indefinido. Isso faz sentido, pois essa classe não existe.

Em segundo lugar, e mais sutilmente, você notará que `$unusedAttribute` e `$testInstance` têm uma cor ligeiramente mais escura que as outras variáveis. Isso indica outro problema, que pode ser revelado ao passar o mouse sobre uma das variáveis:

```
1 <?php
2
3 class DiagnosticsExample
4 {
5     private int $unusedAttribute = 1;
6
7     DiagnosticsExample::$unusedAttribute
8
9     <?php
10    private int $unusedAttribute;
11
12    @var int $unusedAttribute
13
14    $testInstance = new TestClass();
15 }
16 }
17
```

DiagnosticsExample::\$unusedAttribute

<?php

private int \$unusedAttribute;

@var int \$unusedAttribute

Symbol '\$unusedAttribute' is declared but not used. intelephense(1003)

No quick fixes available

\$testInstance = new TestClass();

Figura 7.12: Um pop-up de informações no Intelephense

O pop-up nos diz que `$unsuserAttribute` não é usado em nenhum outro lugar do código. O mesmo se aplica a `$testInstance` também.

Embora forneça algumas regras básicas de detecção de problemas e formatação de código, pode-se dizer claramente que, no momento da escrita, o foco deste plug-in não está no código limpo. No entanto, como o VS Code e este plug-in estão disponíveis gratuitamente, você já tem um PHP IDE decente disponível para começar a codificar.

Integração da ferramenta de qualidade de código no VS Code

Assim como no PhpStorm, é possível integrar algumas ferramentas comuns de qualidade de código no VS Code usando plugins, como para PHPStan (<https://marketplace.visualstudio.com/items?itemName=calismurf2904.vscod-phpstan>), PHP CS Fixer (<https://marketplace.visualstudio.com/items?itemName=junstyle.php-cs-fixer>) e PHPMD (<https://marketplace.visualstudio.com/items?itemName=ecodes.vscod-phpmd>). Portanto, se você deseja codificar com o VS Code, verifique o Marketplace para novos plug-ins de vez em quando.

Resumo

Neste capítulo, aprendemos sobre ferramentas de última geração para ajudá-lo a criar código PHP de alta qualidade. Eles ajudarão você a identificar problemas no início do ciclo de vida de desenvolvimento de software (SDLC), o que economiza muito tempo. A comunidade PHP ainda é vívida e muito produtiva, e não fomos capazes de cobrir todo o software fantástico que existe neste livro. No entanto, com as ferramentas que apresentamos neste capítulo, você agora está bem equipado para sua jornada em direção ao código limpo.

No próximo capítulo, você aprenderá como avaliar a qualidade do código usando as métricas estabelecidas e, claro, as ferramentas necessárias para reuni-las. Veja você lá!

Leitura adicional

Se você quiser experimentar ainda mais ferramentas de qualidade de código, considere os seguintes projetos:

Métricas de qualidade de código

Não seria ótimo se pudéssemos medir a qualidade do nosso software? Os desenvolvedores de software geralmente desejam melhorar seu software repetidamente - mas talvez já seja "bom o suficiente". Como sabemos quando atinge um bom estado?

Métricas de qualidade de software foram introduzidas por pessoas inteligentes nos primórdios da programação. Na década de 1970, eles pensaram sobre esse tema e produziram as ideias que ainda estão em uso hoje. Claro, queremos nos beneficiar desse conhecimento e aplicá-lo em nossos próprios projetos.

Neste capítulo, vamos abordar os seguintes tópicos:

Requerimentos técnicos

Se você passou pelo capítulo anterior e experimentou todas as ferramentas, já tem tudo o que precisa para este capítulo instalado. Caso contrário, certifique-se de fazê-lo antes de executar os próximos exemplos.

Os arquivos de código para este capítulo podem ser encontrados aqui:<https://github.com/PacktPublishing/Clean-Code-in-PHP>

Apresentando métricas de qualidade de código

Nesta seção, você aprenderá como medir a qualidade do software em geral. Veremos algumas das métricas mais usadas no mundo PHP e explicaremos o que elas podem dizer sobre seu código, como coletá-las e quando são úteis ou não.

Aspectos da qualidade de software

Antes de mergulharmos nos números, precisamos primeiro esclarecer uma coisa importante: o que realmente significa qualidade de software? Certamente, todo mundo tem um certo entendimento de qualidade, mas pode ser difícil colocar isso em palavras. Felizmente, já existem modelos existentes, como o modelo FURPS, desenvolvido na Hewlett-Packard na década de 1980. A sigla significa o seguinte:

Outros aspectos de qualidade incluem, entre outros, acessibilidade e conformidade legal. Como você pode ver, este modelo abrange mais aspectos, como experiência do usuário e documentação, do que nós, como desenvolvedores PHP, normalmente trabalhamos. É por isso que podemos olhar para a qualidade do software de dois pontos de vista diferentes: qualidade externa e interna. Vamos entender melhor o que isso significa:

Neste livro, nos concentramos na qualidade do código interno. É por isso que falamos sobre qualidade de código em particular e não usamos o termo mais amplo, qualidade de software.

Métricas de qualidade do código

Agora que entendemos melhor o que significa qualidade de código, vamos agora dar uma olhada em quais métricas de qualidade de código queremos falar nesta seção:

As linhas de código

Contar as Linhas de Código (LOC) em um projeto não é uma métrica de qualidade. No entanto, é uma ferramenta útil para entender o tamanho de um projeto – por exemplo, quando você começa a trabalhar nele. Além disso, como veremos, é utilizado por outras métricas como base para seus cálculos. Também é útil ter uma ideia de quantas linhas de código você está lidando – por exemplo, quando você precisa estimar os esforços de refatoração para determinadas classes.

É por isso que queremos dar uma olhada mais de perto agora. Primeiro, podemos diferenciar ainda mais o LOC:

Aqui, o LOC seria 1. Como temos três instruções executáveis nesta linha, o LLOC contaria como 3, pois o código também pode ser escrito com cada instrução em uma linha:

```
while($i < 5) {
```

```
echo “teste”;
```

```
/* Incrementa em um */
```



```
$i++;
```

```
}
```

No exemplo anterior, destacamos as instruções executáveis. Comentários, linhas vazias e elementos sintáticos como colchetes não são instruções executáveis – é por isso que o comentário de linha inteira e a chave de fechamento no final do loop não são contados como uma linha lógica.

A complexidade ciclomática

Em vez de apenas contar as linhas de código, também podemos medir a complexidade do código – por exemplo, contando o número de caminhos de execução dentro de uma função. Uma métrica comum para isso é a Complexidade Ciclométrica (CC). Foi introduzido no final dos anos 1970, mas ainda é útil. A ideia por trás do nome enigmático é simples: contamos o número de pontos de decisão, que são declarações if, while, for e case. Além disso, a entrada da função também conta como uma instrução.

O exemplo a seguir ilustra como a métrica funciona:

```
// primeiro ponto de decisão  
função algumExample($a, $b)  
{  
    // segundo ponto de decisão  
    if ($a < $b) {  
        eco "1";  
    } senão {  
        eco "2";  
    }  
    // terceiro ponto de decisão
```

```
if ($a > $b) {  
    eco "3";  
} senão {  
    eco "4";  
}  
}
```

O CC para o trecho de código anterior seria 3: a entrada da função conta como o primeiro caminho de decisão e ambas as instruções if também contam como um caminho de decisão cada. No entanto, ambas as declarações else não são consideradas por definição, pois fazem parte das cláusulas if. Essa métrica é especialmente útil para avaliar rapidamente a complexidade do código que você ainda não conhece. Geralmente é usado para verificar uma única função, mas também pode ser aplicado a classes ou até mesmo a um aplicativo inteiro. Se você tiver uma função com CC alto, considere dividi-la em várias funções menores para reduzir o valor.

A complexidade NPath

Uma segunda métrica de complexidade de código é a complexidade NPath. A ideia básica é semelhante ao CC, pois também conta os caminhos de decisão de uma função. No entanto, ele conta todos os caminhos de decisão possíveis e não apenas as quatro instruções (if, while, for e case) que são definidas para o CC. Além disso, o ponto de entrada da função não é contado como um caminho de decisão para essa métrica.

Olhando para o exemplo acima, a complexidade do NPath seria 4, porque temos $2 * 2$ caminhos possíveis através da função: tanto if quanto os else. Portanto, todas as quatro declarações de eco são consideradas caminhos de decisão. Como mencionado anteriormente, a própria chamada de função não é considerada. Agora, se adicionarmos outra instrução if, a complexidade do NPath aumentará para 8. Isso ocorre porque teríamos $2 * 2 * 2$ caminhos possíveis. Em outras palavras, a métrica cresce exponencialmente, então pode rapidamente se tornar bastante alta.

A complexidade NPath descreve o esforço real para testar uma função melhor que o CC, pois nos diz diretamente quantos resultados possíveis da função precisaríamos testar para atingir 100% de cobertura de teste.

métricas de Halstead

Maurice Halstead introduziu um conjunto de oito métricas no final dos anos 1970, que ainda estão em uso hoje e são conhecidas como métricas de Halstead. Eles são baseados apenas no número total e distinto de operadores (por exemplo, ==, != e &&) e operandos (por exemplo, nomes de função, variáveis e constantes), mas, como você verá, eles já dizem muito sobre o código inspecionado.

Não precisamos saber exatamente como essas métricas funcionam. Se você estiver interessado, pode saber mais sobre essas métricas aqui: https://www.verifysoft.com/en_halstead_metrics.html. No entanto, você deve ter uma ideia de quais são as métricas de Halstead:

Esses valores fornecerão uma indicação aproximada do tipo de código com o qual você está lidando. É fácil de entender? Quanto tempo foi gasto para desenvolvê-lo? Quantos bugs podem ser esperados? Porém, sem comparar esses valores com resultados de outros aplicativos, eles não vão te ajudar muito.

O índice de antipadrões de risco de mudança

Outra métrica especialmente útil é o índice Change Risk Anti-Patterns (CRAP). Ele usa o CC e a cobertura de código do código em consideração.

Cobertura de código

Você provavelmente já ouviu muito o termo cobertura de código. É uma métrica usada no contexto de testes automatizados e descreve o número de linhas de código (indicado em porcentagem do número total de linhas) para o qual os testes de unidade foram escritos. Discutiremos essa métrica e seus pré-requisitos novamente mais adiante neste livro, quando lidarmos com esse tópico com mais detalhes.

A combinação dessas duas métricas é bastante útil. O código que não é excessivamente complexo e tem alta cobertura de teste tem muito mais probabilidade de ser livre de bugs e de fácil manutenção do que o código que é complexo e onde não há muitos testes para ele.

O índice de manutenibilidade

Como última métrica nesta seção, veremos o índice de manutenibilidade. Ele fornecerá apenas um valor que indica a manutenibilidade do código inspecionado ou, em outras palavras, informa como será fácil alterá-lo sem introduzir novos bugs. Duas coisas tornam essa métrica particularmente interessante para nós.

Em primeiro lugar, baseia-se nas métricas mencionadas e usa as métricas LOC, Halstead e CC para calcular o índice. Mais uma vez, não precisamos realmente saber a fórmula exata. Se você estiver interessado, pode procurar aqui: https://www.verifysoft.com/en_maintainability.html.

Em segundo lugar, essa métrica retornará um valor que você pode usar para avaliar a qualidade do código diretamente:

Com essa métrica, você não precisa de outro código para compará-la. É por isso que é particularmente útil avaliar rapidamente a qualidade do código.

Nesta seção, passamos por muita teoria. Excelente trabalho até agora – você não vai se arrepender de aprender sobre isso com certeza porque, na próxima seção, mostraremos como coletar essas métricas usando ainda mais ferramentas PHP.

Coletando métricas em PHP

Nesta seção, queremos dar uma olhada nas ferramentas que existem no mundo PHP para coletar métricas de qualidade de código. Como você verá em breve, essas métricas não são apenas números – elas permitirão que você faça suposições sobre quanto esforço será necessário para refatorar o código. Eles também o ajudarão a identificar as partes do código que exigirão mais atenção.

Mais uma vez, selecionamos uma seleção de ferramentas para você:

phploc

Como aprendemos na seção anterior, a abreviação LOC significa linhas de código, então o nome já revela o objetivo principal desta ferramenta. Sendo uma métrica básica, ela já nos diz algumas coisas sobre uma base de código. O phploc também fornece outras métricas, como o CC, por isso vale a pena dar uma olhada mais de perto.

Instalação e uso

O autor desta ferramenta, Sebastian Bergmann, é bem conhecido pelo phpunit, o padrão de fato para testes automatizados no mundo PHP. Ele recomenda não instalá-lo usando o Composer, mas usando o phar diretamente. Discutiremos os prós e os contras dessa abordagem no próximo capítulo. Por enquanto, vamos apenas seguir o conselho do autor e baixar o phar diretamente:

```
$ wget https://phar.phpunit.de/phploc.phar
```

Isso fará o download da versão mais recente do phploc no diretório atual. Depois de baixá-lo, podemos usá-lo diretamente para digitalizar um projeto:

```
$ php phploc.phar src
```

Verificando arquivos únicos

Embora o phploc seja usado em projetos inteiros, também é possível especificar um único arquivo para escanear. Embora as medidas médias não façam sentido porque devem ser usadas em um projeto inteiro, ainda é útil se você precisar descobrir as métricas de LOC ou o CC de uma classe.

O comando anterior irá escanear a pasta src com todas as suas subpastas e

coletar informações sobre ela, que serão apresentadas diretamente na linha de comando conforme Figura 8.1:

```
$ php pholoc.phar src
pholoc 7.0.2 by Sebastian Bergmann.
```

| | |
|-----------------------------------|---------------|
| Directories | 8 |
| Files | 47 |
| Size | |
| Lines of Code (LOC) | 3349 |
| Comment Lines of Code (CLOC) | 199 (5.94%) |
| Non-Comment Lines of Code (NCLOC) | 3150 (94.06%) |
| Logical Lines of Code (LLOC) | 897 (26.78%) |
| Classes | 868 (96.77%) |
| Average Class Length | 18 |
| Minimum Class Length | 0 |
| Maximum Class Length | 117 |
| Average Method Length | 3 |
| Minimum Method Length | 0 |
| Maximum Method Length | 40 |
| Average Methods Per Class | 3 |
| Minimum Methods Per Class | 1 |
| Maximum Methods Per Class | 23 |
| Functions | 29 (3.23%) |
| Average Function Length | 1 |
| Not in classes or functions | 0 (0.00%) |
| Cyclomatic Complexity | |
| Average Complexity per LLOC | 0.22 |
| Average Complexity per Class | 4.89 |
| Minimum Class Complexity | 1.00 |
| Maximum Class Complexity | 51.00 |
| Average Complexity per Method | 2.10 |
| Minimum Method Complexity | 1.00 |
| Maximum Method Complexity | 26.00 |
| Dependencies | |
| Global Accesses | 0 |
| Global Constants | 0 (0.00%) |
| Global Variables | 0 (0.00%) |
| Super-Global Variables | 0 (0.00%) |
| Attribute Accesses | 805 |
| Non-Static | 805 (100.00%) |
| Static | 0 (0.00%) |
| Method Calls | 364 |
| Non-Static | 349 (95.88%) |
| Static | 15 (4.12%) |

Figura 8.1: Um exemplo de saída do phploc (um trecho)

Isso é muito mais informação do que apenas o LOC. As informações são divididas nas seguintes categorias:

Isso é tudo o que precisamos saber sobre a funcionalidade do phploc por enquanto. É uma ferramenta simples de usar, mas útil, que ajuda você a entender rapidamente a qualidade geral do código e a estrutura de um projeto e, portanto, deve fazer parte do seu kit de ferramentas. Porém, não informa como interpretar os números, o que requer alguma experiência.

Depende do PHP

Se houvesse um prêmio para o maior número de métricas combinadas em uma ferramenta, certamente iria para o PHP Depend (PDepend). Abrange todas as métricas que discutimos na seção anterior, além de muitas outras. No entanto, não é a ferramenta mais amigável que existe. Além disso, o site e a documentação do repositório não são perfeitos. No entanto, você deve dar uma olhada.

Instalação e uso

Como antes, esta ferramenta pode ser instalada usando o Composer ou baixando o phar diretamente. Iremos com a instalação baseada no Composer por enquanto:

```
$ composer require pdepend/pdepend --dev
```

Se não houver surpresas desagradáveis, você pode executá-lo diretamente:

```
$ vendor/bin/pdepend --summary-xml=pdepend_summary.xml src
```

Aqui, já podemos ver que o ancestral do PDepend é o JDepend, uma ferramenta de qualidade de código Java, pois a saída é gravada em um arquivo XML. O nome do arquivo é especificado usando a opção `--summary-xml`. Além disso, devemos especificar a pasta a ser verificada como um argumento.

PDepend produz alguns números, no entanto, como pode ser visto no seguinte exemplo de saída:

PDepend 2.10.3

Analizando arquivos de origem:

..... 47

Calculando as métricas de Complexidade Ciclomática:

..... 355

Calculando as métricas do Node Loc:

..... 279

Calculando as métricas de Complexidade NPath:

..... 355

Calculando métricas de herança:

..... 101

Nós pulamos algumas linhas aqui. Os números apenas informam com que frequência cada métrica foi calculada para a pasta especificada, portanto, a saída direta não é particularmente útil. Para ver as métricas reais, precisamos abrir o relatório XML. No nosso caso, o arquivo gerado chama-se `pdepend_summary.xml`.

O relatório XML é muito grande para ser impresso neste livro, então é melhor você mesmo experimentá-lo para vê-lo em toda a sua glória. No entanto, podemos mostrar-lhe como está estruturado:

```
<?xml versão="1.0" codificação="UTF-8"?>
```

```
<métricas>
```

```
<arquivos>
```



```
<file name="/path/to/Namespace/Classname.php"/>

<!-- ... -->

</arquivos>

<nome do pacote="Namespace">

<class name="Classname" fqname="Namespace\Classname">

<file name="/path/to/Namespace/Classname.php"/>

<method name="methodName"/>

<!-- ... -->

</class>

<!-- ... -->

</pacote>

</metrics>
```

O nó <metrics> representa o diretório que foi verificado em sua totalidade. Ele tem os seguintes nós filhos:

Claro, isso não é tudo que o PDepend irá gerar como saída. Para cada nó, ele adicionará dezenas de atributos, que contêm os nomes e valores das métricas calculadas. Este é um nó de exemplo de um relatório XML que foi gerado no código fonte do próprio PDepend:

```
<method name="setConfigurationFile" start="80" end="89"
```

```
ccn="2" ccn2="2" loc="10" cloc="0" eloc="8" lloc="3"  
ncloc="10" npath="2" hnt="15" hnd="21"  
hv="65.884761341681" hd="7.3125" hl="0.13675213675214"  
he="481.78231731105" ht="26.765684295058"  
hb="0.020485472371812" hi="9.0098818928795"  
mi="67.295865328327"/>
```

Você já deve ser capaz de reconhecer algumas métricas, como lloc (LOC) ou ccn (CC Number). Para os outros, você encontrará explicações, ou pelo menos os nomes longos, para as abreviações no relatório XML na documentação online:<https://pdepend.org/documentation/software-metrics/index.html>.

Mais opções

PDepend tem duas opções que você deve conhecer:

```
$ vendor/bin/pdepend --summary-xml=pdepend_summary.xml--  
exclude="Seu\Namespace,Outro\Namespace" src
```

```
$ vendor/bin/pdepend --summary-xml=pdepend_summary.xml--  
ignore="caminho/para/pasta,caminho/para/outro/pasta" src
```

Também pode gerar imagens em formato SVG com mais informações. No entanto, não os abordaremos neste livro, pois existe uma ferramenta melhor para isso, que você encontrará na próxima seção.

PDepend é poderoso, mas ao mesmo tempo difícil de supervisionar. A saída gerada é difícil de ler e, uma vez que o projeto se torna um pouco maior, torna-se inutilizável, a menos que você use outras ferramentas para analisar o arquivo XML. No entanto, você pode precisar das métricas avançadas que ele fornece um dia ou pode trabalhar em um projeto em que já esteja em uso. Então, pelo menos você está preparado agora.

PhpMetrics

Até agora, o mundo das métricas de qualidade do PHP era baseado apenas em texto. Isso vai mudar agora, pois agora vamos dar uma olhada no PhpMetrics, que vai gerar relatórios muito mais adequados para o olho humano e até mesmo interativos.

Instalação e uso

Vamos adicionar PhpMetrics ao seu projeto usando o Composer:

```
$ composer require phpmetrics/phpmetrics --dev
```

Após o download de todos os arquivos, você pode começar imediatamente a gerar seu primeiro relatório:

```
$ vendor/bin/phpmetrics --report-html=phpmetrics_report src
```

A opção `--report-html` especifica a pasta na qual o relatório será criado. Você pode especificar mais de uma pasta a ser verificada fornecendo-as como uma lista separada por vírgulas. Para nosso exemplo, no entanto, usaremos apenas a pasta `src`.

Como resultado, o PhpMetrics irá listar diversas estatísticas que já vão te contar um pouco sobre o código. A Figura 8.2 mostra um trecho da saída, que pode lembrá-lo da saída gerada pelo `phploc`:

Executing system analyzes...

Executing composer analyzes, requesting <https://packagist.org...>

LOC

Lines of code	876
Logical lines of code	762
Comment lines of code	114
Average volume	172.78
Average comment weight	14.48
Average intelligent content	14.48
Logical lines of code by class	35
Logical lines of code by method	11

Object oriented programming

Classes	22
Interface	14
Methods	69
Methods by class	3.14
Lack of cohesion of methods	0.86

Coupling

Average afferent coupling	1.09
Average efferent coupling	2.95
Average instability	0.8
Depth of Inheritance Tree	1.47

Package

Packages	7
Average classes per package	5.14
Average distance	0.18

Figura 8.2: A saída do console PhpMetrics (um trecho)

Para abrir o relatório HTML real que acabou de ser gerado, basta abrir o arquivo `index.html` nessa pasta em seu navegador. Antes de examinarmos mais de perto o relatório gerado, vamos ver quais outras opções úteis o PhpMetrics oferece primeiro:

Abrindo o navegador a partir da linha de comando

Se você estiver usando um sistema operacional baseado em Linux, como o Ubuntu, poderá abrir rapidamente um arquivo HTML na linha de comando da seguinte maneira:

```
$ firefox phpmetrics_report/index.html
```

Em alternativa, consulte o seguinte:

```
$ chromium phpmetrics_report/index.html
```

Entendendo o relatório

Se você abriu um relatório PhpMetrics pela primeira vez, encontrará uma grande variedade de informações. Não vamos nos aprofundar em todos os pequenos detalhes, mas mostraremos quais partes do relatório consideramos mais valiosas para começar.

Para ilustrar melhor o uso do PhpMetrics, escolhemos aleatoriamente um pacote de código aberto existente chamado thephpleague/container como base de código para trabalhar. É um excelente contêiner de injeção de dependência compatível com PSR-11, que tem o tamanho certo para usar como exemplo. A Figura 8.3 mostra a página de visão geral de um relatório de exemplo que geramos para ele:



PhpMetrics

- Overview
- Violations (7)
- Composer
- Size & volume
- Complexity & defects
- Object oriented metrics
- Object relations
- Coupling
- Package oriented metrics
- Package relations

Created at 2022-05-15 23:25:47 , with PHPMetrics v2.8.1 (Jean-François Lépine).

Violations (0 criticals, 2 errors)

7

[View details >](#)

Lines of code

0 →

876

[View details >](#)

Classes

0 →

22

[View details >](#)

Average cyclomatic complexity by class

0 →

5.95

[View details >](#)

Assertions in tests

No JUnit report found. Use the `--junit=<junit.xml>` option to analyse your unit tests. See [documentation of PHPUnit if needed](#)

No details

Average bugs by class

0 →

0.06

[View details >](#)

Maintainability / complexity (with comments)

[download](#)



Each file is symbolized by a circle. Size of the circle represents the Cyclomatic complexity. Color of the circle represents the Maintainability Index.

Large red circles will be probably hard to maintain.

ClassRank ([Google's page rank applied to relations between classes](#))

Page Rank is a way to measure the importance of a class. There is no "good" or "bad" page rank. This metric reflects interactions in your code.

ClassRank

0.04	League\Container\Argument	
	\LiteralArgument	
		83.42 52.74
0.01	League\Container	
	\Exception	
	\ContainerException	
		171 171
0.01	League\Container	
	\Exception	
	\NotFoundException	
		171 171

Figura 8.3: Uma visão geral do relatório PhpMetrics

Principais métricas

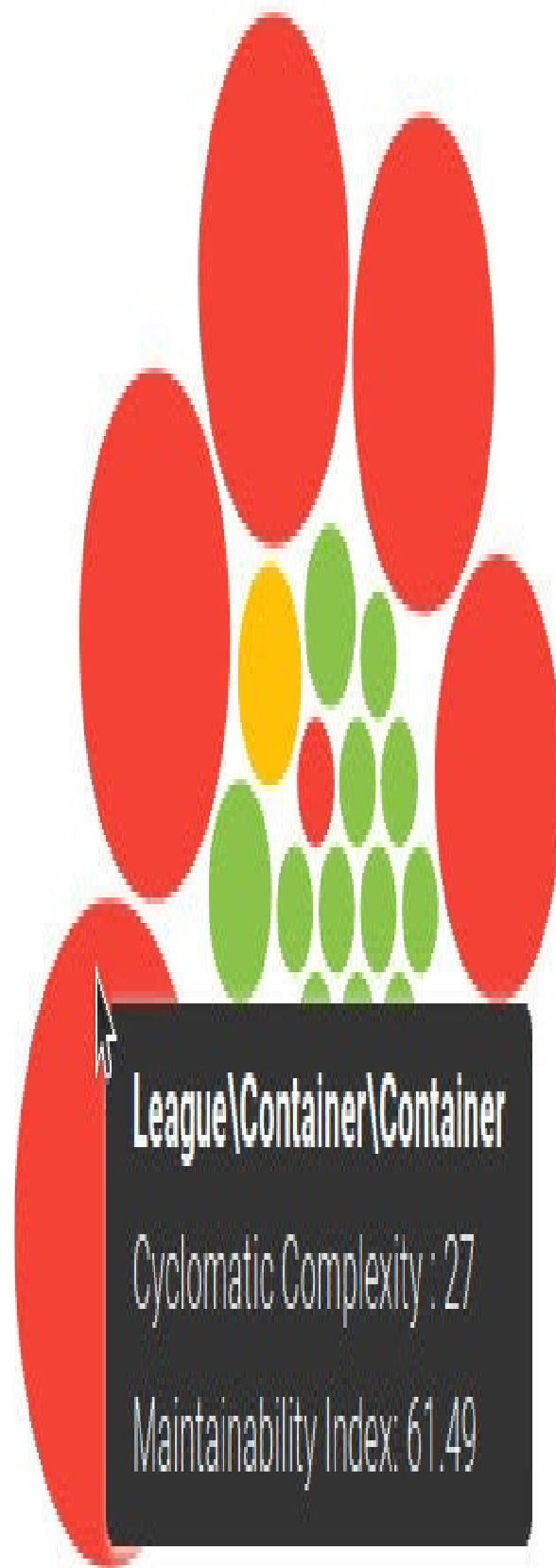
Do lado esquerdo encontra-se o menu onde pode aceder a outras páginas do relatório. A parte superior da página é preenchida com algumas métricas principais, sendo as mais interessantes:

As outras caixas também oferecem informações interessantes, mas as anteriores já são perfeitas para obter uma visão rápida das partes mais problemáticas do código.

Manutenibilidade ou complexidade

Abaixo das principais métricas, o PhpMetrics mostra um diagrama, entre outras coisas, que você certamente já viu ao abrir o relatório pela primeira vez: o gráfico de manutenibilidade/complexidade. Consiste em um círculo colorido para cada namespace do projeto, onde o tamanho do círculo representa o CC de uma classe. Quanto maior o círculo, maior a complexidade. A cor mostra o índice de manutenibilidade, variando de verde (alto) a vermelho (baixo).

Se você passar o mouse sobre um círculo, poderá ver o namespace que o círculo representa e as duas métricas em detalhes:



League\Container\Container

Cyclomatic Complexity : 27

Maintainability Index: 61.49

Figura 8.4: O gráfico de manutenibilidade/complexidade com um pop-up

Esse gráfico é extremamente útil para compreender rapidamente a qualidade geral do código – quanto menos círculos vermelhos grandes houver, melhor. Dessa forma, você pode ver facilmente as partes problemáticas do seu código.

relações de objeto

Quando você seleciona Relações de objeto no menu à esquerda, um gráfico que mostra as relações entre cada namespace aparecerá. Passar o ponteiro do mouse sobre um rótulo de texto destacará suas relações. O gráfico torna-se enorme, por isso não podemos mostrá-lo neste livro em toda a sua beleza, mas podemos pelo menos dar uma primeira impressão:

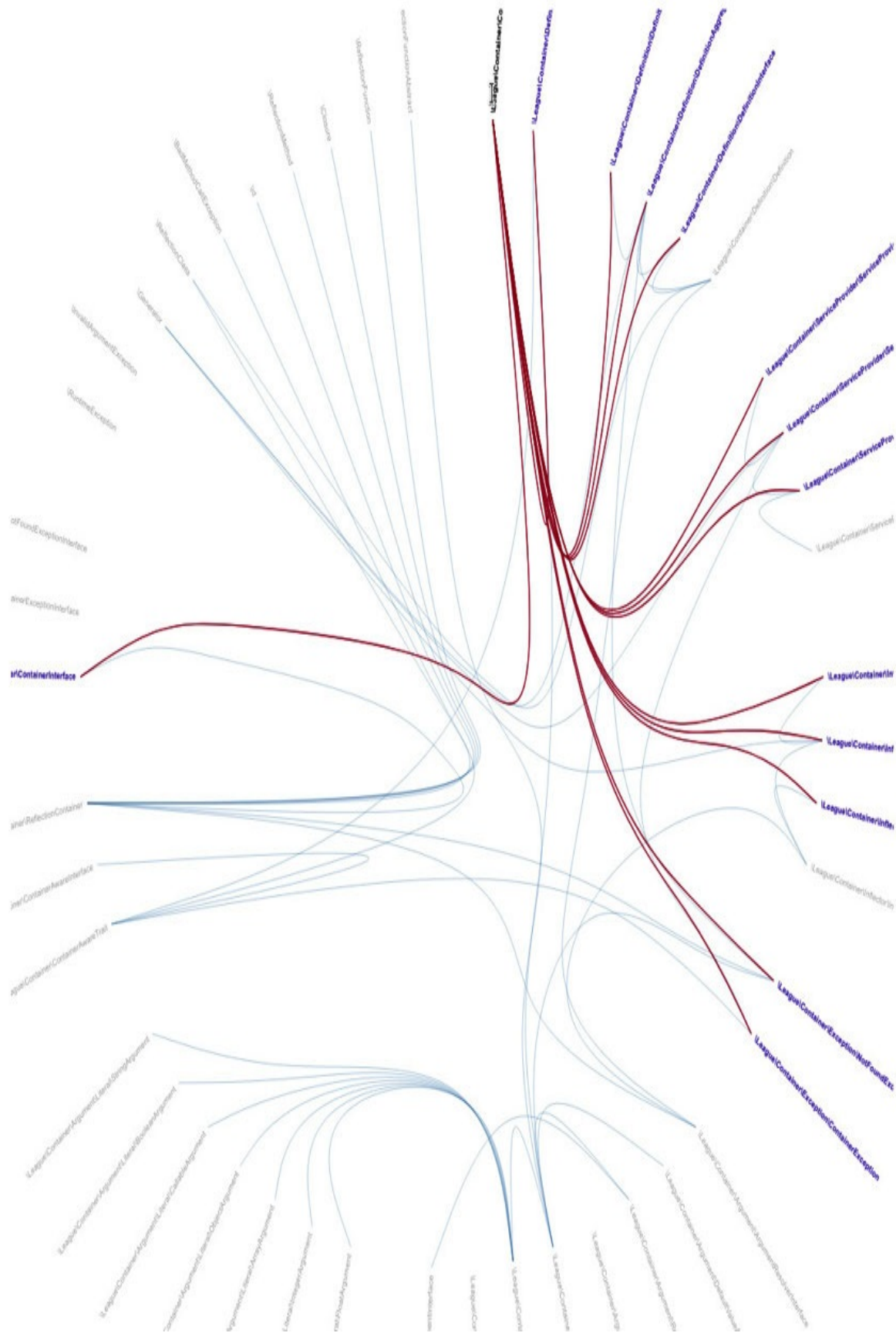


Figura 8.5: Um gráfico de relações de objetos

Acoplamento

O acoplamento de classes indica como elas dependem umas das outras. Existem duas métricas principais:

Métricas orientadas a pacotes

O último gráfico que queremos mostrar é o gráfico Abstractness vs. Instability. Como o nome já indica, mostra a relação entre a abstração e a instabilidade dos pacotes. Foi introduzido por Robert Martin e é baseado em seu trabalho em métricas orientadas a objetos. A Figura 8.6 mostra um exemplo:

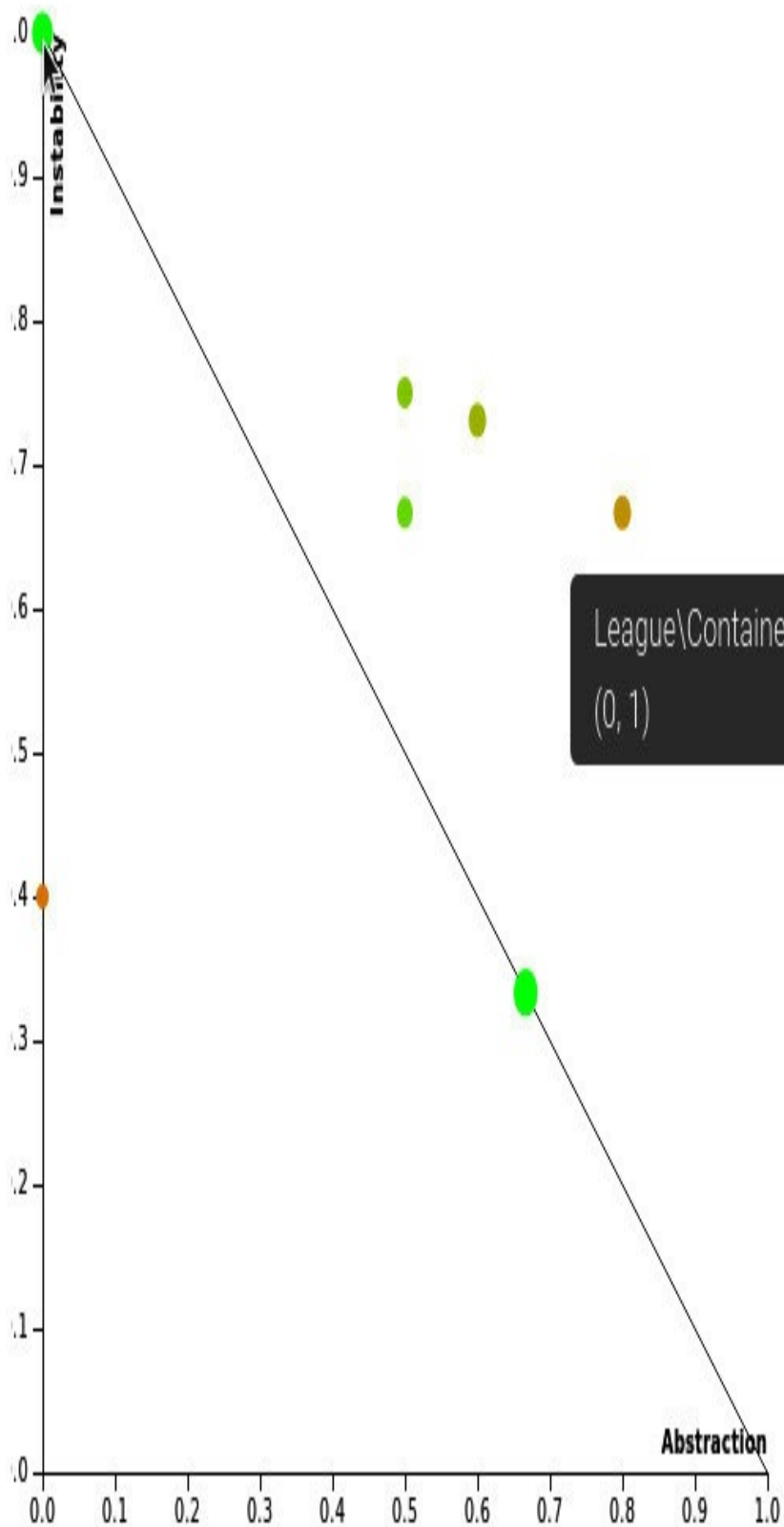


Figura 8.6: Um gráfico Abstração x Instabilidade

Mas o que exatamente esses dois termos significam no contexto do desenvolvimento de software? Vejamos as seguintes definições:

Martin afirmou que os pacotes que são estáveis e, portanto, altamente independentes de outras classes também devem ter um alto nível de A. Vice-versa, os pacotes instáveis devem consistir em classes concretas. Então, em teoria, o A de uma classe pesa seu I. Isso significa que, idealmente, A mais I deveria ser 1 ($A + I = 1$). Essa equação também desenha a linha angular do canto superior esquerdo ao canto inferior direito do gráfico. Você deve se esforçar para manter seus pacotes perto da linha.

No relatório propriamente dito, você encontrará uma tabela abaixo do gráfico onde encontrará os valores com mais detalhes. Se você passar o ponteiro do mouse sobre um círculo, aparecerá um pop-up informando o nome da classe que o círculo representa, bem como o A (o primeiro dígito) e o I (o segundo dígito).

Outra informação

Isso encerra nosso tour pelo PhpMetrics. Há muito mais para descobrir, como, por exemplo, o ClassRank, onde o famoso algoritmo PageRank do Google é usado para classificar as classes de acordo com sua importância (ou seja, o número de interações que elas têm com outras partes do código). Não podemos cobrir tudo neste livro – no entanto, agora você já conhece muitas das métricas. Sua documentação é bastante útil para você. Você encontrará um link para ele

em todas as páginas no canto superior direito.

Os prós e contras do uso de métricas

Nos dois capítulos anteriores deste livro, você aprendeu sobre muitas ferramentas e métricas que existem apenas para ajudá-lo a escrever um software melhor. O conhecimento, a sabedoria e as incontáveis horas de trabalho de centenas, senão milhares, de engenheiros de software podem ser adicionados ao seu projeto em questão de minutos.

O outro lado da moeda é que você já pode se sentir completamente sobrecarregado com o grande número de possibilidades. Quais ferramentas você deve escolher? Em quais métricas você deve se concentrar no futuro?

Se você já tem essa sensação, não se preocupe. Não vamos deixá-lo sozinho em toda essa confusão, mas ajudá-lo a encontrar uma configuração que atenda às suas necessidades durante os próximos capítulos. Para começar, vamos analisar os prós, mas também os contras, do uso de métricas de qualidade de código.

Os prós

Primeiro, cada projeto de software é um trabalho único. Ele cresce com base em certas circunstâncias, como os conjuntos de habilidades dos desenvolvedores e os pacotes ou estruturas disponíveis naquele momento, mas também fatores externos, como prazos, que muitas vezes afetam negativamente a qualidade do código.

As métricas de código ajudam você a obter uma visão geral do estado atual em que um projeto se encontra. Se você, por exemplo, assume um projeto feito por um ex-membro da equipe, deseja saber o que o espera. Tendo uma ideia da qualidade do código, você pode ajustar imediatamente seus esforços estimados em tickets futuros em qualquer direção.

As métricas de qualidade do código também ajudam a entender onde o código precisa ser melhorado. É um excelente treinamento para refatorar seu código e, usando as métricas, você sabe quando obteve sucesso. Independentemente de você estar trabalhando em seu próprio projeto de estimação, deseja contribuir para um projeto de código aberto ou trabalha em equipe, é sempre uma boa conquista finalmente obter mais algumas luzes verdes nos relatórios.

Se você encontrou um trecho de código que precisava urgentemente de refatoração por um motivo válido, mas seu gerente de projeto não queria que você o fizesse, você pode usar as métricas para mostrar a eles como as coisas são terríveis e que é apenas um julgamento baseado em seu próprio opinião. As métricas de código são imparciais e (dolorosamente) honestas.

Por fim, outro caso de uso importante dessas métricas é impedir que você escreva códigos ruins. Às vezes, pode ser um pouco chato escrever um código que siga todas essas regras, mas tenha certeza de que o esforço valerá a pena.

Os contras

Anteriormente, dissemos que os deadlines podem prejudicar a qualidade do código porque eles nos impedem de refatorar code smells ou adicionar mais testes. Embora isso seja verdade, devemos estar cientes de que, uma vez que começam medindo a qualidade de seu código, alguns desenvolvedores começam a refatorar muito mais código do que o necessário porque são recompensados com melhores métricas. Por que isso é problemático?

Por exemplo, imagine que há uma classe em seu projeto atual que possui um baixo índice de manutenibilidade, alta complexidade NPath e, apenas olhando para ela, você pode ver imediatamente como ela é ruim. No entanto, ele amadureceu com o tempo, sendo corrigido com tanta frequência que, em algum momento, provou funcionar sem bugs. Agora, suas ferramentas dizem que esta classe é de má qualidade. Você ainda deve pular nele e começar a refatorá-lo?

Não há, é claro, um claro sim ou não. Como mencionado anteriormente, se você trabalha com código em seu tempo livre, faz sentido (e também é divertido) refatorar uma classe para remover a maioria dos odores de código. Se você estiver trabalhando em projetos comerciais, por exemplo, trabalhando como engenheiro de software para viver, nem sempre terá tempo para fazê-lo. Existem bugs para espremer, que deixam os usuários do seu software insatisfeitos, enquanto, por outro lado, existem recursos para implementar, pelos quais eles estão esperando desesperadamente. No geral, são os clientes satisfeitos que pagam sua conta. Encontrar o ponto ideal entre a velocidade de desenvolvimento e a qualidade do código nunca é fácil - você só precisa estar ciente de que às vezes é necessário tomar a pílula amarga e deixar o código ruim de lado por enquanto.

Não use métricas para competir com colegas, ou pior, para falar mal de ex-desenvolvedores que te deixaram sozinho com o projeto. Esteja ciente de que todos trabalham da melhor maneira possível, com base em suas habilidades. Ninguém tenta deliberadamente escrever código ruim - muitas vezes, isso acontece porque os desenvolvedores nunca ouviram falar sobre os princípios de codificação limpa ou estavam sob tanta pressão de tempo que tiveram que copiar e colar a codificação para deixar seus gerentes ou clientes satisfeitos. Seu ambiente de trabalho deve ser um local de respeito, ajuda e tolerância, não de competição.

Resumo

Este capítulo apresentou algumas das métricas de qualidade de código mais usadas no mundo do PHP. Além disso, apresentamos as ferramentas que ajudam a reuni-los. Claro, há muito mais que não poderíamos cobrir neste livro, mas você não precisa conhecê-los todos – agora você está equipado com uma sólida compreensão das métricas de qualidade de código que o ajudarão em sua rotina diária de trabalho.

Ferramentas e métricas de qualidade de código certamente não são bala de prata para todos os problemas. Por um lado, eles podem ser extremamente úteis para melhorar seu código. Por outro lado, você não deve tomá-los como a medida final. Existem inúmeros exemplos de tipos de software bem-sucedidos que nunca passariam nessas verificações de qualidade, como o WordPress. Certifique-se, porém, de que os criadores do WordPress teriam feito as coisas de maneira diferente se soubessem de antemão.

No próximo capítulo, deixaremos o reino da teoria. Aprenderemos como organizar as ferramentas que introduzimos nos dois últimos capítulos em nossos projetos. Cada projeto é único, por isso oferecemos diferentes sabores para atender às suas necessidades.

Leitura adicional

Organizando as Ferramentas de Qualidade do PHP

Nos últimos dois capítulos, você aprendeu muito sobre métricas de qualidade e como medi-las. Certamente haverá algumas ferramentas que você gostaria de usar em seu futuro ambiente de trabalho, e essas ferramentas funcionam melhor se estiverem perfeitamente integradas para que você nem precise mais pensar em usá-las.

Portanto, neste capítulo, mostraremos como você pode organizar essas ferramentas de forma que sejam mais produtivas e úteis em seu trabalho diário. Isso inclui os seguintes tópicos:

Requerimentos técnicos

Se você seguiu os exemplos dos dois capítulos anteriores, não precisa instalar mais nada. Caso contrário, volte a esses capítulos e instale primeiro todas as ferramentas necessárias.

Todos os exemplos de código podem ser encontrados em nosso repositório GitHub:<https://github.com/PacktPublishing/Clean-Code-in-PHP>.

Instalando ferramentas de qualidade de código usando o Composer

A maioria dos projetos PHP: Hypertext Preprocessor (PHP) atualmente usa o Composer por um bom motivo. Antes de entrar no mundo do PHP em 2012, manter todas as dependências externas (ou seja, código de outros desenvolvedores) atualizadas exigia muito trabalho manual. Os arquivos necessários tiveram que ser baixados dos sites correspondentes e adicionados às pastas corretas do projeto. O carregamento automático (ou seja, a resolução automática de caminhos de arquivo a partir do nome da classe) não era padronizado, se é que estava disponível. Então, geralmente, as classes desejadas precisam ser importadas ativamente usando `require()` ou `require_once()`. Se houvesse algum conflito entre as versões do pacote, você mesmo teria que resolver os problemas de alguma forma.

O Composer simplificou muito esses esforços resolvendo esses problemas. Ele introduziu um repositório central chamado Packagist (<https://packagist.org>), onde todos os pacotes disponíveis são hospedados. Além disso, corrigiu o problema de versão introduzindo restrições de versão. Essas são regras que informam ao Composer quais versões de outros pacotes um pacote suporta, permitindo que o Composer resolva automaticamente as versões corretas. Outro recurso inovador foi o suporte de carregamento automático para pacotes instalados. Lá, agora normalmente usamos `require()` apenas para importar o autoloader do Composer.

Todos esses recursos ajudaram o PHP a competir com outras linguagens da Web, como Python ou Ruby, e sem ele, o PHP provavelmente não seria mais a linguagem mais usada na World Wide Web (WWW), como é hoje. Portanto, queremos dar ao Composer o espaço que ele merece neste livro. Nesta seção,

mostraremos o método de instalação mais usado. Além disso, também veremos outra maneira menos conhecida de usar o Composer em seu projeto.

Instalando ferramentas de qualidade de código usando require-dev

Ao longo dos últimos capítulos, já usamos o Composer para instalar ferramentas na maioria das vezes, então você já deve estar familiarizado com o caso de uso mais comum que existe: adicionar dependências ao seu projeto. Dependências são pacotes de código escritos por outros desenvolvedores que podem ser rapidamente integrados ao seu projeto.

Para recapitular, isso é feito usando a palavra-chave `require` e o nome do pacote. Por exemplo, se você deseja adicionar `PhpMetrics`, pode fazê-lo executando o seguinte comando:

```
$ composer require phpmetrics/phpmetrics --dev
```

Normalmente, os pacotes são identificados pelo nome do(s) desenvolvedor(es) - o chamado fornecedor - e, separados por uma barra, o nome do pacote. No exemplo anterior, o fornecedor e o nome do pacote são idênticos, mas nem sempre é o caso.

Vamos ver a opção `--dev` com mais detalhes agora. Quando executamos o comando `composer require` com esta opção, o Composer irá adicionar o pacote em outra seção do arquivo `composer.json`, chamada `require-dev`. Aqui, você pode ver um trecho de um arquivo `composer.json` típico:

```
{  
  "nome": "fornecedor/pacote",  
  
  ...  
  
  "exigir": {  
    "doutrina/dbal": "^2.10",  
    "monólogo/monólogo": "^2.2",  
  
    ...  
  },  
  
  "exigir-dev": {  
    "phpunit/phpunit": "^9.5",  
    "phpmetrics/phpmetrics": "^2.8",  
  
    ...  
  },  
  
  ...  
}
```

A ideia por trás da seção require-dev é que todos os pacotes dentro desta seção não são necessários para rodar o aplicativo em produção. No ambiente local, ou durante a construção, você certamente precisará do PHPUnit e de todas as nossas queridas ferramentas de qualidade de código; na produção, eles não são mais necessários.

Na verdade, você deve se esforçar para ter o menor número possível de pacotes em produção. Isso ocorre principalmente por dois motivos, a seguir:

Cada pacote adicionado será incluído no mecanismo de carregamento automático do Composer, que custa desempenho a cada solicitação. Internamente, o Composer constrói o chamado classmap, que é um array simples que mapeia um nome de classe para o respectivo local do arquivo. Se você está curioso sobre isso, confira, por exemplo, o arquivo `vendor/composer/autoload_classmap.php`. Dependendo do número de pacotes que seu projeto usa, esse arquivo pode ficar enorme, deixando seu aplicativo lento.

Cada pacote adicional pode introduzir problemas de segurança. Quanto menos código houver, menos vetores de ataque haverá.

Por padrão, o Composer instalará todas as dependências. Portanto, certifique-se de executá-lo usando a opção `--no-dev` para suas compilações de produção para excluir os pacotes em `require-dev` da instalação. No entanto, em seu ambiente local, você não precisa se preocupar com mais nada neste momento.

O método de instalação descrito anteriormente é o que você mais encontrará por bons motivos: não requer nenhuma ferramenta adicional e há apenas uma opção adicional a ser usada ao instalá-lo na produção. Isso o torna um ponto de partida perfeito e geralmente é totalmente suficiente para um pequeno projeto. Outra abordagem que vale a pena conhecer é a instalação global do Composer, que discutiremos na próxima seção.

instalação global

Se você estiver trabalhando em vários projetos simultaneamente em seu sistema local, poderá optar por instalar o Composer e os pacotes globalmente, o que significa que eles não serão instalados em nenhuma pasta raiz do projeto e, portanto, não serão adicionados a nenhum arquivo `composer.json` lá. Em vez disso, tanto o Composer quanto os pacotes serão instalados em uma única pasta, que geralmente é `~/.composer`. Nessa pasta, você encontrará outro arquivo `composer.json` que rastreia os pacotes instalados globalmente, bem como outra pasta `vendor`, onde o código deles está instalado.

Instalar pacotes globalmente requer simplesmente adicionar o modificador global, assim:

```
$ composer global require phpmetrics/phpmetrics
```

Da mesma forma, atualizar todos os pacotes globais também é fácil, conforme demonstrado aqui:

```
$ atualização global do compositor
```

Após a instalação global, ferramentas como o PHP Coding Standards Fixer (PHP-CS-Fixer) podem ser simplesmente executadas sem a necessidade de especificar o caminho, assim:

```
$ php-cs-fixer fix src
```

Para fazer essa abordagem funcionar, no entanto, você precisará adicionar essa pasta global ao caminho de execução. Consulte a documentação do Composer (<https://getcomposer.org/>) para obter mais detalhes sobre como fazer isso para o sistema operacional que você usa.

O uso do recurso de instalação global só deve ser escolhido se você estiver trabalhando sozinho em seus projetos e não usar nenhum pipeline de construção. Se você estiver trabalhando em equipe e/ou usar um pipeline de integração contínua (CI), deverá instalá-lo para cada projeto separadamente.

De acordo com as melhores práticas comuns, como os princípios do Twelve-Factor App (<https://12factor.net>), todas as dependências devem ser declaradas explicitamente e nenhuma dependência global deve ser considerada, pois você nunca pode ter certeza de qual versão será instalada. Embora os pacotes de ferramentas de qualidade de código não façam parte do código do programa real, eles ainda fazem parte do processo de construção. Mesmo pequenas diferenças entre as versões instaladas podem levar a comportamentos imprevistos e podem gerar confusão quando os erros não podem ser reproduzidos localmente.

Além disso, você deseja tornar a instalação inicial de um projeto o mais fácil possível. Fazer com que seus colegas de equipe instalem todas as ferramentas necessárias manualmente é um processo demorado e sujeito a erros que pode levar à frustração.

Pelas razões acima mencionadas, não encorajamos o uso do método de instalação global.

Scripts do compositor

Depois de decidir sobre uma possível maneira de instalar o Composer e usá-lo para baixar as ferramentas desejadas, você deseja começar a usá-las da maneira mais direta possível. DentroCapítulo 11, Integração Contínua, onde falaremos sobre CI, também mostraremos como executar essas ferramentas automaticamente durante o processo de build. Por enquanto, porém, queremos mostrar como o Composer pode ajudá-lo a executá-los manualmente sempre que necessário.

Vamos considerar o seguinte exemplo: como primeiro passo, gostaríamos de executar o PHP-CS-Fixer para corrigir automaticamente o código em nossa pasta src. Posteriormente, desejamos executar o PHPStan com nível 1 em nosso código também. Certamente você poderia executar as duas etapas separadamente, mas queremos adicionar um pouco mais de conforto e executar as duas ferramentas de uma só vez.

Para conseguir isso, podemos utilizar a seção de scripts do arquivo composer.json na raiz do projeto. Lá, temos que adicionar as ferramentas que queremos executar sob um nome de comando conciso, como analise. O exemplo a seguir mostra como isso poderia ser:

```
{
```

```
...
```

```
"scripts": {
```

```
"analisar": [
```

```
"origem de correção do fornecedor/bin/php-cs-fixer",  
"fornecedor/bin/phpstan analise --level 1 src"  
]  
}  
}
```

Usamos a notação de matriz JavaScript Object Notation (JSON) aqui para adicionar cada comando em uma linha separada, o que torna mais fácil de ler e manter do que escrever tudo em uma linha.

Se você deseja compartilhar esses comandos do Composer, também pode adicionar um texto de descrição curta, que é exibido quando você executa a lista de compositores para ver uma lista de comandos disponíveis. Para fazer isso, você precisa adicionar a seção `script-descriptions` ao seu arquivo `composer.json`. Para o comando de análise introduzido anteriormente, poderia ser assim:

```
{  
...  
"scripts": {  
...  
},  
"scripts-descrições": {  
"analyze": "Executar limpeza e análise de código"
```

```
}
```

```
}
```

Ao instalar as ferramentas em uma subpasta, encontramos uma maneira adequada de organizar nossas ferramentas de qualidade de código sem permitir que elas interfiram nas dependências de nossos aplicativos. Mas e se, por qualquer motivo, você não estiver usando o Composer em seu projeto ou não gostar do fato de ter dois arquivos `composer.json` em seu repositório? Na próxima seção, apresentaremos uma forma alternativa que não faz uso do Composer.

Instalando ferramentas de qualidade de código como arquivos phar

O Composer não é a única maneira possível de adicionar ferramentas de qualidade de código ao seu projeto. Nesta seção, mostraremos como adicionar as ferramentas como arquivos phar.

Já encontramos phar nos capítulos anteriores. Significa PHP Archive e pode ser considerado um projeto PHP inteiro dentro de um único arquivo. Tecnicamente, é um arquivo ZIP autoexecutável que contém o código-fonte do aplicativo, além de todas as dependências necessárias. A grande vantagem é que o código necessário está disponível imediatamente após o download, para que você possa usar instantaneamente qualquer arquivo phar sem precisar se preocupar com o Composer ou dependências. Além disso, os arquivos phar são suportados por todas as versões modernas do PHP.

Isso torna o uso de arquivos phar bastante útil, pois você pode tratá-los como binários. Normalmente, você pode baixar as muitas ferramentas PHP que apresentamos a você até agora como arquivos phar diretamente e colocá-los em qualquer diretório que desejar. No entanto, não há uma maneira unificada de fornecer esses arquivos; portanto, consulte a documentação oficial de cada ferramenta.

Vamos dar uma olhada em como fazer isso para a ferramenta `phploc`, que introduzimos em [Capítulo 7, Ferramentas de qualidade de código](#). De acordo com seu repositório GitHub, você pode simplesmente baixá-lo no site do `PHPUnit`, pois ambos são do mesmo autor. O trecho de código a seguir mostra como você

pode fazer isso:

```
$ wget https://phar.phpunit.de/phploc.phar -O phploc
```

Observe que instalamos a ferramenta com o nome `phploc` e não `phploc.phar`. A opção `-O` permite especificar um nome de arquivo diferente daquele que você está baixando. A extensão `.phar` não é necessária para executar a ferramenta, então você pode economizar um pouco de esforço de digitação aqui.

Phar e somas de verificação

Baixar e executar arquivos da Internet sempre inclui o risco de que eles possam ser corrompidos e infectados por códigos maliciosos. É por isso que os autores das ferramentas costumam gerar somas de verificação (por exemplo, por meio de algoritmos de hash, como Secure Hash Algorithm 256 (SHA256)) dos downloads e publicá-los em seus sites para que você possa usá-los para verificar a integridade do download. Verifique os sites oficiais das ferramentas que você está pensando em usar para descobrir se elas oferecem somas de verificação e como verificá-las.

Claro, você pode baixá-lo usando qualquer método que desejar, seja `curl` ou pelo navegador. Depois de baixá-lo, você pode executá-lo imediatamente usando sua instalação PHP local, assim:

```
$ php phploc src
```

Se você não quiser digitar php toda vez, você precisa tornar o arquivo phar executável, que no Linux, por exemplo, ficaria assim:

```
$ chmod +x phploc
```

Depois, basta executar o seguinte comando para executar o phploc:

```
$ ./phploc src
```

Mantendo seus arquivos phar organizados

Agora, não queremos apenas baixar os arquivos phar - também queremos mantê-los organizados em nosso projeto para que qualquer outro desenvolvedor não precise fazer nenhum trabalho manual antes de usá-los. A escolha mais óbvia é adicionar esses arquivos ao seu repositório, e é exatamente isso que veremos agora. No exemplo a seguir, usaremos o Git, mas essa abordagem funcionaria com qualquer outro sistema de controle de versão (VCS).

Geralmente não é recomendável armazenar arquivos grandes no Git porque eles podem afetar negativamente o desempenho. O GitHub, por exemplo, bloqueia arquivos com mais de 100 megabytes (MB). No entanto, os arquivos phar que usamos geralmente têm apenas alguns MB de tamanho, portanto, adicioná-los não deve ter efeitos colaterais negativos.

Git Armazenamento de Arquivos Grandes (Git LFS)

Se você precisar armazenar arquivos grandes no Git, considere usar o Git LFS, que foi projetado exatamente para esse caso de uso. Para nossas necessidades, porém, não precisamos usá-lo.

Você pode escolher livremente onde adicionar arquivos phar ao seu projeto. Um lugar comum é a pasta raiz; no entanto, como isso ficará muito cheio com o tempo, recomendamos o uso de uma pasta separada para armazená-los. Um bom lugar seria a pasta de ferramentas novamente, assim como usamos na seção anterior. Você não precisa considerar mais nada; basta adicioná-los ao repositório

como qualquer outro arquivo.

Vamos supor que você copiou o arquivo `phploc` para a pasta de ferramentas e o tornou executável conforme descrito anteriormente. Então, você apenas executaria da seguinte maneira:

```
$ ferramentas/phploc src
```

O uso de arquivos `phar` é fácil e não interfere nas dependências do aplicativo. No entanto, eles não são perfeitos: se você quiser atualizá-los, você precisa procurar o URL de download Uniform Resource Locator, baixar o arquivo `phar` e validar sua soma de verificação manualmente todas as vezes - para cada ferramenta. Na próxima seção, mostraremos como facilitar esse processo apresentando outra ferramenta de gerenciamento de dependência: `Phive`.

Gerenciando arquivos phar usando Phive

Na seção anterior, aprendemos como usar arquivos phar em vez de usar o Composer para instalar nossas ferramentas de qualidade de código. Essa abordagem funciona bem, mas requer algum trabalho extra caso você queira atualizá-los.

Phive é uma ferramenta que assume esse trabalho extra. Vamos instalá-lo imediatamente.

Naturalmente, o próprio Phive também pode ser baixado como phar. Os seguintes comandos irão baixá-lo sob o nome phive e torná-lo executável:

```
$ wget https://github.com/phar-io/phive/releases/download/0.15.1/phive-0.15.1.phar -O phive
```

```
$ chmod +x phive
```

Observe que este método de instalação não é muito seguro. Verifique o site da ferramenta (<https://phar.io>) para saber como instalá-la com segurança e como disponibilizá-la globalmente.

Para fins de demonstração, o download simples funciona muito bem. Uma vez que o arquivo é baixado e se torna executável, você pode começar a usar Phive

diretamente para instalar as primeiras ferramentas. Vamos usar o `phploc`, que introduzimos no capítulo anterior, para demonstrar como funciona, da seguinte forma:

```
$ ./phive instala o phploc
```

Baixar verificação

Phive não só se encarrega da instalação como também da verificação dos downloads. Isso é feito automaticamente durante o processo de instalação. No entanto, isso exige que o fornecedor disponibilize as somas de verificação, que também é a principal razão pela qual nem todas as ferramentas podem ser gerenciadas por meio do Phive.

Como você viu anteriormente, a instalação de uma ferramenta é feita apenas com o comando `install`. As quatro coisas a seguir aconteceram agora:

Phive baixou a versão mais recente do `phploc` e verificou sua soma de verificação.

O arquivo `phar` foi armazenado em uma pasta compartilhada (geralmente localizada em sua pasta pessoal sob o nome `.phive`).

Phive então criou um link simbólico para essa pasta compartilhada. Um link

simbólico é uma referência no sistema de arquivos para que um arquivo ou diretório possa aparecer em vários diretórios, embora seja armazenado em apenas um local. Por padrão, este link simbólico é armazenado na pasta de ferramentas, que será gerada caso não exista.

Outra pasta `.phive` foi criada na pasta raiz do seu projeto, que é usada para armazenar as informações sobre quais ferramentas foram baixadas.

Os links simbólicos aparecem apenas como executáveis “reais” em seu diretório, enquanto o arquivo original permanece em um único local. Se você não quiser usar links simbólicos, poderá instalar uma cópia de arquivo usando a opção `--copy`.

Após a instalação, a execução do `phploc` é simples, como podemos ver aqui:

```
$ ferramentas/phploc src
```

Phive oferece comandos mais úteis. Basta executar o seguinte código sem nenhum comando para obter uma lista deles:

```
$ ./cinco
```

Aqui, apresentamos os mais importantes:

Adicionando Phive ao seu projeto

Se você trabalha em equipe, não deseja apenas instalar localmente os arquivos phar. Phive tem você coberto aqui também. Duas etapas são necessárias para adicionar corretamente o Phive ao seu projeto, conforme a seguir:

Adicione a pasta `.phive` na raiz do seu projeto ao seu repositório. O arquivo `phars.xml` contém todas as informações necessárias (como o arquivo `composer.lock`).

Certifique-se de que a pasta de ferramentas não esteja sob controle de versão (por exemplo, usando um arquivo `.gitignore`). Você explicitamente não deseja adicionar os próprios arquivos phar ao seu repositório.

Feito isso, na próxima vez que o projeto for retirado do repositório, as ferramentas podem ser instaladas executando o seguinte comando:

```
$ ./phive install
```

Esse comando pode ser facilmente integrado a outros fluxos de trabalho, por exemplo, como um script `post-install-cmd` adicional no arquivo `composer.json`.

Isso já é tudo o que você precisa saber sobre o Phive para começar a usá-lo. Como de costume, recomendamos que você leia a documentação oficial porque não podemos cobrir todos os recursos que ela fornece neste livro.

Resumo

O Composer é uma parte indispensável do mundo PHP atual. A abordagem usual para adicionar ferramentas de qualidade de código ao seu projeto é adicioná-las à seção require-dev das dependências, o que funciona bem em muitos casos.

No entanto, o Composer não é a única maneira que existe. Portanto, neste capítulo, apresentamos mais duas opções para gerenciar suas ferramentas de qualidade de código: adicionando os arquivos phar manualmente ao seu projeto ou utilizando o Phive para gerenciar os arquivos phar.

Você provavelmente está ansioso para aplicar todo o seu conhecimento adquirido ao seu código agora. No entanto, a refatoração implacável pode fazer mais mal do que bem, e clicar em todas as partes do seu aplicativo após cada alteração para verificar se algo está quebrado custará muito tempo e pode ser muito frustrante. Assim, no próximo capítulo, mostraremos como os testes automatizados podem ajudá-lo aqui.

Teste automatizado

Se você leu todos os capítulos deste livro desde o início, terá não apenas uma ideia do embasamento teórico, mas também um bom conjunto de ferramentas à mão que o ajudarão a escrever um ótimo PHP: Hypertext Preprocessor (PHP) código. Claro, você pode simplesmente refatorar todo o código existente, provavelmente usando alguns recursos de manipulação de código automatizado que nossas ferramentas oferecem.

Você não escreverá um código perfeito na primeira tentativa - geralmente são necessárias várias iterações até que você esteja satisfeito. E como você nunca para de aprender, refatorará partes do seu código mesmo meses ou anos depois. No entanto, mesmo as ferramentas de qualidade de código mais sofisticadas não o impedirão de realizar uma tarefa tediosa: testar seu código depois de fazer alterações para garantir que ele ainda funcione conforme o esperado. É por isso que, neste capítulo, queremos apresentá-lo aos testes automatizados.

Por meio de testes automatizados, você poderá verificar se suas melhorias no código não quebraram sua funcionalidade de maneira rápida e confiável. Este é um dos pilares da escrita de código limpo, pois permite refatorar o código com confiança.

O tópico de testes automatizados merece um ou dois livros inteiros, então podemos apenas arranhar a superfície aqui. No entanto, como estamos convencidos de que você se beneficiará muito com isso em seu trabalho diário, esperamos que este capítulo o desperte o desejo de aprender mais sobre esse interessante tópico.

As seções a seguir fornecerão uma boa visão geral:

Requerimentos técnicos

Além dos requisitos técnicos dos capítulos anteriores, você precisará instalar a extensão Xdebug PHP. Forneceremos mais informações sobre esse tópico na seção correspondente, Sobre a cobertura do código, posteriormente neste capítulo.

Os arquivos de código para este capítulo podem ser encontrados aqui:<https://github.com/PacktPublishing/Clean-Code-in-PHP>

Por que você precisa de testes automatizados

Embora PHPUnit, a estrutura padrão de teste de unidade para PHP, exista desde 2006, testes automatizados ainda não são usados em todos os projetos PHP atualmente. Muito potencial é desperdiçado aqui porque os testes automatizados têm muitos benefícios, como os seguintes:

TDD

TDD é uma forma de desenvolver código que combina a escrita de testes e o código real ao mesmo tempo. A ideia básica é simples e muitas vezes referida como vermelho/verde/refator: antes de escrever qualquer código para um novo recurso ou até mesmo uma correção de bug, o primeiro passo é escrever um teste que verifique o resultado esperado. Como você ainda não tem nenhum código escrito, os testes falharão (indicado pela cor vermelha). Na segunda etapa, você escreve o código, sem se preocupar muito em deixá-lo perfeito, até que os testes passem (verde). Como agora você já tem testes funcionando, pode facilmente melhorar (refatorar) o código até ficar satisfeito.

O paradigma TDD garante que você terá todo o seu código coberto com testes e que o código já está escrito de forma totalmente testável. Porém, não leve as coisas muito a sério: há momentos em que você só quer experimentar sem ter um objetivo claro em mente — por exemplo, quando brinca com uma nova interface de programação de aplicativo (API). Nesse caso, você não precisa seguir o TDD.

Refatoração mais fácil com testes

Se você iniciar um projeto no verde (ou seja, escrevê-lo do zero), poderá ter o conforto de obter feedback imediato de suas ferramentas de qualidade de código assim que começar a codificá-lo. Esta é uma grande ajuda, mas mesmo as melhores ferramentas não o impedirão de tomar más decisões e escrever códigos que você desejará desfazer em algum momento.

Isso acontece com todo mundo e não deve desencorajá-lo. Você aprende algo novo todos os dias e, à medida que suas habilidades pessoais evoluem, seu código também. Se você observar seu código de 1 ano atrás, provavelmente desejará refatorá-lo imediatamente.

E claro, não apenas suas habilidades, mas todo o ecossistema PHP melhora constantemente. Muitas coisas que são padrão hoje eram simplesmente inexistentes no passado. Novos pacotes ou recursos de linguagem estão constantemente sendo introduzidos e você deseja usá-los em seus projetos para não ficar preso às velhas técnicas para sempre.

Portanto, o código muda com o tempo - isso é completamente normal e nós, como desenvolvedores, devemos aceitar a mudança; nenhuma parte do nosso código jamais será final. Referimo-nos à alteração do código existente como refatoração. A parte interessante da refatoração é que o código é alterado, mas o software parece inalterado para o usuário. Todo o trabalho acontece “sob o capô”. Se você, por exemplo, atualizou a estrutura do seu projeto para a versão mais recente e os usuários não perceberam mudanças diretas, então você fez bem o seu trabalho.

A refatoração tem benefícios; caso contrário, não o faríamos. Se bem feito, pode levar, por exemplo, a um melhor desempenho, maior segurança ou geralmente permite que um aplicativo seja escalável na nuvem. E, no entanto, a refatoração geralmente vem com uma conotação ruim. Os gerentes, em particular, tendem a pensar que a refatoração significa alterar o código apenas porque há outro hype no mundo do desenvolvimento da Web que os engenheiros desejam seguir e horas preciosas de trabalho são desperdiçadas.

Sejamos honestos: claro, isso também acontece. A linha é muitas vezes difícil de traçar. Por exemplo, imagine que seu dever seja manter um aplicativo PHP antigo, mas perfeitamente funcional, que usa o padrão Singleton para instanciação de objetos. Se você só precisa fazer pequenas alterações ocasionalmente, não há necessidade de refatorá-lo para usar DI. No entanto, se você precisar implementar mudanças contínuas, como adicionar novos módulos e testes para isso, pode ser uma boa escolha fazê-lo.

Freqüentemente, você terá que justificar seu trabalho de refatoração. Em vez disso, é útil falar sobre a manutenção da integridade do sistema no código. Todo mundo concorda com o fato de que as máquinas precisam de manutenção: as peças são substituídas, o lubrificante deve ser renovado e assim por diante. No entanto, por algum motivo, nosso software deve funcionar para sempre.

Tendo bons argumentos para refatoração agora, queremos entender como o teste pode nos ajudar aqui. Para conseguir isso, vamos dar uma olhada mais de perto nos diferentes tipos de teste que existem na próxima seção.

Tipos de testes automatizados

Embora os testes de unidade sejam provavelmente o tipo mais conhecido de teste automatizado, há mais a descobrir. Nesta seção, apresentaremos os tipos de teste mais comuns (e importantes). Um conceito de teste bem conhecido é a pirâmide de teste, que é mostrada aqui:

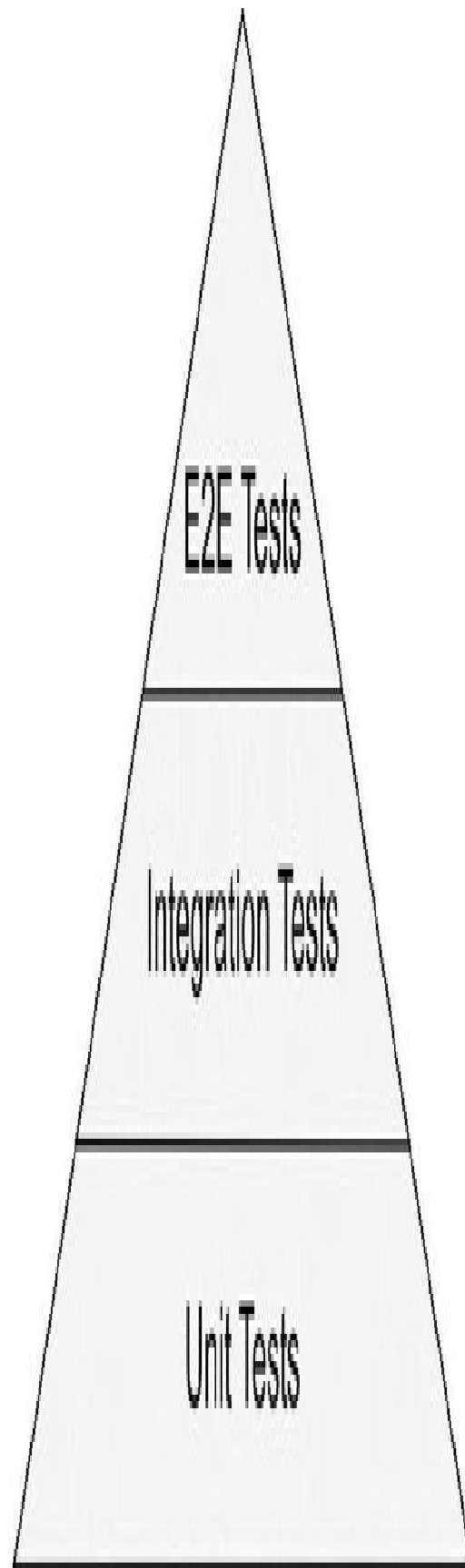


Figura 10.1: Pirâmide de teste

Esse conceito mostra basicamente três tipos de testes, ou seja, testes de ponta a ponta (ou testes E2E em resumo), testes de integração e testes de unidade. Explicaremos cada tipo de teste e sua posição na pirâmide de teste nas seções a seguir.

testes de unidade

Como o nome indica, os testes de unidade tratam de testar pequenas unidades de código. É uma prática recomendada escrever apenas um teste para cada funcionalidade de um objeto; caso contrário, os testes ficarão maiores e mais difíceis de entender e manter. Isso também mantém os testes pequenos, e é por isso que geralmente há muitos deles. Dependendo do tamanho do projeto, ter centenas ou milhares de testes de unidade é completamente normal, por isso é importante mantê-los executando o mais rápido possível. Normalmente, eles não devem demorar mais do que alguns microssegundos cada.

Os testes de unidade devem ser executados isoladamente, o que significa que, nos testes, os objetos testados não interagem com nenhum outro serviço externo, como bancos de dados ou APIs. Isso é obtido falsificando as dependências externas, o que é chamado de zombaria no jargão de teste de unidade. Simplificando, substituímos objetos externos - como serviços ou repositórios usados em nosso objeto de teste - por objetos fictícios (ou fictícios, em resumo). Esses objetos simulam o comportamento das dependências que substituem durante o tempo de execução dos testes de unidade. Isso garante que um teste não falhe repentinamente apenas porque, por exemplo, alguns dados no banco de dados, dos quais nosso teste dependia, foram alterados.

Como os testes desse tipo são pequenos, rápidos e não dependem de dependências externas, é relativamente fácil criar uma configuração de teste para eles. Eles são extremamente úteis porque podem dizer em segundos se suas últimas alterações no código causaram algum problema ou não. É por isso que eles são a base da pirâmide de testes.

Se você é novo em testes, faz sentido começar com PHPUnit, pois é o padrão da indústria no mundo PHP. Se você iniciar um novo projeto, é mais provável que o PHPUnit seja usado. Existem outras estruturas de teste que têm suas vantagens exclusivas, como Pest (<https://pestphp.com>). Depois de entender o conceito de teste de unidade com PHPUnit, nós o encorajamos a experimentá-los também.

Uma desvantagem dos testes de unidade é o fato de que eles não interagem entre si. Isso pode até fazer com que todos os seus testes passem enquanto seu aplicativo está quebrado, apenas porque a interação entre as classes não foi testada corretamente.

Para ilustrar esse problema, criamos um aplicativo de demonstração elementar. Vamos dar uma olhada nas partes mais importantes dele.

Código-fonte do aplicativo de demonstração

Você encontrará o código-fonte completo no repositório GitHub deste livro:

<https://github.com/PacktPublishing/Clean-Code-in-PHP>

Primeiramente, criamos uma classe rudimentar, chamada MyApp, da seguinte forma:

```
<?php
```

```

classe MyApp
{
função pública __construct(
privado meuRepositório $meuRepositório
) {
}

função pública run(): string
{
$dataArray = $this->meuRepositório->getData();
return $dataArray['value_1'] .
$dataArray['value_2'];
}
}

```

O método MyRepository é injetado por meio do construtor. O único método, run, usa o repositório para buscar dados e concatená-los. É importante observar que MyClass espera que uma determinada estrutura de array seja retornada por MyRepository. Isso não é recomendado, mas você ainda encontrará muito isso “na natureza”. Portanto, serve perfeitamente como demonstração.

MeuRepositório parece com isso:

```
<?php  
classe MeuRepositório  
{  
função pública getData(): matriz  
{  
Retorna [  
'value_1' => 'alguns dados...',  
'value_2' => 'e mais alguns dados'  
];  
}  
}
```

Na vida real, MyRepository buscaria os dados de uma fonte de dados externa, como um banco de dados. Para o nosso exemplo, ele retorna um array codificado. Se o método run de MyClass for executado, ele retornará alguns dados... e mais alguns dados.

Claro, também adicionamos testes (usando PHPUnit) para as classes anteriores. Por brevidade, mostraremos apenas os casos de teste no trecho de código a seguir, não todas as classes de teste:

```
função pública testRun(): void  
{
```



```

// Arranjo

$repositoryMock =
$this->createMock(MeuRepositório::classe);

$repositoryMock
->espera($this->uma vez())
->método('getData')
->vaiRetornar([
'valor_1' => 'um',
'valor_2' => 'b'
]);

// Agir

$appTest = new MyApp($repositoryMock);

$resultado = $appTest->run();

// Afirmar

$this->assertEquals('ab', $resultado);

}

função pública testGetDataReturnsAnArray(): void
{
// Arranjo

$repositoryTest = new MeuRepositório();

// Agir

```

```
$resultado = $repositórioTest->getData();  
  
// Afirmar  
  
$this->assertIsArray($resultado);  
  
$this->assertCount(2, $resultado);  
  
}
```

O padrão Arrange-Act-Assert (AAA)

Você deve ter notado que adicionamos três linhas de comentários a ambos os nossos casos de teste: Arrange, Act e Assert. Fizemos isso para demonstrar o padrão mais utilizado para escrever testes de unidade: o padrão AAA. Mesmo sem nunca ter escrito um único teste de unidade, isso ajuda você a entender como eles funcionam.

Em primeiro lugar, os objetos de teste e os pré-requisitos necessários, como objetos fictícios, são preparados (Arrange). Em segundo lugar, o trabalho real do objeto sob teste é executado (Act). Por fim, garantimos que os resultados do teste atendam às nossas expectativas (assert). Se uma das asserções não for atendida, todo o teste falhará.

Duas coisas são dignas de nota aqui, conforme exposto a seguir:

Em `testRun()`, criamos uma simulação `$repositoryMock` em vez de usar o método real `MyRepository`. Isso ocorre porque presumimos que `MyRepository`

normalmente buscaria os dados de uma fonte de dados externa e não queremos escrever testes de unidade que tenham dependências externas.

`testGetDataReturnsAnArray()` não testa o repositório muito bem. Apenas verificamos se o resultado é um array e se possui duas entradas, mas não quais chaves do array são retornadas.

Agora, imagine que, por qualquer motivo, um colega desenvolvedor decidiu que as chaves de array `value_1` e `value_2` eram muito longas e as renomeou como `val1` e `val2`. Se agora executarmos nosso aplicativo, é claro que ele será interrompido, conforme ilustrado aqui:

```
$ php index.php
```

```
PHP Warning: Chave de matriz indefinida "value_1" em
```

```
/home/curtis/clean-
```

```
code/chapter10/unit_tests_fail/src/MyApp.php na linha 18
```

```
PHP Warning: Chave de matriz indefinida "value_2" em
```

```
/home/curtis/clean-
```

```
code/chapter10/unit_tests_fail/src/MyApp.php na linha 18
```

No entanto, se você executar os testes, eles ainda passarão, como vemos aqui:

```
$ fornecedores/bin/phpunit testes
```

PHPUnit 9.5.20 #StandWithUkraine

.. 2/2 (100%)

Hora: 00:00.008, Memória: 6,00 MB

OK (2 testes, 4 afirmações)

Isso ilustra que ter testes de unidade é importante, mas não significa necessariamente que não iremos mais introduzir bugs porque eles podem estar com defeito ou testar coisas erradas, como em nosso exemplo.

Frequentemente, objetos como repositórios que interagem com sistemas externos não são testados porque requerem uma configuração de teste mais complexa, por exemplo, usando um banco de dados de teste adicional com dados falsos. Se apenas substituirmos tal objeto por uma simulação, o teste funcionará corretamente. Se houver mudanças significativas no objeto original posteriormente e a simulação não for atualizada para refletir essas mudanças, podemos acabar em uma situação como acabamos de descrever.

Para superar esse problema, precisamos de uma maneira adicional de testar nossas classes sem substituir as dependências por mocks. Para isso, apresentaremos o segundo tipo de teste da pirâmide de teste — testes de integração — na próxima seção.

testes de integração

O segundo tipo de teste que queremos examinar são os testes de integração. Ao contrário dos testes de unidade, que supostamente não usam dependências externas, com esse tipo de teste queremos fazer o contrário: queremos testar o código como ele normalmente rodaria, sem substituir nada por mocks.

Você já deve ter testemunhado suítes de teste de unidade que fizeram uso de um banco de dados de teste ou alguma API externa. Tecnicamente falando, esses testes não são mais testes de unidade, mas testes de integração (ou testes funcionais, como também são chamados). Poderíamos teoricamente usar o PHPUnit para esses testes também, ou usar ferramentas de teste específicas que assumem muito trabalho de base para nós.

O trecho de código a seguir mostra um exemplo de teste de integração:

função pública productIsSaved (Testador \$ testador)

{

\$produto = new Produto();

\$produto->setId(123);

\$product->setName('Cafeteira USB');

\$produto->salvar();

```
$this->testador->seeInDatabase(  
  
    'produtos',  
  
    ['id' => 123, 'name' => 'Cafeteira USB']  
  
);  
  
}
```

A função anterior mostra como seria um teste de integração se usássemos a ferramenta de teste Codeception (consulte a próxima caixa de informações para obter mais informações). Um objeto chamado \$tester é passado para o teste que é um objeto Helper que oferece a funcionalidade que precisamos executar - por exemplo, verificações de banco de dados. Depois de executar o método save no objeto de teste \$product, usamos este objeto Helper para validar se os dados que esperamos foram realmente gravados no banco de dados.

Codeception

Codeception (<https://codeception.com>) combina uma variedade de tipos de teste, como unidade, integração e até testes E2E em uma ferramenta. Sob o capô, é baseado em ferramentas existentes, como PHPUnit. Ele oferece módulos para todos os principais frameworks e, portanto, integra-se bem na maioria dos projetos PHP.

O uso de testes de integração torna a configuração do teste mais complexa porque devemos garantir que as dependências externas que usamos estejam sempre em um estado confiável. Por exemplo, se você precisar contar com um determinado usuário em seu banco de dados, certifique-se de que ele sempre tenha os mesmos dados, como o identificador de usuário (ID) que você testa;

caso contrário, seus testes falharão. Isso geralmente requer a criação de um novo banco de dados de teste antes de cada execução de teste, para garantir que nenhuma sobra de execuções de teste anteriores atrapalhe nosso teste. Além disso, precisamos executar migrações de banco de dados para garantir que o esquema do banco de dados de teste esteja atualizado. Finalmente, temos que preenchê-lo com dados de teste, o que é chamado de seeding.

A principal desvantagem desse tipo de teste é a velocidade de execução. As transações do banco de dados são lentas (em comparação com o uso de objetos fictícios) e precisamos preparar o banco de dados de teste a cada execução de teste. Testes de integração também tendem a quebrar mais facilmente, ou se tornam esquisitos (instáveis) porque a interação com outras dependências rapidamente se torna muito complexa: se o teste anterior alterou o banco de dados de uma forma que o próximo teste não esperava, sua execução de teste falha, embora o código não mudou.

Por exemplo, um novo teste é adicionado ao conjunto de testes que verifica se uma classe atualiza um conjunto de dados de uma determinada maneira. Como este é um teste de integração, ele usará o banco de dados de teste para ele e alterará esse conjunto de dados específico. Após a execução deste teste, no entanto, os dados alterados ainda estarão no banco de dados de teste. Se outro teste executado após esse novo teste estiver contando com os dados anteriores, ele falhará. Apesar da complexidade adicional de sua configuração de teste, os testes de integração garantem que a integração dos objetos testados no contexto do aplicativo funcione bem. É por isso que eles devem ser parte integrante de sua estratégia de teste, sendo a segunda camada na pirâmide de testes.

Você encontrará muitos testes de integração quando se trata de testar repositórios, modelos ou controladores. No entanto, eles não podem testar a interação entre o PHP e o navegador. Como usamos PHP principalmente para construir aplicações web, este é um aspecto que não devemos esquecer. Felizmente, o último tipo de teste da pirâmide de testes cobre exatamente esse problema.

testes E2E

Para este tipo de teste, deixaremos o domínio do PHP por um momento. Com os testes E2E, queremos garantir que todo o fluxo do servidor para o cliente (por exemplo, o navegador) e de volta para o servidor esteja funcionando bem. Basicamente, simulamos um usuário sentado em frente ao computador e clicando em nosso aplicativo.

Para conseguir isso, primeiro precisamos de um ambiente de teste reproduzível. Assim como nos testes de integração, devemos garantir que o aplicativo que queremos testar esteja sempre no mesmo estado. Isso significa que precisamos garantir que o mesmo conjunto de dados (por exemplo, postagens de blog ou artigos em uma loja) esteja disponível em cada execução de teste.

Em segundo lugar, precisamos automatizar a interação do usuário com nosso ambiente de teste. Aqui é onde as coisas ficam interessantes: não precisamos apenas do aplicativo, mas também de um servidor web local e um navegador para executá-lo e simular a interação do usuário. O servidor da Web adiciona mais complexidade à configuração do teste, mas geralmente não é um bloqueador. Para interação com o usuário, precisamos usar o chamado navegador sem cabeça. Esse navegador pode interagir com um servidor sem precisar abrir uma janela do navegador. Este é um recurso extremamente útil porque podemos usá-lo na linha de comando sem ter que instalar um sistema operacional completo com uma interface gráfica do usuário (GUI), como Ubuntu Desktop ou Windows. Isso nos economiza muito tempo de instalação e nos ajuda a não aumentar ainda mais a complexidade.

No momento em que escrevo, o Google Chrome é a escolha preferida, pois não é

apenas o mecanismo de navegador mais usado atualmente, mas também fornece um modo sem cabeça, ou seja, pode funcionar como um navegador sem cabeça. Com estruturas modernas como Cypress, é fácil automatizar a interação do usuário com nosso aplicativo. Pense nisso como um script que informa ao navegador qual URL (Uniform Resource Locator) abrir, em qual botão clicar e assim por diante. O exemplo a seguir mostra um teste Cypress simplificado:

```
describe('Login do aplicativo', function () {  
  
  it('logou com sucesso', function () {  
  
    cy.visit('http://localhost:8000/login')  
  
    cy.get('#nome de usuário')  
  
    .type(' teste@teste.com ')  
  
    cy.get('#senha')  
  
    .type('supersecret')  
  
    cy.get('#enviar')  
  
    .click()  
  
    cy.url()  
  
    .deve conter,  
  
    'http://localhost:8000/home')  
  
  })  
  
})
```

Cipreste

A estrutura de teste do Cypress (<https://www.cypress.io/>) torna a escrita de testes E2E muito fácil, já que ele lida com a configuração e a comunicação com o navegador headless para você. Sim — os testes devem ser escritos em JavaScript, mas isso não deve impedir você de tentar.

O objeto `cy` representa o testador, que executa determinadas etapas. No exemplo de código anterior, primeiro ele abre a página de login de um aplicativo fictício, preenche os campos `#username` e `#password` em um formulário de login e o envia clicando no botão `#submit`. Como última etapa, verifica se o login foi bem-sucedido e se o testador foi encaminhado para a tela inicial. Todas essas ações são executadas em um navegador real, rodando em segundo plano. Usando essa tecnologia, é possível escrever suítes de teste que literalmente clicam em nosso aplicativo, assim como um ser humano faria. Eles não testam apenas o código PHP, mas também o código front-end — um erro de JavaScript, por exemplo, interromperá rapidamente os testes. Mesmo que você não consiga consertar o bug sozinho, você ainda pode informar aos engenheiros de front-end de sua equipe que há um problema.

Os frameworks modernos tornam muito mais fácil escrever testes do que acontecia com tecnologias mais antigas, como o Selenium. Na verdade, é tão confortável hoje em dia que também as pessoas que não são desenvolvedores, mas têm uma base técnica sólida, como engenheiros de garantia de qualidade (QA), podem facilmente escrever seus próprios conjuntos de testes. Essa abordagem exige muita pressão das equipes porque os desenvolvedores precisam escrever menos testes, e o pessoal do controle de qualidade pode configurar seus testes conforme necessário, sem ter que esperar pelos desenvolvedores.

É claro que os testes E2E têm algumas desvantagens, e é por isso que eles são apenas a terceira camada de testes na pirâmide de testes: o ambiente de teste é mais complexo e precisa de mais trabalho para configurar, especialmente se um

banco de dados ou qualquer API externa for usado. Esse tipo de teste também é o mais lento, pois envolve o navegador, além das configurações de teste dos tipos de teste anteriores. Por fim, esses testes podem ser facilmente interrompidos porque, geralmente, as estruturas de teste usam atributos de id e classe de linguagem de marcação de hipertexto (HTML), ou mesmo seletores de modelo de objeto de documento (DOM), para navegar pelo DOM e encontrar elementos com os quais interagir. Assim, uma pequena alteração no DOM pode interromper rapidamente todo o seu conjunto de testes.

Objetos de página

Se você estiver interessado em criar testes E2E sustentáveis, verifique o conceito de objetos de página (<https://www.martinfowler.com/bliki/PageObject.html>).

A pirâmide de testes na prática

Com testes de unidade, integração e E2E, agora você conhece os três tipos de teste mais importantes e seus prós e contras. A abordagem sugerida de ter testes de unidade como uma grande base, uma boa quantidade de testes de integração e, finalmente, alguns testes E2E é um bom ponto de partida.

No entanto, você não precisa segui-lo rigidamente o tempo todo, pois cada projeto é diferente: se você, por exemplo, deseja começar a testar um aplicativo ainda completamente não testado, a introdução de testes de unidade exigiria muito esforço de refatoração para tornar as classes testável. Essa refatoração provavelmente introduzirá mais bugs no início do que resolverá.

Nesse caso, começar com uma boa cobertura de teste E2E será mais rápido e seguro. Uma vez que as partes principais do aplicativo possam ser testadas automaticamente, você pode começar com segurança com a refatoração e introduzir testes de unidade e/ou integração. Se o seu aplicativo quebrar devido à refatoração necessária, seus testes E2E o cobrirão.

No final deste capítulo, listaremos mais alguns tipos de teste para você avaliar caso tenha interesse. Por enquanto, os três tipos de teste que abordamos neste capítulo são os mais importantes e devem ser suficientes para você começar.

Há uma questão importante que ainda não abordamos: quanto do seu código você realmente precisa testar? Discutiremos isso na próxima seção.

Sobre a cobertura do código

Agora que exploramos os diferentes tipos de teste, você provavelmente deseja começar a escrever testes imediatamente. Mas, antes de deixar este livro de lado para aprender a codificar, vamos terminar este capítulo com a questão de quanto do seu código você deve testar.

Parte da resposta está no conceito de cobertura de código, que já mencionamos brevemente em [Capítulo 8, Métricas de qualidade de código](#) quando falamos sobre métricas de qualidade de código. Vamos dar uma olhada nisso agora.

Entendendo a cobertura do código

Cobertura de código mede a proporção de código que é coberto com testes. Quanto maior a cobertura do código, melhor - se houver mais testes, é menos provável que o software contenha bugs e será mais difícil introduzir novos despercebidos. Maior cobertura de código também é um possível indicador de melhor qualidade de código - como discutimos em uma seção anterior deste capítulo, o código testado deve ser escrito de uma determinada maneira, o que geralmente leva a uma melhor qualidade.

Geralmente, o grau de cobertura é expresso simplesmente usando a porcentagem do código testado, ou seja, de 0% (totalmente não testado) a 100% (cobertura completa do código). Mas como podemos medir a cobertura do código? Para isso, usaremos o PHPUnit, pois ele pode criar um relatório de cobertura de código para nós. No entanto, requer uma extensão PHP adicional para a funcionalidade de cobertura de código. Para este capítulo, decidimos usar o Xdebug, o depurador e criador de perfil padrão do PHP.

Configurando o Xdebug

O Xdebug é uma extensão para PHP, então precisa ser carregado como um módulo. Como sua instalação é um pouco mais complexa e depende principalmente do sistema operacional em que você está executando o PHP, consulte a documentação oficial em <https://xdebug.org> sobre como instalá-lo e configurá-lo. Também há muitos tutoriais disponíveis na internet.

Se você refatorar seu código, provavelmente desejará saber quais implicações de desempenho as alterações tiveram. Você melhorou o tempo de execução ou piorou ainda mais? Com o chamado profiler, você pode medir o tempo de execução de cada função em detalhes e ver onde os gargalos estão ocultos.

Não podemos cobrir este tópico em nosso livro, mas como ao longo deste capítulo já trabalhamos com o Xdebug, você também pode querer verificar seus recursos de criação de perfil: <https://xdebug.org/docs/profiler>. Outros serviços comerciais que oferecem mais conforto são, por exemplo, Tideways (<https://tideways.com>) ou Fogo Negro (<https://www.blackfire.io>).

Alternativas Xdebug

Observe que você pode usar outras extensões para isso, como PCOV (<https://github.com/krakjoe/pcov>), que funciona melhor se você quiser apenas fazer relatórios de cobertura de código. No entanto, o Xdebug é um depurador incrivelmente útil que você deve conhecer - caso contrário, recomendamos que você verifique alguns tutoriais sobre ele.

Como gerar relatórios de cobertura de código

Para demonstrar como criar um relatório de cobertura de código, usaremos nosso pequeno aplicativo de demonstração da seção anterior deste capítulo. Para acompanhar o exemplo, verifique-o no GitHub, execute `composer install` e verifique se o Xdebug está instalado com o modo definido como cobertura.

Antes de começarmos a gerar um relatório, vamos ver quais formatos de relatório o PHPUnit tem a oferecer. Ele pode gerar relatórios em uma variedade de formatos que você provavelmente não precisará agora, como Clover, Cobertura, Crap4J ou o formato PHPUnit XML. Eles podem se tornar mais relevantes, porém, quando você começa a integrar o PHPUnit com outras ferramentas.

No entanto, não queremos fazer isso neste livro, por isso estamos interessados apenas nos dois formatos mais acessíveis: texto e HTML. O formato de texto pode ser impresso diretamente na linha de comando, o que é útil quando você deseja resultados instantâneos ou para integrar PHPUnit em seu pipeline de construção, enquanto o formato HTML oferece mais informações.

Para nosso exemplo, queremos gravar ambos os formatos de relatório em uma nova pasta chamada `reports` na raiz do projeto. Embora você possa gerá-los usando as várias opções de tempo de execução do PHPUnit, queremos usar o arquivo de configuração `phpunit.xml` para definir o que gerar em cada execução de teste. O trecho de código a seguir mostra uma versão mínima, reduzida para facilitar a leitura. Em nosso repositório GitHub, você encontrará o arquivo `phpunit.xml` completo:


```
<?xml versão="1.0" codificação="UTF-8"?>

<phpunit bootstrap="vendor/autoload.php">

<testsuites>

<nome da suíte de teste="padrão">

<diretório>testes</diretório>

</testsuite>

</testsuites>

<cobertura>

<incluir>

<directory suffix=".php">src</directory>

</include>

<relatório>

<html outputDirectory="relatórios/cobertura" />

<text outputFile="reports/coverage.txt" />

</relatório>

</coverage>

</phpunit>
```

Além da configuração básica, que inclui uma definição da pasta de testes que precisamos para as execuções de teste regulares, adicionamos o nó <coverage>. Isso contém dois nós filhos: <includes> e <report>. É importante especificar ao

usar o nó `<includes>` qual diretório e extensão de arquivo devem ser usados para coletar informações de cobertura de código. Caso contrário, o PHPUnit não gerará nenhum relatório, mas também não reclamará sobre a falta de informações. Isso pode ser bastante confuso às vezes.

Além disso, precisamos dizer ao PHPUnit onde escrever quais relatórios. Usamos o nó `<report>` para isso e, como você pode ver, especificamos os relatórios HTML e de texto a serem gravados na pasta `reports` na raiz do nosso projeto, que será criada caso não exista.

PHPUnit espera que o arquivo de configuração seja nomeado `phpunit.xml` e esteja na raiz do projeto. Se isso foi feito, você pode executar rapidamente a geração dos relatórios executando o seguinte comando sem mais opções ou argumentos:

```
$ fornecedor/bin/phpunit
```

Após a execução do comando anterior, você encontrará uma pasta de relatórios gerada na raiz do seu projeto. Ele deve conter duas coisas: primeiro, um arquivo `coverage.txt` que contém um relatório em formato de texto e, segundo, uma pasta de cobertura que contém um relatório HTML.

Cobertura de código é cara

Usar o Xdebug para gerar relatórios de cobertura de código retardará a execução de seu conjunto de testes porque o Xdebug precisa coletar muitos dados e não foi

criado para desempenho. Portanto, recomendamos que você ative o Xdebug e a geração de relatórios apenas se necessário, mas mantenha-o desativado durante as execuções de teste regulares.

O relatório de texto é curto, mas já informa o quão bem seus testes cobrem seu aplicativo, conforme mostrado na captura de tela a seguir:

```
$ cat reports/coverage.txt
```

```
Code Coverage Report:
```

```
2022-06-21 21:02:07
```

```
Summary:
```

```
Classes: 50.00% (2/4)
```

```
Methods: 50.00% (3/6)
```

```
Lines: 50.00% (6/12)
```

```
CodeCoverageExample\MyApp
```

```
Methods: 100.00% ( 2/ 2) Lines: 100.00% ( 3/ 3)
```

```
CodeCoverageExample\MyOtherClass
```

```
Methods: 0.00% ( 0/ 2) Lines: 33.33% ( 2/ 6)
```

```
CodeCoverageExample\MyRepository
```

```
Methods: 100.00% ( 1/ 1) Lines: 100.00% ( 1/ 1)
```

```
$
```

Figura 10.2: Relatório de cobertura de código de texto

Para obter mais detalhes, abra o arquivo `reports/coverage/index.html` em seu navegador. Deve ficar assim:

/home/curtis/clean-code/chapter10/code_coverage/src / [\(Dashboard\)](#)

	Code Coverage								
	Lines			Functions and Methods			Classes and Traits		
Total		54.55%	6 / 11		60.00%	3 / 5		50.00%	2 / 4
 MyApp.php		100.00%	3 / 3		100.00%	2 / 2		100.00%	1 / 1
 MyOtherClass.php		40.00%	2 / 5		0.00%	0 / 1		0.00%	0 / 1
 MyRepository.php		100.00%	1 / 1		100.00%	1 / 1		100.00%	1 / 1
 UntestedClass.php		0.00%	0 / 2		0.00%	0 / 1		0.00%	0 / 1

Legend

Low: 0% to 50% Medium: 50% to 90% High: 90% to 100%

Generated by [php-code-coverage 9.2.15](#) using [PHP 8.1.2](#) and [PHPUnit 9.5.20](#) at Mon Jun 20 22:22:52 UTC 2022.

Figura 10.3: Relatório de cobertura de código HTML

Lá você encontrará as mesmas informações do relatório de texto, mas com uma visualização melhor. Além disso, o relatório é interativo. Por exemplo, se você clicar no link `MyOtherClass.php` à esquerda, será direcionado para um relatório detalhado dessa classe, conforme mostrado na captura de tela a seguir:

/home/curtis/clean-code/chapter10/code_coverage/src / MyOtherClass.php

	Code Coverage									
	Lines			Functions and Methods				Classes and Traits		
Total		40.00%	2 / 5		0.00%	0 / 1	CRAP		0.00%	0 / 1
MyOtherClass		40.00%	2 / 5		0.00%	0 / 1	4.94		0.00%	0 / 1
doSomething		40.00%	2 / 5		0.00%	0 / 1	4.94			

```
1 <?php
2
3 declare(strict_types=1);
4
5 namespace CodeCoverageExample;
6
7 class MyOtherClass
8 {
9     public function doSomething(int $a): int
10    {
11        if ($a > 0) {
12            return $a + 1;
13        }
14
15        if ($a === 0) {
16            return $a;
17        }
18
19        return $a - 1;
20    }
21 }
```

Legend

Executed Not Executed Dead Code

Figura 10.4: Relatório de cobertura de código HTML – exibição de classe

Duas coisas são dignas de nota aqui: primeiro, na seção Funções e Métodos, você já deve ter reconhecido a métrica CRAP que introduzimos em Capítulo 8, Métricas de qualidade de código. Aqui, você pode finalmente vê-lo em ação.

Em segundo lugar, o relatório mostra detalhadamente quais linhas foram acessadas durante o teste (fundo verde) e quais não foram (fundo vermelho). Se houvesse alguma linha inacessível (por exemplo, outra instrução após a última instrução return), ela seria exibida como Dead Code (fundo amarelo). O código morto pode ser removido com segurança.

Agora você tem uma boa visão geral da cobertura de código do seu projeto. Se os arquivos aparecerem com uma barra vermelha, eles não foram executados durante a execução do teste, então é aqui que você pode melhorar seu conjunto de testes.

Usando a anotação @covers

Há um problema com a cobertura de código: ela informa qual código foi executado durante o teste, mas isso não significa que o código executado também foi testado (ou seja, usando asserções). Isso é algo que o PHPUnit não pode determinar automaticamente. Isso significa que, mesmo que seus relatórios de cobertura de código mostrem 100% e barras verdes em todos os lugares, isso não significa que seu código foi bem testado. Ele foi executado apenas durante a execução do seu conjunto de testes.

Para contornar esse problema, é recomendável usar a anotação @covers no nível da classe, conforme ilustrado aqui:

```
/**
```

```
* @covers MeuRepositório
```

```
*/
```

```
classe MyRepositoryTest estende TestCase
```

```
{
```

função pública testGetDataReturnsAnArray(): void

{

// ...

}

}

Isso melhora a precisão de nossos testes porque, ao usar a anotação `@covers`, declaramos explicitamente qual código deve ser testado por nossos testes. Por exemplo, digamos que a classe em teste usa um serviço externo. Você só quer testar aquela classe, não o serviço que ela usa, então você escreve apenas asserções que verificam a classe em teste. Sem a anotação `@covers`, o PHPUnit ainda incluiria o serviço externo no relatório de cobertura de código porque foi executado durante o teste.

Você pode usar `@covers` também em um nível de método; no entanto, isso pode causar problemas se, por exemplo, você refatorar uma classe e extrair métodos para alguma outra classe. Se você esquecer de ajustar a anotação `@covers` no nível do método aqui, o relatório de cobertura não será mais preciso.

Para forçar o uso da anotação `@covers`, use a opção `forceCoversAnnotation` no arquivo `phpunit.xml`. Se estiver definido como verdadeiro, os testes que não usam a anotação serão marcados como arriscados; não falham, mas aparecem separadamente nos relatórios como algo a ser melhorado. Dessa forma, seus colegas desenvolvedores (e você) não esquecerão de usá-lo.

o que testar

Agora sabemos como obter informações detalhadas sobre quanto do nosso código é testado. Então, você deve se esforçar para obter uma cobertura de código completa agora? 100% deveria ser sua meta?

Como vimos nos testes de nosso aplicativo de exemplo em uma parte anterior deste capítulo, escrever testes para uma classe não significa automaticamente que você realmente testa todos os aspectos dela. Aqui, infelizmente, nem mesmo medir a cobertura do código ajudará. No entanto, pode ajudá-lo a identificar testes que não testam nada. Especialmente quando muitos mocks são usados em um caso de teste, pode acontecer que apenas o mock seja testado, mas nenhum “código real”. Considere o seguinte caso de teste, que é um teste válido que passaria:

```
função pública testUselessTestCase(): void
```

```
{
```

```
    $repositoryMock =
```

```
    $this->createMock(MeuRepositório::classe);
```

```
$repositoryMock
```

```
->método('getData')
```

```
->vaiRetornar([
```

```
'valor_1' => 'um',
```

```
'valor_2' => 'b'
```

```
]);
```

```
$this->assertEquals(
```

```
[
```

```
'valor_1' => 'um',
```

```
'valor_2' => 'b'
```

```
],
```

```
$repositoryMock->getData()
```

```
);
```

```
}
```

Este exemplo é simplificado, mas demonstra como os relatórios de cobertura de código podem ajudar porque este teste não adicionaria uma única linha testada à nossa taxa de cobertura de código. Infelizmente, nenhuma ferramenta pode dizer (ainda) quais testes estão bem escritos e quais devem ser melhorados ou mesmo inúteis, como no nosso exemplo.

Seguindo o princípio de Pareto, almejar 80% de cobertura de código já deve melhorar bastante sua base de código, e isso pode ser alcançado com um esforço razoável. Concentre-se no código que torna seu aplicativo especial, geralmente chamado de lógica de negócios. Este é o código que requer mais de sua atenção.

Princípio de Pareto

O princípio de Pareto afirma que 80% dos resultados são alcançados com 20%

do esforço total. Os 20% restantes dos resultados requerem mais trabalho quantitativamente, com 80% do esforço total.

Há também um código trivial que realmente não precisa ser testado. Um exemplo comum é o teste de getters e setters. Se esses métodos contiverem mais lógica, faz sentido testá-los, é claro. Mas se forem apenas funções simples que definem ou retornam o valor de uma propriedade, é uma perda de tempo escrever testes para elas. Ainda assim, você precisaria fazer isso se desejasse obter 100% de cobertura de código.

Outros exemplos são arquivos de configuração, fábricas ou definições de rota. É suficiente usar E2E ou testes de integração, que garantem o funcionamento geral do aplicativo. Eles implicitamente (isto é, sem usar afirmações concretas) testam todo o código cola, que é todo o código que mantém seu aplicativo unido, mas é tedioso para testar.

Particularmente, os testes E2E geralmente não são contados na métrica de cobertura de código porque é tecnicamente difícil fazê-lo. Se você os tiver, eles adicionarão uma camada extra de cobertura de teste que não pode ser medida. Você não pode se gabar de 100% de cobertura de código, mas sabe que todos os diferentes tipos de teste estão à sua disposição e esse deve ser nosso objetivo número um.

Resumo

Neste capítulo, discutimos por que você deve usar testes automatizados e como eles melhoram a qualidade do seu código. Cobrimos os três principais tipos de teste, que são testes de unidade, testes de integração e testes E2E, com seus prós e contras, possíveis armadilhas e nossas recomendações sobre como usá-los. Por fim, você aprendeu sobre o conceito de cobertura de código e como usá-lo em seus próprios projetos.

Juntamente com o conhecimento do capítulo anterior sobre ferramentas de qualidade de código e como organizá-las, no próximo capítulo, podemos finalmente começar a combinar todas essas ferramentas em um processo que ajuda a executá-las de maneira estruturada e confiável - construir pipelines .

Leitura adicional

Existem mais tipos de teste por aí do que poderíamos cobrir neste capítulo. Se você acha o mundo dos testes automatizados tão fascinante quanto os autores, talvez queira verificar também outros tipos de teste, como os seguintes:

Integração contínua

Você aprendeu a teoria sobre como escrever código PHP limpo: código Hypertext Preprocessor (PHP) e agora conhece as ferramentas e métricas necessárias que nos ajudam a atingir e manter altos níveis de qualidade. No entanto, o que ainda falta é a integração de todas essas tecnologias em um fluxo de trabalho que facilite seu trabalho diário.

Nas páginas a seguir, detalharemos a integração contínua (CI) e aprenderemos com exemplos de como configurar um fluxo de trabalho automatizado simples, mas eficaz.

Além disso, mostraremos como configurar localmente uma seleção de ferramentas de qualidade de código de forma que elas ofereçam o máximo de suporte, sem a necessidade de executá-las manualmente. Além disso, mostraremos algumas práticas recomendadas sobre como adicionar esses fluxos de trabalho a um projeto existente.

Os principais tópicos que abordaremos estão listados aqui:

Requerimentos técnicos

Além da configuração dos capítulos anteriores, você precisará de uma conta GitHub para poder seguir todos os exemplos. No entanto, isso não terá custos adicionais, pois estamos usando apenas o plano gratuito.

O aplicativo de exemplo que usaremos neste capítulo pode ser baixado do repositório GitHub para este livro: <https://github.com/PacktPublishing/Clean-Code-in-PHP/tree/main/ch11/example-application>.

Por que você precisa de CI

Escrever software é um processo demorado e, portanto, caro. Se você desenvolver software para se divertir, isso “só” lhe custará seu tempo de lazer. Se você trabalha para uma empresa (seja terceirizado ou funcionário em tempo integral), o tempo é ainda mais valioso, pois você é pago por ele. Na verdade, as empresas querem reduzir custos e, portanto, não querem gastar mais dinheiro em um recurso do que o necessário.

Uma grande parte do nosso trabalho diário é corrigir defeitos. Entregar software livre de bugs é algo que provavelmente todos os desenvolvedores gostariam de alcançar. Não cometemos erros deliberadamente, mas eles sempre acontecerão. Existem maneiras, no entanto, de reduzir os custos dos bugs.

Os custos de um bug

Um bug é consideravelmente caro porque não agrega valor ao produto. Portanto, pretendemos detectar esses bugs o mais cedo possível - quanto mais cedo os detectarmos, menos custos eles causarão. A captura de tela a seguir mostra como os custos para corrigir um bug aumentam significativamente quanto mais tarde ele aparece no processo de desenvolvimento:

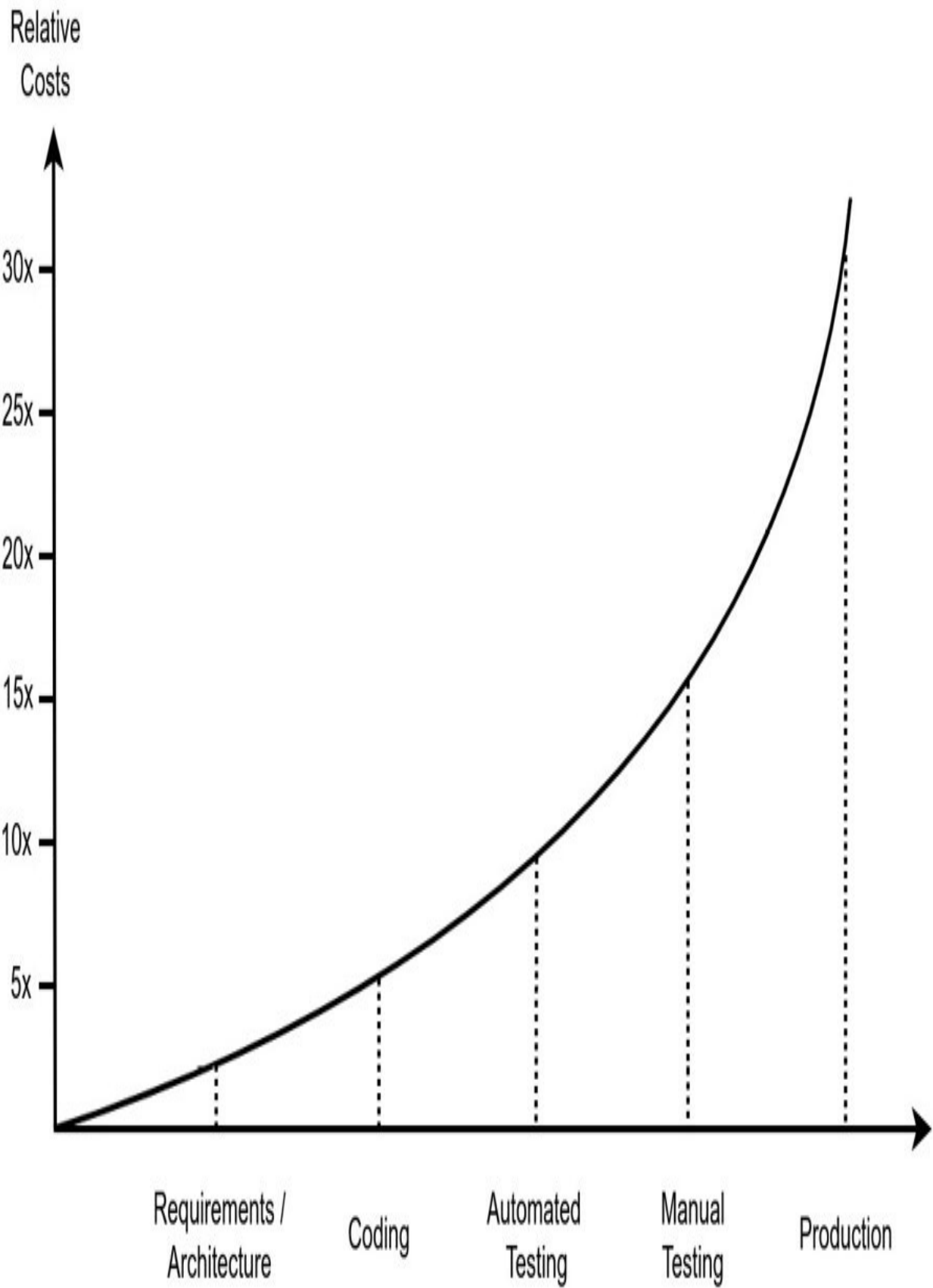


Figura 11.1: Custos relativos estimados de correção de um bug com base no tempo de sua detecção

Mas qual é a razão para o aumento maciço de custos ao longo do tempo? E por que os bugs custam dinheiro?

Nos estágios iniciais, os custos decorrem principalmente do tempo necessário para resolver o problema. Se um bug pudesse ter sido evitado apenas por meio de requisitos melhores, por exemplo, isso exigiria menos esforço, pois foi descoberto durante o teste manual. Se um bug foi encontrado na produção, muitas pessoas estão envolvidas em corrigi-lo: primeiro, um funcionário do helpdesk precisa reconhecer o bug relatado por um cliente e repassá-lo ao engenheiro de garantia de qualidade (QA), que o reproduz e escreve um relatório adequado bilhete de bug.

Esse ticket é então atribuído ao gerente de produto, que, após ter tempo para reproduzir e verificar também o defeito, o planeja para o próximo sprint. O ticket acaba sendo atribuído a um desenvolvedor, que precisará de algum tempo para reproduzi-lo e corrigi-lo. Mas não acabou aqui porque a correção de bug provavelmente requer uma revisão de código de outro desenvolvedor e é verificada duas vezes pelo gerente de produto ou engenheiro de controle de qualidade antes que possa finalmente ser lançada.

Uma vez “escapado” do ambiente local do desenvolvedor, toda essa sobrecarga aumenta significativamente os custos do defeito. Além disso, se o bug já estiver em produção, pode fazer com que os clientes não queiram mais usar o produto porque não estão mais satisfeitos com ele. Isso se chama rotatividade de clientes.

Mesmo que você não esteja trabalhando em um produto comercial, mas, por exemplo, em um projeto de código aberto, o conceito pode ser traduzido em tempo ou esforço. Um bug levará a um relatório de problema que primeiro você precisa ler e entender, provavelmente fazendo mais algumas perguntas e esperando a resposta do autor do ticker. Se o seu software tiver muitos bugs, as pessoas o usarão menos e todos os seus esforços anteriores podem ter sido em vão em algum momento.

Como prevenir erros

Felizmente, agora temos uma caixa de ferramentas completa ao seu lado que pode nos ajudar a encontrar bugs em seu código antes que outra pessoa o faça. Nós apenas temos que usá-lo - e isso já é um problema porque nós, desenvolvedores, geralmente somos pessoas preguiçosas.

Obviamente, você pode executar todas as ferramentas manualmente antes de cada implantação. Digamos que você queira implantar algum código para produção. Depois de mesclar o código na ramificação principal, as seguintes etapas devem ser executadas para garantir que nenhum código quebrado seja entregue à produção:

Usando o PHP linter para garantir a correção sintática do código

Executar um verificador e fixador de estilo de código para manter o estilo de código alinhado

Encontrar possíveis problemas usando a análise de código estático

Execução de todos os conjuntos de testes automatizados para garantir que seu código ainda funcione

Criação de relatórios para as métricas de qualidade de código usadas

Limpando a pasta de compilação e criando um arquivo do código para implantar

Esta é uma lista de coisas para se manter em mente. Ninguém faria isso por um longo período sem cometer erros em algum momento, então, naturalmente, você começaria a escrever scripts que o ajudassem a executar essas etapas de uma só vez. Isso já é uma boa melhoria, e também a usaremos um pouco mais adiante neste capítulo.

Apresentando CI

A execução de todas as etapas da seção anterior em seu ambiente local levará algum tempo e, durante a execução das verificações, você dificilmente poderá trabalhar em mais alguma coisa, então terá que esperar até que elas sejam concluídas. Então, por que não carregar todo esse fluxo de trabalho em outro servidor dedicado?

Isso é exatamente o que a CI faz: descreve o processo automatizado de reunir todos os componentes necessários de seu aplicativo em um produto para que possa ser implantado nos ambientes desejados. Durante o processo, verificações automatizadas garantirão a qualidade geral do código. É importante ter em mente que, se uma das verificações falhar, toda a compilação será considerada falha.

Existem muitas ferramentas de CI disponíveis, como o Jenkins, que geralmente é auto-hospedado (ou seja, operado por você ou por alguém da sua equipe ou empresa). Ou você pode escolher serviços pagos, como GitHub Actions, GitLab CI, Bitbucket Pipelines ou CircleCI.

Você lerá frequentemente a abreviação CI/CD, e também a usaremos ao longo deste livro. CD significa entrega contínua, um conceito que abordaremos no final deste capítulo. Por enquanto, você não precisa se preocupar com isso, no entanto.

Configurar uma dessas ferramentas parece muito trabalhoso, mas também traz alguns grandes benefícios, como os seguintes:

Esperamos que os pontos anteriores lhe dêem uma ideia de quanto você se beneficiará com o uso do IC. Uma parte integrante de cada ferramenta de CI/CD é o chamado pipeline de construção, que explicaremos em detalhes na próxima seção.

O pipeline de construção

Na seção anterior, listamos as várias etapas necessárias para deixar nosso código pronto para ser enviado para produção. No contexto do CI, uma combinação dessas etapas é o que chamamos de pipeline de construção: ele pega a entrada (no nosso caso, todo o código do aplicativo), executa-a por meio de várias ferramentas e cria os chamados artefatos de construção a partir dela . Eles são o resultado de uma compilação - geralmente, isso inclui a entrega (um pacote do código do aplicativo que está pronto para ser movido para o ambiente desejado), além de dados adicionais, como logs de compilação, relatórios e assim por diante.

O diagrama a seguir fornece uma visão geral esquemática da aparência de um pipeline de construção típico. Como não é executado em seu ambiente local, requer duas etapas adicionais: criar um ambiente de construção e construir um aplicativo:

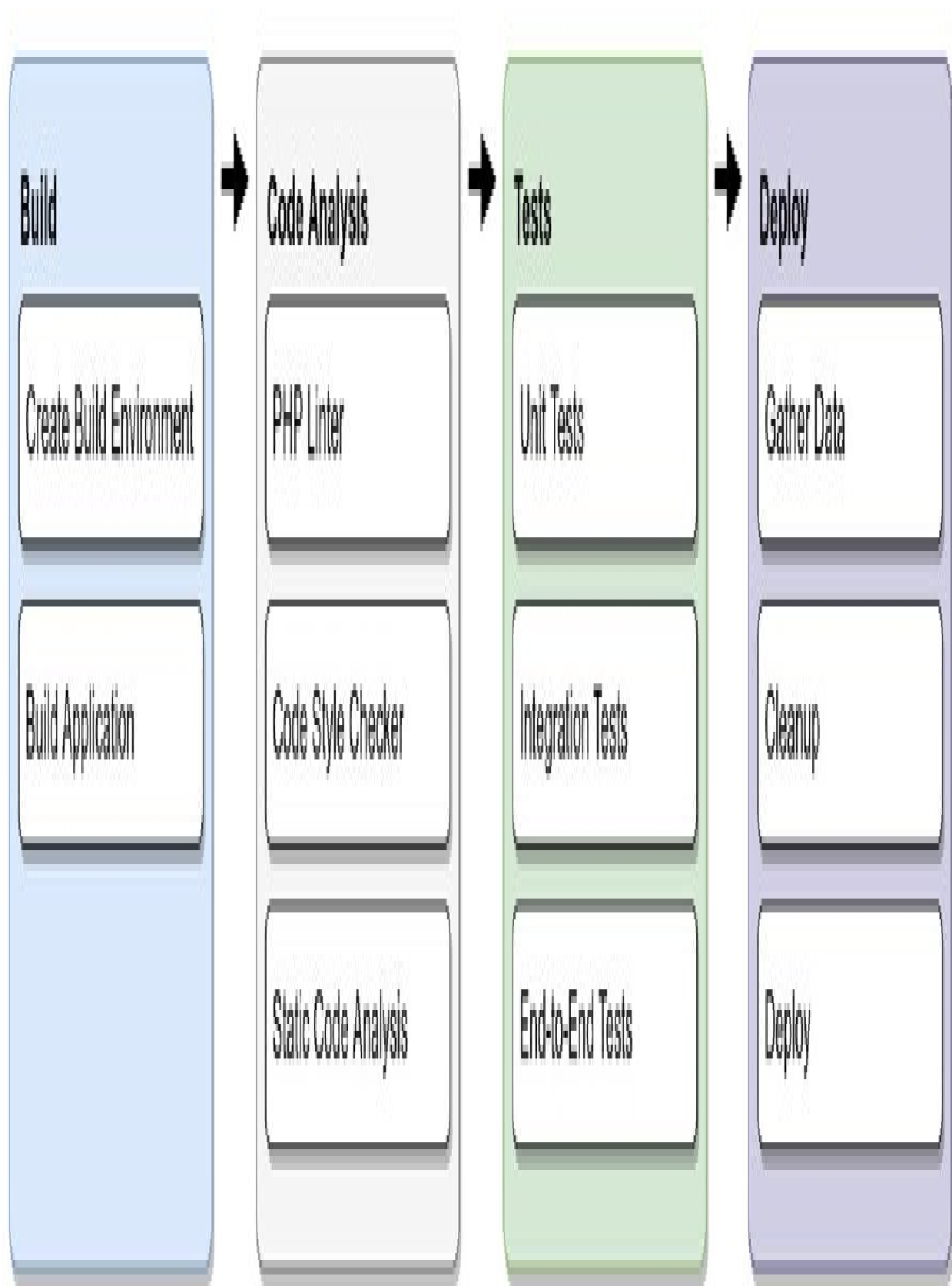


Figura 11.2: Esquema de um pipeline de CI

Outros idiomas em andamento

Neste livro, daremos apenas uma olhada nas partes do pipeline relacionadas ao PHP, mas um aplicativo da Web moderno não consiste apenas em PHP. Especialmente para o código voltado para o front-end, há todo um outro universo de ferramentas que também precisa fazer parte do pipeline. No entanto, no final, o processo é muito semelhante.

Nas próximas seções, entraremos em mais detalhes sobre cada estágio de construção. Vamos mantê-lo teórico no início e depois daremos exemplos de implementação técnica mais adiante neste capítulo.

Etapa 1: construir o projeto

O pipeline de CI requer uma instância de construção dedicada de nosso aplicativo, onde podemos executar todas as ferramentas e verificações isoladas. Isso pode ser dividido em duas etapas: criar um ambiente de construção e executar as ferramentas de construção necessárias.

Criando um ambiente de compilação

Para construir um aplicativo em outro lugar que não seja o sistema de desenvolvimento local, primeiro precisamos criar um ambiente de construção. Como exatamente o ambiente é fornecido depende da ferramenta de CI/CD usada. Isso pode ser um servidor dedicado que oferece espaços de trabalho separados para cada projeto ou um ambiente Docker completamente em contêiner, que é ativado toda vez que você precisar e dura apenas a duração da compilação.

Uma vez que existe um ambiente de construção, precisamos baixar todo o código fonte lá, mas sem pacotes externos ou outras dependências por enquanto. Provavelmente, seu código será armazenado em um repositório Git e hospedado em um servidor Git privado ou em um serviço comercial. O download de uma cópia de uma ramificação específica de um repositório é chamado de checkout.

Temos que prestar atenção em qual ramificação do repositório o código é retirado. Isso depende do que você deseja construir. Se você pretende verificar o código de uma solicitação pull (PR) (um conjunto de alterações no código, que alguém solicita para ser integrado à base de código), é necessário verificar a ramificação desse recurso. Se você deseja carregar a versão mais recente do seu aplicativo para produção, verifique a ramificação principal.

Branch principal versus master

Ao longo da história dos computadores, os termos mestre e escravo foram amplamente usados, seja para configuração de disco rígido, replicação de banco

de dados ou, é claro, Git. No entanto, esses termos são prejudiciais para muitas pessoas, então o pessoal de tecnologia da informação (TI) decidiu finalmente mudar isso para termos menos ofensivos, como principal e réplica. O GitHub, por exemplo, por padrão chama a ramificação principal simplesmente principal em vez de master hoje em dia. Você ainda encontrará repositórios por aí que usam o antigo nome do branch. Neste livro, porém, vamos nos ater à nova terminologia.

Não se preocupe, no entanto, se o seu projeto não estiver hospedado usando o Git - esta etapa ainda é necessária porque precisamos obter o código no servidor CI/CD. Seja via Git, Mercurial, Subversion (SVN), ou mesmo download direto do arquivo, não importa no final. O resultado desta etapa é ter o código que desejamos implantar prontamente disponível no servidor CI/CD para que possamos começar a instalar as dependências na próxima etapa.

Construindo o aplicativo

Construir o aplicativo é semelhante a instalá-lo em um novo sistema. Na etapa anterior, verificamos se o código-fonte está disponível no ambiente. Nesta etapa, precisamos executar todas as etapas de compilação necessárias. Isso geralmente inclui o seguinte:

Para reduzir o tempo de compilação, muitas ferramentas modernas de CI/CD oferecem armazenamento em cache. Se ativados, eles manterão as dependências em armazenamento temporário após o primeiro download. Geralmente, é uma boa ideia ativá-lo, se não estiver ativado por padrão.

Etapa 2 - Análise de código

Cobrimos ferramentas de qualidade de código em Capítulo 7, Ferramentas de qualidade de código, em detalhe. Agora, é hora de adicionar essas ferramentas ao nosso pipeline para garantir que elas sejam executadas para cada alteração introduzida.

PHP linter

Se você mesclar o código em outra ramificação, sempre pode acontecer que o código seja interrompido. O Git tem algoritmos de mesclagem muito sofisticados, mas ainda assim não pode fazer magia. Dado que você tem um conjunto de testes com alta cobertura, alguns testes certamente serão interrompidos se houver um erro de sintaxe causado pela mesclagem. Então, por que devemos executar essa etapa extra? Recomendamos porque o PHP linter tem duas vantagens: roda muito rápido e verifica todos os arquivos PHP, independentemente de haver testes para eles ou não.

Queremos que nosso pipeline falhe rapidamente se algum problema for detectado. Portanto, antes de executar qualquer tarefa de execução longa, faz sentido executar uma verificação rápida de sintaxe no início. Eles quebrariam de qualquer maneira e você perderia um tempo valioso. Como regra geral, quanto mais rápida a verificação for executada, mais cedo ela aparecerá no pipeline.

Verificador de estilo de código

Depois de verificar a sintaxe do código, é hora de verificar o estilo do código. Essa operação também é rápida, por isso faz sentido executá-la no início do pipeline. Para o nosso exemplo, iremos com o PHP Coding Standards Fixer (PHP-CS-Fixer), que já introduzimos em [Capítulo 7, Ferramentas de qualidade de código](#).

Há uma diferença sutil, mas importante, entre executar o PHP-CS-Fixer localmente e no pipeline CI/CD: para o último, usaremos apenas para verificar o código, mas não para corrigi-lo. Não queremos que o pipeline altere nosso código, apenas para analisá-lo. Ou seja, nosso pipeline apenas verificará se o código está formatado corretamente (conforme as regras que definimos), mas não tentará corrigi-lo automaticamente; se alguma das regras for violada, a compilação falhou.

Nenhuma regra está dizendo que o pipeline de CI/CD não deve alterar o código. No entanto, adiciona complexidade para confirmar automaticamente as alterações no repositório durante o processo. Além disso, requer um aplicativo muito bem testado e você precisa confiar que a ferramenta de sua escolha não quebra nada. Normalmente, eles funcionam bem, mas você quer arriscar?

Em seu ambiente local, faz sentido executar o fixer juntamente com o verificador de estilo de código. Discutiremos a configuração local em uma das próximas seções deste capítulo.

Análise de código estático

Neste ponto, sabemos que nosso código está sintaticamente correto e formatado de acordo com nossas regras. Ambas as verificações anteriores geralmente são executadas rapidamente, portanto, nossa compilação falharia antecipadamente se algum desses problemas fáceis de detectar tivesse ocorrido.

Agora, é hora de executar as tarefas mais lentas. A análise de código estático geralmente demora um pouco mais do que as duas anteriores, mas não é tão lenta quanto executar os testes automatizados. Essencialmente, esta etapa não é tão diferente da verificação de estilo de código e linting: se as regras que definimos anteriormente forem violadas, a compilação falhará.

Se você está introduzindo CI em um projeto existente, o desafio é encontrar o ponto ideal no relatório de erros. Por um lado, você deseja manter os desenvolvedores satisfeitos e não forçá-los a corrigir dezenas de problemas introduzidos por outros desenvolvedores em cada arquivo que tocam. Por outro lado, você precisa definir o limite suficientemente rígido para impor pelo menos alguma refatoração a cada alteração de código.

Não há regra de ouro aqui, infelizmente, e você precisa experimentar as configurações. Posteriormente, quando a maioria dos problemas dos relatórios de análise de código estático forem resolvidos, você precisará restringir um pouco as regras de relatório de erros para que seu projeto não fique estagnado em um determinado nível.

Fase 3 – Testes

Depois que nosso código atinge esse ponto no pipeline, estamos convencidos de que ele está sintaticamente correto, segue nossas diretrizes de estilo de código e não apresenta falhas gerais de acordo com nossas regras de análise de código estático. Então, agora vamos executar a etapa do pipeline, que geralmente leva mais tempo: os testes automatizados.

Como introduzimos no capítulo anterior, há mais do que apenas testes de unidade a serem considerados. Frequentemente, um projeto como um web service tem pelo menos alguns testes de integração para garantir que o serviço esteja funcionando bem como um todo, incluindo as transações do banco de dados. Ou, se seu projeto for um aplicativo da web clássico, você pode ter um conjunto de testes E2E que utiliza um navegador para clicar virtualmente nele.

Queremos aplicar aqui a mesma abordagem que fizemos para as etapas de compilação: comece com os testes de execução rápida e continue com os mais lentos. Se os testes de unidade já estiverem falhando, você não precisa esperar pelos resultados dos testes E2E. Assim, para os testes, a ordem de execução normalmente seria esta:

testes de unidade

testes de integração

testes E2E

Se apenas um teste de qualquer tipo falhar, a compilação será marcada como falha. Se todos passarem, já passamos pela parte mais crítica do pipeline. Nosso aplicativo está pronto para ser implantado, então agora é hora de limpar e preparar a entrega.

Estágio 4 - Implantar

Nosso código foi verificado minuciosamente por várias ferramentas e estamos confiantes de que está de acordo com nossos padrões. Agora podemos preparar os artefatos de construção e finalmente implantar o aplicativo. Vamos dar uma olhada no que precisa ser feito para isso.

Juntando informação

Nas etapas anteriores, todas as ferramentas que usamos produziram algum tipo de dado, seja por meio da escrita na saída padrão (stdout), ou se você configurou isso, criando relatórios que resumem as ações realizadas.

Você pode, por exemplo, fazer upload de relatórios gerados para armazenamento dedicado ou enviá-los para seu repositório. Ou você pode usar o relatório de cobertura de código do PHPUnit e criar automaticamente um selo de cobertura de código a partir dele, que pode ser adicionado ao README do seu projeto GitHub.

O caso de uso mais importante, no entanto, é depurar se algum dos estágios falhou. Você nunca pode ter uma saída de depuração suficiente caso algo dê errado, então é uma boa ideia definir o detalhamento de suas ferramentas para um nível mais alto. Sua ferramenta de CI/CD geralmente cuida para que tudo o que foi gravado no stdout seja disponibilizado após a execução do pipeline de construção.

Limpar

Antes de carregarmos o aplicativo em algum lugar, queremos ter certeza de que ele não contém nenhum lastro desnecessário. Isso inclui remover logs ou relatórios de estágios anteriores ou excluir as ferramentas de qualidade de código. Lembre-se de que devemos implantar apenas o código necessário para a execução do aplicativo - as ferramentas de desenvolvimento, como o PHPUnit, não são criadas com a segurança em mente (<https://phpunit.readthedocs.io/en/9.5/installation.html#webserver>).

Implantar

Para implantar o código no ambiente de destino, precisamos envolvê-lo em um ativo que possa ser facilmente movido para lá. Esse ativo também é chamado de entrega. O tipo que escolhemos para a entrega depende de como o aplicativo deve ser implantado na produção. Um tipo comum para tal entrega é simplesmente um arquivo do código que deve ser implantado.

Por exemplo, se seu ambiente de produção estiver sendo executado em um servidor da Web local clássico, precisamos criar um arquivo do código do aplicativo, carregá-lo no servidor de destino e extraí-lo de lá.

O padrão de fato hoje é um ambiente containerizado usando o Docker. Depois que a instância de compilação tiver sido totalmente testada, uma imagem do Docker poderá ser criada a partir dela. Essa imagem será carregada em um repositório de imagens, como Amazon Web Services Elastic Container Repository (AWS ECR). Esse repositório de imagens hospeda todas as suas imagens, para que possam ser usadas para criar novos contêineres quando necessário. Essa abordagem abriu o caminho para aplicativos da Web altamente escaláveis como os que temos hoje, portanto, projetar seu aplicativo para ser dockerizável desde o início valerá a pena se seu aplicativo, em algum momento, precisar ser dimensionado.

Contêineres e imagens

Se você é novo no Docker, o conceito de contêineres e imagens pode ser confuso. Resumindo, uma imagem contém todos os dados, mas é somente leitura

e não pode ser usada sozinha. Para torná-lo utilizável no ambiente Docker, é necessário um contêiner que contenha todas as informações da imagem e ofereça a funcionalidade necessária para se conectar a outros contêineres de sua configuração. Como você pode criar quantos contêineres precisar de uma imagem, também pode considerá-la um modelo de contêiner.

Se você quiser saber mais sobre o Docker, recomendamos que consulte a documentação oficial em <https://docs.docker.com/get-started/overview>. Também existem muitos tutoriais, cursos online e outras informações que podem ser encontradas na internet.

Agora temos uma boa ideia de como seria um pipeline de construção. Antes de começarmos a configurar nosso pipeline de exemplo, mais uma coisa precisa ser esclarecida: quando ele será executado?

Integrando o pipeline ao seu fluxo de trabalho

Depois de configurar todas as etapas necessárias, finalmente precisamos integrar o pipeline ao seu fluxo de trabalho. As ferramentas de CI/CD geralmente oferecem opções diferentes sobre quando um pipeline é executado. No começo, isso pode, é claro, ser feito manualmente clicando em um botão. Isso não é muito confortável, no entanto. Se você usar repositórios Git hospedados, como GitHub, GitLab ou Bitbucket, poderá conectá-los ao seu pipeline de compilação e iniciar a compilação sempre que um PR for criado ou uma ramificação for mesclada na ramificação principal.

Para grandes projetos em que a compilação leva horas, também é comum executar uma compilação para a base de código atual à noite (as chamadas compilações noturnas). Os desenvolvedores obtêm seus comentários do pipeline no dia seguinte.

A execução de uma compilação requer algum tempo e, claro, os desenvolvedores não devem se sentar na frente das telas e esperar até que possam continuar com seu trabalho. Eles devem ser informados assim que a compilação for bem-sucedida ou falhar. Todas as ferramentas de CI/CD hoje em dia oferecem múltiplas formas de notificar o desenvolvedor, principalmente por e-mail e por mensagens em ferramentas de chat como Slack ou Microsoft Teams. Além disso, eles geralmente também oferecem exibições de painel, onde você pode ver o status de todas as compilações em uma tela.

Agora você deve ter uma boa ideia de como seria um pipeline de construção para o seu projeto. Portanto, é hora de mostrar um exemplo prático.

Construindo um pipeline com GitHub Actions

Depois de aprender sobre todas as etapas do IC, é hora de praticar. A apresentação de todos os recursos de uma ou mais ferramentas de CI/CD está fora do escopo deste livro; no entanto, ainda queremos mostrar a você como pode ser fácil configurar um pipeline de construção funcional. Para manter a barreira de entrada o mais baixa possível para você e evitar custos, decidimos usar o GitHub Actions.

GitHub Actions não é uma ferramenta CI clássica como Jenkins ou CircleCI, mas sim uma maneira de criar fluxos de trabalho em torno de repositórios GitHub. Com um pouco de criatividade, você pode fazer muito mais do que “apenas” um pipeline CI/CD clássico. Vamos nos concentrar apenas nesse aspecto, é claro.

Você provavelmente já possui uma conta no GitHub e, caso contrário, obter uma não custará nada. Você pode usar o GitHub Actions gratuitamente até 2.000 minutos por mês no momento da escrita para repositórios públicos, o que o torna um ótimo playground ou uma ferramenta útil para seus projetos de código aberto.

Exemplo de projeto

Criamos um pequeno aplicativo de demonstração para usar durante este capítulo. Você irá encontrar isso aqui: <https://github.com/PacktPublishing/Clean-Code-in->

PHP/tree/main/ch11/example-application. Observe que ele não serve para nenhum outro propósito além de demonstrar o uso básico do GitHub Actions e dos ganchos do Git.

GitHub Actions em poucas palavras

O GitHub Actions não oferece interface de usuário sofisticada onde você pode configurar todos os estágios. Em vez disso, tudo é configurado por meio de arquivos YAML Ain't Markup Language (YAML) que são armazenados diretamente no repositório. Como um desenvolvedor PHP, você provavelmente tem experiência com o uso de arquivos YAML para todos os tipos de configurações - caso contrário, não se preocupe, pois eles são fáceis de entender e usar.

As ações do GitHub são organizadas em torno de fluxos de trabalho. Um fluxo de trabalho é acionado por determinados eventos e contém um ou mais trabalhos a serem executados se o evento ocorrer. Uma tarefa consiste em uma ou mais etapas que executam uma ação cada.

Os arquivos devem ser armazenados na pasta `.github/workflows` de um repositório. Vamos dar uma olhada nas primeiras linhas do arquivo `ci.yml`, que será nosso workflow de CI:

Nome: Integração Contínua

sobre:

`workflow_dispatch:`

Empurre:

galhos:

- a Principal

pull_request:

empregos:

canalização:

run-on: ubuntu-latest

degraus:

- nome: ...

usos: ...

Isso já é bastante informação. Vamos analisar linha por linha:

Temos o básico do fluxo de trabalho configurado agora, então podemos começar a adicionar os estágios de construção.

Etapa 1 - Construir o projeto

As etapas são o que torna o GitHub Actions tão poderoso: aqui, você pode escolher entre uma grande quantidade de ações já existentes para usar em seu fluxo de trabalho. Eles são organizados no GitHub Marketplace (<https://github.com/marketplace>). Vamos adicionar algumas etapas ao fluxo de trabalho YAML, como segue:

degraus:

```
#####
```

```
# Fase 1 - Construir #
```

```
#####
```

```
- nome: Check-out da última revisão
```

```
usa: ações/ checkout@v3
```

```
- nome: Instalar PHP
```

```
usa: shivammathur/ setup-php@v2
```

```
com:
```

```
versão do php: '8.1'
```

```
cobertura: pcov
```

As ações mantidas pelo GitHub devem ser encontradas no namespace de ações. Em nosso exemplo, são ações/checkout, que é usado para fazer check-out do

repositório. Não precisamos especificar nenhum parâmetro por enquanto, pois esta ação usará automaticamente o repositório no qual este arquivo de fluxo de trabalho está localizado.

A anotação `@V3` é usada para especificar a versão principal a ser usada. Para ações/check-out, esta seria a versão 3. Observe que a versão secundária mais recente é sempre usada, que no momento da redação seria a versão 3.0.2.

A outra ação, `shivammathur/setup-php`, é fornecida por uma das muitas pessoas excelentes que disponibilizam seu trabalho como código aberto. Para esta etapa, estamos usando a palavra-chave `with` para especificar parâmetros adicionais. Neste exemplo, usamos a opção `php-version` para ter o PHP 8.1 instalado na máquina Ubuntu selecionada anteriormente. Usando o parâmetro de cobertura, podemos dizer ao `setup-php` para habilitar a extensão `pcov` para gerar relatórios de cobertura de código.

Parâmetros de ação

Ambas as ações apresentadas anteriormente oferecem muito mais parâmetros do que podemos descrever aqui. Você pode encontrar mais informações sobre suas funcionalidades procurando-as no Marketplace.

Em relação à formatação, utilizamos comentários e linhas em branco entre as etapas para deixar o arquivo mais legível. Não há convenção e cabe a você decidir como formatar seus arquivos YAML posteriormente.

O próximo passo é a instalação das dependências do projeto. Para PHP, isso geralmente significa executar `composer install`. Observe que não usamos a opção `--no-dev` porque precisamos instalar as dependências do desenvolvedor para realizar todas as verificações de qualidade. Vamos removê-los no final do pipeline novamente.

Gerenciamento de dependência

Usamos o fluxo de trabalho do Composer neste capítulo como exemplo para gerenciar nossas ferramentas de qualidade de código, porque essa é a maneira mais comum. No entanto, ambas as outras formas de organizar as ferramentas de qualidade de código que introduzimos em [Capítulo 9, Organizando Ferramentas de Qualidade do PHP](#), também funcionaria com o GitHub Actions. Nesse capítulo, também explicamos a opção `--no-dev` em detalhes.

É assim que as próximas etapas podem ser:

- name: Obtém o diretório de cache do compositor

id: composer-cache

run: echo "::set-output name=dir::\$(cache de configuração do compositor arquivos-dir)"

- nome: Dependências de cache

usa: ações/ cache@v2

com:

caminho: \${{ steps.composer-cache.outputs.dir }}

chave: \${{ runner.os }}-composer-\${{

hashFiles('**/composer.lock') }}

chaves de restauração: \${{ runner.os }}-composer-

- nome: Instalar dependências do compositor

executar: instalação do compositor

As ações do GitHub exigem algum trabalho manual para possibilitar o armazenamento em cache das dependências do Composer. Na primeira etapa, armazenamos a localização do diretório de cache do Composer, que obtemos do Composer usando o comando `config cache-files-dir`, em uma variável de saída chamada `dir`. Note `id: composer-cache` aqui — precisaremos disso para referenciar a variável na próxima etapa.

Em seguida, acessamos essa variável na próxima etapa usando a referência `steps.composer-cache.outputs.dir` (uma combinação do valor `id` que definimos na etapa anterior e o nome da variável) para definir o diretório que deve ser armazenado em cache pelas ações/ação de cache. `key` e `restore-key` são usados para gerar chaves de cache exclusivas, ou seja, as entradas de cache onde nossas dependências do Composer são armazenadas.

Por último, usamos o parâmetro `run` para executar diretamente a instalação do composer, como se fôssemos executá-lo localmente em uma máquina Ubuntu. É importante ter isso em mente: você pode, mas não precisa usar as ações existentes do GitHub para cada etapa — você também pode executar comandos shell puros (ou comandos equivalentes em executores do Windows).

Também existem ações no Marketplace que assumem a escrita de comandos,

como php-actions/composer. Não temos uma solução preferida aqui; ambos funcionarão bem.

Como queremos executar testes de integração na API do nosso aplicativo de exemplo, precisamos ter um servidor web em execução. Para o nosso caso de uso simples, é totalmente suficiente usar o servidor web interno do PHP, que podemos começar a usar na seguinte etapa:

- name: Iniciar o servidor da Web interno do PHP

execute: php -S localhost:8000 -t public &

A opção -S diz ao binário do PHP para iniciar um servidor web que está escutando no endereço localhost e na porta 8000. Como começamos na pasta raiz do nosso projeto, precisamos definir uma pasta raiz do documento (a pasta onde o servidor web procura arquivos para executar) usando a opção -t. Aqui, queremos usar a pasta pública, que contém apenas o arquivo index.php. É uma boa prática não armazenar nenhum outro código na pasta raiz do documento, pois isso torna mais difícil para os invasores invadirem nosso aplicativo.

PHP servidor web embutido

Por favor, note que o servidor web embutido do PHP deve ser usado apenas para fins de desenvolvimento. Nunca deve ser usado em produção, pois não foi construído pensando em desempenho ou segurança.

Você certamente notou o e comercial (&) no final do comando. Isso diz ao Linux

para executar o comando, mas não esperar por seu término. Sem ele, nosso fluxo de trabalho ficaria paralisado neste ponto porque o servidor da Web não é encerrado sozinho, pois precisa continuar ouvindo as solicitações até que executemos nossos testes de integração em um estágio posterior.

A configuração do nosso ambiente de construção está concluída. Agora, é hora de executar as primeiras verificações de qualidade de código em nosso aplicativo de amostra.

Etapa 2 - Análise de código

No primeiro estágio de compilação, criamos nosso ambiente de compilação e verificamos nosso código de aplicativo. Neste ponto, o aplicativo deve estar completamente funcional e pronto para ser testado. Agora, queremos fazer algumas análises de código estático.

A abordagem padrão é usar ações dedicadas do GitHub para cada ferramenta. A vantagem é que mantemos as ferramentas de desenvolvimento longe do ambiente de construção, pois elas serão executadas em containers Docker separados que serão descartados logo após o uso. No entanto, existem algumas desvantagens nessa abordagem.

Primeiramente, a cada ação, introduzimos mais uma dependência, e contamos com o autor para mantê-la atualizada e não perder o interesse em mantê-la depois de um tempo. Além disso, adicionamos alguma sobrecarga, já que as imagens do Docker geralmente são muitas vezes maiores que a ferramenta real. E, por último, quando a configuração de nosso aplicativo fica mais complicada, executar as ferramentas de qualidade de código em contêineres separados do Docker pode causar problemas, simplesmente porque não é o mesmo ambiente que o ambiente de construção. Às vezes, pequenas diferenças podem causar problemas que o mantêm ocupado por horas ou dias em resolvê-los.

Como vimos na seção anterior, podemos simplesmente executar comandos de shell do Linux no ambiente de compilação, portanto, nada contra a execução de nossas ferramentas de qualidade de código diretamente no ambiente de compilação - só precisamos nos certificar de removê-los posteriormente para que não seja lançado em produção.

Em nosso aplicativo de exemplo, adicionamos PHP-CS-Fixer e PHPStan à seção require-dev do arquivo composer.json. Ao adicionar as seguintes linhas de código ao nosso fluxo de trabalho YAML, permitiremos que sejam executadas nas próximas etapas:

```
#####
```

```
# Etapa 2 - Análise de Código #
```

```
#####
```

```
- nome: Code Style Fixer
```

```
execute: vendor/bin/php-cs-fixer fix --dry-run
```

```
- nome: Análise de Código Estático
```

```
execute: fornecedor/bin/phpstan
```

Não precisamos de muitos parâmetros ou opções aqui, pois nosso aplicativo de exemplo fornece os arquivos de configuração .php-cs-fixer.dist.php e phpstan.neon, que ambas as ferramentas procurarão por padrão. Somente para PHP-CS-Fixer usaremos a opção --dry-run porque queremos apenas verificar problemas durante o pipeline de CI/CD, e não resolvê-los.

Definindo o escopo das verificações

Para nosso pequeno aplicativo de exemplo, não há problema em executar as verificações anteriores em todos os arquivos porque elas serão executadas

rapidamente. No entanto, se nosso aplicativo crescer ou desejarmos introduzir CI/CD em um aplicativo existente (que discutiremos mais adiante neste capítulo), basta executar essas verificações nos arquivos que foram alterados apenas no último commit. A ação a seguir pode ser útil para você neste caso:<https://github.com/marketplace/actions/changed-files>.

Se nem o PHP-CS-Fixer nem o PHPStan relatarem problemas, podemos executar com segurança os testes automatizados na próxima etapa: os testes.

Fase 3 – Testes

Nosso código foi minuciosamente analisado e verificado quanto a bugs e erros de sintaxe, mas precisamos verificar se há erros lógicos em nosso código. Felizmente, temos alguns testes automatizados para garantir que não introduzimos nenhum bug inadvertidamente.

Pelas mesmas razões das ferramentas de qualidade de código no Estágio 2, não queremos usar uma ação dedicada para executar nossos conjuntos de teste do PHPUnit. Simplesmente executamos o PHPUnit como faríamos em nosso sistema de desenvolvimento local. O uso do arquivo phpunit.xml claramente se mostra útil aqui, pois não precisamos nos lembrar de todas as opções a serem usadas aqui. Vamos dar uma olhada no fluxo de trabalho YAML primeiro, como segue:

```
#####
```

```
# Fase 3 - Testes #
```

```
#####
```

```
- nome: Testes de unidade
```

```
execute: vendor/bin/phpunit --testsuite Unidade
```

```
- nome: Testes de Integração
```

```
execute: vendor/bin/phpunit --testsuite API
```

A única coisa que vale a pena notar aqui é que não apenas executamos todos os testes, mas os dividimos em duas suítes de teste: Unit e Api. Como nossos testes de unidade devem ser executados mais rapidamente, queremos executá-los (e falhar) primeiro, seguidos pelos testes de integração mais lentos. Observe que não adicionamos nenhum teste E2E, pois nosso aplicativo não é executado no navegador, mas é um mero serviço da web.

Dividimos os testes usando o arquivo de configuração phpunit.xml. O fragmento de código a seguir mostra seu nó `<testsuites>`, onde separamos as suítes por seu diretório (Api e Unit):

```
<testsuites>

<testsuite name="Api">

<diretório>testes/Api</diretório>

</testsuite>

<testsuite name="Unidade">

<diretório>testes/Unidade</diretório>

</testsuite>

</testsuites>
```

Também configuramos o PHPUnit para criar relatórios de cobertura de código, conforme ilustrado aqui:

```
<coverage processUncoveredFiles="false">
```



```
<incluir>

<directory suffix=".php">src</directory>

</include>

<relatório>

<html outputDirectory="relatórios/cobertura" />

<text outputFile="reports/coverage.txt" />

</relatório>

</coverage>
```

Para criar esses relatórios, o PHPUnit usará automaticamente a extensão pcov, que configuramos na Etapa 1. Eles serão gravados na pasta de relatórios, da qual cuidaremos na próxima etapa.

Isso já é tudo o que precisa ser feito para a etapa de testes. Se nossos testes não descobrirem nenhum erro, podemos ir para o último estágio de nosso pipeline e encerrar tudo.

Estágio 4 - Implantar

Nosso aplicativo agora é completamente verificado e testado. Antes de estarmos prontos para implantá-lo em qualquer ambiente que imaginarmos, precisamos primeiro remover as dependências de desenvolvimento. Felizmente, isso é muito fácil, como podemos ver aqui:

```
#####
```

```
# Estágio 4 - Implantar #
```

```
#####
```

- nome: Remover dependências do desenvolvedor

execute: composer install --no-dev --optimize-autoloader

Executar `composer install --no-dev` simplesmente excluirá todas as dependências dev da pasta do fornecedor. Outro recurso digno de nota é a opção `--optimize-autoloader` do Composer: como na produção, não adicionaremos ou alteraremos nenhuma classe ou namespace como faríamos no desenvolvimento, o autoloader do Composer pode ser otimizado não verificando nenhuma alteração e, portanto, acesso ao disco, acelerando um pouco.

Como última etapa, queremos criar artefatos de compilação: um artefato é a entrega — esse é o código que pretendemos implantar. O outro artefato são os relatórios de cobertura de código que criamos no Estágio 3. O GitHub Actions não manterá nenhum dado adicional além das informações de registro exibidas no site do GitHub após a execução do fluxo de trabalho YAML, portanto,

precisamos garantir que eles sejam armazenados em o fim. O código é ilustrado no trecho a seguir:

- name: Criar artefato de lançamento

usa: ações/ upload-artifact@v2

com:

nome: lançamento

caminho: |

público/

origem/

provedor/

- nome: Criar artefato de relatórios

usa: ações/ upload-artifact@v2

com:

nome: relatórios

caminho: relatórios/

Usamos a ação actions/upload-artifacts para criar dois arquivos ZIP (chamados aqui de artefatos): release e reports. O primeiro contém todos os arquivos e diretórios que precisamos para executar nosso aplicativo em produção e nada mais. Omitimos todos os arquivos de configuração na pasta raiz do nosso projeto, até mesmo os arquivos composer.json e composer.lock. Não precisamos mais deles, pois nossa pasta de provedores já existe.

O artefato de relatórios conterá apenas a pasta de relatórios. Após a compilação, você pode simplesmente baixar os dois arquivos ZIP separadamente no GitHub. Mais sobre isso na próxima seção.

Integrando o pipeline ao seu fluxo de trabalho

Depois de adicionar o fluxo de trabalho YAML à pasta `.github/workflows` (por exemplo, `.github/workflows/ci.yml`), você só precisa confirmar e enviá-lo para o repositório. Configuramos nosso pipeline para ser executado a cada PR aberto ou sempre que alguém envia um commit para o ramo principal.

Quando você abre <https://github.com> vá para a página do seu repositório, você encontrará uma visão geral do seu último fluxo de trabalho executado na guia Ações, conforme mostrado na captura de tela a seguir:

Workflows

[New workflow](#)

All workflows

Showing runs from all workflows

All workflows

[Continuous Integration](#)

34 workflow runs

Event ▾

Status ▾

Branch ▾

Actor ▾

 Add Deploy stageContinuous Integration #34: Commit a04fa8b pushed
by carstenwindler

main



22 hours ago

...



43s

 Add Deploy stageContinuous Integration #33: Commit c85b0ea pushed
by carstenwindler

main



22 hours ago

...



42s

 Add Deploy stageContinuous Integration #32: Commit 6b1dd1a pushed
by carstenwindler

main



23 hours ago

...



45s

 Add Deploy stageContinuous Integration #31: Commit 4c2682d pushed
by carstenwindler

main



23 hours ago

...



41s

 Add Deploy stageContinuous Integration #30: Commit 7fe06da pushed
by carstenwindler

main



yesterday

...



43s

Figura 11.3: A página do repositório em github.com

A marca de seleção verde marca as execuções bem-sucedidas, enquanto a cruz vermelha - é claro - marca as que falharam. Você também pode ver quando eles foram executados e quanto tempo isso levou. Ao clicar nos três pontos no lado direito de cada entrada, você encontrará mais opções, por exemplo, onde você pode excluir a execução do fluxo de trabalho. Clicando no título da execução, que é a mensagem de confirmação correspondente da execução, você entrará na página Resumo, conforme mostrado na captura de tela a seguir:

✓ Add Deploy stage Continuous Integration #34

Re-run all jobs



Summary

Jobs

✓ pipeline

Triggered via push 22 hours ago

Status

Total duration

carstenwindler pushed a04fa8b main

Success

43s

Billable time

Artifacts

1m

2

ci.yml

on: push

✓ pipeline

33s



Artifacts

Produced during runtime

Name

Size



release

953 KB



reports

764 KB



Figura 11.4: A página de resumo da execução do fluxo de trabalho

Aqui, você pode ver todos os trabalhos do fluxo de trabalho. Como nosso exemplo consiste apenas em um trabalho, pipeline, você verá apenas um. Nesta página, você também pode encontrar quaisquer artefatos gerados (como nossos artefatos de lançamento e relatórios) e baixá-los ou excluí-los. O GitHub oferece apenas espaço em disco limitado gratuitamente, portanto, exclua-os quando estiver ficando sem espaço.

Outra informação importante é o tempo faturado. Embora nosso trabalho tenha executado apenas 43 segundos no total, o GitHub deduzirá 1 minuto do seu uso mensal. O GitHub oferece um plano gratuito generoso, mas você deve verificar seu uso de tempos em tempos. Você pode encontrar mais informações sobre isso na página de configurações do usuário na seção Faturamento e planos (<https://github.com/settings/billing>).

Se você quiser ver exatamente o que acontece durante a execução do fluxo de trabalho, por exemplo, se algo der errado, clique no trabalho do pipeline para obter uma visão geral detalhada de todas as etapas, conforme ilustrado na captura de tela a seguir:

✓ Add Deploy stage Continuous Integration #34

[Re-run all jobs](#)

Summary

Jobs

✓ pipeline

pipeline

succeeded 22 hours ago in 33s



- > ✓ Set up job 1s
- > ✓ Checkout latest revision 1s
- ▼ ✓ Install PHP 1s
 - 1 ▶ Run shivammathur/setup-php@v2
 - 6 /usr/bin/bash /home/runner/work/_actions/shivammathur/setup-php/v2/src/scripts/run.sh
 - 7
 - 8 ==> Setup PHP
 - 9 ✓ PHP Found PHP 8.1.7
 - 10
 - 11 ==> Setup Tools
 - 12 ✓ composer Added composer 2.3.9
 - 13
 - 14 ==> Setup Coverage
 - 15 ✓ pcov.enabled=1 Added to php.ini
 - 16 ✓ coverage: pcov PCOV 1.0.11 enabled as coverage driver
 - 17
 - 18 ==> #StandWithUkraine
 - 19 ✓ read-more <https://setup-php.com/support-ukraine>
- > ✓ Get composer cache directory 0s
- > ✓ Cache dependencies 1s
- > ✓ Install composer dependencies 7s

Figura 11.5: página de detalhes do trabalho

Cada etapa pode ser expandida e recolhida para obter informações adicionais sobre o que exatamente aconteceu durante sua execução. Na captura de tela anterior, expandimos a etapa Instalar PHP para ver o que a ação fez em detalhes.

Parabéns, agora você tem um pipeline de CI funcionando para o seu projeto! Isso encerra nosso pequeno tour pelo GitHub Actions. Claro, você pode estender o pipeline como quiser, por exemplo, fazendo upload do artefato de lançamento para um servidor SSH File Transfer Protocol (SFTP) ou um bucket do AWS Simple Storage Service (S3). Há muito mais do que pode ser feito, então certifique-se de experimentar.

Na próxima seção, mostraremos como você pode configurar seu pipeline local. Isso economizará algum tempo e provavelmente até custos, evitando fluxos de trabalho desnecessários por meio de verificações antecipadas.

Seu pipeline local – Git hooks

Depois de configurar com sucesso um pipeline de CI/CD simples, mas já muito útil, agora queremos executar algumas etapas já no ambiente de desenvolvimento local, mesmo antes de confirmá-las no repositório. Isso pode soar como trabalho duplo agora - por que deveríamos executar as mesmas ferramentas duas vezes?

Lembre-se da Figura 11.1 – Custos relativos estimados de correção de um bug com base no tempo de sua detecção, desde o início do capítulo: quanto mais cedo detectarmos um bug, menos custos ou menos esforço ele causará. Claro, se encontrarmos um bug durante o pipeline de CI/CD, isso ainda será muito mais cedo do que no ambiente de produção.

O pipeline não vem de graça, no entanto. A construção de nosso aplicativo de exemplo foi rápida e levou apenas um minuto. Imagine, no entanto, uma configuração completa do Docker que já leva um tempo considerável para criar todos os contêineres necessários. E agora, ele falha ao construir apenas por causa de um pequeno bug que você poderia ter resolvido em 2 minutos se não tivesse esquecido de executar testes de unidade antes de confirmar seu código. Você pode ter acabado de fazer uma merecida pausa para chá ou café, apenas para descobrir que a compilação falhou por causa disso quando você voltou. Isso é irritante e um desperdício de dinheiro, bem como de poder computacional.

Exatamente essas verificações de execução rápida, como testes de unidade, um sniffer de código ou análise de código estático, são o que queremos executar antes de iniciarmos uma compilação completa para nossas alterações. Não

podemos confiar em nós mesmos para executar essas verificações automaticamente porque somos humanos. Esquecemos as coisas, mas as máquinas não.

Se você usa o Git para o seu desenvolvimento, o que a maioria dos desenvolvedores faz hoje, podemos utilizar a funcionalidade interna dos ganchos do Git para automatizar essas verificações. Git hooks são scripts de shell que são executados automaticamente em determinados eventos, como antes ou depois de cada confirmação.

Para nossas necessidades, o hook pre-commit é particularmente útil. Ele será executado toda vez que você executar o comando `git commit` e poderá abortar o commit se o script executado retornar um erro. Nesse caso, nenhum código seria adicionado ao repositório.

Configurando ganchos do Git

A configuração manual dos ganchos do Git requer algum conhecimento de shell script, então queremos usar um pacote chamado CaptainHook para nos ajudar. Com esta ferramenta, podemos instalar qualquer gancho que quisermos e até usar alguns recursos avançados sem a necessidade de dominar o Linux.

Você pode facilmente baixar o Phar usando Phive (veja Capítulo 9, Organizando Ferramentas de Qualidade do PHP, para mais informações sobre isso), ou use o Composer para instalá-lo, como faremos agora:

```
$ composer require --dev captainhook/captainhook
```

Em seguida, precisamos criar um arquivo `captainhook.json`. Este arquivo contém a configuração do hook para seu projeto. Como esse arquivo será adicionado ao repositório, garantimos que outros desenvolvedores de nossa equipe possam usá-lo. Para criar este arquivo, poderíamos executar o seguinte comando:

```
$ fornecedor/bin/captainhook configurar
```

O CaptainHook fará algumas perguntas e gerará um arquivo de configuração com base em suas respostas. No entanto, você pode pular esta etapa e criar um arquivo diretamente, como faremos agora. Abra seu editor favorito e escreva o seguinte código:

```
{  
  "configuração": {  
    "falha no primeiro erro": verdadeiro  
  },  
  "pré-commit": {  
    "habilitado": verdadeiro,  
    "ações": [  
      {  
        "action": "fornecedor/bin/php-cs-fixer  
corrigir --dry-run"  
      },  
      {  
        "ação": "fornecedor/bin/phpstan"  
      }  
    ]  
  }  
}
```

Cada gancho tem sua própria seção. Na seção de gancho pré-confirmação, habilitado pode ser verdadeiro ou falso - o último desabilita o gancho, mas mantém a configuração no arquivo, o que pode ser útil para fins de depuração.

ações contêm os comandos reais a serem executados. Como você pode ver, esses comandos são como os que você já conhece de Capítulo 7, Ferramentas de qualidade de código.

Cada ação que você deseja executar precisa ser escrita em uma seção de ação separada. No exemplo anterior, configuramos PHP-CS-Fixer e PHPStan para serem executados no pré-confirmação.

Uma vez que temos arquivos de configuração adicionais para ambas as ferramentas, não precisamos especificar nenhuma opção adicional, exceto dizer ao PHP-CS-Fixer para fazer apenas uma simulação—isto é, apenas nos informar quando uma violação de estilo de código for encontrada.

Na seção de configuração, você pode especificar outros parâmetros de configuração. Queremos interromper a execução do gancho imediatamente após a ocorrência de um erro, portanto, definimos fail-on-first-error como verdadeiro. Caso contrário, o CaptainHook executaria primeiro todas as verificações e, em seguida, informaria os resultados. Claro que isso é apenas uma questão de gosto pessoal.

Documentação do CaptainHook

Não podemos listar todos os recursos do CaptainHook neste livro. No entanto, recomendamos que você verifique a documentação oficial em <https://captainhookphp.github.io/captainhook> para saber mais sobre esta ferramenta.

Como terminamos agora com a configuração, armazene este arquivo JavaScript Object Notation (JSON) sob o nome captainhook.json na pasta raiz do projeto. Isso já é tudo o que temos que fazer em relação à configuração.

Só precisamos instalar os ganchos agora, ou seja, gerar arquivos de gancho em .git/hooks. Isso pode ser feito simplesmente assim:

```
$ fornecedor/bin/captainhook install -f
```

Usamos a opção -f aqui, que significa força. Sem essa opção, o CaptainHook nos pediria cada gancho separadamente, se quisermos instalá-lo. Observe que o CaptainHook instalará um arquivo para cada gancho do Git que ele suporta, mesmo para aqueles que você não configurou. Esses ganchos não farão nada, no entanto.

Para testar o gancho de pré-confirmação, você pode executá-lo manualmente sem precisar confirmar nada usando o seguinte comando:

```
$ vendor/bin/captainhook hook:pre-commit
```

Comandos semelhantes estão disponíveis para todos os outros ganchos suportados pelo CaptainHook. Se você fez alterações no arquivo captainhook.json, não se esqueça de instalá-lo usando o comando install -f novamente.

Para garantir que os ganchos sejam instalados no ambiente de desenvolvimento local, você pode adicionar o seguinte código à seção de scripts de seu arquivo `composer.json`:

```
"post-autoload-dump": [  
  
    "if [ -e fornecedor/bin/captainhook ]; então  
  
    fornecedor/bin/captainhook install -f -s; fi"  
  
]
```

Usamos o evento `post-autoload-dump` do Composer para executar o comando `install -f`. O comando será executado toda vez que o autoloader do Composer for atualizado, o que acontecerá sempre que a instalação do composer ou a atualização do composer for executada. Dessa forma, garantimos que os hooks sejam instalados ou atualizados regularmente nos ambientes de desenvolvimento de qualquer pessoa que trabalhe neste projeto. Usando `if [ -e vendor/bin/captainhook ]`, verificamos se o binário CaptainHook existe e evitamos interromper a compilação do CI se ele não estiver instalado.

Ganchos Git na prática

Concluimos a configuração do gancho de pré-confirmação, testamos e instalamos. Agora, estamos prontos para vê-lo em ação: se você fizer qualquer alteração no código do aplicativo - por exemplo, adicionando uma linha em branco em algum lugar no arquivo `ProductController.php` - e depois tentar confirmar as alterações, o gancho de pré-confirmação deve ser executado. Se as alterações violarem o padrão PSR-12, a etapa PHP-CS-Fixer deverá falhar, conforme mostrado na captura de tela a seguir:

```
$ git add src/Controller/ProductController.php
```

```
$ git commit -m "Some arbitrary changes"
```

```
pre-commit:
```

```
- vendor/bin/php-cs-fixer fix --dry-run : failed
```

```
failed to execute: vendor/bin/php-cs-fixer fix --dry-run
```

```
1) src/Controller/ProductController.php
```

```
Checked all files in 0.007 seconds, 14.000 MB memory used
```

```
Loaded config default from "/home/curtis/dev/github-actions-test/.php-cs-fixer.dist.php".
```

```
Using cache file ".php-cs-fixer.cache".
```

```
$ □
```

Figura 11.6: O hook pre-commit falha

Corrigindo problemas de estilo de código automaticamente

Obviamente, você pode remover a opção `--dry-run` ao executar o PHP-CS-Fixer para permitir que ele corrija os problemas automaticamente. Na verdade, esta é uma prática comum e nós encorajamos você a experimentar o mesmo. No entanto, requer um pouco mais de trabalho, porque você deve informar ao usuário que seus arquivos alterados foram corrigidos e que precisam ser confirmados novamente. Para manter este exemplo simples, decidimos omiti-lo.

Agora sabemos que o arquivo `ProductController.php` deve ser corrigido. Podemos deixar o PHP-CS-Fixer fazer o trabalho, conforme demonstrado aqui:

```
$ vendor/bin/php-cs-fixer fix
```

```
Loaded config default from "/home/curtis/dev/github-actions-test/.php-cs-fixer.dist.php".
```

```
Using cache file ".php-cs-fixer.cache".
```

```
1) src/Controller/ProductController.php
```

```
Fixed all files in 0.007 seconds, 14.000 MB memory used
```

```
$
```

Figura 11.7: Usando PHP-CS-Fixer para corrigir automaticamente problemas de estilo de código

ProductController.php foi alterado novamente agora e essas alterações adicionais ainda não foram testadas - ou seja, ainda não foram adicionadas ao commit. As mudanças anteriores ainda são encenadas, no entanto. A captura de tela a seguir mostra como seria se você executasse `git status` neste ponto:

```
$ git status
```

```
On branch main
```

```
Your branch is ahead of 'origin/main' by 5 commits.
```

```
(use "git push" to publish your local commits)
```

```
Changes to be committed:
```

```
(use "git restore --staged <file>..." to unstage)
```

```
    modified:   src/Controller/ProductController.php
```

```
Changes not staged for commit:
```

```
(use "git add <file>..." to update what will be committed)
```

```
(use "git restore <file>..." to discard changes in working directory)
```

```
    modified:   src/Controller/ProductController.php
```

```
$ █
```


Figura 11.8: Alterações não preparadas

Tudo o que precisa ser feito agora é adicionar o arquivo `ProductController.php` novamente e executar `git commit` novamente, conforme demonstrado na captura de tela a seguir:

```
$ git add src/Controller/ProductController.php
```

```
$ git commit -m "Some arbitrary changes"
```

```
pre-commit:
```

```
- vendor/bin/php-cs-fixer fix --dry-run : done  
- vendor/bin/phpstan : done
```

```
On branch main
```

```
Your branch is ahead of 'origin/main' by 5 commits.  
(use "git push" to publish your local commits)
```

```
nothing to commit, working tree clean
```

```
$ █
```

Figura 11.9: passes de hook pré-commit

Ambas as etapas do gancho de pré-confirmação passam agora. Tudo o que você precisa fazer agora é git push as alterações confirmadas.

Uso avançado

O exemplo anterior foi muito básico. Claro, há muito mais que você pode fazer no ambiente de desenvolvimento local. Você poderia, por exemplo, adicionar mais ferramentas como o detector de copiar e colar `phpcpd` ou o detector de bagunça `phpmd`, ambos introduzidos em [Capítulo 7, Ferramentas de qualidade de código](#).

Se seus testes não forem muito lentos (o que exatamente isso significa depende da sua paciência e da de seus colegas de equipe), considere executar seus testes localmente também. Mesmo se você tiver testes de execução lenta, poderá separá-los em vários conjuntos de testes e executar apenas os testes de execução rápida no pré-confirmação.

Você também deve considerar a execução de verificações de qualidade de código apenas em arquivos modificados e não em todo o projeto, como fizemos em nosso exemplo. `CaptainHook` fornece o espaço reservado útil `{ $STAGED_FILES }`, que contém todos os arquivos preparados. É muito conveniente de usar, como podemos ver aqui:

```
{
```

```
"pré-commit": {
```

"habilitado": verdadeiro,

"ações": [

{

"action": "fornecedor/bin/php-cs-fixer fix

{\${STAGED_FILES}|do tipo:php} --dry-run"

},

{

"ação": "analisar fornecedor/bin/phpstan

{\${STAGED_FILES}|do tipo:php}"

}

```
]
```

```
}
```

```
}
```

O exemplo anterior executa as verificações apenas em arquivos PHP modificados. Isso tem dois benefícios principais: em primeiro lugar, é mais rápido porque você não precisa verificar o código que não tocou. A aceleração, é claro, depende do tamanho da sua base de código.

Em segundo lugar, especialmente se você estiver trabalhando em um projeto existente e apenas começou a introduzir essas verificações, executá-las em toda a base de código não é uma opção porque você precisaria corrigir muitos arquivos de uma só vez. Discutiremos isso com mais detalhes na próxima seção.

Excursion – Adicionando CI ao software existente

Se você trabalha em uma empresa, nem sempre começará do zero, ou seja, construirá um novo projeto do zero. Na verdade, muito provavelmente será o contrário: ao ingressar em uma empresa, você será adicionado a uma equipe que já trabalha em um ou mais projetos há muito tempo.

Você provavelmente já se deparou com os termos software legado ou sistema legado. Em nosso contexto, eles descrevem um software que existe há muito tempo e ainda é usado em processos críticos de negócios. Ele não atende mais aos padrões de desenvolvimento modernos, portanto não pode ser facilmente atualizado ou alterado. Com o tempo, torna-se tão frágil e difícil de manter que nenhum desenvolvedor quer mais tocá-lo. O que torna ainda pior é o fato de que, como o sistema cresceu ao longo do tempo, ele tem tantas funcionalidades que nenhuma parte interessada (isto é, os usuários) gostaria de perdê-la. Portanto, substituir um sistema legado não é tão fácil.

Não surpreendentemente, o software legado tem uma conotação ruim, mas provavelmente tudo o que o sistema precisa é de alguma “atenção”. Pense nisso como a restauração de uma máquina antiga, onde as peças antigas são substituídas por outras modernas, enquanto o exterior permanece inalterado. Alguns desenvolvedores acham ainda mais desafiador trabalhar em tal software. Ele percorreu um longo caminho, ganhou seu dinheiro e - provavelmente (pelo menos em parte) - contribuiu para o sucesso da empresa, por isso merece algum respeito.

Portanto, se você tiver que trabalhar em um projeto desses, não desista

rapidamente. Com este livro, fornecemos a você o conhecimento e as ferramentas necessárias para começar a colocá-lo em forma novamente - só vai demorar um pouco mais e você pode nunca atingir um nível perfeito. Mas ser perfeito não é necessário de qualquer maneira.

Passo a passo

Comece adicionando testes de integração e E2E. Ambos os tipos de teste geralmente requerem nenhuma ou apenas algumas alterações no código, mas trarão grandes benefícios, pois cobrem indiretamente uma grande quantidade de código sem a necessidade de escrever testes de unidade. Depois de cobrir os caminhos críticos (ou seja, os fluxos de trabalho mais usados) do aplicativo com testes, você pode começar a refatorar as classes e começar a introduzir testes de unidade adicionais. Os testes irão ajudá-lo a descobrir bugs e efeitos colaterais rapidamente, sem ter que clicar repetidamente no aplicativo.

Apresentar um estilo de código como o PSR-12 é, como você já sabe, tão fácil quanto executar uma ferramenta como PHP-CS-Fixer em toda a base de código de uma só vez. O commit resultante será, obviamente, enorme, então você deve concordar com o congelamento de código com qualquer outro desenvolvedor antes de fazê-lo. Um congelamento de código significa que todos confirmam suas alterações no repositório para que sua refatoração não cause grandes conflitos de mesclagem quando verificarem as alterações posteriormente.

Para decidir qual código refatorar, pretendemos usar uma ou mais das muitas ferramentas de qualidade de código que você já conhece. Indo com PHPStan no nível 0 é uma boa escolha. Você também pode considerar o Salmo, pois ele também pode resolver alguns problemas automaticamente. Dependendo do tamanho do projeto, a lista de erros pode ser assustadoramente longa. Fazendo uso do recurso de linha de base, conforme descrito em [Capítulo 7, Code Quality Tools](#), pode ser uma ajuda cosmética aqui, mas apenas ocultará e não resolverá os problemas de código.

Você não precisa se apressar. Se você configurar seu pipeline de CI/CD para verificar apenas arquivos modificados, poderá começar a melhorar o código ao longo do tempo, peça por peça. Isso deixa você com o problema de que, depois de tocar em um arquivo, você deve refatorá-lo para atender às regras. Especialmente para classes antigas, mas enormes, isso pode ser problemático. Dentro do Capítulo 7, Ferramentas de qualidade de código, no entanto, explicamos como você pode excluir arquivos ou partes do código das verificações realizadas. Você pode até configurar um pipeline que permita ignorar as verificações de uma determinada palavra-chave (por exemplo, ignorar ci) na mensagem de confirmação. Essa abordagem, no entanto, deve ser apenas o último recurso — caso contrário, você nunca começará a refatorar o código antigo. É preciso algum autocontrole dos desenvolvedores para não fazer mau uso desse recurso também.

Com o tempo, a equipe que trabalha no projeto ganhará mais confiança e, com a crescente cobertura de teste, começará a refatorar cada vez mais código. Certifique-se de instalar um pipeline local também, para manter os tempos de espera curtos.

Uma visão do CD

Eventualmente, seu pipeline de CI funcionará tão bem que você poderá confiar totalmente nele. Isso impedirá o envio confiável de código quebrado para a produção e, em algum momento, você se verá fazendo menos verificações manuais se a implantação foi bem. Nesse ponto, você pode pensar em usar CD: isso descreve a combinação de ferramentas e processos para implantar código em qualquer ambiente automaticamente.

Um fluxo de trabalho normal é que, sempre que as alterações forem mescladas em uma determinada ramificação (por exemplo, principal para o ambiente de produção), o pipeline de CI/CD será acionado automaticamente. Se as alterações passarem em todas as verificações e testes, o processo é tão confiável que o código é implantado no destino desejado sem testar mais o resultado da compilação manualmente.

Se você já teve a oportunidade de trabalhar em um ambiente assim, com certeza não vai querer perder. Além de um ótimo pipeline de CI/CD e 99% de confiança nele, ele requer mais alguns processos para reagir rapidamente se uma implantação tiver problemas. Mesmo as melhores ferramentas não podem evitar erros lógicos ou problemas de infraestrutura que só aparecerão sob uma carga maior.

Sempre que houver um problema após a implantação, sua equipe deve ser a primeira a perceber! Você não só precisa confiar totalmente no pipeline, mas também na configuração de monitoramento e registro. Existem muitos conceitos e ferramentas por aí, e finalmente estamos deixando o tópico de qualidade de

código aqui, entrando nos domínios das operações de desenvolvimento (DevOps) e administração do sistema. No entanto, queremos fornecer uma breve orientação sobre alguns conceitos-chave nos quais você pode se aprofundar, como segue:

Certamente, o CD está além do escopo de escrever um código PHP limpo. No entanto, achamos que é um objetivo que vale a pena almejar, pois acelerará muito o seu desenvolvimento e apresentará uma variedade de ferramentas e conceitos fascinantes.

Resumo

Esperamos que, depois de ler este capítulo, você esteja tão convencido quanto nós de que a CI é extremamente útil e, portanto, uma ferramenta obrigatória em sua caixa de ferramentas. Explicamos os termos necessários em torno deste tópico, bem como as diferentes etapas de um pipeline, não apenas na teoria, mas também na prática, criando um pipeline simples, mas funcional, usando o GitHub Actions. Finalmente, demos a você uma visão geral do CD.

Agora você tem uma grande base de conhecimento e ferramentas para escrever um ótimo código PHP. É claro que o aprendizado nunca para, e há muito mais conhecimento disponível para você descobrir que não caberia neste livro.

Se você fez do desenvolvimento de software PHP sua profissão, geralmente trabalha em equipes de desenvolvedores. E mesmo se você estiver mantendo seu próprio projeto de código aberto, você interagirá com outras pessoas, por exemplo, quando elas enviarem alterações ao seu código. CI é um bloco de construção importante, mas não é a única coisa que você precisa considerar para uma configuração de equipe bem-sucedida.

Para nós, esse tópico é tão importante que dedicamos os próximos dois capítulos à introdução de técnicas modernas de colaboração que ajudarão você a escrever um ótimo código PHP ao trabalhar em equipe. Esperamos vê-lo no próximo capítulo!

Leitura adicional

Se você deseja saber mais, dê uma olhada nos seguintes recursos:

Trabalhando num time

O principal objetivo deste livro é permitir que você escreva um código que possa ser entendido, mantido e estendido por você e por outras pessoas. Na maioria das vezes, ser um desenvolvedor PHP significa que você não trabalha sozinho em um projeto ou ferramenta. E mesmo que você tenha começado a escrever código sozinho, as chances são altas de que, em algum momento, outro desenvolvedor se junte a você - seja em um produto comercial ou em seu pacote de código aberto, onde outros desenvolvedores começam a adicionar novos recursos ou correções de bugs.

Sempre haverá várias maneiras de realizar uma tarefa no desenvolvimento de software. Isso é o que torna o trabalho em equipe mais desafiador quando você deseja escrever um código limpo em conjunto. Neste capítulo, você encontrará várias dicas e práticas recomendadas sobre como configurar padrões e diretrizes de codificação. Também falaremos sobre como as revisões de código irão melhorar o código e garantir que as diretrizes sejam mantidas.

Também exploraremos o tópico de padrões de projeto com mais detalhes no final deste capítulo. Esses padrões podem ajudar sua equipe a resolver problemas típicos de desenvolvimento de software porque oferecem soluções bem testadas.

Este capítulo incluirá as seguintes seções:

Requerimentos técnicos

Se você acompanhou os capítulos anteriores, não precisa de nenhuma configuração adicional.

Os exemplos de código para este capítulo podem ser encontrados em nosso repositório GitHub:<https://github.com/PacktPublishing/Clean-Code-in-PHP>.

Padrões de codificação

Nos capítulos anteriores, você aprendeu muito sobre como escrever código de alta qualidade. No entanto, não é suficiente se você fizer isso sozinho. Quando você trabalha em equipe, provavelmente terá o problema de que outros desenvolvedores têm uma compreensão diferente da qualidade e estão em um nível de habilidade diferente do seu.

Isso é prejudicial ao seu código porque pode levar a compromissos preguiçosos, onde as partes envolvidas concordam em uma maneira, apenas para ter paz. Portanto, se você deseja trabalhar de maneira eficaz em equipe, padronize seu trabalho o máximo possível.

Faz sentido começar com a fruta mais fácil: formatação de código. Isso se resume ao básico, como concordar quantos espaços devem ser usados para recuar linhas ou onde as chaves devem ser colocadas. Mas por que isso é importante?

Já abordamos brevemente este tópico em capítulo 5, Otimizando Seu Tempo e Separando Responsabilidades. No entanto, queremos expandi-lo neste ponto. A principal vantagem de ter um padrão de codificação comum (também chamado de estilo de codificação) é reduzir o atrito cognitivo ao ler o código.

Atrito cognitivo

O atrito cognitivo basicamente descreve o esforço mental necessário para o nosso cérebro processar informações. Imagine, por exemplo, que você lê um livro, onde todos os parágrafos foram escritos em uma fonte, tamanho ou espaçamento de linha diferente. Você ainda seria capaz de lê-lo, mas logo se tornaria irritante ou cansativo. O mesmo se aplica à leitura de código.

Introduzir um padrão de codificação em um projeto é relativamente fácil, graças às ferramentas que já apresentamos anteriormente neste livro. Por outro lado, concordar com os outros sobre um padrão comum exige mais trabalho. É por isso que, nesta seção, queremos mostrar a você como alinhar facilmente em um padrão de codificação comum.

Indo com os padrões existentes

Estabelecer padrões junto com outras pessoas pode ser um processo longo e doloroso. Porém, hoje em dia, você não discute mais sobre o tamanho de uma folha de papel. Nos países europeus, o padrão DIN A4 é amplamente aceito, enquanto em outros países, como os EUA, você usaria o tamanho de carta dos EUA sem perguntar por quê. A maioria das pessoas aceita essas medidas e seguir esses padrões torna a vida um pouco mais fácil – uma coisa a menos com que se preocupar.

O mesmo se aplica aos padrões de codificação, que definem como você formata seu código. Claro, você pode discutir por horas com seus colegas de equipe sobre se tabulações ou espaços devem ser usados para o recuo. Ambos os lados apresentarão argumentos válidos e você nunca encontrará a resposta certa, pois simplesmente não há certo e errado aqui. E uma vez que você tenha resolvido a questão sobre indentação, o próximo tópico a ser discutido pode ser a colocação de colchetes. Eles devem aparecer na mesma linha ou na seguinte?

Não precisamos necessariamente concordar com todos os detalhes de um padrão, mas, sem dúvida, economiza tempo e nervos usar as normas existentes. No ecossistema PHP, já existem padrões de codificação que você pode utilizar. Um grande benefício adicional de fazer isso é que os farejadores de código têm conjuntos de regras integrados para esses padrões. Na próxima seção, falaremos sobre provavelmente o padrão de codificação mais conhecido para PHP.

PHP-FIG e PSR

O próprio PHP não possui padrão de codificação oficial. Historicamente, cada grande framework PHP que existiu, ou ainda existe hoje, introduziu algum tipo de padrão porque os desenvolvedores rapidamente perceberam que usá-los trazia seus benefícios.

Porém, como cada projeto utilizava seus próprios padrões, o mundo PHP acabou com uma mistura de diferentes padrões de formatação. Em 2009, quando o PHP-FIG (PHP Framework Interoperability Group (PHP-FIG) foi formado, que consistia em membros de todos os projetos e frameworks PHP importantes da época, eles queriam resolver exatamente esses tipos de problemas.

Naquela época, o Composer estava se tornando cada vez mais importante e foram introduzidos pacotes que podiam ser facilmente usados em diferentes estruturas. Para manter o código um tanto consistente, uma maneira mútua de escrever código foi acordada: nasceram as Recomendações padrão do PHP (PSRs).

Para fazer o autoloader do Composer funcionar, era necessário concordar sobre como nomear classes e diretórios. Isso foi feito com a primeira recomendação padrão, PSR-0 (sim, os nerds começam a contar em 0), que acabou sendo substituído pelo PSR-4.

A primeira recomendação do Padrão de Codificação foi introduzida com PSR-1 e PSR-2. PSR-2 foi posteriormente substituído por PSR-12, que continha regras para recursos de linguagem de versões mais recentes do PHP.

Embora o PSR-12 aborde o estilo do código, ele não cobre as convenções de nomenclatura ou como estruturar o código. Muitas vezes, isso ainda é predefinido pela estrutura que você usa. A estrutura do Symfony, por exemplo, tem seu próprio conjunto de Padrões de Codificação que são baseados nos já mencionados PSR-4 e PSR-12, mas adicionam outras diretrizes, por exemplo, convenções sobre nomenclatura ou documentação. Mesmo se você não usar um framework e apenas escolher componentes individuais para construir um aplicativo, você pode considerar usar estas diretrizes, que você encontrará no site do Symfony: <https://symfony.com/doc/current/contributing/code/standards.html>.

estilo de codificação PER

PSR-12 foi lançado em 2019 e, portanto, não cobre mais os recursos mais recentes do PHP. Portanto, no momento em que este livro foi escrito, o PHP-FIG lançou o PER Coding Style 1.0.0 (PER é a abreviação de PHP Extended Recommendation). É baseado no PSR-12 e contém algumas adições a ele. No futuro, o PHP-FIG não planeja mais lançar novos Padrões de Codificação relacionados a PSRs, mas novas versões deste PER, se necessário. É muito provável que as ferramentas de qualidade de código apresentadas neste livro adotem o novo PER em breve. Você encontrará mais informações sobre isso aqui: <https://www.php-fig.org/per/coding-style>.

Com o tempo, o PHP-FIG introduziu mais de uma dúzia de recomendações, e outras estão sendo feitas. Eles cobrem tópicos como como integrar log, cache e clientes HTTP, para citar apenas alguns. Você encontrará uma lista completa no site oficial: <https://www.php-fig.org>.

Problemas com PHP-FIG e PSR

O PHP-FIG não deve ser considerado a autoridade oficial do PHP, e nenhum PSR deve ser tomado como indiscutível. De fato, muitos frameworks importantes como Symfony ou Laravel não fazem mais parte do PHP-FIG, pois os padrões recomendados interferiram demais em seus internos. Olhando para todos os PSRs que estão disponíveis hoje, você pode considerá-los como sua própria metaestrutura. No entanto, isso não diminui a relevância de muitas recomendações - só queremos que você não as aceite cegamente como concedidas.

Aplicação de padrões de codificação em seu IDE

Existem várias maneiras de impor padrões de codificação. No capítulo anterior, Capítulo 11, Integração contínua, explicamos como garantir que nenhum código formatado incorretamente possa estragar a base do código. Isso funcionou bem, mas requer uma etapa adicional, mesmo se permitirmos que nossas ferramentas corrijam a formatação do código automaticamente porque precisamos confirmar os arquivos alterados novamente. Então, não seria realmente útil se nosso editor de código, ou IDE, nos ajudasse a formatar o código enquanto o escrevíamos?

Os editores de código modernos geralmente têm funcionalidade integrada que o ajuda a aderir aos seus padrões de codificação preferidos, se você os configurar. Se não for incorporada, essa funcionalidade pode ser fornecida pelo menos com plug-ins.

Existem duas maneiras básicas pelas quais seu editor pode apoiá-lo:

Reformatar o código ao salvar o arquivo é uma maneira muito conveniente de garantir que seu código atenda aos padrões de codificação. Como configurar isso depende de qual IDE você está usando, então não vamos detalhar isso neste livro.

Ainda recomendamos o uso de ganchos Git e integração contínua como a segunda camada de verificações para garantir que nenhum código mal formatado seja enviado para o repositório do projeto. Você nunca pode ter certeza se um membro da equipe desativou acidentalmente ou voluntariamente a reformatação

automatizada ou não se importou com as partes destacadas do código.

Padrões de Codificação são sobre como formatar o código de forma consistente. Mas isso não é tudo o que você deve concordar ao trabalhar em equipe – na próxima seção, mostraremos quais outros aspectos valem a pena concordar.

Diretrizes de codificação

Na seção anterior, falamos sobre por que você deve introduzir os Padrões de Codificação. Uma vez feito isso, você deve considerar a criação de diretrizes de codificação. Ambos os tópicos parecem muito familiares e, de fato, são. No entanto, enquanto os padrões de codificação geralmente se concentram em como formatar o código, as diretrizes de codificação definem como escrever o código. Isso, é claro, inclui definir qual padrão de codificação usar, mas abrange muito mais, como você aprenderá nesta seção.

O que exatamente significa como escrever código? Normalmente, há mais de uma maneira de alcançar as coisas ao escrever um software. Pegue o amplamente conhecido padrão model-view-controller (MVC), por exemplo. Ele é usado para dividir a lógica do aplicativo em três tipos de elementos interconectados – os modelos, as visualizações e os controladores. No entanto, ele não define explicitamente onde colocar a lógica de negócios. Deve estar localizado dentro dos controladores, ou melhor, dentro dos modelos?

Não há uma resposta clara certa ou errada para esta pergunta. Nossa recomendação, no entanto, seria a abordagem de modelos gordos e controladores magros: a lógica de negócios não deve ser escrita nos controladores, pois eles são o elemento de ligação entre as visualizações e o código específico do problema. Além disso, os controladores geralmente contêm muito código específico da estrutura e é uma boa prática manter isso fora da lógica de negócios o máximo possível.

Independentemente da nossa recomendação, deve estar definido nas diretrizes de

codificação do seu projeto como você acha que sua equipe deve lidar com essa questão. Caso contrário, você provavelmente acabará tendo ambas as abordagens em sua base de código.

Normalmente, as diretrizes de codificação abrangem questões como nomear métodos, funções e propriedades. Como você deve saber pela famosa citação “Existem apenas duas coisas difíceis na ciência da computação: invalidação de cache e nomeação de coisas”, encontrar os nomes certos não é um problema trivial. Portanto, ter convenções sobre esse tópico pelo menos reduz o tempo dos desenvolvedores tentando criar um nome adequado. Além disso, assim como os padrões de codificação, eles ajudam a reduzir o atrito cognitivo.

As diretrizes de codificação ajudam os desenvolvedores menos experientes em sua equipe, ou aqueles que acabaram de começar, a ter uma solução em mãos que, de outra forma, precisariam procurar no código ou na Internet. Também ajuda a escrever código sustentável evitando más práticas, como já discutimos em Capítulo 3, Código, Não Faça Acrobacias. Para ajudá-lo a começar a configurar o primeiro conjunto de regras, daremos alguns exemplos na próxima seção.

Exemplos de diretrizes de codificação

Começar com uma folha de papel (virtual) em branco é difícil, portanto, nesta seção, coletamos uma lista de exemplos do mundo real do que poderia fazer parte de suas diretrizes de codificação. Observe que esta coleção de regras, embora sejam baseadas nas melhores práticas, não pretende ser perfeita ou a única verdade. Em vez disso, queremos dar a você um bom ponto de partida para discussões e exemplos sobre quais tópicos devem ser esclarecidos usando as diretrizes de codificação.

Convenções de nomenclatura

Ao usar convenções de nomenclatura, garantimos que certos elementos do nosso código sejam nomeados de maneira uniforme e compreensível. Isso reduz o atrito cognitivo e facilita a integração de novos membros da equipe.

Serviços, repositórios e modelos

Escrito em UpperCamelCase. Use o tipo como o sufixo.

aqui estão alguns exemplos:

Eventos

Escrito em UpperCamelCase. Use o tempo correto para indicar se o evento é acionado antes ou depois do evento real.

aqui estão alguns exemplos:

Propriedades, variáveis e métodos

Escrito em minúsculaCamelCase.

aqui estão alguns exemplos:

testes

Escrito em minúsculaCamelCase. Use a palavra teste como um prefixo.

aqui estão alguns exemplos:

Características

Escrito em UpperCamelCase. Use o adjetivo para descrever para que o traço é usado.

aqui estão alguns exemplos:

Interfaces

Escrito em UpperCamelCase. Use a palavra Interface como sufixo.

Aqui estão alguns exemplos:

Convenções gerais do PHP

Mesmo que você já use padrões de codificação como o PSR-12, há certos aspectos que eles não cobrem. Vamos pegar alguns deles nesta seção.

Comentários e Blocos de Documentos

Evite comentários se possível, pois eles tendem a ficar desatualizados e, assim, mais confundem do que ajudam. Guarde apenas comentários que não possam ser substituídos por nomes autoexplicativos ou por simplificação de código, assim fica mais fácil de entender e não requer mais o comentário.

Adicione DocBlocks apenas se eles adicionarem informações, como anotações para as ferramentas de qualidade de código. Particularmente desde o PHP 8, a maioria dos DocBlocks pode ser substituída por dicas de tipo, que todos os IDEs modernos entenderão. Se você usar dicas de tipo, a maioria dos DocBlocks pode ser removida:

```
// Bloco de Documentação Redundante
```

```
/**
```

```
 * @param int $propriedade
```

```
 * @return void
```

```
 */
```



```
public function setProperty(int $propriedade): void {  
  
    // ...  
  
}
```

Freqüentemente, os DocBlocks são gerados automaticamente pelo IDE. Se não forem atualizados, são, na melhor das hipóteses, inúteis ou podem até estar totalmente errados:

```
// DocBlock inútil  
  
/**  
  
 * @param $propriedade  
  
 */  
  
public function setProperty(int $propriedade): void {  
  
    // ...  
  
}  
  
//DocBlock errado  
  
/**  
  
 * @param string $propriedade  
  
 */  
  
public function setProperty(int $propriedade): void {  
  
    // ...  
  
}
```

DocBlocks ainda devem ser usados para informações que não podem ser fornecidas pelos recursos da linguagem PHP até agora, como especificar o conteúdo de um array ou marcar uma função como obsoleta:

```
// Bloco de Documentos Útil
```

```
/**
```

```
 * @return string[]
```

```
*/
```

```
public function getList(): array {
```

```
    Retorna [
```

```
    'foo',
```

```
    'bar',
```

```
];
```

```
}
```

```
/**
```

```
 * @deprecated use a função fooBar() em vez disso
```

```
*/
```

```
public function foo(): bool {
```

```
    // ...
```

```
}
```

Sobre DocBlocks

Os DocBlocks foram introduzidos para, entre outras coisas, compensar parcialmente as deficiências de digitação fraca em versões anteriores do PHP. O padrão de fato foi introduzido pelo projeto phpDocumentor (<https://www.phpdoc.org/>) e, como tal, é compatível com muitas ferramentas, como IDEs e analisadores de código estático. Usando digitação estrita, muitas vezes não é mais necessário usar DocBlocks, a menos que você queira usar phpDocumentor em seu projeto.

operadores ternários

Cada parte deve ser escrita em uma única linha para aumentar a legibilidade. Exceções podem ser feitas para declarações muito curtas:

// Exemplo de declaração curta

```
$isFoo? 'foo' : 'barra';
```

// Notação usual

```
$éLongerVariable
```

```
? 'longerFoo'
```

```
: 'barra mais longa';
```

Não use operadores ternários aninhados, pois eles são difíceis de ler e depurar:

```
// Exemplo para operadores aninhados
```

```
$numero > 0 ? 'Positivo': ($numero < 0 ? 'Negativo':  
'Zero');
```

Construtor

Use a promoção de propriedade do construtor para classes mais curtas, se estiver trabalhando com PHP 8+. Mantenha a vírgula após a última propriedade, pois isso facilitará a adição ou comentário de linhas:

```
// Antes do PHP 8+
```

```
classe ExemploDTO  
{  
    string pública $nome;  
    função pública __construct(  
        string $nome  
    ) {  
        $este->nome = $nome;  
    }  
}
```

```
// Desde PHP 8+

classe ExemploDTO

{

função pública __construct(

string pública $nome,

) {}

}
```

Matrizes

Sempre use a notação de array curta e mantenha a vírgula após a última entrada (consulte a seção anterior, Construtor, para obter uma explicação):

```
// Notação antiga

$meuArray = array(

'primeira entrada',

'segunda entrada'

);

// Notação curta do array

$meuArray = [

'primeira entrada',
```

```
'segunda entrada',
```

```
];
```

estruturas de controle

Sempre use parênteses, mesmo para one-liners. Isso reduz o atrito cognitivo e facilita a adição de mais linhas de código posteriormente:

```
// Mau
```

```
if ($instrução === verdadeiro)
```

```
  faça alguma coisa();
```

```
// Bom
```

```
if ($declaração === verdadeiro) {
```

```
  faça alguma coisa();
```

```
}
```

Evite instruções else e retorne antecipadamente, pois isso é mais fácil de ler e reduz a complexidade do seu código:

```
// Mau
```

```
if ($instrução) {
```

```
// A instrução foi bem-sucedida

Retorna;

} senão {

// A instrução não foi bem-sucedida

Retorna;

}

// Bom

if (!$instrução) {

// A instrução não foi bem-sucedida

Retorna;

}

// A instrução foi bem-sucedida

Retorna;
```

Manipulação de exceção

Blocos catch vazios devem ser evitados, pois eles engolem mensagens de erro silenciosamente e, portanto, podem dificultar a localização de bugs. Em vez disso, registre a mensagem de erro ou, pelo menos, escreva um comentário explicando por que a exceção pode ser ignorada:

```
// Mau
```

```
tentar {  
  
    $this->someUnstableCode();  
  
} catch (Exceção $exceção) {}  
  
// Bom  
  
tentar {  
  
    someUnstableCode();  
  
} catch (Exceção $exceção) {  
  
    $this->logError($exception->getMessage());  
  
}
```


Padrões arquitetônicos

As diretrizes de codificação não estão limitadas a como formatar elementos de código ou nome. Eles também podem ajudá-lo a controlar como o código é escrito em um sentido arquitetônico.

Modelos gordos, controladores magros

Se for usado o padrão MVC, a lógica de negócios deve estar localizada dentro de modelos ou classes semelhantes, como serviços ou repositórios. Os controladores devem conter o mínimo de código possível para receber ou transferir dados entre as visualizações e os modelos.

Código independente de estrutura

No contexto da abordagem de modelos gordos e controladores magros, você provavelmente encontrará o termo lógica de negócios independente de estrutura. Isso significa que o código que contém suas regras de negócios deve usar o mínimo possível de recursos da estrutura subjacente. Isso facilita muito as atualizações do framework ou até mesmo as migrações para outros frameworks.

Princípio de responsabilidade única

Classes e métodos devem ter apenas uma responsabilidade. Ver Capítulo 2, Quem decide quais são as “boas práticas”?, para mais informações sobre este princípio.

Diretrizes da estrutura

Neste livro, queremos nos concentrar em escrever código limpo em PHP. Muitas vezes, porém, você estará trabalhando com estruturas e, embora seja essencial incluí-las também nas diretrizes, não queremos entrar em muitos detalhes aqui.

No entanto, a seguir, você encontrará uma lista de perguntas que devem lhe dar uma boa ideia sobre quais tópicos relacionados à estrutura incluir em suas diretrizes:

Como as respostas a essas perguntas dependem muito da estrutura usada, não podemos fornecer recomendações aqui. Você precisará definir as diretrizes junto com sua equipe. Na próxima seção, daremos algumas ideias sobre como fazer isso.

Configurando diretrizes

O processo de definição das diretrizes de codificação leva tempo e muitas vezes requer vários workshops nos quais as regras são discutidas. Isso requer moderação, por exemplo, por um líder técnico; caso contrário, você pode ficar preso em discussões intermináveis.

Não se preocupe se você não conseguir chegar a um acordo imediato sobre todos os tópicos. Lembre-se de que as pessoas em sua equipe têm formações, experiências e níveis de habilidade diferentes – e ninguém abandonará diretamente suas formas pessoais de codificação apenas porque de repente surgiram regras que eles não entendem ou aceitam.

Certifique-se de configurar um processo que verifique periodicamente se as diretrizes precisam ser atualizadas. Talvez algumas regras fiquem desatualizadas com o tempo ou novos recursos de linguagem devam ser incluídos. Um ponto de ação em uma reunião de equipe regular seria uma boa oportunidade para isso.

As diretrizes devem ser facilmente acessíveis por escrito, como em um wiki ou na base de conhecimento interna da empresa, que deve ser capaz de rastrear o histórico da versão. Cada membro da equipe deve ser capaz de escrever comentários sobre ele para que as perguntas ou problemas possam ser resolvidos assim que aparecerem. Por fim, todos os membros da equipe devem ser informados automaticamente sobre novas alterações.

Depois que sua equipe concordar com um conjunto de regras, certifique-se de

utilizar as ferramentas de qualidade de código que você aprendeu nos capítulos anteriores para verificar automaticamente se as regras são respeitadas. Você pode, por exemplo, usar o PHPStan para detectar blocos catch vazios ou o PHPMD para aplicar sem usar outro.

Como podemos garantir que nossas diretrizes de codificação sejam aplicadas? Obviamente, devemos usar nossas ferramentas de qualidade de código sempre que possível. Mas e se essas ferramentas não incluíssem as regras que gostaríamos de aplicar? Com um pouco de pesquisa na Internet, você poderá encontrar uma implementação de terceiros para eles. Ou, se não conseguir encontrar nada, você mesmo pode escrever regras personalizadas, já que todos os analisadores de código estático são extensíveis.

Para regras muito complicadas de verificar automaticamente, temos que verificar manualmente se elas são usadas corretamente. Isso pode acontecer em revisões de código, e achamos que elas são tão importantes que merecem uma seção própria neste capítulo.

A configuração das diretrizes de codificação sozinha será apenas uma perda de tempo se você não se certificar de que elas sejam mantidas. Podemos automatizar a verificação de todas as regras relacionadas ao estilo de codificação e também um bom número de diretrizes de codificação. Mas, no momento, para as regras que abordam as diretrizes do framework ou aspectos arquitetônicos, a automação não é mais possível, e nós, humanos, devemos entrar, assumindo as verificações. Nesse ponto, as revisões de código entram em ação. Vamos dar uma olhada mais de perto na próxima seção.

Revisões de código

O processo de verificação manual do código de outros desenvolvedores é chamado de revisão de código. Isso inclui todas as alterações, ou seja, não apenas novas funcionalidades, mas também correções de bugs ou até mesmo alterações simples de configuração.

Uma revisão é sempre feita por pelo menos um colega desenvolvedor e geralmente acontece no contexto de uma solicitação pull, pouco antes de o código de um recurso ou ramificação de correção de bug ser mesclado na ramificação principal; somente se o revisor aprovar as alterações, elas se tornarão parte do aplicativo real.

Nesta seção, discutiremos o que você deve procurar nas revisões de código, por que elas são tão importantes e como devem ser feitas para torná-las uma ferramenta de sucesso em seu kit de ferramentas.

Por que você deve fazer revisões de código

Pode parecer um pouco óbvio, porque é disso que trata todo este livro. No entanto, nunca é demais enfatizar – as revisões de código irão melhorar a qualidade do seu código. Vamos examinar mais de perto o porquê:

Se você ainda não está convencido dos benefícios das revisões de código, confira a próxima seção, na qual falaremos mais sobre o que as revisões de código devem abranger – e quais não.

O que as revisões de código devem cobrir

Quais aspectos devemos verificar ao fazer revisões de código?

Sua equipe também deve considerar se o teste do código em seu ambiente de desenvolvimento local deve fazer parte do processo de revisão ou não. Não há nenhuma recomendação clara sobre isso.

Práticas recomendadas para revisões de código

Embora as revisões de código tenham muitos benefícios e possam ser implementadas com bastante facilidade, existem algumas armadilhas das quais você deve estar ciente e as melhores práticas estabelecidas que tornarão as revisões ainda mais bem-sucedidas.

Quem deve revisar o código?

Em primeiro lugar, quem deveria fazer as revisões de código? Claro, isso também depende da sua configuração. Se você trabalha em equipe com outro desenvolvedor PHP, essa certamente deve ser a primeira pessoa a perguntar. Dessa forma, você constrói conhecimento de domínio compartilhado; embora seu colega não tenha trabalhado diretamente em seu tíquete, ele pelo menos tem uma ideia do que você trabalhou.

No entanto, entrar em contato com membros de outras equipes (se houver) de tempos em tempos evita ficar preso em uma bolha e promove o compartilhamento de conhecimento. Se você não tiver certeza sobre determinados tópicos, peça ajuda aos especialistas do domínio. Frequentemente, isso inclui alterações relacionadas a desempenho, arquitetura ou segurança.

Automatizar

Uma coisa que as revisões de código não devem cobrir é se os Padrões de Codificação são mantidos. DentroCapítulo 7, Ferramentas de qualidade de código introduzimos as ferramentas necessárias para fazer isso automaticamente e, emCapítulo 11, Integração Contínua, nós os integramos em um pipeline de CI.

Certifique-se de que apenas as solicitações pull sejam revisadas nas quais todas as verificações (como farejadores de código, analisadores de código e testes automatizados) foram aprovadas. Caso contrário, você gastará muito tempo em tópicos que nem deveriam ser discutidos.

Evite longas revisões de código

Quantas linhas deve ter a alteração de código que precisa ser revisada? Estudos sugerem que 200 a 400 linhas devem ser o máximo, pois a concentração do revisor diminui com o tempo. Portanto, tente manter as alterações individuais relativamente pequenas. Também é muito mais provável que o revisor encontre tempo para revisar mudanças menores, como para uma longa tapeçaria de diferenças.

As revisões de código, mesmo as menores, exigirão um tempo em que o revisor não conseguirá escrever o código. Mas quanto tempo deve ser gasto? Novamente, isso depende da sua configuração. Um bom número aproximado é de no máximo 60 minutos para evitar o cansaço do revisor. Deixe espaço suficiente para o revisor revisar o código linha por linha. As revisões devem ser aceitas como parte de seu trabalho diário ou elas rapidamente se tornarão um fardo, então ninguém deve se apressar em fazê-las.

Fique humano

Como formular o feedback é crucial para tornar as revisões bem-sucedidas. Cuidado com o tom e tente evitar acusações como “Isso está errado!” ou um absoluto não-ir, “Isso é estúpido.” Os desenvolvedores, especialmente os menos experientes, não devem ficar ansiosos para permitir que seu código seja revisado.

Lembre-se de que um ser humano lerá seus comentários. Muitas vezes funciona bem se você os escrever da perspectiva do “eu”, por exemplo, “Eu não entendo esta linha, você pode explicar?” ou “Acho que também poderíamos fazer assim...”

Não se esqueça de usar as críticas para elogiar peças bem feitas. Um rápido “Ótima ideia!” ou “Gosto muito da sua abordagem” ou “Obrigado pela limpeza do código” mostra sua apreciação pelo trabalho do outro desenvolvedor e aumenta sua motivação.

Normalmente, as revisões de código são feitas escrevendo comentários na plataforma Git que você usa. Mas, claro, você também pode fazê-los cara a cara. Alguns desenvolvedores apreciam o feedback direto mais do que apenas comentários porque o texto escrito carece de muitas metainformações, como o tom de voz ou a expressão facial.

Não exagere, mas também não seja descuidado

Lembre-se do princípio de Pareto e não exagere. Talvez ainda existam pequenas partes no código que você mudaria, mas que não estão explicitamente erradas, pois seguem todos os padrões da equipe. A programação ainda é uma questão de estilo pessoal, e ter discussões intermináveis em uma revisão de código levará à frustração sem maiores benefícios.

No entanto, não aceite alterações que degradem a integridade geral do sistema. Se você estiver convencido de que uma alteração é prejudicial ou viola as diretrizes de codificação, não aprove as alterações. Em caso de dúvida, envolva outro desenvolvedor.

Abrace as mudanças

Por fim, se você acha que um problema discutido em uma revisão deve fazer parte das diretrizes, anote-o e aborde-o na próxima reunião de equipe, sem mencionar o outro desenvolvedor diretamente. Talvez você estivesse certo e as diretrizes serão alteradas para evitar o problema no futuro.

Mas você também pode estar errado, e o resto da equipe não vê isso como um problema. Se você não consegue apresentar argumentos e exemplos convincentes, também deve aceitar essas decisões.

Garantir que as revisões de código sejam feitas

Durante o trabalho diário estressante, com correções de bugs de alta prioridade e prazos apertados, é fácil esquecer de fazer revisões de código. Felizmente, todos os provedores de serviços do Git oferecem funcionalidades para ajudá-lo aqui:

Definição de pronto

Se você trabalha com metodologias ágeis, provavelmente já ouviu o termo definição de pronto. Aqui, a equipe concorda com uma lista de ações que devem ser executadas antes que uma tarefa seja concluída.

Uma definição típica de pronto contém verificações como se o teste foi escrito ou se a documentação foi atualizada. Você também pode utilizar isso para as revisões de código.

Novamente, nossas ferramentas Git nos ajudam fornecendo modelos para solicitações pull (também chamadas de solicitações de mesclagem). Esses são os textos que serão usados para preencher automaticamente a descrição da solicitação pull.

Como isso funciona depende do software que você usa, por isso não podemos fornecer instruções exatas aqui. O texto a seguir, no entanto, mostra um exemplo de como poderia ser:

Definição de Feito

Revisor

[] Mudanças de código revisadas

1. Diretrizes de codificação mantidas

- 2. Funcionalidade considerada
- 3. O código é bem projetado
- 4. Legibilidade e complexidade consideradas
- 5. Nenhum problema de segurança encontrado
- 6. Padrão de codificação e diretrizes mantidas

☐ Alteração testada manualmente

Desenvolvedor

☐ Critérios de aceitação atendidos

☐ Testes automatizados escritos ou atualizados

☐ Documentação escrita ou atualizada

O que será incluído na lista de verificação depende de você e de sua equipe. Se usados como modelo, esses itens sempre aparecerão na descrição do pull request por padrão. Destina-se a ser usado tanto pelo revisor quanto pelo desenvolvedor para não esquecer o que precisa ser feito antes de aprovar a solicitação pull e integrá-la à ramificação principal.

Algumas ferramentas, como o GitHub, usam uma linguagem de marcação no estilo Markdown para esses modelos. Eles exibirão as caixas de seleção (os dois colchetes antes de cada item) como caixas de seleção clicáveis no navegador e acompanharão se foram clicadas ou não. Voilà! Sem muito trabalho, você criou uma lista de verificação útil e fácil de usar!

Conclusão das revisões de código

Esperamos que esta seção tenha fornecido bons insights sobre como as revisões de código podem ser benéficas para sua equipe e para você. Como eles podem ser introduzidos sem esforço, vale a pena experimentá-los. As práticas recomendadas nesta seção ajudarão você a evitar alguns dos problemas que as revisões de código podem ter.

Mas, como sempre, eles também têm algumas desvantagens: as revisões levam muito tempo e podem gerar conflitos entre os membros da equipe. Estamos convencidos de que o tempo gasto compensa, porque os aspectos positivos superam em muito os negativos. Os conflitos provavelmente aconteceriam de qualquer maneira, e as análises são apenas o medidor para desabafar. Isso não pode ser totalmente evitado se você trabalhar em equipe, mas deve ser tratado com antecedência com seu gerente. É seu trabalho lidar com esses tipos de problemas.

Na última parte deste capítulo, veremos os padrões de projeto com mais detalhes. Eles podem atuar como diretrizes sobre como resolver problemas gerais no desenvolvimento de software.

Padrões de design

Padrões de design são soluções comumente usadas para problemas que ocorrem regularmente no desenvolvimento de software. Como desenvolvedor, mais cedo ou mais tarde você se deparará com esse termo, caso ainda não o tenha feito – e não sem motivo, pois esses padrões são baseados nas melhores práticas e provaram sua utilidade.

Nesta seção, falaremos mais sobre os diferentes tipos de padrões de projeto e por que eles são tão importantes que se tornaram parte deste livro. Além disso, apresentaremos alguns padrões de design comuns que são amplamente usados em PHP.

Entendendo os padrões de projeto

Vamos dar uma olhada mais de perto nos padrões de projeto agora. Eles podem ser considerados modelos para resolver problemas particulares e são nomeados de acordo com a solução que fornecem. Por exemplo, neste capítulo, você aprenderá sobre o padrão Observer, que pode ajudá-lo a implementar uma maneira de observar mudanças em objetos. Isso é muito útil quando você escreve código, mas também quando projeta software com outros desenvolvedores. É muito mais fácil usar um nome curto para nomear um conceito do que ter que explicá-lo todas as vezes.

Porém, não confunda padrões de projeto com algoritmos. Os algoritmos definem etapas claras que precisam ser seguidas para resolver um problema, enquanto os padrões de projeto descrevem como implementar a solução em um nível superior. Eles não estão vinculados a nenhuma linguagem de programação.

Você também não pode adicionar padrões de design ao seu código como adicionaria um pacote do Composer, por exemplo. Você precisa implementar o padrão por conta própria e tem certos graus de liberdade para fazer isso.

No entanto, os padrões de projeto não são a única solução para todos os problemas, nem pretendem oferecer as soluções mais eficientes. Sempre leve esses padrões com cautela – muitas vezes, os desenvolvedores desejam implementar um determinado padrão apenas porque o conhecem. Ou, como diz o ditado: "Se tudo o que você tem é um martelo, tudo parece um prego."

Normalmente, os padrões de projeto são divididos em três categorias:

Nas páginas seguintes, veremos alguns exemplos de implementações para explicar a ideia por trás dos padrões de projeto.

Padrões de design comuns em PHP

Agora queremos apresentar alguns dos padrões de design mais amplamente usados no mundo do PHP. Escolhemos um padrão de cada uma das três categorias Criacional, Estrutural e Comportamental, que discutimos na seção anterior.

Método de fábrica

Imagine o seguinte problema: você precisa escrever um aplicativo que seja capaz de gravar dados em arquivos usando diferentes formatos. Em nosso exemplo, queremos oferecer suporte a CSV e JSON, mas também a outros formatos no futuro. Antes de gravar os dados, gostaríamos de aplicar alguma filtragem, o que sempre deve acontecer, independentemente do formato de saída escolhido.

Um padrão aplicável para resolver este problema seria o Factory Method. É um padrão criacional, pois lida com a criação de objetos.

A ideia principal desse padrão é que as subclasses podem implementar diferentes maneiras de atingir o objetivo. É importante observar que não usamos o operador new na classe pai para instanciar nenhuma subclasse, como você pode ver na classe a seguir:

```
classe abstrata AbstractWriter  
  
{  
  
public function write(array $data): void  
  
{  
  
$encoder = $this->createEncoder();  
  
// Aplique alguma filtragem que sempre deve acontecer,  
  
// independentemente do formato de saída.
```



```

array_walk(
    $dados,
    função (&$valor) {
        $valor = str_replace('dados', '', $valor);
    }
);

// Para fins de demonstração, repetimos o resultado
// aqui, em vez de escrevê-lo em um arquivo
echo $encoder->encode($dados);
}

função protegida abstrata createEncoder(): Codificador;
}

```

Observe o método createEncoder – este é o método fábrica que deu o nome ao padrão, pois ele atua, de certo modo, como uma fábrica para novas instâncias. É definida como uma função abstrata, portanto precisa ser implementada por uma ou mais subclasses.

Para ser flexível o suficiente para formatos futuros, pretendemos usar classes Encoder separadas para cada formato. Mas primeiro, definimos uma interface para essas classes para que sejam facilmente intercambiáveis:

codificador de interface

```
{  
função pública codificar(array $dados): string;  
}
```

Em seguida, criamos uma classe Encoder para cada formato que implementa a interface Encoder; primeiro, criamos o JsonEncoder:

```
classe JsonEncoder implementa Encoder  
{  
função pública codificar(array $dados): string  
{  
// a codificação real acontece aqui  
// ...  
return $encodedString;  
}  
}
```

Em seguida, criamos o CsvEncoder:

```
classe CsvEncoder implementa Encoder  
{
```

```
função pública codificar(array $dados): string
{
    // a codificação real acontece aqui

    // ...

    return $encodedString;
}
}
```

Agora, precisamos criar uma subclasse da classe `AbstractWriter` para cada formato que queremos suportar. No nosso caso, é o `CsvWriter` primeiro:

```
classe CsvWriter estende AbstractWriter
{
    public function createEncoder(): Codificador
    {
        $encoder = new CsvEncoder();

        // aqui, mais trabalho de configuração ocorreria

        // por exemplo, configurando o delimitador

        return $codificador;
    }
}
```

E segundo, é JsonWriter:

```
classe JsonWriter estende AbstractWriter
{
public function createEncoder(): Codificador
{
retornar novo JsonEncoder();
}
}
```

Observe que ambas as subclasses sobreescrevem apenas o método de fábrica createEncoder. Além disso, os novos operadores ocorrem apenas nas subclasses. O método write permanece inalterado, pois é herdado de AbstractWriter.

Finalmente, vamos juntar tudo isso em um script de exemplo:

```
function factoryMethodExample(AbstractWriter $writer)
{
$exemploDados = [
'conjunto1' => ['dados1', 'dados2'],
```

```
'conjunto2' => ['dados3', 'dados4'],  
  
];  
  
$writer->write($exampleData);  
  
}  
  
echo "Saída usando o CsvWriter: ";  
  
factoryMethodExample(new CsvWriter());  
  
echo "Saída usando o JsonWriter: ";  
  
factoryMethodExample(new JsonWriter());
```

A função `factoryMethodExample` primeiro recebe `CsvWriter` e, para a segunda execução, `JsonWriter` como parâmetros. A saída ficará assim:

Saída usando o `CsvWriter`:

3,4

1,2

Saída usando o `JsonWriter`:

```
[["3","4"],["1","2"]]
```

O padrão `Factory Method` nos permite mover a instanciação da classe `Encoder` da classe pai `AbstractWriter` para as subclasses. Fazendo isso, evitamos o acoplamento apertado entre `Writer` e `Encoder`, ganhando muito mais flexibilidade. Como desvantagem, o código se torna mais complexo, pois temos que introduzir interfaces e subclasses para implementar esse padrão.

Injeção de dependência

O próximo padrão que queremos introduzir é um padrão estrutural chamado Dependency Injection (DI). Isso nos ajuda a implementar uma arquitetura fracamente acoplada inserindo dependências nas classes já no momento da construção, em vez de instanciá-las dentro da classe.

O código a seguir mostra como uma dependência, neste exemplo, um Logger clássico, é instanciada no construtor:

```
classe InstanciationExample
{
    Logger privado $logger;

    função pública __construct()
    {
        $this->logger = new FileLogger();
    }
}
```

O código em si funciona perfeitamente bem, mas os problemas começam quando você deseja substituir o FileLogger por uma classe diferente. Embora já usemos a interface Logger para a propriedade \$logger, que teoricamente torna mais fácil trocá-la por outra implementação, codificamos o FileLogger no construtor.

Agora, imagine que você usou esse registrador em quase todas as aulas; substituí-lo por uma implementação diferente do Logger causa algum esforço, pois você teria que mexer em cada arquivo que o usa.

Não ser capaz de substituir o FileLogger também dificulta a criação de testes de unidade para a classe. Você não pode substituí-lo por uma simulação, mas também não deseja gravar informações em seus logs reais durante as execuções de teste. Se você quiser testar se o log funciona corretamente, precisará criar algumas soluções alternativas em seu código que também são usadas na produção.

DI nos obriga a pensar sobre quais e quantas dependências devem ser usadas em uma classe. É considerado um cheiro de código (ou seja, um indicador de código mal estruturado) quando o construtor leva consideravelmente mais do que três ou quatro dependências como parâmetro porque indica que a classe viola o princípio da responsabilidade única (o “S” em SOLID). Isso também é conhecido como aumento do escopo: o escopo de uma classe aumenta lenta mas constantemente com o tempo.

Vejamos agora como o DI resolveria os problemas mencionados anteriormente:

```
classe ConstrutorInjeção  
  
{  
  
  Logger privado $logger;  
  
  public function __construct(Logger $logger)  
  
  {
```

```
$this->logger = $logger;
```

```
}
```

```
}
```

promoção de propriedade do construtor

Observe que não usamos a promoção de propriedade do construtor propositadamente aqui para melhor visualização.

A diferença para o código anterior não parece ser tão grande. Tudo o que fizemos foi passar a instância do Logger como um parâmetro para o construtor, em vez de instanciá-la diretamente. O benefício é enorme: agora podemos alterar a instância que é injetada (se ela implementar a interface Logger) sem tocar na classe real.

Imagine que você não deseja mais que a classe faça login no sistema de arquivos, mas sim em um sistema de gerenciamento de logs como o Graylog, que gerencia todos os logs de seus diferentes aplicativos em um só lugar. Tudo o que você precisa fazer é criar o GraylogLogger, que também implementa a interface Logger, mas grava os logs nesse sistema em vez de em arquivos. Em seguida, você simplesmente injeta GraylogLogger em vez de FileLogger em todas as classes que devem usá-lo – parabéns, você acabou de alterar a maneira como seus aplicativos registram informações sem tocar nas classes reais.

Da mesma forma, podemos trocar facilmente uma dependência por um objeto fictício em nossos testes de unidade. Esta é uma grande melhoria em relação à testabilidade.

A instanciação do Logger, no entanto, não importa qual implementação você escolheu, ainda tem que acontecer em outro lugar. Acabamos de movê-lo para fora da classe InjectionExample. A dependência é injetada quando a classe é instanciada:

```
$constructorInjection = new ConstructorInjection(  
    new FileLogger()  
);
```

Normalmente, você encontraria esse tipo de instanciação dentro de uma classe Factory. Esta é uma classe que implementa, por exemplo, o padrão Simple Factory e cuja única função é criar instâncias de uma determinada classe, com todas as dependências necessárias.

Padrão de fábrica simples

Não discutiremos esse padrão com mais detalhes neste livro porque ele é, bem, muito simples. Você pode encontrar mais informações sobre isso aqui:<https://designpatternsphp.readthedocs.io/en/latest/Creational/SimpleFactory/>

A injeção não precisa necessariamente acontecer por meio do construtor. Outra abordagem possível é a chamada injeção de setter:

```
classe SetterInjection
{
    Logger privado $logger;
    função pública __construct()
    {
        // ....
    }
    função pública setLogger(Logger $logger): void
    {
        $this->logger = $logger;
    }
}
```

A injeção da dependência aconteceria então usando o método setLogger. O mesmo que para a injeção do construtor, isso provavelmente aconteceria dentro da classe Factory.

O seguinte é um exemplo de como essa fábrica poderia ser:

```
classe SetterInjectionFactory
{
```

```
função pública createInstance(): SetterInjection
{
    $setterInjection = new SetterInjection();
    $setterInjection->setLogger(new FileLogger());
    return $setterInjection;
}
```

Contêiner de injeção de dependência

Você provavelmente já se perguntou como gerenciar todas as fábricas necessárias, especialmente em um projeto maior. Para isso, o contêiner DI foi inventado. Não faz parte do padrão DI, mas está intimamente relacionado, por isso queremos apresentá-lo aqui.

O contêiner de DI atua como armazenamento central para todos os objetos que são trazidos para suas classes de destino usando padrões de DI. Ele também contém todas as informações necessárias para instanciar os objetos.

Ele também pode armazenar instâncias criadas para que não precise instanciá-las duas vezes. Por exemplo, você não criaria uma instância de FileLogger para cada classe que a usa, pois isso resultaria em várias instâncias idênticas. Você prefere criá-lo uma vez e depois passá-lo por referência a suas classes de destino.

contentor DI

Mostrar toda a funcionalidade de um contêiner de DI moderno excederia este livro. Se você estiver interessado em aprender mais sobre este conceito, recomendamos que você verifique o pacote `phpleague/container`: <https://container.thephpleague.com>. É pequeno, mas rico em recursos e possui uma excelente documentação que pode apresentar a você conceitos mais interessantes, como provedores de serviços ou infletores.

O conceito de contêiner DI foi adotado em todos os principais frameworks PHP hoje em dia, então você provavelmente já usou esse contêiner. Você provavelmente não notou, porque geralmente está escondido no fundo do seu aplicativo e às vezes também é chamado de contêiner de serviço.

PSR-11 – Interface de contêiner

O contêiner DI é tão importante para o ecossistema PHP que ganhou seu próprio PSR: <https://www.php-fig.org/psr/psr-11>.

Observador

O último padrão que queremos apresentar neste livro é o padrão Observer. Como um padrão comportamental, seu principal objetivo é permitir uma comunicação eficiente entre os objetos. Uma tarefa comum a ser implementada é acionar uma determinada ação em um objeto quando o estado de outro objeto muda. Uma mudança de estado pode ser algo tão simples quanto uma mudança de valor de uma propriedade de classe.

Vamos começar com outro exemplo: você tem que enviar um e-mail para a equipe de vendas toda vez que um cliente cancela sua assinatura para que eles sejam informados e possam tomar medidas para manter o cliente.

Como você faz isso melhor? Você pode, por exemplo, configurar um trabalho recorrente que verifica dentro de um determinado intervalo de tempo (por exemplo, a cada 5 minutos) se houve cancelamentos desde a verificação. Isso funcionaria, mas dependendo do tamanho de sua base de clientes, o trabalho provavelmente não retornaria nenhum resultado na maioria das vezes. Se o intervalo entre duas verificações, por outro lado, for muito longo, você poderá perder um tempo valioso até a próxima verificação.

Agora, as vendas podem não ser a coisa mais urgente do mundo (os vendedores geralmente discordam aqui), mas você certamente entendeu. Não seria ótimo se pudéssemos enviar o e-mail assim que o cliente cancelasse a assinatura? Então, ao invés de verificar regularmente as mudanças, só nos informamos no momento em que a mudança acontece?

O código pode se parecer com este exemplo simplificado:

```
classe ClienteConta
{
função pública __construct(
serviço de correio privado $mailService
) {}
função pública cancelSubscription(): void
{
// Código necessário para o cancelamento real
// ...
$this->mailService->sendEmail(
' vendas@exemplo.com ',
'A conta xy cancelou a assinatura'
);
}
}
```

Exemplo simplificado

O exemplo foi simplificado. Você deve, por exemplo, não codificar o endereço

de e-mail.

Essa abordagem certamente funcionaria, mas tem uma desvantagem: a chamada de MailService é codificada diretamente na classe e, portanto, está fortemente acoplada a ela. E agora, a classe CustomerAccount tem que cuidar de outra dependência, o que aumenta o esforço de manutenção, pois os testes precisam ser estendidos, por exemplo. Se mais tarde não quisermos mais enviar este e-mail ou mesmo enviar um e-mail adicional para outro departamento, CustomerAccount deverá ser alterado novamente.

Usando uma abordagem fracamente acoplada, o objeto CustomerAccount armazenaria apenas uma lista de outros objetos que deveria notificar em caso de alteração. A lista não é codificada e os objetos que precisam ser notificados devem ser anexados a essa lista durante a fase de inicialização.

O objeto que queremos observar (no exemplo anterior, CustomerAccount) é chamado de sujeito. O sujeito é responsável por informar os observadores. Nenhuma alteração de código seria necessária sobre o assunto para adicionar ou remover observadores, portanto, essa abordagem é muito flexível.

O código a seguir mostra um exemplo de como a classe CustomerAccount pode implementar o padrão Observer:

```
use SplSubject;
```

```
use SplObjectStorage;
```

```
use SplObserver;
```

```
classe CustomerAccount implementa SplSubject
{
$observadores SplObjectStorage privados;

função pública __construct()
{
$this->observadores = new SplObjectStorage();
}

função pública attach(SplObserver $observer): void
{
$this->observadores->attach($observadores);
}

public function detach(SplObserver $observer): void
{
$this->observadores->detach($observador);
}

função pública notificar(): void
{
foreach ($this->observadores como $observador) {
$observador->atualizar($este);
}
}
```



```
função pública cancelSubscription(): void  
  
{  
  
    // Código necessário para o cancelamento real  
  
    // ...  
  
    $this->notify();  
  
}  
  
}
```

Muita coisa aconteceu aqui, então vamos passar por isso pouco a pouco. A primeira coisa notável é que a classe usa as interfaces `SplSubject` e `SplObserver`, bem como a classe `SplObjectStorage`. Como a classe `CustomerAccount` implementa a interface `SplSubject`, ela deve fornecer os métodos `attach`, `detach` e `notification`.

Também usamos o construtor para inicializar a propriedade `$observers` como `SplObjectStorage`, que armazenará todos os observadores da classe `CustomerAccount`. Felizmente, o SPL já fornece a implementação desse armazenamento, portanto, não precisamos fazer isso.

Biblioteca PHP Padrão

Falamos sobre a Standard PHP Library (SPL) em [Capítulo 3](#), Métricas de qualidade de código. O fato de a SPL incluir essas entidades mostra a importância do padrão Observer, bem como a utilidade dessa biblioteca.

Os métodos attach e detach são requeridos pela interface SplSubject. Eles são usados para adicionar ou remover observadores. Sua implementação é fácil – só precisamos encaminhar o objeto SplObserver para SplObjectStorage em ambos os casos, o que assume o trabalho necessário para nós.

O método de notificação deve chamar o método de atualização em todos os objetos SplObserver armazenados em SplObjectStorage. Isso é tão simples quanto usar um loop foreach para iterar todas as entradas do SplObserver e chamar seu método de atualização, passando uma referência ao assunto usando \$this.

O código a seguir mostra como esse observador pode se parecer:

```
classe CustomerAccountObserver implementa SplObserver
{
    função pública __construct(
        serviço de correio privado $mailService
    ) {}

    atualização de função pública(CustomerAccount|SplSubject
        $splAssunto): void
    {
        $this->mailService->sendEmail(
            ' vendas@exemplo.com ',
```

```
'Conta ' . $splSubject->id . ' cancelou  
a assinatura'  
  
);  
  
}  
  
}
```

O observador, sem surpresa, implementa a interface `SplObserver`. O único método necessário é `update`, que é chamado do assunto no método `notify`. Como a interface requer o parâmetro `$splSubject` para implementar a interface `SplSubject`, temos que usar essa dica de tipo de parâmetro. Caso contrário, levaria a um erro PHP.

Como sabemos que, nesse caso, o objeto é na verdade um objeto `CustomerAccount`, também podemos adicionar essa dica de tipo. Isso permitirá que nosso IDE nos ajude com o preenchimento de código adequado; não é necessário adicioná-lo embora.

Como você pode ver, toda a lógica referente ao envio de e-mail agora foi movida para `CustomerAccountObserver`. Em outras palavras, eliminamos com sucesso o forte acoplamento entre `CustomerAccount` e `MailService`.

A última coisa que precisamos fazer é anexar `CustomerAccountObserver`:

```
$mailService = new MailService();
```

```
$observer = new CustomerAccountObserver($mailService);
```

```
$contacliente = new Contacliente();
```

```
$contacliente->attach($observador);
```

Novamente, este exemplo de código é simplificado. Em um aplicativo do mundo real, todos os três objetos seriam instanciados em fábricas dedicadas e reunidos por um contêiner de DI.

O padrão Observer ajuda você a desacoplar objetos com uma quantidade relativamente baixa de trabalho. Ele tem algumas desvantagens embora. A ordem na qual os observadores são atualizados não pode ser controlada; assim, você não pode usá-lo para implementar a funcionalidade onde a ordem é crucial. Em segundo lugar, ao desacoplar as classes, não é mais óbvio apenas observar o código ao qual os observadores estão vinculados.

Para resumir o tópico dos padrões de projeto, veremos os padrões que ainda são bastante comuns hoje, mas provaram ter desvantagens muito significativas para serem recomendados. Cortina para os Antipadrões!

Antipadrões

Nem todo padrão de design resistiu ao teste do tempo. Tudo evolui, assim como o desenvolvimento de software e o PHP. Alguns padrões que tiveram sucesso no passado foram substituídos por versões mais novas e/ou melhores.

O que antes era a abordagem padrão para resolver um problema alguns anos atrás pode não ser mais a solução certa. A comunidade PHP continua aprendendo e melhorando, mas esse conhecimento ainda não está distribuído uniformemente. Assim, para tornar mais óbvio quais padrões devem ser evitados, eles são frequentemente referidos como Antipadrões – isso soa claramente como algo que você não gostaria de ter em seu código, certo?

Como é esse antipadrão? Vejamos o primeiro exemplo.

solteiro

Antes de a DI se tornar cada vez mais popular no mundo PHP, já tínhamos que lidar com o problema de como criar instâncias de forma eficaz e como disponibilizá-las nos escopos de outras classes. O padrão Singleton oferecia uma solução rápida e fácil que geralmente se parecia com isto:

```
$instance = Singleton::getInstance();
```

O método estático `getInstance` é surpreendentemente simples:

```
classe Singleton
{
    private static ?Singleton $instance = null;

    função estática pública getInstance(): Singleton
    {
        if (self::$instância === null) {
            self::$instância = new self();
        }

        return self::$instância;
    }
}
```

}

Se o método for executado, é verificado se uma instância da classe já foi criada. Se sim, será devolvido; caso contrário, será criado previamente. Essa abordagem também é chamada de inicialização preguiçosa. Ser preguiçoso é uma coisa boa aqui porque só é inicializado quando necessário, economizando recursos.

Além disso, o método armazena a nova instância na propriedade estática `$instance`. Isso é notável, pois a abordagem só é possível porque as propriedades estáticas podem ter um valor sem exigir uma instância de classe. Em outras palavras, podemos armazenar a instância de uma classe em sua própria definição de classe. Além disso, no PHP, todos os objetos são passados como referência, ou seja, como ponteiro para o objeto na memória. Ambas as peculiaridades nos ajudam a garantir que a mesma instância seja sempre retornada.

O Singleton na verdade é bastante elegante; como também usa um método estático, também não precisa de instância da classe Singleton. Dessa forma, ele pode ser executado literalmente em qualquer lugar do seu código, sem nenhuma preparação adicional.

A facilidade de uso é uma das principais razões pelas quais Singleton acabou se tornando um Anti-Padrão, já que leva ao aumento do escopo. Explicamos esse problema na seção sobre DI.

Outro problema é a capacidade de teste: é muito difícil substituir a instância por um objeto simulado, portanto, escrever testes de unidade para código que usa o padrão Singleton tornou-se muito mais complexo.

Hoje em dia, você deve usar DI junto com um contêiner de DI. Não é tão fácil de usar quanto o Singleton, mas isso nos ajuda a pensar duas vezes antes de usar outra dependência em uma classe.

No entanto, isso não significa que o padrão Singleton não deva ser usado. Pode haver razões válidas para implementá-lo ou, pelo menos, mantê-lo em um projeto legado. Apenas esteja ciente dos riscos.

Localizador de serviços

O segundo padrão que pode ser considerado problemático é o Service Locator:

```
classe ServiceLocatorExample
{
    função pública __construct(
        privado ServiceLocator $serviceLocator
    ) {}

    função pública fooBar(): void
    {
        $someService = $this->serviceLocator
            ->get(AlgumServiço::classe);
        $someService->doSomething();
    }
}
```

Nesta classe de exemplo, injetamos ServiceLocator durante o tempo de construção do objeto. Em seguida, é usado em toda a classe para buscar as dependências necessárias. Nesse sentido, DI e Service Locator são implementações do princípio de inversão de dependência (o “D” em SOLID): eles movem o controle sobre suas dependências para fora do escopo da classe,

ajudando-nos a obter uma arquitetura fracamente acoplada.

Mas, se precisarmos injetar apenas uma dependência em vez de muitas, não é uma ótima ideia? Bem, a desvantagem do padrão Service Locator é que ele oculta as dependências da classe atrás da instância ServiceLocator. Enquanto com DI você pode ver claramente quais dependências são usadas olhando para o construtor, você não pode fazer isso ao injetar apenas ServiceLocator.

Ao contrário do DI, ele não nos obriga a questionar quais dependências devem ser usadas em uma classe, pois, para classes maiores, você pode perder rapidamente a visão geral de quais dependências são usadas em uma classe. Esta é basicamente uma das principais desvantagens que identificamos para o padrão Singleton.

Mais uma vez, não queremos ser dogmáticos quando se trata do uso do padrão Service Locator. Pode haver situações em que seja apropriado usá-lo - apenas manuseie-o com cuidado.

Resumo

Neste capítulo, discutimos a importância dos padrões e diretrizes. Os padrões de codificação ajudam você a se alinhar com outros desenvolvedores sobre como o código deve ser formatado e você aprendeu sobre os padrões existentes que vale a pena adotar.

As diretrizes de codificação ajudam sua equipe a se alinhar sobre como escrever software. Embora essas diretrizes sejam altamente individuais para cada equipe, fornecemos a você um bom conjunto de exemplos e práticas recomendadas para criar as diretrizes de sua equipe. Com revisões de código, você também sabe como manter a qualidade.

Por fim, apresentamos a você o mundo dos padrões de projeto. Estamos confiantes de que conhecer pelo menos uma boa parte desses padrões ajudará você a projetar e escrever código de alta qualidade junto com os membros de sua equipe. Há muito mais para explorar neste tópico, e você encontrará links para algumas ótimas fontes no final deste capítulo.

Isso quase encerra nossa empolgante jornada pelos muitos aspectos do código limpo em PHP. Temos certeza de que agora você deseja usar todos os seus novos conhecimentos em seu trabalho diário o mais rápido possível. No entanto, antes de fazer isso, aguarde o último capítulo, quando falamos sobre a importância da documentação.

Leitura adicional

Criando Documentação Eficaz

Muitos desenvolvedores consideram a documentação um fardo em vez de uma atividade significativa. Isso é compreensível, pois muitas vezes a documentação não é mais atualizada depois de escrita. Logo, ele está cheio de declarações erradas e informações desatualizadas, o que de fato é algo que ninguém deseja.

Estamos convencidos de que a documentação é importante demais para abandoná-la. Se bem feito, será uma adição valiosa e um bloco de construção importante para escrever código limpo, especialmente ao trabalhar em equipe.

Portanto, no último capítulo deste livro, queremos dar a você algumas ideias sobre como escrever uma documentação que seja prática e sustentável.

Vamos cobrir os seguintes tópicos principais neste capítulo:

Requerimentos técnicos

Para este capítulo, não há requisitos técnicos adicionais. Todos os exemplos de código podem ser encontrados em nosso repositório
GitHub:<https://github.com/PacktPublishing/Clean-Code-in-PHP>.

Por que a documentação é importante

Bem-vindo ao último capítulo deste livro. Você percorreu um longo caminho e, antes de encerrar este livro, por enquanto, queremos chamar sua atenção para o tópico frequentemente negligenciado da criação de documentação. Vamos convencê-lo nas páginas seguintes de que a documentação não precisa necessariamente ser cansativa e irritante, mas tem benefícios valiosos.

Por que a documentação é importante

Por que deveríamos realmente criar qualquer documentação? Nosso código ou nossos testes já não têm documentação suficiente? Há alguma verdade nesses pensamentos e discutiremos esse tópico mais adiante nesta seção. No entanto, ao longo dos anos, inúmeros desenvolvedores nunca pararam de criar inúmeros documentos, então deve haver algo sobre isso.

Criamos documentação porque podemos facilitar o trabalho de outras pessoas com nosso software. Trata-se de contexto, que não pode ser facilmente extraído da leitura do código de algumas classes. A documentação geralmente não é apenas sobre o quê ou como, mas também sobre o porquê.

Conhecer a motivação ou os fatores externos que levam a uma decisão é crucial para entender e aceitar porque um projeto foi construído de uma determinada maneira. Por exemplo, você pode reclamar sobre seu ex-colega que implementou um download frágil e acionado por cronjob de arquivos de valores separados por

vírgula (CSV) de um servidor FTP (File Transfer Protocol) externo, a menos que você saiba na documentação que o cliente foi simplesmente incapaz de fornecer um terminal de interface de programação de aplicativo (API) REpresentational State Transfer (REST) para entregar os dados antes do prazo do projeto.

Um novo colega que inicia seu projeto certamente ficará feliz em ter pelo menos alguma documentação para ler, para não precisar perguntar (e provavelmente atrapalhar) outros desenvolvedores para cada pequena dúvida. E não vamos esquecer nossos eus do futuro, que não tocam nesse projeto há muitos meses e devem corrigir um bug crítico agora. Se ao menos soubéssemos o que nossos eus do passado fizeram no passado... e por quê.

Se você criar software de código aberto (OSS), a documentação também é importante. Se você precisar avaliar vários pacotes de terceiros para decidir qual usar em um projeto, é mais provável que um pacote seja considerado se tiver uma boa documentação. Não seria uma pena se você investisse inúmeras horas em uma ferramenta, mas ninguém a usasse porque não tem ou não tem boa documentação?

Por fim, se você vive do desenvolvimento de software, deve considerar que é parte do dever de um desenvolvedor profissional escrever a documentação. É para isso que você é pago.

Documentação do desenvolvedor

Quando pensamos em documentação, geralmente é a documentação do usuário que vem primeiro à nossa mente: textos longos, difíceis de ler e chatos sobre como usar todos os recursos de um produto de software, como, por exemplo, um processador de texto . É claro que essa documentação existe por um bom motivo, mas isso não deve nos interessar no contexto deste livro, pois geralmente não é escrita por (e para) desenvolvedores.

Documentação do software é um campo extenso e, como tal, não pode ser totalmente coberto neste capítulo. Em vez disso, queremos nos concentrar na documentação que o apoia no processo de desenvolvimento e permite que você escreva um código limpo, conforme descrito na lista não exaustiva a seguir:

Na próxima seção, gostaríamos de examinar mais de perto como a escrita desses tipos de documentação pode ser facilitada.

Criando documentação

A documentação pode ser escrita de várias maneiras. Não existe uma abordagem correta e muitas vezes é determinada pelas ferramentas já em uso, como o serviço de repositório ou o wiki da empresa. Ainda assim, existem algumas dicas e truques que o ajudarão a escrevê-lo e mantê-lo, e queremos apresentá-los a você nesta seção.

Documentos de texto

Vamos nos concentrar primeiro nos típicos documentos de texto escritos manualmente. A abordagem clássica é configurar um wiki, pois estes têm a grande vantagem de poderem ser acessados e utilizados até mesmo por pessoas menos técnicas. Isso os torna uma ótima escolha para as empresas. Os wikis modernos, auto-hospedados ou software como serviço (SaaS), oferecem muita segurança e recursos úteis, como comentários em linha ou controle de versão. Eles também podem se conectar com muitas ferramentas externas, como sistemas de tickets.

Outra opção é manter a documentação próxima ao seu código, adicionando-a ao repositório, por exemplo, em uma subpasta. Essa também é uma abordagem válida, especialmente para equipes menores ou projetos de código aberto. Você não deve usar formatos inchados, como Word ou Portable Document Format (PDF), e sim se concentrar em formatos baseados em texto, como Markdown. Eles são muitas vezes menores e as alterações neles são fáceis de rastrear por meio do histórico de controle de versão.

O ponto crucial com a documentação escrita manualmente é mantê-la atualizada. Arquivos de texto ou wikis são pacientes e não esquecem e, com o tempo, muitas páginas de documentação praticamente se acumulam em seu armazenamento. Fica problemático quando não está claro qual documentação está correta e qual está desatualizada. Uma vez em dúvida, não é mais confiável.

A única maneira de resolver esse problema é configurar um processo que garanta que os documentos sejam atualizados. No capítulo anterior, já apresentamos uma forma possível: revisões de código em combinação com uma Definição de

Pronto (DoD). Isso garante que, sempre que estivermos prestes a adicionar algum código novo ou alterado à nossa base de código, sejamos lembrados por uma lista de verificação para atualizar a documentação, se necessário.

Em particular, a arquitetura do sistema e do software é documentada por meio de diagramas. Portanto, na próxima seção, queremos mostrar como criá-los de maneira eficaz.

Diagramas

Um bom diagrama geralmente é muito mais revelador do que um longo texto. Existem muitas ferramentas de diagramação gratuitas disponíveis e você pode optar por desenhar manualmente os diagramas ou gerá-los a partir de definições de texto.

Desenhar diagramas manualmente

A maneira clássica de criar um diagrama é usando uma ferramenta de diagramação que permite desenhá-lo manualmente. Essas ferramentas foram projetadas especificamente para auxiliá-lo no processo de criação, por exemplo, oferecendo modelos e conjuntos de ícones ou mantendo as setas de conexão entre os objetos se eles forem movidos.

Uma ferramenta versátil que queremos apresentar a você neste capítulo é o diagrams.net (<https://www.diagrams.net>). Na verdade, também o usamos para criar ilustrações para este livro. Ele oferece uma biblioteca de elementos que, por exemplo, podem ser usados para criar diagramas como diagramas e fluxogramas Unified Modeling Language (UML). Ele também oferece ícones para os provedores de nuvem mais populares, como Google Cloud Platform (GCP), Amazon Web Services (AWS) e Microsoft Azure.

Se você pretende usá-lo, recomendamos salvar seus diagramas como Scalable Vector Graphics (SVG). O SVG é baseado em Extensible Markup Language (XML) e, embora o XML seja bastante detalhado, ele ainda consome menos espaço em disco do que formatos gráficos como Portable Network Graphics (PNG).

Mais importante, ele pode ser carregado e alterado no editor repetidamente, para que você não precise começar de novo toda vez que o sistema for alterado. A maioria dos ambientes de desenvolvimento integrado (IDEs) e todos os navegadores exibem arquivos SVG como imagens gráficas que podem até ser dimensionadas indefinidamente e, se necessário, podem ser facilmente exportadas para os formatos de imagem mais populares.

Gerando diagramas a partir de definições

Nem todo mundo gosta de usar editores complicados para desenhar diagramas, no entanto. Felizmente, há uma variedade de ferramentas de diagramação que podem gerar diagramas a partir de definições. Para demonstrar como eles funcionam em geral, escolhemos Mermaid.js (<https://mermaid-js.github.io>) como um exemplo. Ele é escrito em JavaScript e utiliza uma linguagem inspirada no Markdown para definir os diagramas.

Antes de verificarmos as vantagens dessa abordagem, vamos primeiro dar uma olhada em um exemplo simples de um fluxograma:

gráfico LR

A{Você sabe como escrever um ótimo código PHP?} --> B[Não]

A --> C[Sim]

C --> E(Incrível!)

B --> D{Você leu Código Limpo em PHP?} --> F[Não]

D --> G[Sim]

G --> H(Por favor, leia novamente)

F --> I(Por favor, leia)

O código anterior renderizaria um diagrama, conforme mostrado aqui:

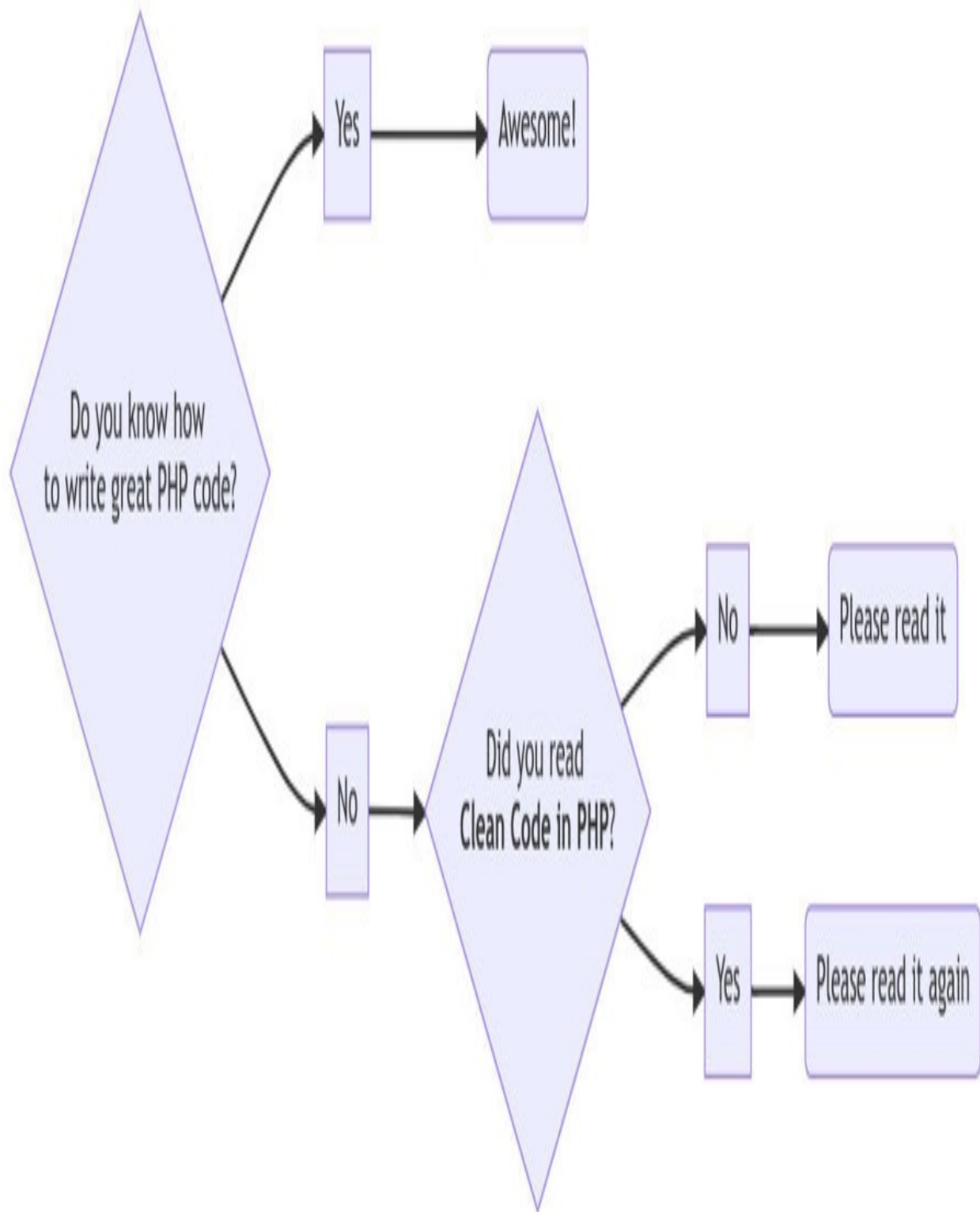


Figura 13.1: Exemplo de diagrama de sereia

As ferramentas de geração de diagramas ajudam você a criar vários tipos de diagramas, como diagramas de sequência, gráficos de Gantt ou até mesmo os conhecidos gráficos de pizza. Você não precisa pensar em como estilizá-los ou como eles são organizados. O trabalho principal é assumido pela ferramenta de diagramação. Obviamente, o Mermaid.js oferece muitas maneiras de afetar a aparência dos diagramas gerados.

Como as definições do diagrama são blocos de texto simples, elas podem ser adicionadas ao repositório de código. As alterações neles são confortavelmente rastreáveis por meio do histórico de versões. Os diagramas de sereia se integram especialmente bem em documentos Markdown, pois os IDEs mais populares podem exibir esses diagramas diretamente no documento por meio de extensões adicionais.

Por último, se você quiser apenas brincar com as possibilidades do Mermaid, você pode usar o Mermaid Live Editor (<https://mermaid.live>) para entender melhor como funciona.

alternativas de sereia

Outras ferramentas de diagramação dignas de nota são PlantUML (<https://plantuml.com>), que oferece tipos de diagramas ainda mais práticos para documentar a arquitetura de software, e Diagramas (<https://diagrams.mingrammer.com>), que é forte na documentação da arquitetura

do sistema.

Geradores de documentação

Provavelmente, a melhor documentação é aquela que não precisamos criar nós mesmos e ainda é tão útil quanto o conteúdo escrito por humanos. Infelizmente, isso continuará sendo um sonho por enquanto, embora não saibamos aonde o aprendizado de máquina (ML) nos levará no futuro.

Neste momento, já podemos usar ferramentas para criar documentação a partir do nosso código. Pelo menos, podemos usá-los para agregar informações que estão espalhadas pelas várias classes de nossos projetos.

Documentação da API

Nesta seção, mostraremos como criar documentação a partir do código usando um exemplo de documentação de API. Caso sua aplicação forneça uma API, é fundamental ter a documentação atualizada da mesma. Escrever tal documentação é um processo demorado e sujeito a erros, mas podemos pelo menos torná-lo um pouco mais fácil.

Existem muitas abordagens para documentar APIs. Neste livro, apresentaremos um formato que se tornou cada vez mais popular: OpenAPI. Esse formato, anteriormente conhecido como Swagger, descreve todos os aspectos de uma API em um documento YAML Ain't Markup Language (YAML), que pode ter esta aparência:

openapi: 3.0.0

informações:

título: 'API do produto'

versão: '0.1'

caminhos:

/produtos:

pegue:

operationId: getProductsUsingAnnotations

Parâmetros:

-

nome: limite

em: consulta

descrição: 'Quantos produtos devolver'

obrigatório: falso

esquema:

tipo: inteiro

respostas:

'200':

descrição: 'Retorna os dados do produto'

Isso pode ser um pouco demais de informação à primeira vista. Não se preocupe - não é tão complicado. Em poucas palavras, o YAML anterior descreve a API do produto em sua versão 0.1, que oferece um endpoint, /product. Este endpoint pode ser chamado utilizando o verbo GET do Hypertext Transfer Protocol (HTTP) e aceita o parâmetro opcional limit, que é do tipo integer e deve ser escrito na consulta da URL (por exemplo, /product?limit=50). Se tudo correr bem, o endpoint retornará com o código HTTP 200.

Documentação OpenAPI

O formato OpenAPI é bastante extenso, portanto não podemos abordá-lo neste livro. Se você estiver interessado em aprender mais sobre isso, consulte a documentação oficial:<https://oai.github.io/Documentation>.

Como um benefício bem-vindo, IDEs como o PhpStorm por padrão oferecem suporte na gravação desses arquivos YAML, verificando a validade do esquema. Se você, por exemplo, escrevesse `operation` em vez de `operationId`, o IDE destacaria o uso incorreto.

Você pode escrever um arquivo YAML manualmente ou gerá-lo. Queremos dar uma olhada mais de perto no último caso de uso. Para conseguir isso, precisamos da ajuda de um pacote do Composer chamado `swagger-php` (<https://github.com/zircote/swagger-php>). Consulte a documentação do pacote para saber como instalá-lo.

Claro, o pacote não pode criar documentação magicamente do nada. Em vez disso, o `swagger-php` analisa metainformações que são escritas diretamente no código PHP, seja como anotações DocBlock ou, com o PHP 8.1, como atributos. Em outras palavras, precisamos ter certeza de que as metainformações já estão lá antes de podermos gerar o arquivo YAML.

Como são essas informações? Vejamos o primeiro exemplo, usando anotações:

```
/**  
  
 * @OA\Info(  
  
 * title="API do produto",  
  
 * versão="0.1"  
  
 * )
```



```
*/
```

```
classe ProductController
```

```
{
```

```
/**
```

```
* @OA\Get(
```

```
* caminho="/produto",
```

```
* operationId="getProducts",
```

```
* @OA\Parameter(
```

```
* nome="limite",
```

```
* in="consulta",
```

```
* description="Quantos produtos devo devolver",
```

```
* obrigatório=falso,
```

```
* @OA\Schema(
```

```
* tipo="inteiro"
```

```
* )
```

```
* ),
```

```
* @OA\Resposta(
```

```
* resposta="200",
```

```
* description="Retorna os dados do produto"
```

```
* )
```

```
* )
```

```
*/
```

```
função pública getProducts(): array
```

```
{
```

```
// ...
```

```
}
```

```
}
```

Com base nas informações contidas nos DocBlocks, o swagger-php retornará a documentação de nossa API como um arquivo YAML que será exatamente igual ao exemplo anterior. Mas por que deveríamos usar swagger-php quando poderíamos escrever o YAML diretamente?

De fato, nem todo mundo quer ter grandes blocos de documentação dentro do código e, dependendo do nível de detalhe que você deseja documentar, eles podem se tornar muito maiores do que no exemplo anterior. Se você pensar em uma API com muitos endpoints espalhados por vários controladores em seu código, você já deve ter percebido os benefícios: todas as metainformações necessárias são armazenadas perto do código, portanto, se forem feitas alterações no endpoint, é muito mais fácil para o desenvolvedor simplesmente corrigir as anotações do DocBlock do que fazer essas alterações em algum documento ou wiki adicional. Como os comentários fazem parte do código, as alterações também já estão sob controle de versão. No final, a decisão de usar o swagger-php depende de você ou da equipe.

Na seção de documentação Inline deste capítulo, discutiremos por que os DocBlocks não são o melhor lugar para armazenar metainformações. Desde o PHP 8.0, felizmente temos um lugar melhor para eles - ou seja, atributos, sobre os quais já falamos em Capítulo 6, PHP está evoluindo - depreciações e

revoluções.

Antes de discutirmos por que eles são a melhor opção, vamos dar uma olhada em como nosso endpoint seria documentado usando atributos:

```
use OpenApi\Attributes como OAT;
```

```
#[OAT\Info(
```

```
    versão: '0.1',
```

```
    título: 'API do produto',
```

```
)]
```

```
classe ProductController
```

```
{
```

```
    #[OAT\Get(
```

```
        caminho: '/v2/produto',
```

```
        id da operação: 'getProducts',
```

```
        parâmetros: [
```

```
            novo OAT\Parameter(
```

```
                nome: 'limite',
```

```
                descrição: 'Quantos produtos devolver',
```

```
                em: 'consulta',
```

```
                obrigatório: falso,
```

```
esquema: novo OAT\Eschema(  
    tipo: 'inteiro'  
),  
),  
],  
respostas: [  
    novo OAT\Resposta(  
        resposta: 200,  
        descrição: 'Retorna os dados do produto',  
    ),  
]  
)]  
função pública getProducts(): array  
{  
    // ...  
}  
}
```

Reconhecidamente, a sintaxe do atributo pode parecer um pouco estranha. Recomendamos o uso de atributos em vez de anotações, no entanto, porque eles vêm com vantagens convenientes. Em primeiro lugar, eles são códigos reais; eles são analisados pelo interpretador PHP, e seu IDE poderá ajudá-lo a escrevê-los. Na primeira linha do exemplo anterior, você pode ver que precisamos importar o

namespace OpenApi\Attributes para fazer este exemplo funcionar.

Nesse namespace, você encontrará classes reais referenciadas aqui. Os arquivos estão localizados dentro da pasta vendor do seu projeto. Isso permite que você use recursos como autocompletar e receberá um feedback imediato de seu IDE se algo não estiver correto, o que facilita muito a redação dessa documentação.

Como última etapa, você precisa gerar um arquivo YAML a partir do código. Essa etapa pode, é claro, ser automatizada no pipeline de CI, que introduzimos em Capítulo 11, Integração contínua. Você pode encontrar exemplos de uso em nosso repositório Git para este livro.

Você pode se perguntar: o que posso fazer com esta documentação da API? Com certeza, ele já pode funcionar como documentação para outros desenvolvedores, mas tem muito mais do que isso. Você pode, por exemplo, importá-lo para clientes HTTP como Insomnia ou Postman. Dessa forma, você pode começar a interagir imediatamente com a API sem precisar procurar o esquema exato.

Outro caso de uso é ajudá-lo a escrever testes funcionais para sua API. Existem pacotes como PHP Swagger Test (<https://github.com/byjg/php-swagger-test>) ou Espectador (<https://github.com/hotmeteor/spectator>) que pode ajudá-lo a escrever testes em relação à Especificação OpenAPI (OAS), que também pode ser considerada um contrato. Você pode testar, por exemplo, se o objeto retornado para um código de status HTTP específico é igual ao especificado naquele contrato.

Por último, e provavelmente o caso de uso mais importante, é usar a especificação OAS com Swagger UI (<https://github.com/swagger-api/swagger-ui>)

ui), que é uma documentação visual e interativa de sua API.

A captura de tela a seguir mostra como seria nossa API de exemplo:

Product API

0.1

OAS3

/openapi-attributes.json

default



GET /product



Parameters

Try it out

Name	Description
------	-------------

limit	How many products to return at one time
-------	---

integer
(query)

limit

Responses

Code	Description	Links
200	Returns the product data	No links

Figura 13.2: IU do Swagger

Explorar todas as possibilidades de OpenAPI e Swagger UI iria além do escopo de nosso livro. Recomendamos que você verifique ambas as ferramentas se quiser aprender mais sobre elas.

Alternativas OpenAPI

Existem outros formatos, como RESTful API Modeling Language (RAML) (<https://raml.org>) ou API Blueprint (<https://apiblueprint.org>) que você poderia usar, e não somos opinativos em relação a nenhuma solução.

Documentação em linha

Um caso especial é a documentação que muitos de nós fazemos regularmente desde que começamos a escrever software: comentários. Eles são escritos diretamente no código, onde os desenvolvedores podem vê-los imediatamente, então esse parece ser um bom lugar para colocar a documentação. Mas os comentários devem realmente ser vistos ou usados para documentação?

Em nossa opinião, os comentários geralmente devem ser evitados. Vejamos alguns argumentos nas próximas páginas.

Anotações não são código

Comentários não fazem parte do código. Embora seja possível analisar comentários por meio da API de reflexão do PHP, eles originalmente não foram criados para armazenar metainformações. Idealmente, seu software ainda deve funcionar da mesma forma depois de remover todos os comentários.

Hoje, porém, isso muitas vezes não é mais o caso. Estruturas e pacotes como mapeadores relacionais de objeto (ORMs) usam anotações DocBlock para armazenar informações neles, como definições de rota ou relações entre objetos de banco de dados. Algumas ferramentas de qualidade de código usam anotações para controlar seu comportamento em determinadas partes do código.

O PHP não pode lançar mensagens de erro se suas anotações estiverem erradas. Se eles servirem a um propósito importante, esperamos que seus testes detectem o bug antes de serem implantados na produção. Uma escolha melhor são os atributos, que são uma construção de linguagem real. Discutimos isso com mais detalhes anteriormente neste capítulo, quando falamos sobre a documentação da API.

Código ilegível

Além disso, como já discutimos em Capítulo 12, Trabalhando em equipe, os comentários costumam ser um indicador de código muito complexo. Em vez de explicar seu código, você deve tentar escrever um código que não precise ser comentado em primeiro lugar.

Pode ser um exercício divertido comprimir uma função inteira em uma única linha - por exemplo, usando alguns operadores ternários aninhados quádruplos ou uma cláusula if assustadoramente complicada que ninguém entenderá. Você vai se arrepender de tê-lo escrito no momento em que há um bug de alta prioridade no ambiente de produção, e cabe a você corrigi-lo sem ter a menor ideia do que sua obra-prima enigmática deve fazer.

Ou, pior ainda, seu novo colega em seu primeiro turno de plantão tem a honra de depurar tarde da noite, quando os alertas continuam chegando. Existem maneiras melhores de iniciar uma relação de trabalho.

comentários desatualizados

Um comentário é rapidamente escrito, mas também rapidamente esquecido. Como os comentários não são analisados pelo interpretador PHP, você não será informado quando eles não estiverem mais corretos - por exemplo, quando a função que eles deveriam explicar for reescrita e de repente servir a um propósito diferente. Não há outra maneira de validar os comentários do que um desenvolvedor tentando ler e entender seu significado e compará-lo com o código da função real.

No momento em que escrevo, isso pode não parecer um problema, mas imagine voltar a uma aula depois de um ano e encontrar um comentário que você não entende mais. Por que você escreveu isso em primeiro lugar? E se você não sabe por quê, como é que qualquer outra pessoa pode saber?

Comentários desatualizados são informações erradas dentro do seu código. Eles distraem e custam caro, pois o tempo do desenvolvedor não vem de graça. Portanto, pense duas vezes antes de adicioná-los.

comentários inúteis

Tente evitar comentários que digam o óbvio e não acrescentem mais informações. O trecho de código a seguir é um exemplo real disso:

```
// grava a string no arquivo de log  
  
file_put_contents($logFileName, $someString)
```

Embora possa ser considerado um bom gesto que o desenvolvedor tenha dedicado tempo para explicar a função `file_put_contents`, isso não agrega valor ao código. Se você não conhece uma função, pode procurá-la. Fora isso, é apenas uma linha de código desnecessária que você precisa verificar ao ler o código.

Às vezes não é fácil traçar uma linha entre comentários úteis e inúteis. Você pode usar revisões de código para lidar com esse problema; como discutido em Capítulo 12, Trabalhando em equipe, ter alguém de sua equipe fazendo revisões honestas de seu código ajudará a evitar comentários como esses.

DocBlocks errados ou inúteis

Já discutimos DocBlocks e o que os torna problemáticos em Capítulo 12, Trabalhando num time, quando introduzimos as diretrizes de codificação. Resumindo, como os DocBlocks são basicamente comentários (mas seguindo uma certa estrutura), eles podem ficar desatualizados ou simplesmente dar errado rapidamente se, por exemplo, os parâmetros de uma chamada de função mudarem e as alterações necessárias não forem atualizadas no DocBlock também. Seu IDE pode lançar avisos, mas o PHP não.

Com a introdução de dicas de tipos melhores no PHP, muitos DocBlocks podem ser simplesmente removidos. A redundância não é benéfica e pode confundir o leitor se o código real divergir das anotações.

TODO comentários

Os comentários não são um local adequado para armazenar tarefas. Você provavelmente conhece comentários como este:

```
// refatoração TODO assim que a nova versão do serviço XY for lançada
```

Embora alguns IDEs possam ajudá-lo a gerenciar seus comentários TODO, essa abordagem só funcionará se você for o único no projeto. Assim que você está Trabalhando em Equipe, usando uma ferramenta de gestão de trabalho como JIRA, Asana ou mesmo Trello, escrever tal comentário é simplesmente uma forma de criar dívida técnica, ou seja, você está descarregando a tarefa em algum dia incerto no futuro. Espera-se que alguém o conserte um dia - na maioria das vezes, isso não acontecerá.

Em vez de um comentário, considere criar uma tarefa em sua ferramenta de gerenciamento de trabalho preferida. Dessa forma, fica transparente para seus colegas, e fica muito mais fácil planejar esse trabalho.

Quando comentar é útil

Depois de discutir o que não deve ser comentado, resta algum caso de uso em que os comentários sejam úteis? De fato, não tantos, mas há algumas ocasiões em que os comentários ainda fazem sentido, como nos seguintes casos:

Por favor, tenha em mente que não queremos ser dogmáticos. Se você sentir que um comentário é necessário em algum momento, escreva-o. Ele ainda pode ser excluído, provavelmente após discutir o tópico com outro desenvolvedor na revisão do código.

Testes como documentação

Os desenvolvedores que escrevem testes costumam dizer que esses testes também funcionam como documentação. Nós também fizemos a reclamação em Capítulo 10, Testes Automatizados quando falamos sobre os benefícios dos testes automatizados.

Se você não sabe qual é o propósito de uma classe, pode pelo menos inferir seu comportamento esperado a partir dos testes, porque é exatamente isso que os testes fazem: eles fazem afirmações com as quais o código será testado. Olhando para essas afirmações, você sabe o que o código deve fazer.

Se os testes falharem, você pelo menos saberá que há uma discrepância entre as afirmações e o código real e não poderá confiar neles agora. A menos que as falhas de teste não sejam geralmente ignoradas em seu projeto, você pode ter certeza de que alguém as corrigirá em breve - ou, em outras palavras, a documentação implícita será atualizada.

Se todos os testes forem aprovados, você saberá que pode confiar na implementação da classe, desde que os testes sejam bem escritos e não testem apenas a implementação de um objeto fictício, conforme discutido em Capítulo 10, Testes Automatizados, quando falamos sobre testes de unidade.

Certamente, os testes de leitura e compreensão não são a forma mais fácil de documentação, mas podem ser uma fonte confiável da verdade (SOT) se não houver outra documentação. Eles devem, no entanto, não ser o único tipo de

documentação em seu projeto.

Resumo

Escrever código limpo não é apenas saber como fazer você mesmo, mas também garantir que outros desenvolvedores também sigam esse caminho. Para poder fazer isso, eles precisam conhecer as regras que se aplicam ao projeto.

Neste capítulo, discutimos como criar documentação que pode ajudá-lo a atingir esse objetivo. Discutimos as melhores práticas para escrever documentação manualmente, bem como criar diagramas informativos e, ao mesmo tempo, de fácil manutenção. Por fim, apresentamos maneiras de gerar documentação a partir do código e elaboramos os prós e contras da documentação inline.

Parabéns! Você chegou ao final deste livro. Esperamos que você tenha gostado de lê-lo e agora esteja totalmente motivado para escrever um código limpo.

Você provavelmente não terá sucesso com isso imediatamente. Fortalecer suas habilidades de codificação é um processo que pode ser frustrante e às vezes até difícil de fazer ao trabalhar em um projeto comercial. Tente ser paciente e, com o tempo, você ficará cada vez melhor.

Ler apenas um livro sobre código limpo certamente não é suficiente. Ao longo deste livro, muitas vezes conseguimos apenas arranhar os tópicos superficiais, e encorajamos você a se aprofundar nos tópicos que lhe interessam - e naqueles que você pode não considerar interessantes em primeiro lugar. Requer mais estudos, uma mente aberta e a vontade de aceitar feedback de outras pessoas

para aumentar suas habilidades.

No entanto, estamos convencidos de que, com este livro, oferecemos a você mais do que um ponto de partida sólido para sua futura jornada como um grande desenvolvedor PHP. Ficaríamos felizes se você pensasse assim também.